Cookbook

SharinPix offers open APIs to easily meet your business requirements.
Below is an extract of popular SharinPix implementation steps and code for common use cases. The code is hosted on Github - you can use the “Deploy to Salesforce” button to copy the code directly to your organization. Please note that the SharinPix AppExchange package needs to be installed and up-to-date for the deployment to work. Secondly, make sure the SharinPix secret is correctly set up - this is outlined in the SharinPix API Configuration section of the Getting Started document.

Prerequisites

If you have not done so yet, please check out the Getting Started document to have an idea how to make use of these cookbooks. The SharinPix for Objects section will be particularly helpful to make use of some demos.

What are SharinPix Image Abilities ?

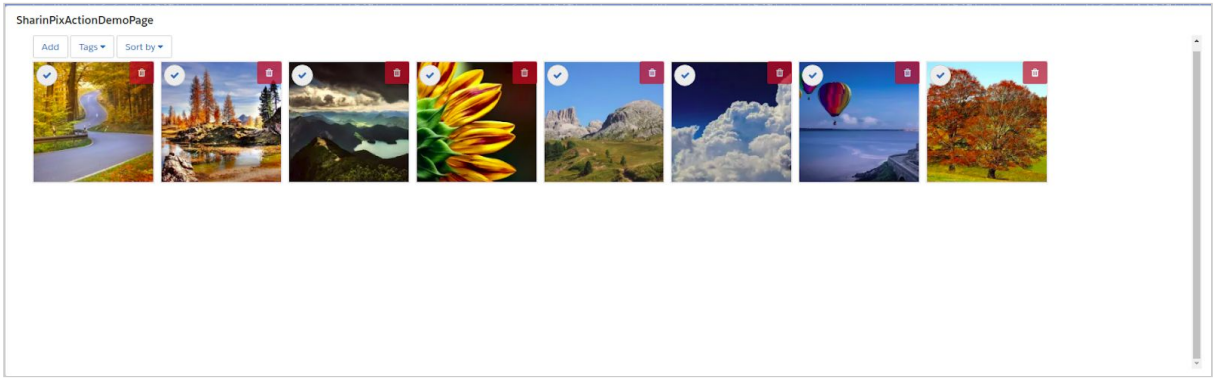
SharinPix abilities, to put in simple terms, are designations that have the possibility to expand or restrict the features enabled on the SharinPix Album.

Find below a list of abilities and the features they enable.

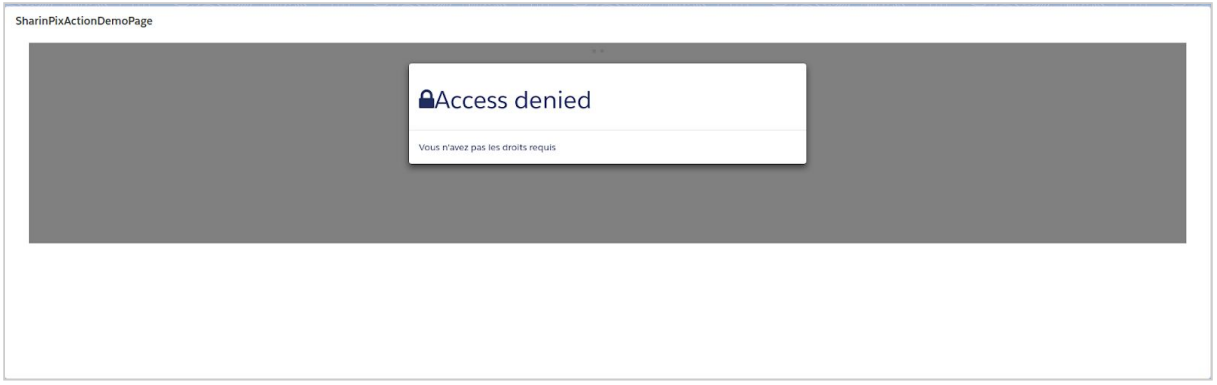
1. **See**

The **see** ability allows the images on a SharinPix Album to be accessible and visible.

Result on the SharinPix Album when **see** is **enabled**



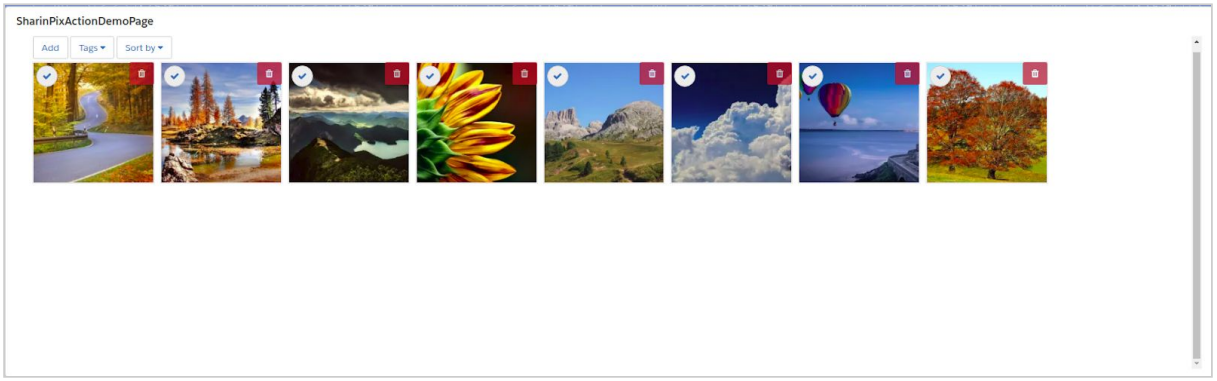
Result on the SharinPix Album when **see** is **disabled.**



2. **image_list**

The image_list ability allows the SharinPix Album to display images for all users.

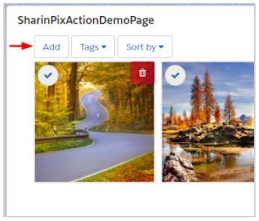
The result on the SharinPix Album when the **image_list** ability is enabled.



3. **image_upload**

The **image_upload** ability allows images to be added to the SharinPix Album.

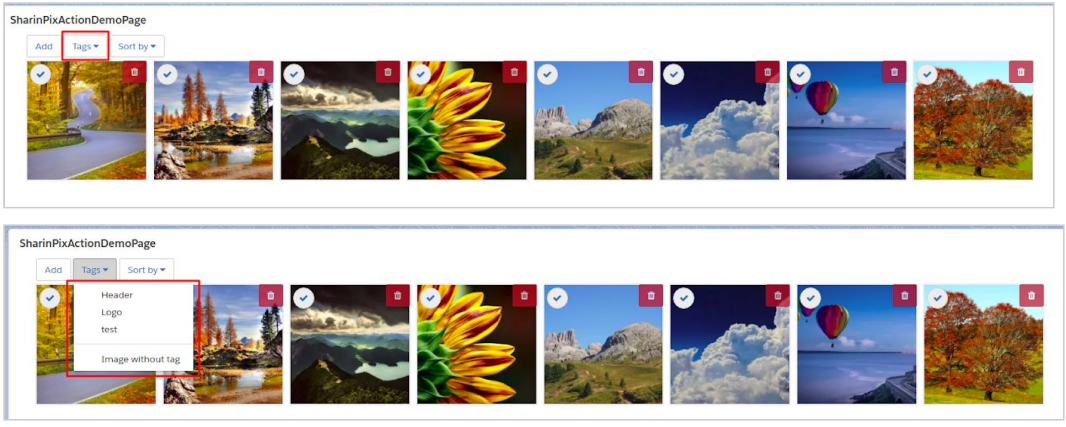
The result on the SharinPix Album when **image_upload** is enabled.



4. **image_tag**

The **image_tag** ability allows images to be tagged with different labels.

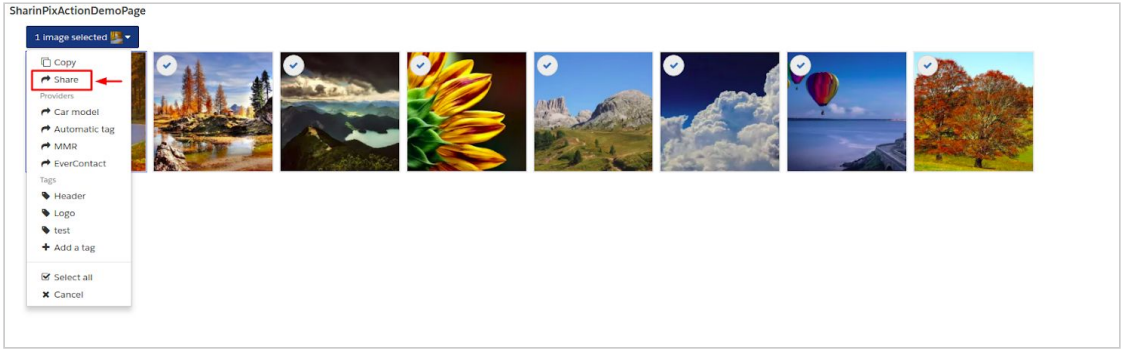
The result on the SharinPix Album when the **image_tag** ability is enabled.



5. **share**

The **share** ability allows the creation of a link that makes it possible for selected images to be shared publicly.

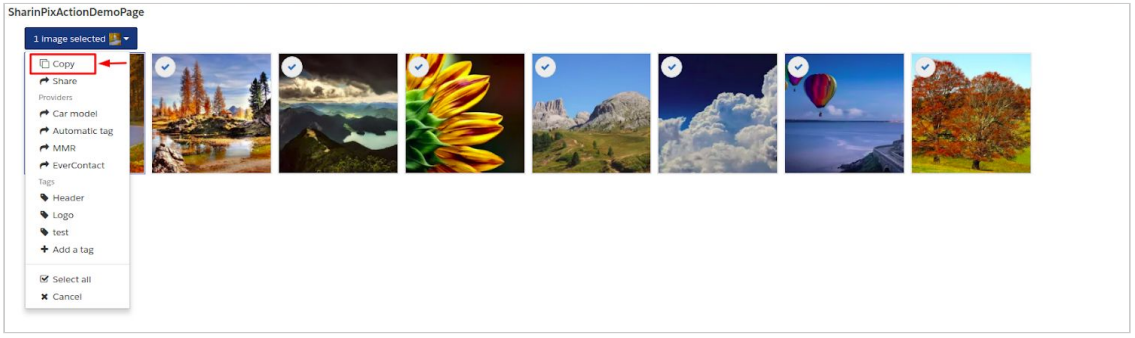
The result on the SharinPix Album when **share** is enabled is shown below.



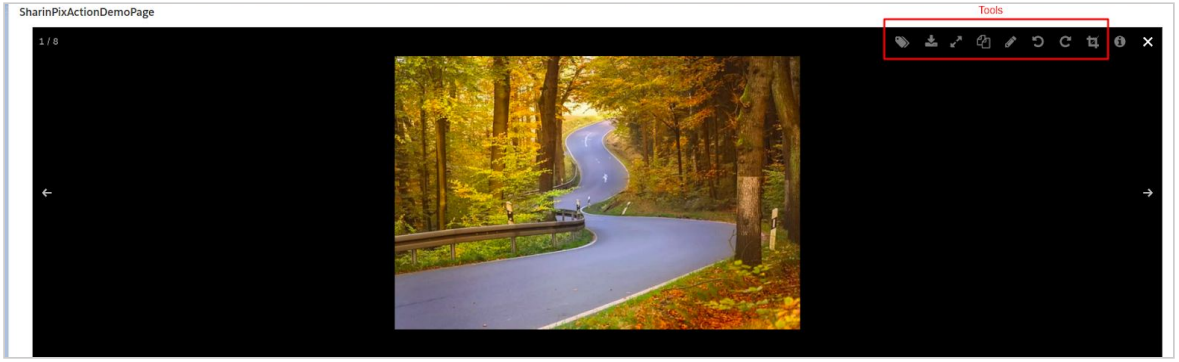
6. **image_copy**

The **image_copy** ability allows the user to select one or more images and copy them from one SharinPix Album and paste them onto another SharinPix Album.

The result on the SharinPix Album when **image_copy** is enabled is shown below.



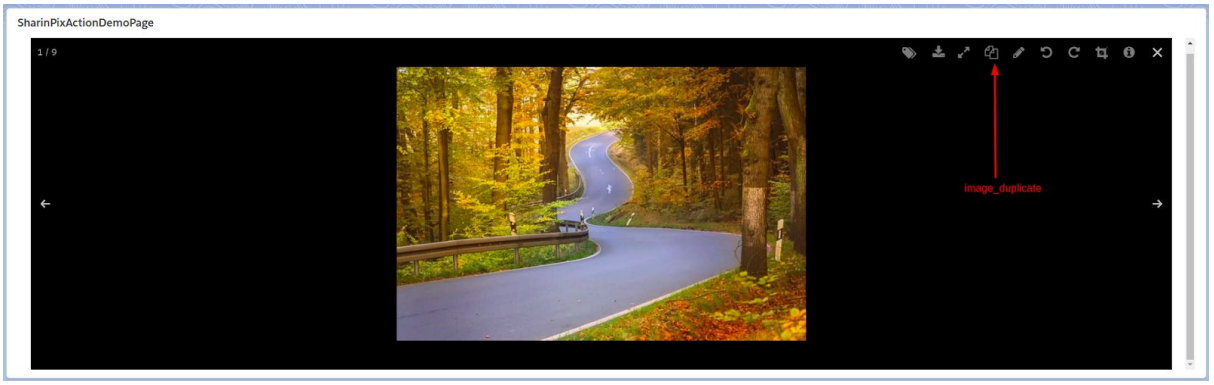
The abilities below define what set of tools will be made available for all users within the SharinPix toolbar. The SharinPix toolbar can be accessed when clicking upon an image in the SharinPix Album. The figure below shows the SharinPix toolbar along with the chosen image.



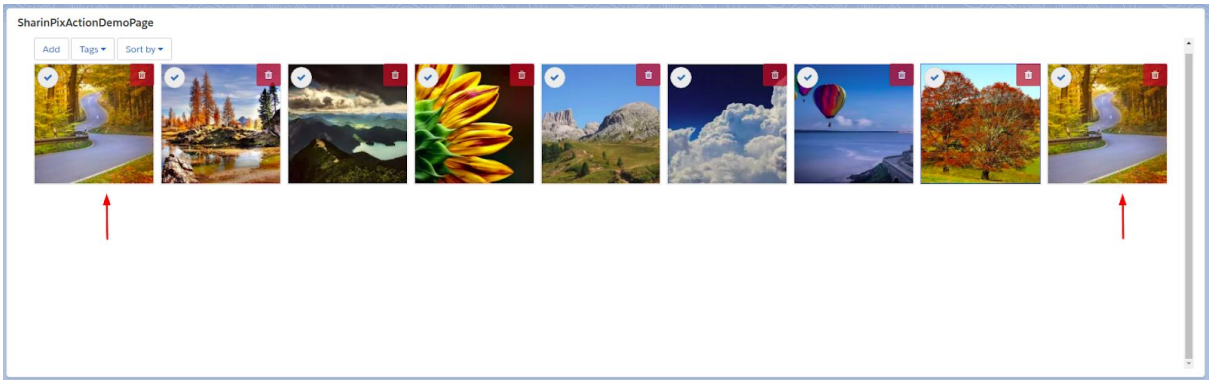
7. Image_duplicate

The ability **image_duplicate**, as its name suggests, duplicates(makes an exact copy) of a selected image.

The figure below indicates the icon for the **image_duplicate** ability.



The figure below shows the SharinPix Album after an image has been duplicated.



8. Image_annotate

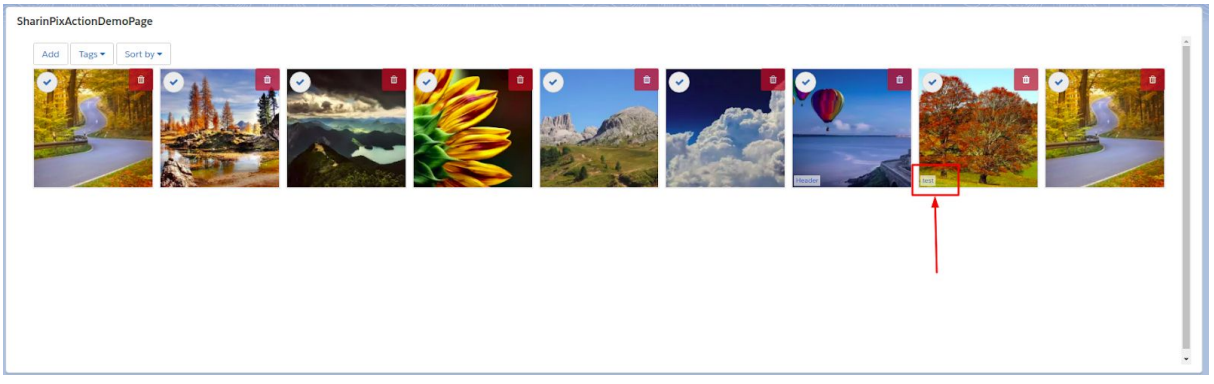
The ability **image_annotate** allows the user to assign a label to an image and thus allowing the images in a SharinPix Album to be filtered according to this label.

The figure below illustrates the icon for the **image_annotate** ability.

The subsequent figures illustrate how an annotation/label is assigned to an image.

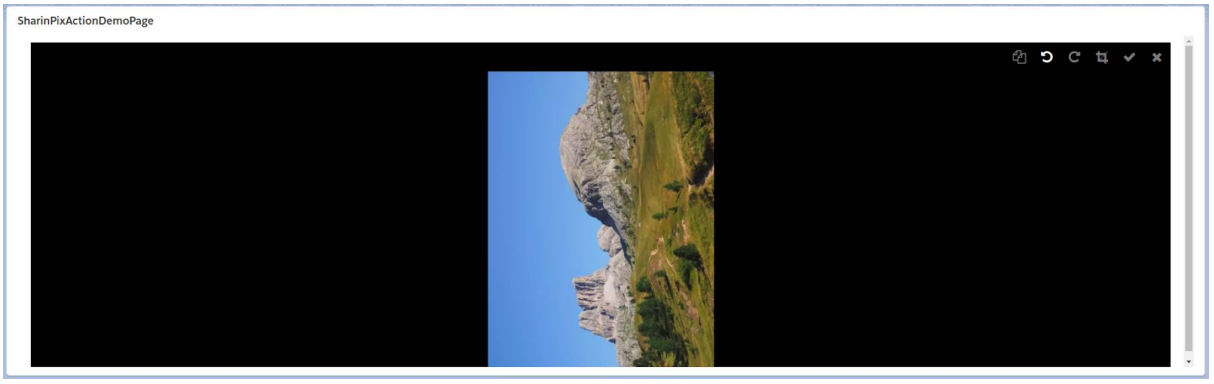
Select label.

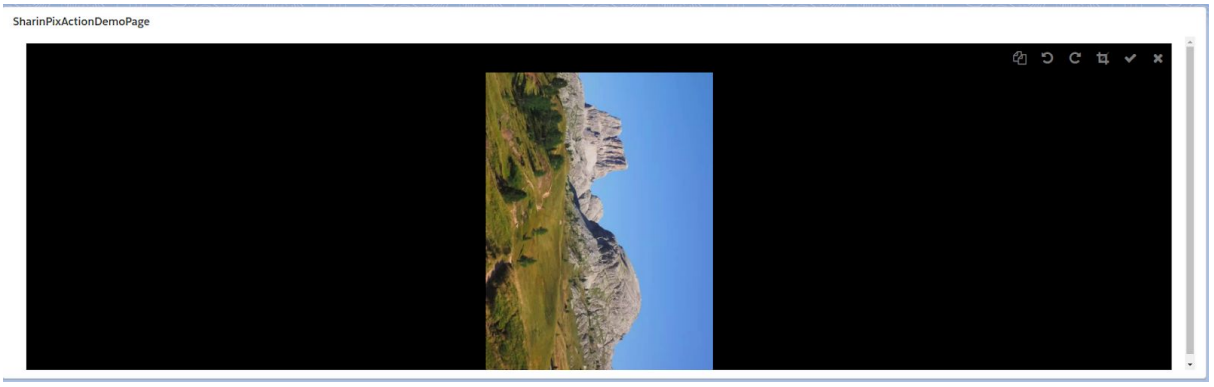
Label is displayed at the left-hand-side foot of the image.



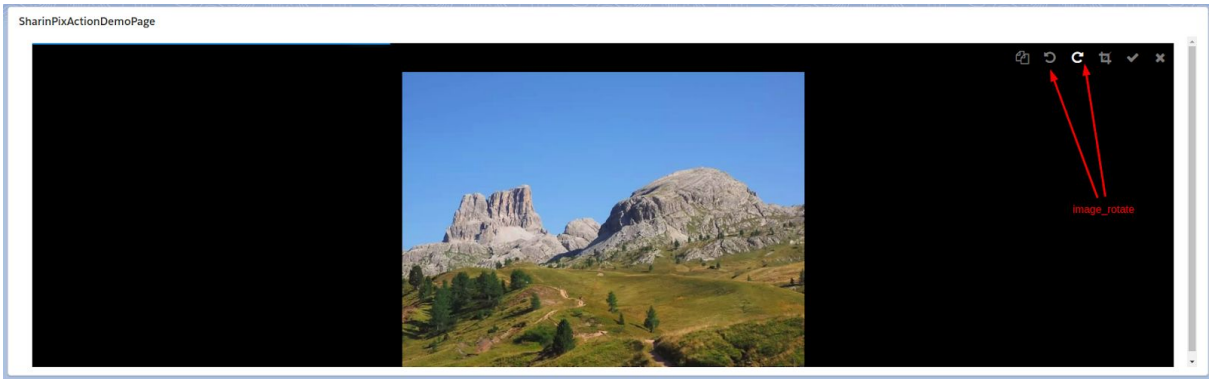
9. image_rotate

The **image_rotate** ability, as its name suggests, allows the user to rotate the image in a clockwise or anti-clockwise direction.





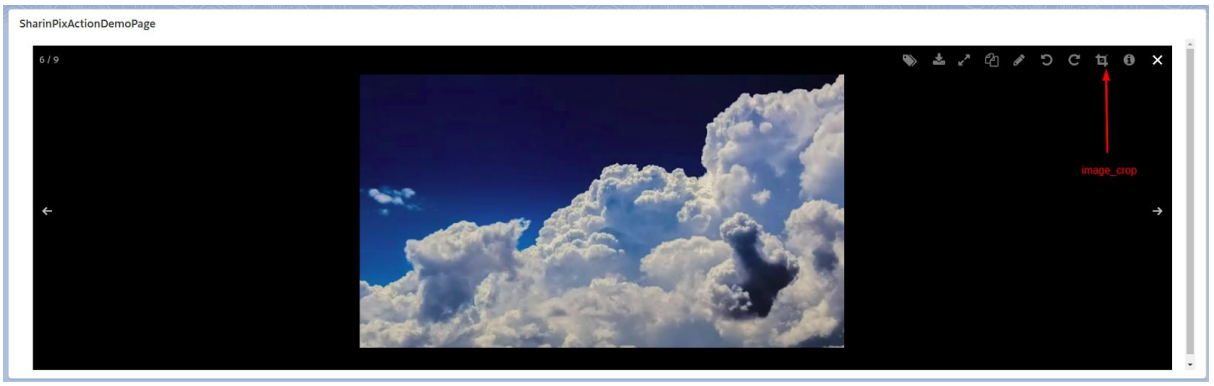
The figure below shows the icons for the **image_rotate** ability.



10. image_crop

The **image_crop**, as its name suggests, allows the user to crop any part of a selected image.

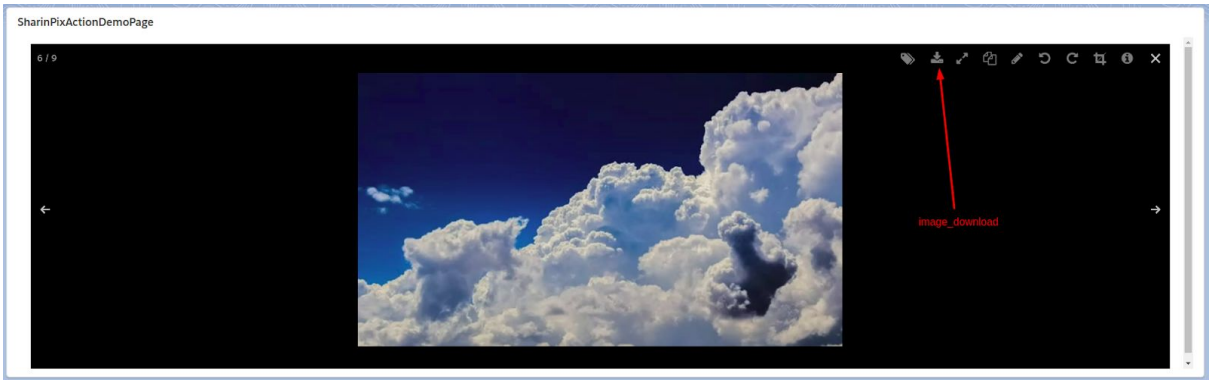
The figure below shows icon for the ability **image_crop**.



11. image_download

The **image_download** ability, as its name suggests, allows the user to download a selected image on his/her own device.

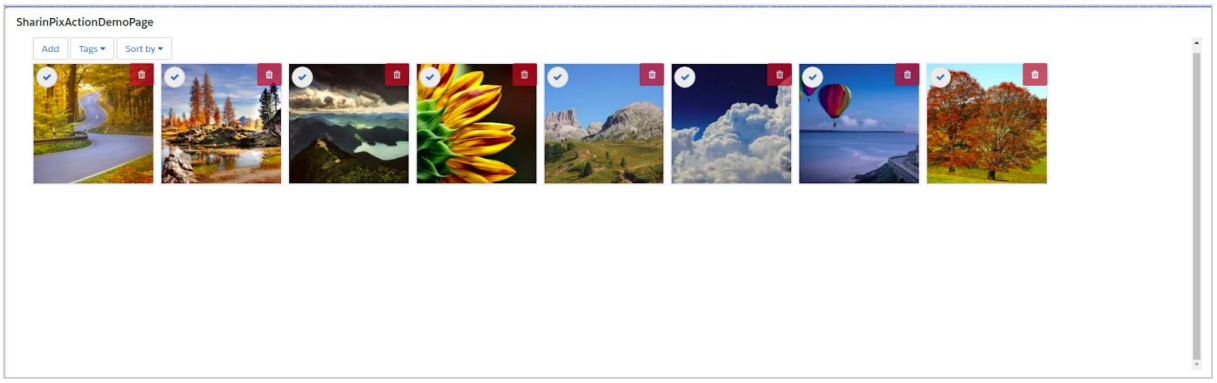
The figure below shows the icon for the **image_download** ability.



12. Image_delete

The **image_delete** ability, as its name suggests, allows the user to delete image(s) from the SharinPix Album.

The figure below shows the SharinPix Album when the ability **image_delete** is enabled.
a selected image

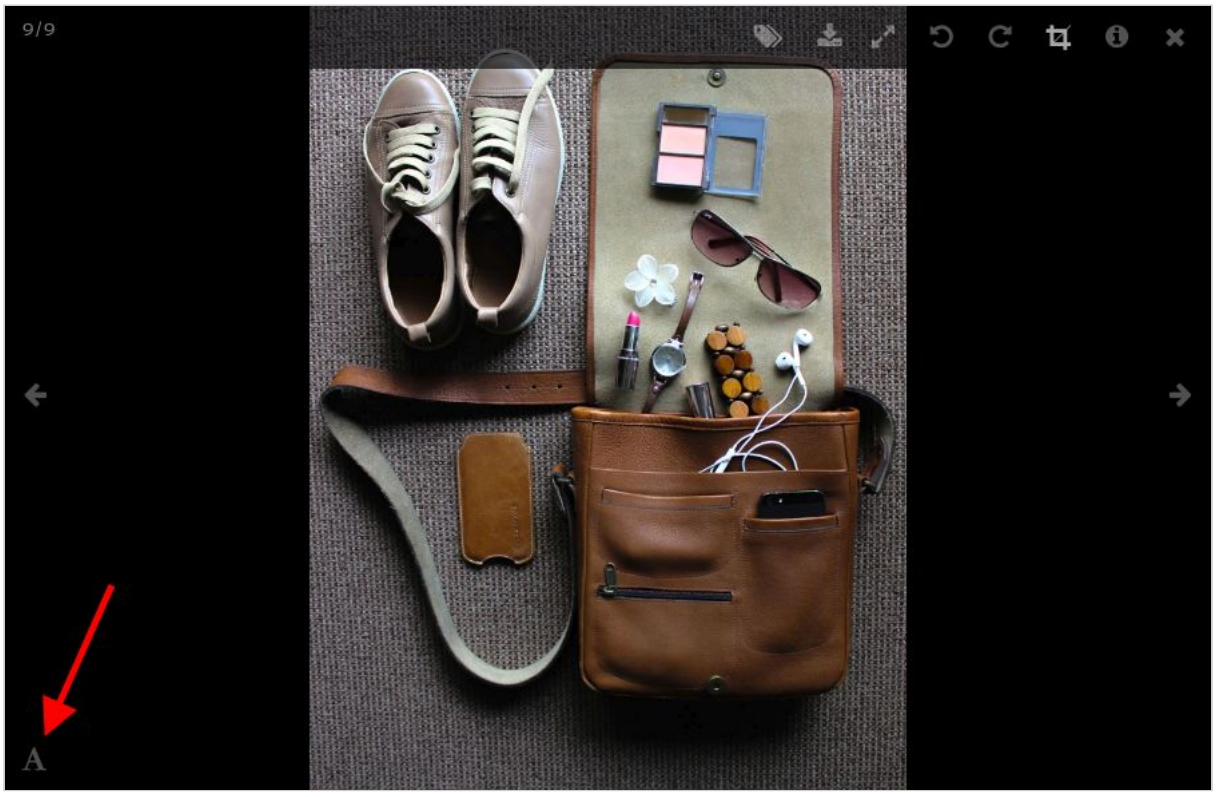


The figure below shows the SharinPix Album when the ability **image_delete** is disabled.

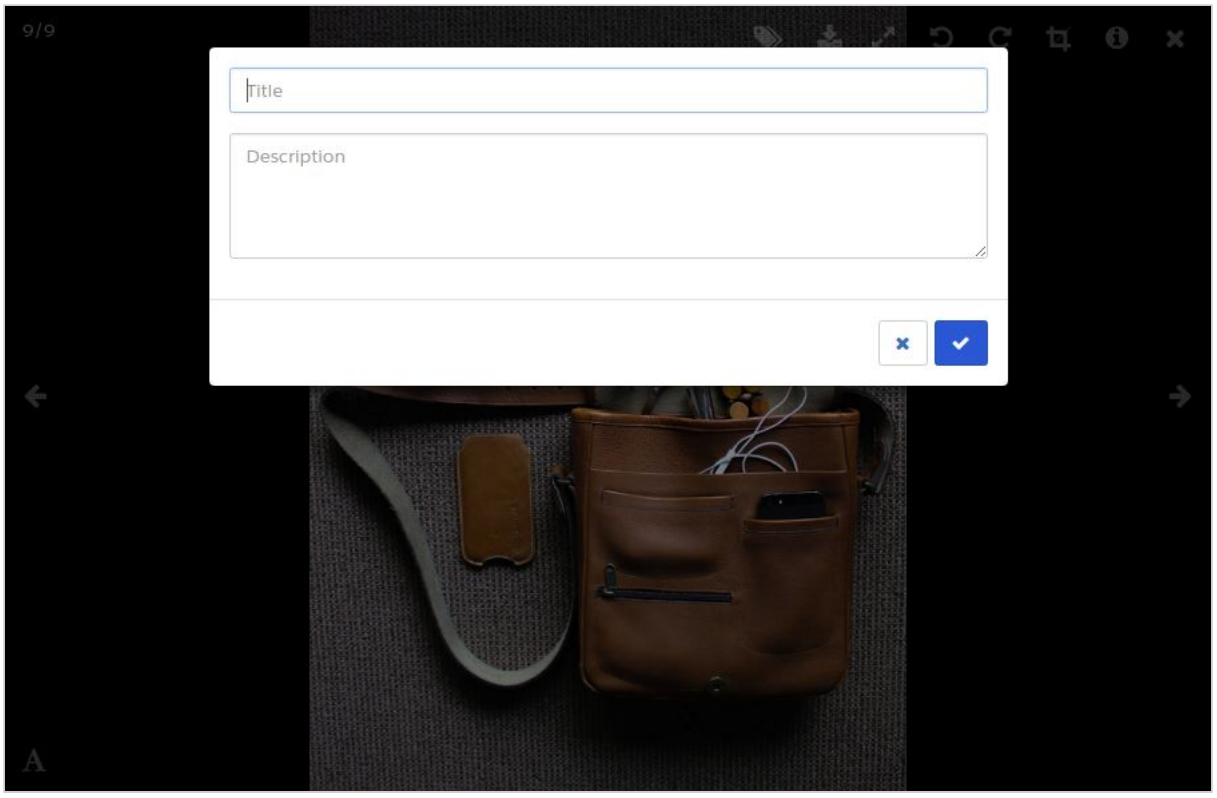
13. image_caption

The **image_caption** ability allows the user to add a title and a description to an image image(s) from the SharinPix Album.

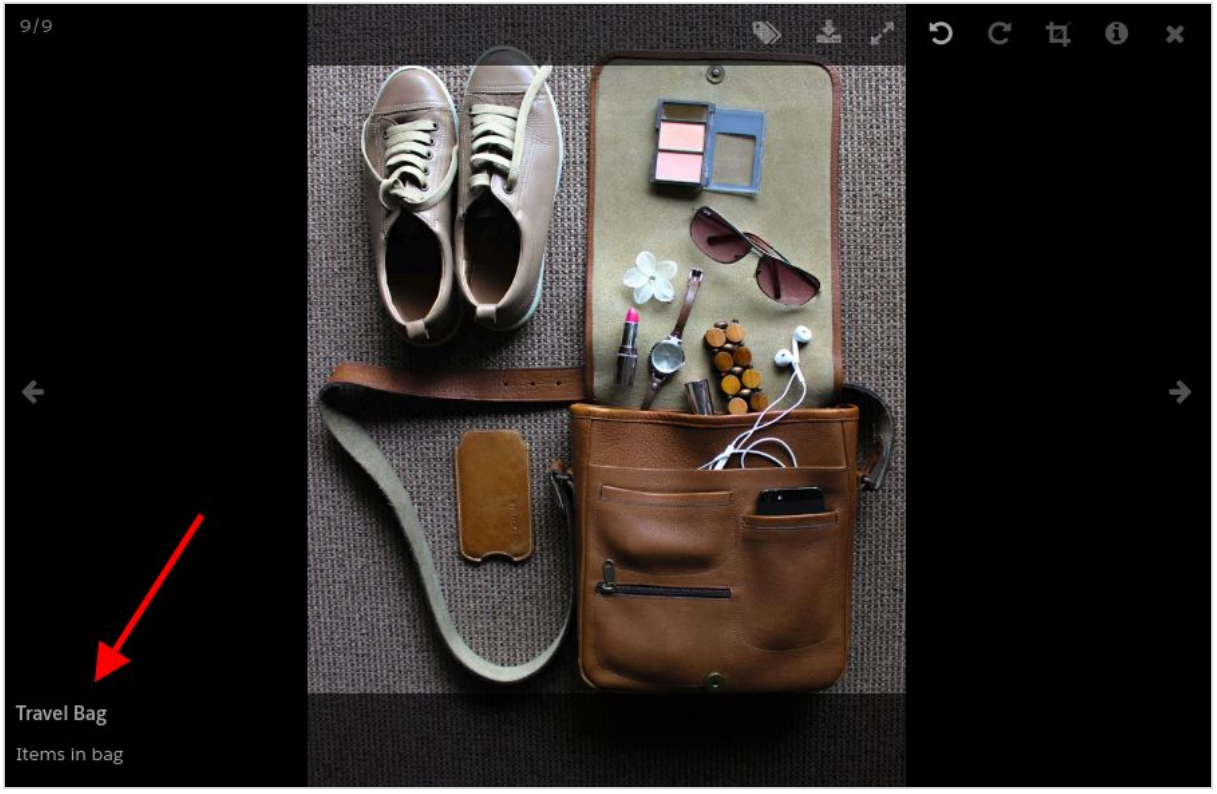
Select an image when **image_caption** is enabled. The icon as indicated by the red arrow in the figure below represents the image-captioning feature.



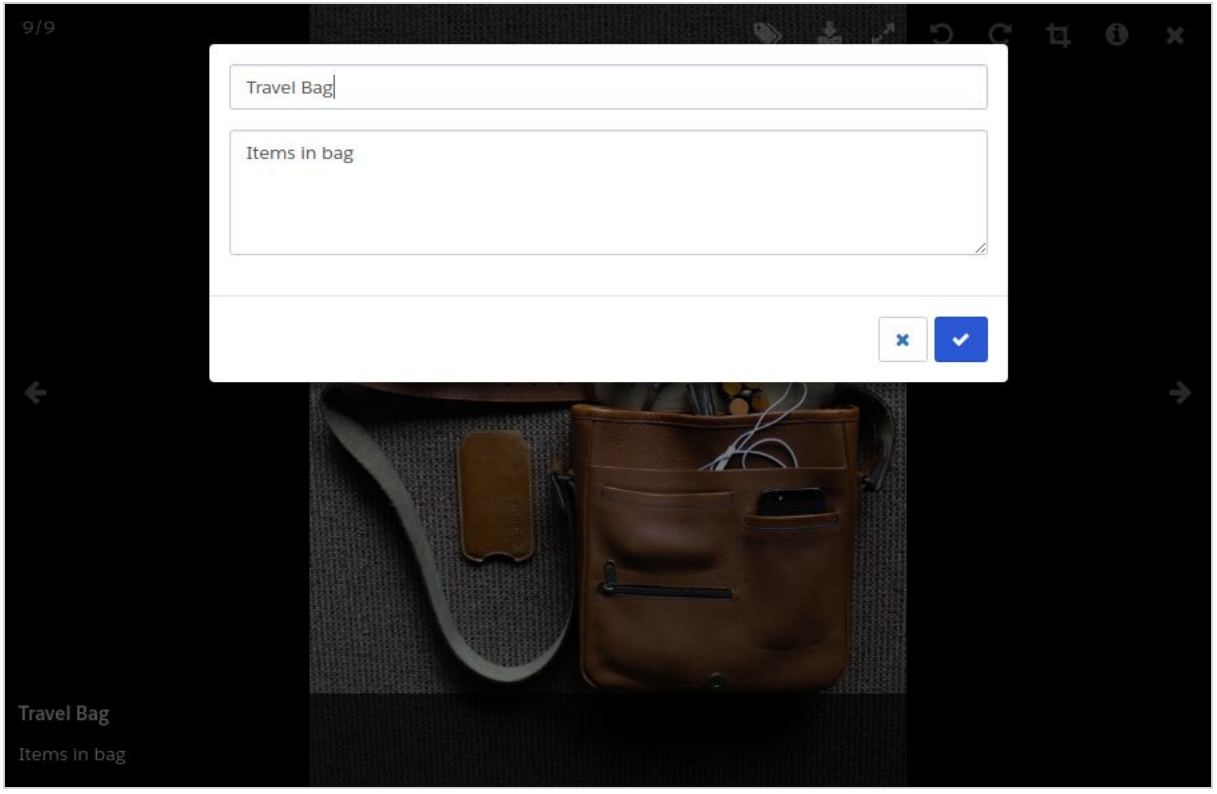
Upon clicking on this icon, a prompt for the title and description of the image is shown.



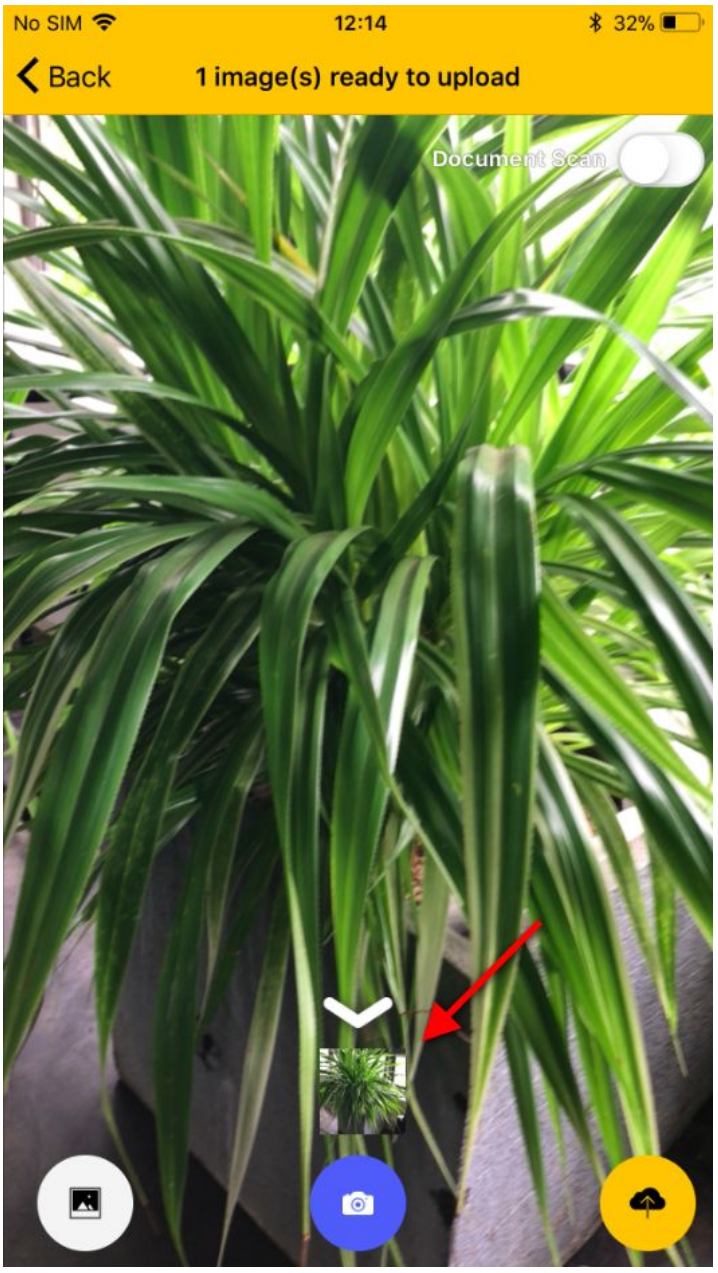
To save the title and description entered, click on the checkmark icon (✓). Once saved, the title and description are displayed at the left footer of the image viewer.



It is possible to edit the title and description by clicking at the same point as indicated in the figure above.



The figure below shows how the image-captioning feature is used within the SharinPix Mobile App. Once an image is snapped, you will be required to click on the image thumbnail as indicated below.

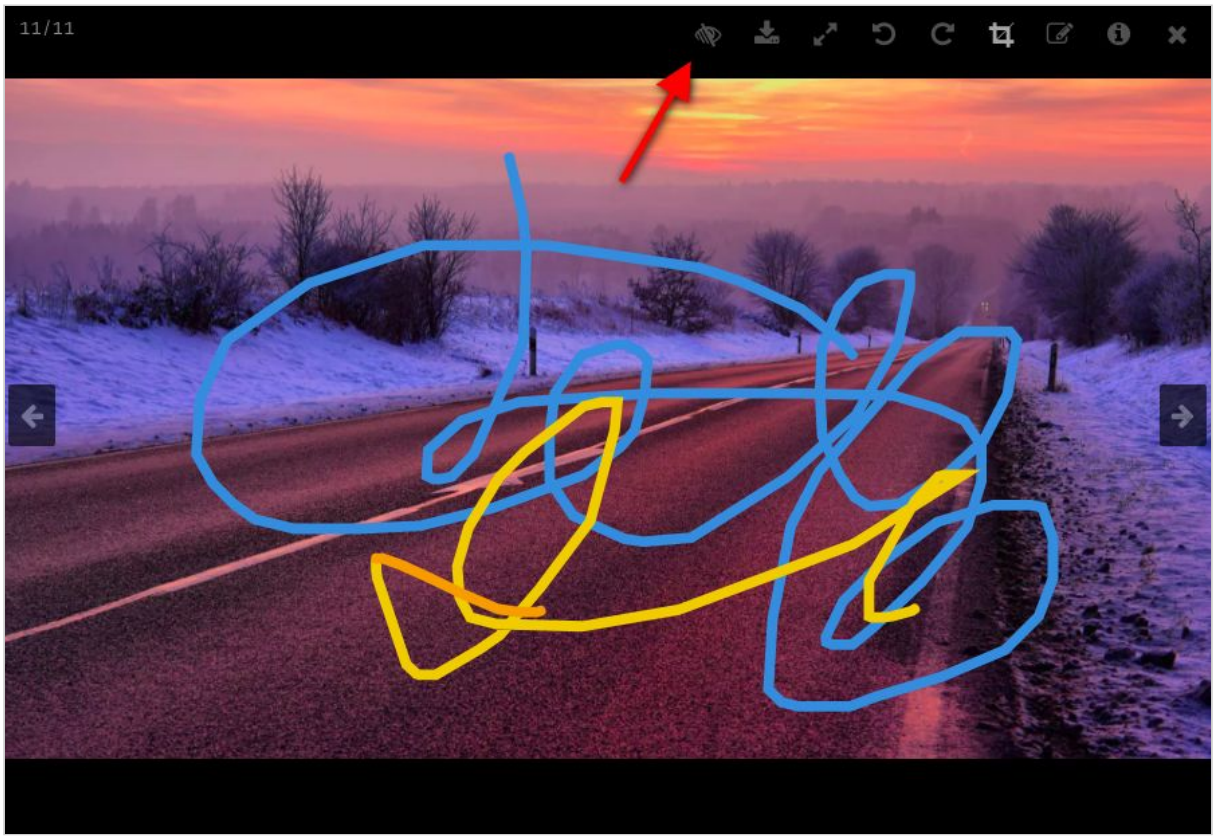


The figure below displays the text input where the title and description are entered respectively.

14. annotation_toggle

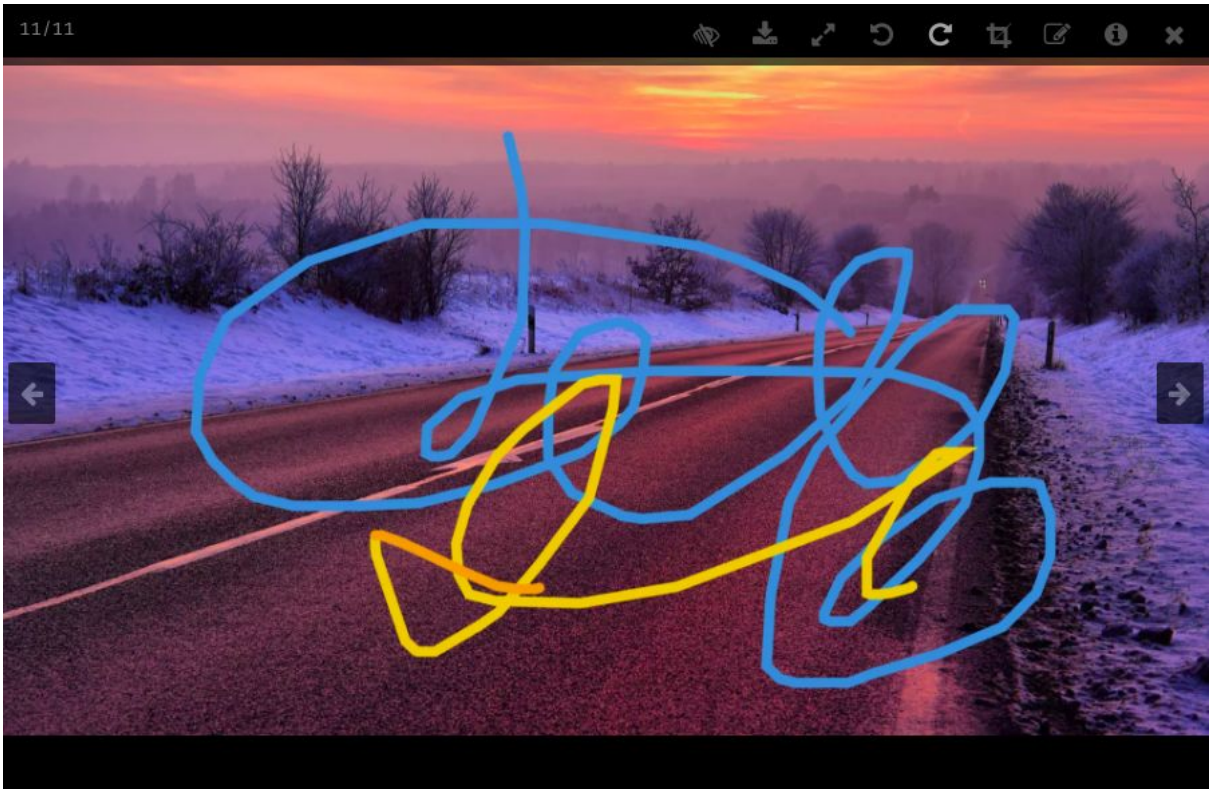
The **annotation_toggle** ability allows the user to enable or disable the annotation toggle which has the possibility to show or hide the annotations on an image.

The figure below shows the annotation toggle when it is enabled.

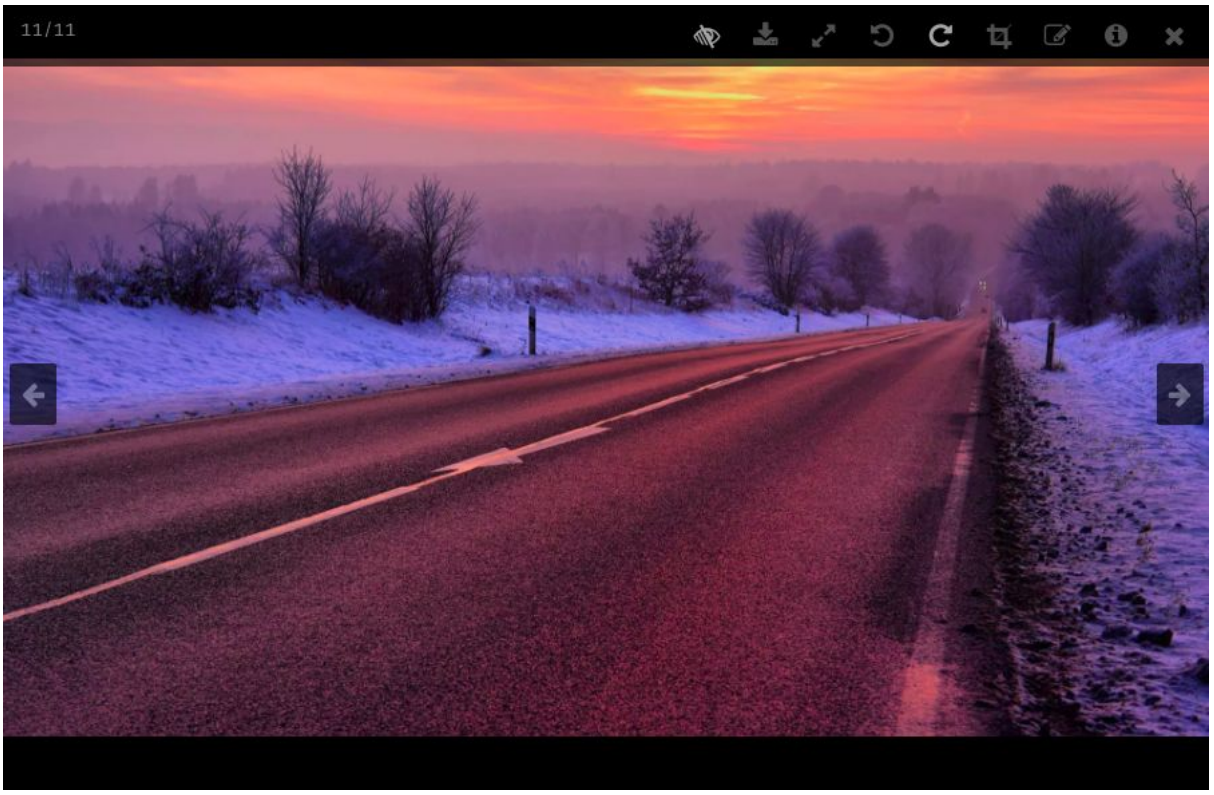


Upon clicking on this **annotation toggle**, it is possible to toggle the visibility of the annotations present on the image.

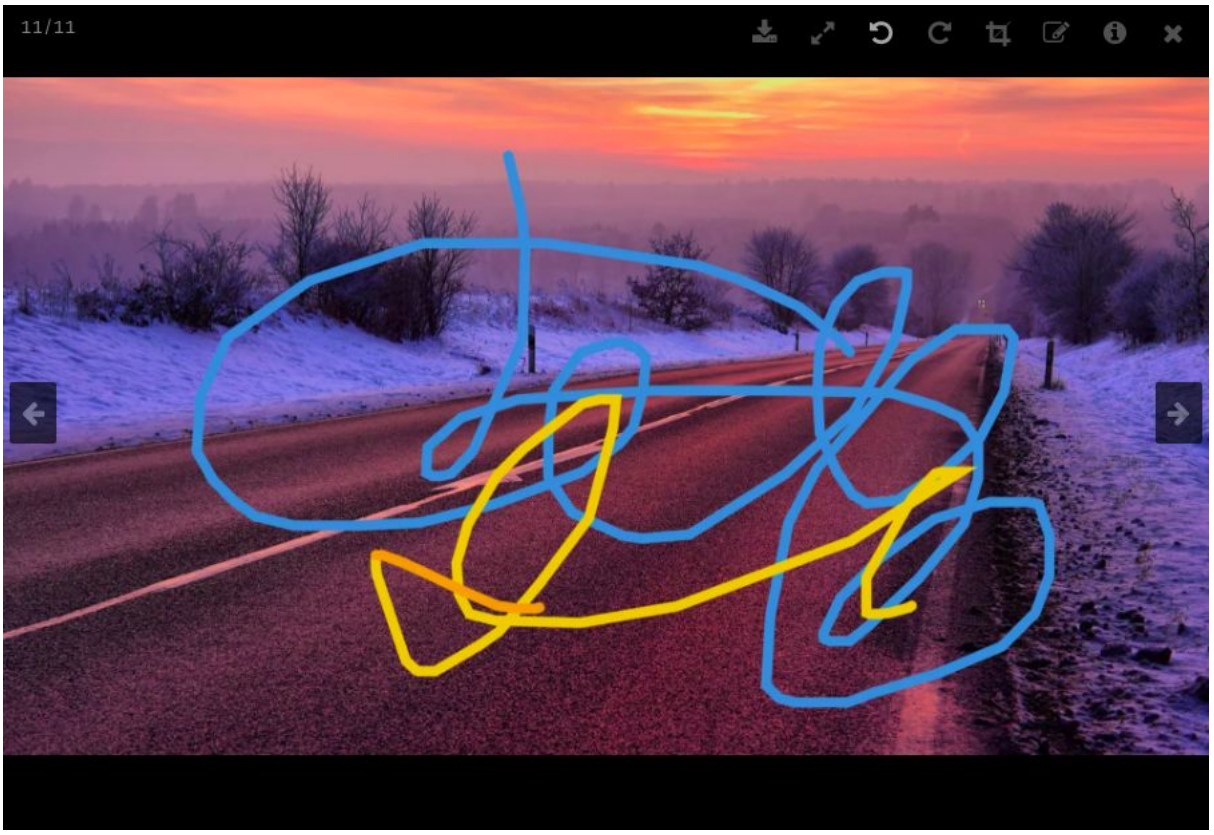
Toggle to show annotations.



Toggle to hide annotations.



However when the ability **annotation_toggle** is set to **false,** the annotation toggle is not present. The figure below illustrates when the **annotation_toggle** ability is set to false. It can be observed that the toggle is not present.


11/11

SharinPix Image Abilities using an Apex Controller

In the subsequent demos you will often and invariably come across a specific code snippet inside an Apex controller, as shown in the figure below.

```
public class SharinPixActionDemoCtrl {
    public String parameters {get; set; }

    public SharinPixActionDemoCtrl(ApexPages.StandardController controller){
        Id accId = controller.getId();

        Map<String, Object> params = new Map<String, Object> {
            'abilities' => new Map<String, Object> {
                accId => new Map<String, Object> {
                    'Access' => new Map<String, Object> {
                        'see' => true,
                        'image_list' => true,
                        'image_upload' => true,
                        'image_tag' => true,
                        'image_delete' => true
                    },
                        'Tags' => new Map<String, Object>{
                            'test' => new Map<String, String>{
                                'en' => 'test'
                            }
                        }
                }
            }
        };
        parameters = JSON.serialize(params);
    }    }
```

The Map Collection **params** contains a set of abilities/accesses, Tags and actions that define what the SharinPix Album is authorized and able to do.

These “parameters” (params) are then fed to the Visualforce Page containing the SharinPix Album.

```
<apex:page standardController="Account" extensions="SharinPixActionDemoCtrl">
    <apex:canvasApp developerName="Albums" height="500px" parameters="{! parameters }" width="100%"/>
</apex:page>
```

Understanding how Image Abilities are defined and enabled

In the previous section, we have how parameters define what the SharinPix Album is authorized and able to do. In this section, we will see how it is possible to selectively enable and disable specific SharinPix Image Abilities on a SharinPix Album.

As we have seen above, the Map Collection **params** contains a set of abilities/accesses, Tags, and actions. However, in the present context, we will take a deeper look at the **abilities** defined for a SharinPix Album.

(NOTE: An explanation on the structure of the SharinPix Parameters inside an Apex Controller can be found here.)

The code snippet below indicates where these abilities are defined inside the Map Collection params.

```
public class SharinPixActionDemoCtrl {
    public String parameters {get; set; }

    public SharinPixActionDemoCtrl(ApexPages.StandardController controller){
        Id accId = controller.getId();

        Map<String, Object> params = new Map<String, Object> {
            'abilities' => new Map<String, Object> {
                accId => new Map<String, Object> {
                    'Access' => new Map<String, Object> {
                        'see' => true,
                        'image_list' => false,
                        'image_upload' => true,
                        'image_tag' => true,
                        'image_delete' => true
                    },
                    'Tags' => new Map<String, Object>{
                      'test' => new Map<String, String>{
                             'en' => 'test',
                            'fr' => 'test'
                      }
                    },
                    'Actions' => new List<String>{
                        'Add to description'
                    }
                }
            }
        };
        parameters = JSON.serialize(params);
    }
}
```

The “Access” Map Collection defines the **abilities** as keys and contains Boolean values(True or False) to indicate whether the corresponding ability is enabled or disabled.

The figure below shows that the ability **image_upload** is enabled while the ability **image_list** is disabled.

```
'Access' => new Map<String, Object> {
                        'see' => true,
                        'image_list' => false,
                        'image_upload' => true,
                        'image_tag' => true,
                        'image_delete' => true
                    }
```

SharinPix Image Abilities using a Visualforce Page only

In the previous section, we have seen how to define the SharinPix Image Abilities inside an Apex Controller. However, it is also possible to define these same abilities within a Visualforce Page only without requiring the use of an Apex Controller.

(NOTE: An explanation on the structure of SharinPix Parameters inside a Visualforce Page can be found here.)
The code snippet represents the minimum amount of code to embed a SharinPix Album while defining and selectively enabling/disabling specific SharinPix Image Abilities.

```
<apex:page standardController="Contact">
    <sharinpix:TagAction recordId="{!$CurrentPage.Parameters.Id}" />
    <apex:canvasApp developerName="Albums" namespacePrefix="sharinpix" height="500px"
        parameters="{
            abilities:{
            '{!CASESAFEID($CurrentPage.Parameters.Id)}':
                {
                    Access: {image_upload:true,image_list:true,see:true,image_tag:true},
                    Tags:{'MainPicture':{'en': 'Main Picture'},'BusinessCard':{'en':
'Business Card'}},
                    Action:{'Add to description'},
                    Display: {tags: true}
                }
            }
        }"
    width="100%"/>
</apex:page>
```

The Access Map collection defines the abilities as keys and its Boolean values (true/false) that indicate whether the corresponding ability is enabled(true) or disabled(false).

Add SharinPix action

What is an action ?

In SharinPix, an ***action*** refers to an added feature that allows the user to perform a given specific task.

What are the types of action ?

1. **Email Action**

   The Email Action allows the user to select an image and send it as an email. The email address of the recipient is extracted from the *Email* field.
   Then the recipient will receive an email along with the newly-uploaded image.

   Find a step-by-step implementation of the email action: here.

2. **Delete Action**

   The Delete Action allows the user to select multiple images and delete them all at once.

   Find a step-by-step implementation of the delete action: here.

3. **Description Action**

   The Description Action allows the user to use a Salesforce field to display a primary picture for a record.

   Find a step-by-step implementation of the Description action: here.

4. **Task Action**

   The Task Action allows the user to select images and create Tasks from them.

   Find a step-by-step implementation of the Task Action: here

How to enable an action ?

Actions can be added and removed by modifying the Apex class controller of the Visualforce Page. More specifically, the Map data structure (defined as **params**) must be configured as displayed in the figure below.

```
public class SharinPixActionDemoCtrl {
    public String parameters {get; set; }

    public SharinPixActionDemoCtrl(ApexPages.StandardController controller){
        Id accId = controller.getId();

        Map<String, Object> params = new Map<String, Object> {
            'abilities' => new Map<String, Object> {
                accId => new Map<String, Object> {
                    'Access' => new Map<String, Object> {
                        'see' => true,
                        'image_list' => false,
                        'image_upload' => true,
                        'image_tag' => true,
                        'image_delete' => true
                    },
                    'Tags' => new Map<String, Object>{
                      'test' => new Map<String, String>{
                              'en' => 'test',
                             'fr' => 'test'
                      }
                    },
                    'Actions' => new List<String>{
                        'Add to description'
                    }
                }
            }
        };
        parameters = JSON.serialize(params);
    }
}
```

In the above figure, the Description Action is enabled by adding the String ‘Add to description’ to the **Actions** List.

The table shown below represents the Actions and their corresponding labels. (e.g **Description Action** corresponds to **Add to description**.

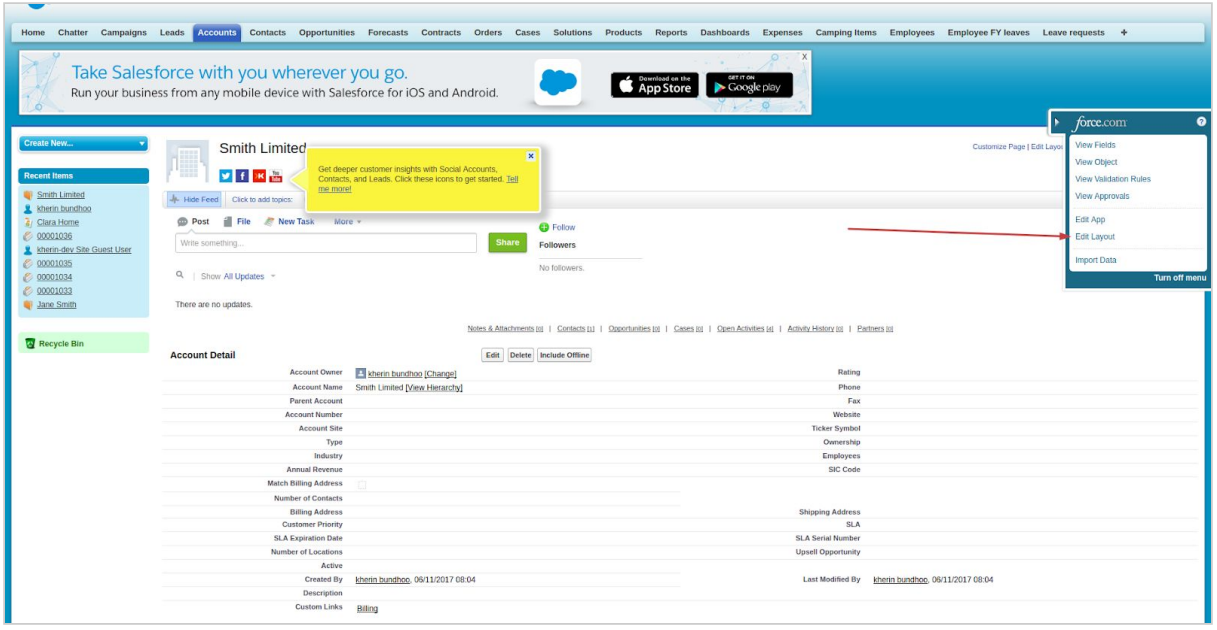
Action	Label
Email	Send an email
Delete	Delete images
Task	Create a task
Description	Add to description

Adding Visualforce Page

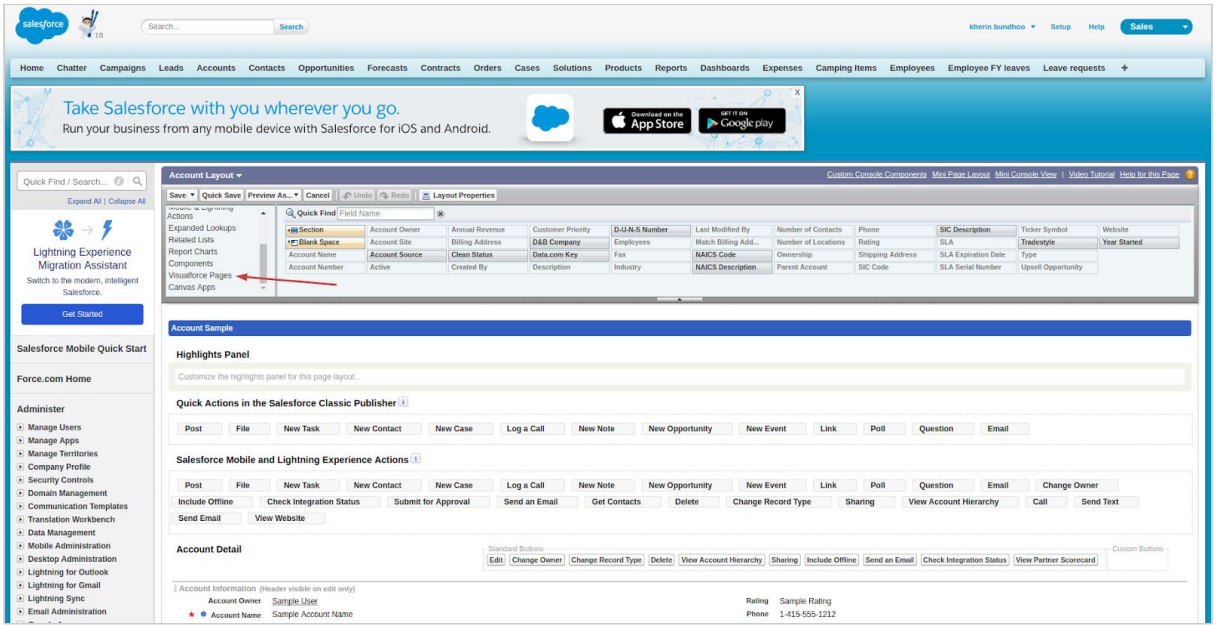
Each cookbook example comes with a visualforce page that will be added to your Salesforce environment once deployed. The visualforce page must be added to the correct object(the object name will indicated in each cookbook example). Below are the steps to add the visualforce page.

The ensuing content assumes that you already deployed a cookbook example.

- ## Inserting Visualforce Page in Salesforce Classic
    1. Go the object, in this case it is Account.
    2. Select a record.
    3. Select “Edit layout” on the right-hand-side menu.



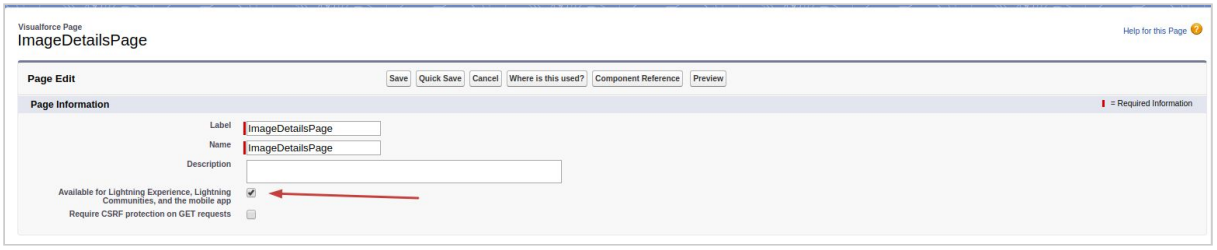
4. Select “Visualforce Pages”.



5. Select the relevant Visualforce Page and drop it inside the section of your choice.
6. Edit the Properties of the Visualforce Page and set its height to 500 pixels.
7. Save and exit.
8. Proceed with the instructions of the cookbook example.

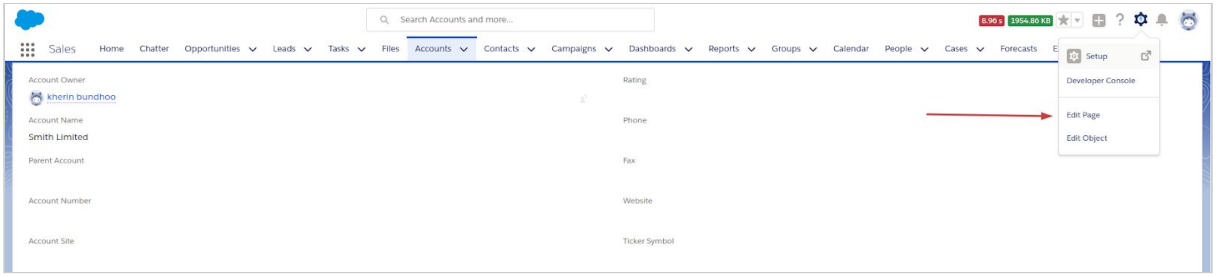
- Inserting Visualforce Page as a Lightning component
    1. Go to Setup Home. Type “Visualforce Pages” inside the *Quick Find box*.

2. Find the relevant Visualforce Page and click on “edit”.
3. Enable the lightning experience. (The Label and Name fields will correspond to the selected Visualforce Page)

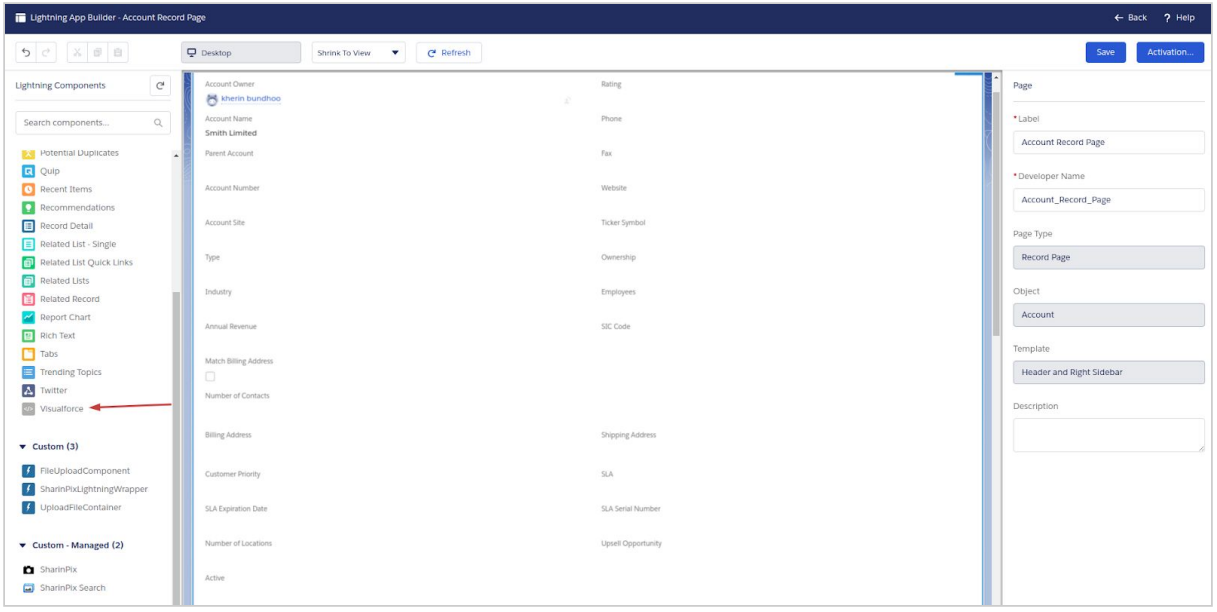


4. Save and exit.
5. Go to Salesforce Home and select the relevant Object. (In this case: Account)
6. Select a record.

7. Edit Page.



8. Select “Visualforce Page”



9. Select the relevant Visualforce Page name.
10. Save and exit.
11. Proceed with the instructions of the cookbook example.

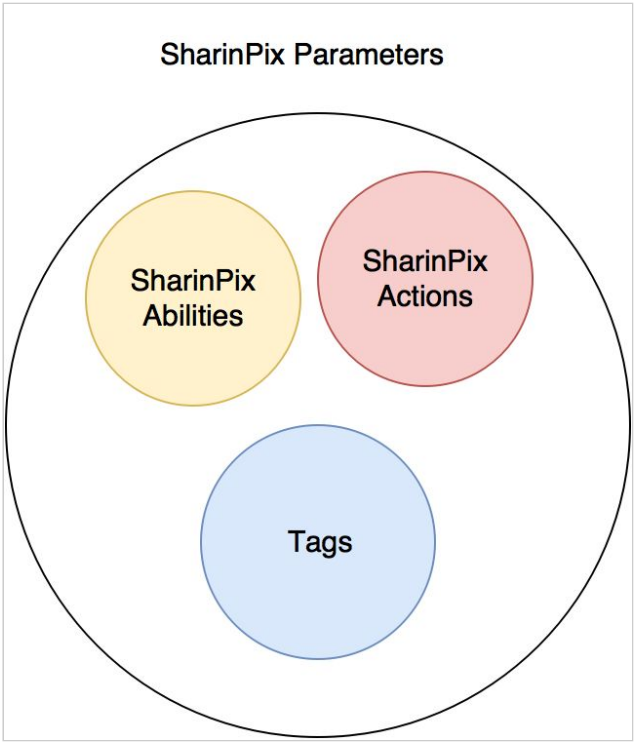
SharinPix Parameters

SharinPix Parameters represent a fundamental set of information that:

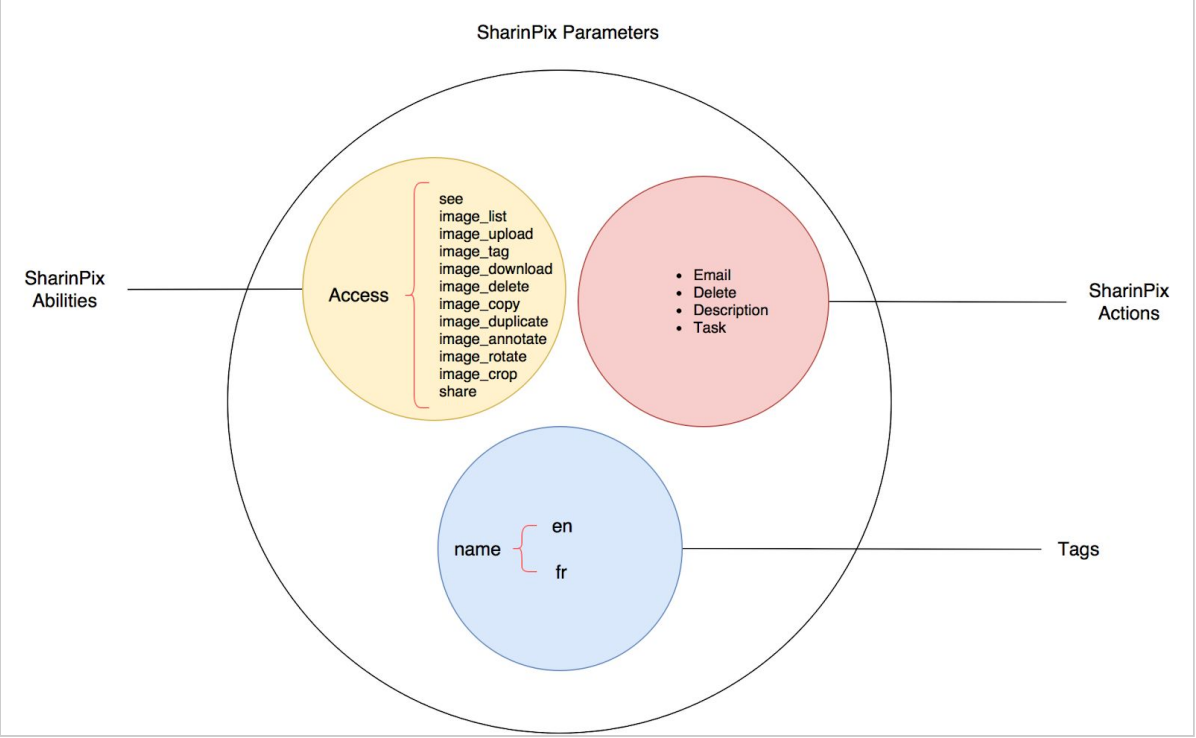
1. Define the rights to access features (SharinPix Image Abilities).
2. Define the SharinPix Actions of an Album.
3. Define which Tag labels will be made available on an Album.

Structure of SharinPix Parameters

- First depth-level



- Second depth-level



While the two precedent diagrams illustrated graphically the inner structure and elements of the SharinPix Parameters, in the next two sections we will dive into the actual syntax and implementation of the SharinPix Parameters inside an Apex Controller as well as a Visualforce Page.

Syntax of SharinPix Parameters inside Apex Controller

The figure below demonstrates the implementation of SharinPix Parameters inside an Apex Controller.

```
public class SharinPixActionDemoCtrl {
    public String parameters {get; set; }

    public SharinPixActionDemoCtrl(ApexPages.StandardController controller){
        Id accId = controller.getId();

        Map<String, Object> params = new Map<String, Object> {
            'abilities' => new Map<String, Object> {
                accId => new Map<String, Object> {
                    'Access' => new Map<String, Object> {
                        'see' => true,
                        'image_list' => false,
                        'image_upload' => true,
                        'image_tag' => true,
                        'image_delete' => true
                    },
                    'Tags' => new Map<String, Object>{
                      'test' => new Map<String, String>{
                              'en' => 'test',
                             'fr' => 'test'
                      }
                    },
                    'Actions' => new List<String>{
                        'Add to description'
                    }
                }
            }
        };
        parameters = JSON.serialize(params);
    }
}
```

1. **Id** of the SharinPix Album.

The **Id** corresponds to the id of the record where the Visualforce Page is currently in. The **Id** is also used as a unique identifier for the SharinPix Album, thus, it is also qualifies as an **Album Id.**

2. **Access** - the rights to specific SharinPix features.
3. **Tags** - Labels along with their corresponding translations in english(en) and in french(fr).
4. **Action** - Label of the SharinPix Action

Structure of the Parameter values

- Id

```
Map<String, Object> params = new Map<String, Object> {
          'abilities' => new Map<String, Object> {
              accId => new Map<String, Object> {
                   'Access' => new Map<String, Object> {
                       'see' => true,
                       'image_list' => false,
                       'image_upload' => true,
                       'image_tag' => true,
                       'image_delete' => true
                   },
                   'Tags' => new Map<String, Object>{
                     'test' => new Map<String, String>{
                            'en' => 'test',
                           'fr' => 'test'
                     }
                   },
                   'Actions' => new List<String>{
                       'Add to description'
                   }
              }
          }
      };
```

- Access

```
Map<String, Object> params = new Map<String, Object> {
          'abilities' => new Map<String, Object> {
              accId => new Map<String, Object> {
                   'Access' => new Map<String, Object> {
                       'see' => true,
                       'image_list' => false,
                       'image_upload' => true,
                       'image_tag' => true,
                       'image_delete' => true
                   },
                   'Tags' => new Map<String, Object>{
                     'test' => new Map<String, String>{
                            'en' => 'test',
                           'fr' => 'test'
                     }
                   },
                   'Actions' => new List<String>{
                       'Add to description'
                   }
              }
```

```
        }
    };
```

- Tags

```
Map<String, Object> params = new Map<String, Object> {
        'abilities' => new Map<String, Object> {
            accId => new Map<String, Object> {
                'Access' => new Map<String, Object> {
                    'see' => true,
                    'image_list' => false,
                    'image_upload' => true,
                    'image_tag' => true,
                    'image_delete' => true
                },
                'Tags' => new Map<String, Object>{
                  'test' => new Map<String, String>{
                          'en' => 'test',
                         'fr' => 'test'
                  }
                },
                'Actions' => new List<String>{
                    'Add to description'
                }
            }
        }
    };
```

- Action

```
Map<String, Object> params = new Map<String, Object> {
        'abilities' => new Map<String, Object> {
            accId => new Map<String, Object> {
                'Access' => new Map<String, Object> {
                    'see' => true,
                    'image_list' => false,
                    'image_upload' => true,
                    'image_tag' => true,
                    'image_delete' => true
                },
                'Tags' => new Map<String, Object>{
                  'test' => new Map<String, String>{
                          'en' => 'test',
                         'fr' => 'test'
                  }
                },
                'Actions' => new List<String>{
                    'Add to description'
                }
            }
```

```
            }
};
```

Syntax of SharinPix Parameters inside Visualforce Page

While the above implementation requires both an Apex Controller and a Visualforce Page to display a SharinPix Album, the present section will show how it is possible to define the SharinPix Parameters while using a Visualforce Page only.

The figure below illustrates the syntax used to implement a SharinPix Album and its respective SharinPix Parameters inside a Visualforce Page which is related to a Contact record.

```
<apex:page standardController="Contact">
    <sharinpix:TagAction recordId="{!$CurrentPage.Parameters.Id}" />
    <apex:canvasApp developerName="Albums" namespacePrefix="sharinpix" height="500px"
        parameters="{
            abilities:{
            '{!CASESAFEID($CurrentPage.Parameters.Id)}':
                {
                    Access: {image_upload:true,image_list:true,see:true,image_tag:true},
                    Tags:{'MainPicture':{'en': 'Main Picture'},'BusinessCard':{'en':
'Business Card'}},
                    Action:{'Add to description'},
                    Display: {tags: true}
                }
            }
        }"
    width="100%"/>
</apex:page>
```

1. **Id** of the SharinPix Album.

The **Id** corresponds to the id of the record where the Visualforce Page is currently located in. The **Id** is also used as a unique identifier for the SharinPix Album, thus, it also qualifies as an **Album Id.**

2. **Access** - the rights to specific SharinPix features.
3. **Tags** - Labels along with their corresponding translations in english(en) and in french(fr).
4. **Action** - Label of the SharinPix Action

Structure of the Parameter values

- Id

```
<apex:page standardController="Contact">
    <sharinpix:TagAction recordId="{!$CurrentPage.Parameters.Id}" />
    <apex:canvasApp developerName="Albums" namespacePrefix="sharinpix" height="500px"
        parameters="{
            abilities:{
            '{!CASESAFEID($CurrentPage.Parameters.Id)}':
                {
                    Access: {image_upload:true,image_list:true,see:true,image_tag:true},
                    Tags:{'MainPicture':{'en': 'Main Picture'},'BusinessCard':{'en':
'Business Card'}},
                    Action:{'Add to description'},
                    Display: {tags: true}
                }
            }
        }"
    width="100%"/>
</apex:page>
```

- Access

```
<apex:page standardController="Contact">
    <sharinpix:TagAction recordId="{!$CurrentPage.Parameters.Id}" />
    <apex:canvasApp developerName="Albums" namespacePrefix="sharinpix" height="500px"
        parameters="{
            abilities:{
            '{!CASESAFEID($CurrentPage.Parameters.Id)}':
                {
                    Access: {image_upload:true,image_list:true,see:true,image_tag:true},
                    Tags:{'MainPicture':{'en': 'Main Picture'},'BusinessCard':{'en':
'Business Card'}},
                    Action:{'Add to description'},
                    Display: {tags: true}
                }
            }
        }"
    width="100%"/>
</apex:page>
```

- Tags

```
<apex:page standardController="Contact">
    <sharinpix:TagAction recordId="{!$CurrentPage.Parameters.Id}" />
    <apex:canvasApp developerName="Albums" namespacePrefix="sharinpix" height="500px"
        parameters="{
            abilities:{
            '{!CASESAFEID($CurrentPage.Parameters.Id)}':
                {
                    Access: {image_upload:true,image_list:true,see:true,image_tag:true},
                    Tags:{'MainPicture':{'en': 'Main Picture'},'BusinessCard':{'en':
'Business Card'}},
                    Action:{'Add to description'},
                    Display: {tags: true}
                }
            }
        }"
    width="100%"/>
</apex:page>
```

- Action

```
<apex:page standardController="Contact">
    <sharinpix:TagAction recordId="{!$CurrentPage.Parameters.Id}" />
    <apex:canvasApp developerName="Albums" namespacePrefix="sharinpix" height="500px"
        parameters="{
            abilities:{
            '{!CASESAFEID($CurrentPage.Parameters.Id)}':
                {
                    Access: {image_upload:true,image_list:true,see:true,image_tag:true},
                    Tags:{'MainPicture':{'en': 'Main Picture'},'BusinessCard':{'en':
'Business Card'}},
                    Action:{'Add to description'},
                    Display: {tags: true}
                }
            }
        }"
    width="100%"/>
</apex:page>
```

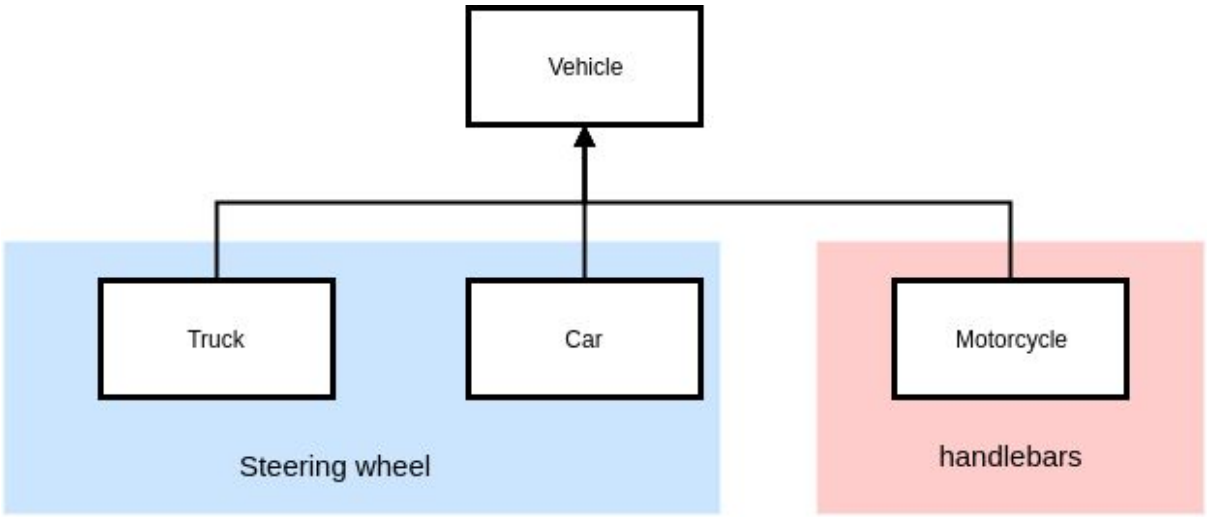
Dynamically assign tags based on Record Type

- Purpose

The following demo will illustrate the possibility to integrate a single Visualforce Page(embedded with the SharinPix app) along with specific sets of Tag labels corresponding to the correct Record Type. These Tags will be dynamically assigned depending on which Record Type the Visualforce page is currently on. This feature proves invaluable as it removes the need to have a Visualforce Page for each Record Type and thus reduces repetition.

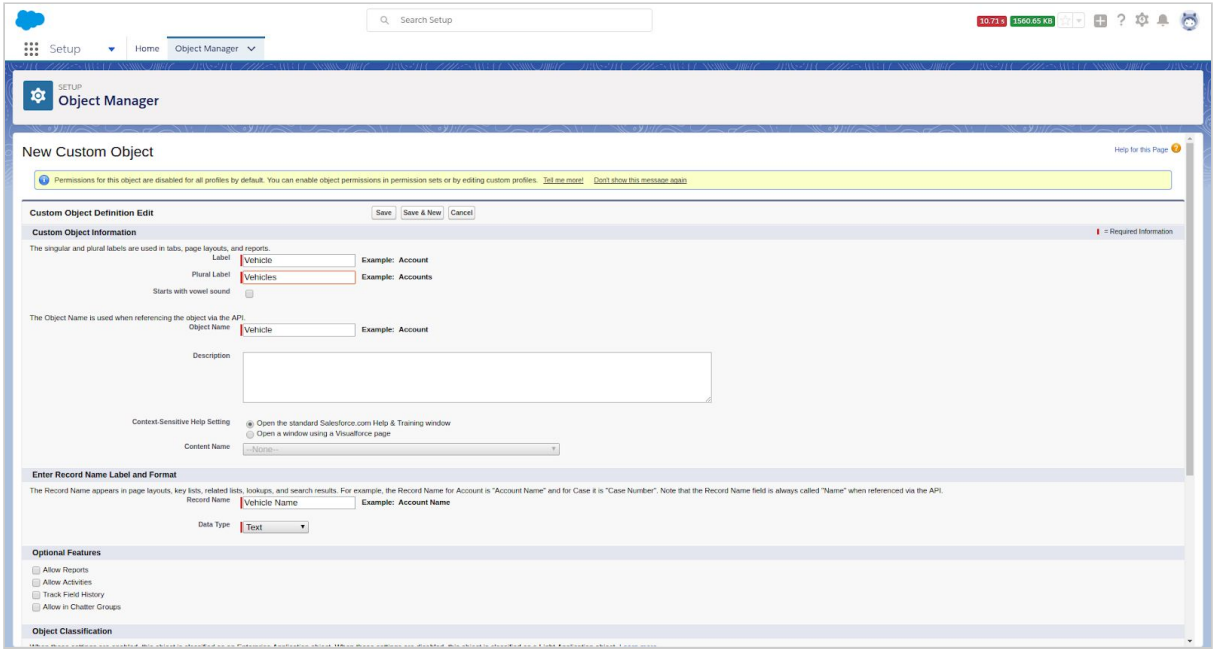
- Structure

For instance, a custom object labelled **Vehicle** has the following Record Types: **Car, Truck** and **Motorcycle**. While those three Record Types can all be qualified as vehicles, they however possess particularities that require specific **tags** to describe them. The tag **steering wheel** might be applicable to a **Truck** but not to a **Motorcycle.** This demo shows how it is possible to dynamically assign tags to relevant Record Types.



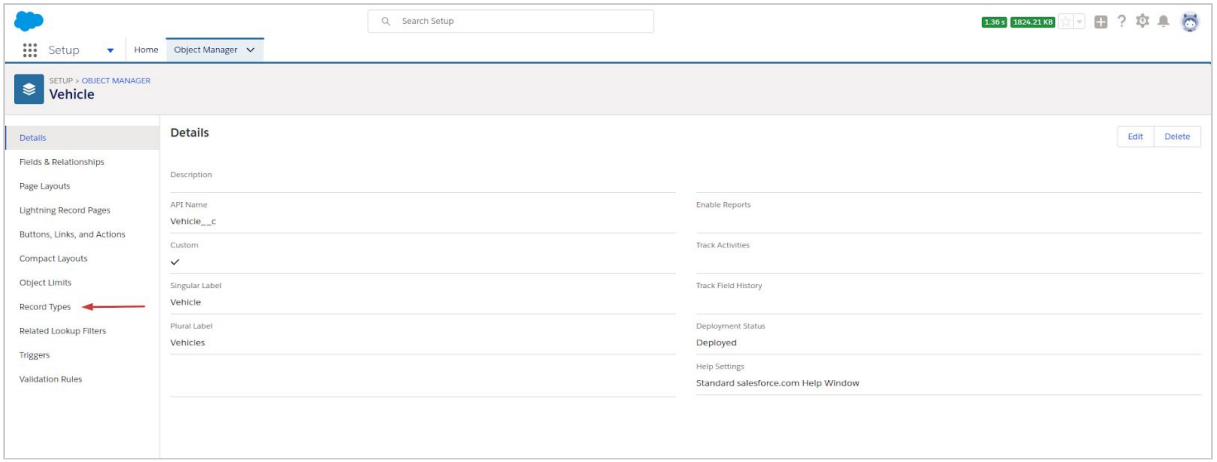
- Demo

1. Create a custom object **Vehicle.**

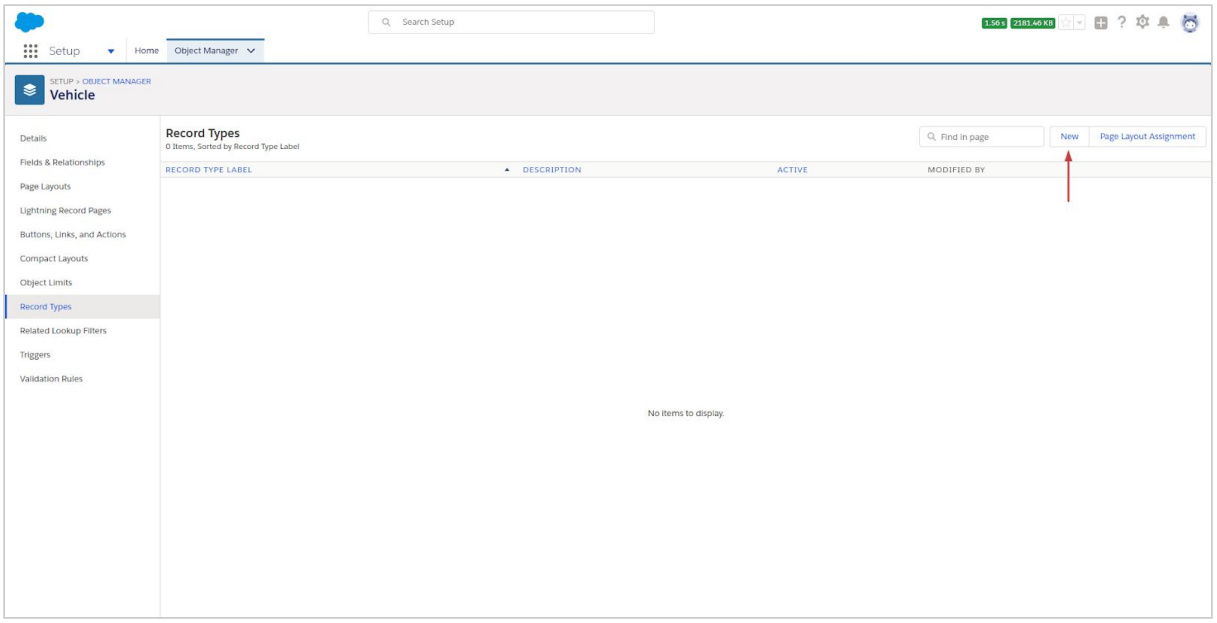


Proceed with the rest of the configurations and finally click on save.

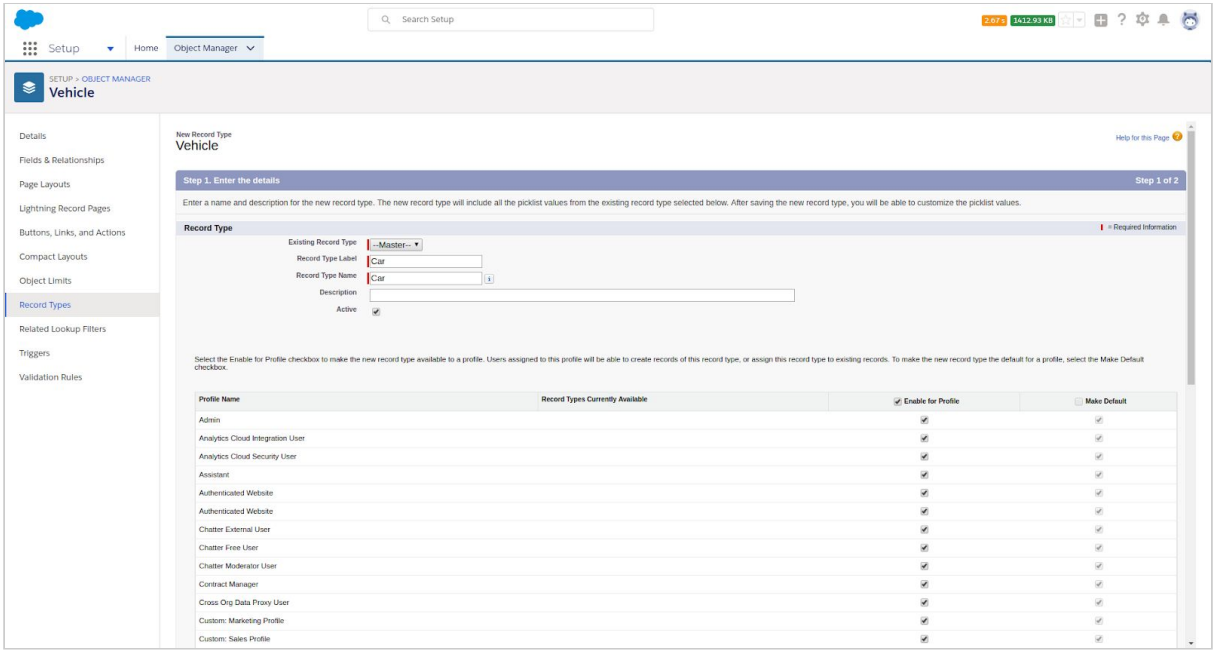
2. Select **Record Types**.



3. Click on **New**

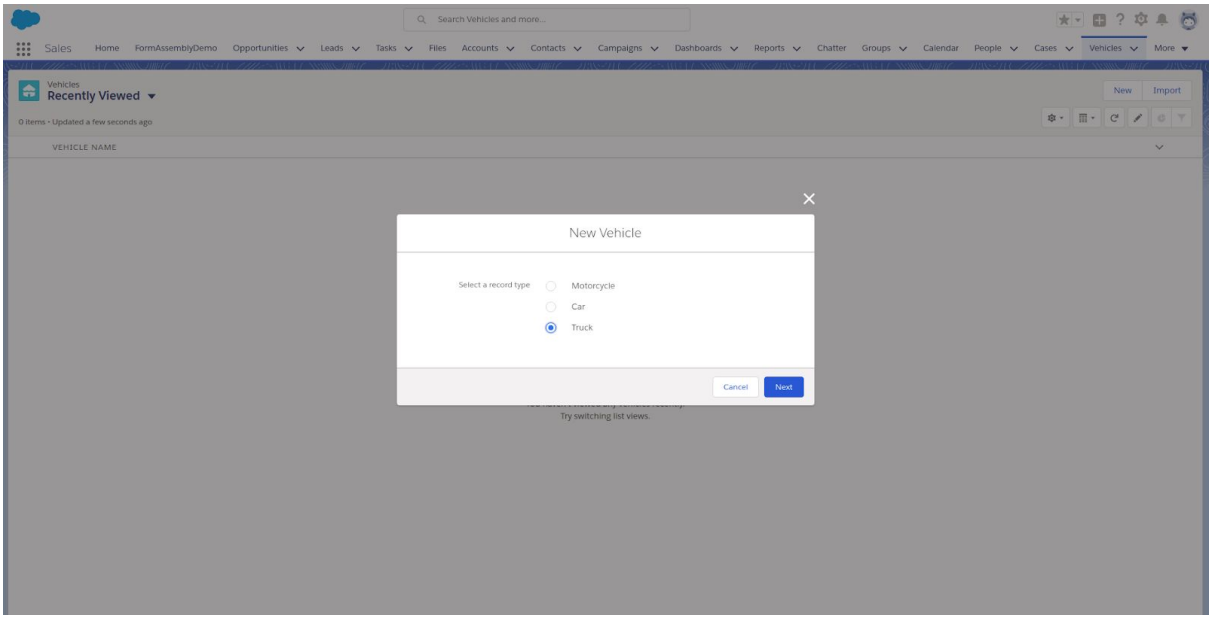


4. Now we will create the first Record Type: **Car**. Make sure the checkbox **Active** is checked. And the picklist **Existing Record Type** is set to **Master**.
Proceed to the rest of the configurations.



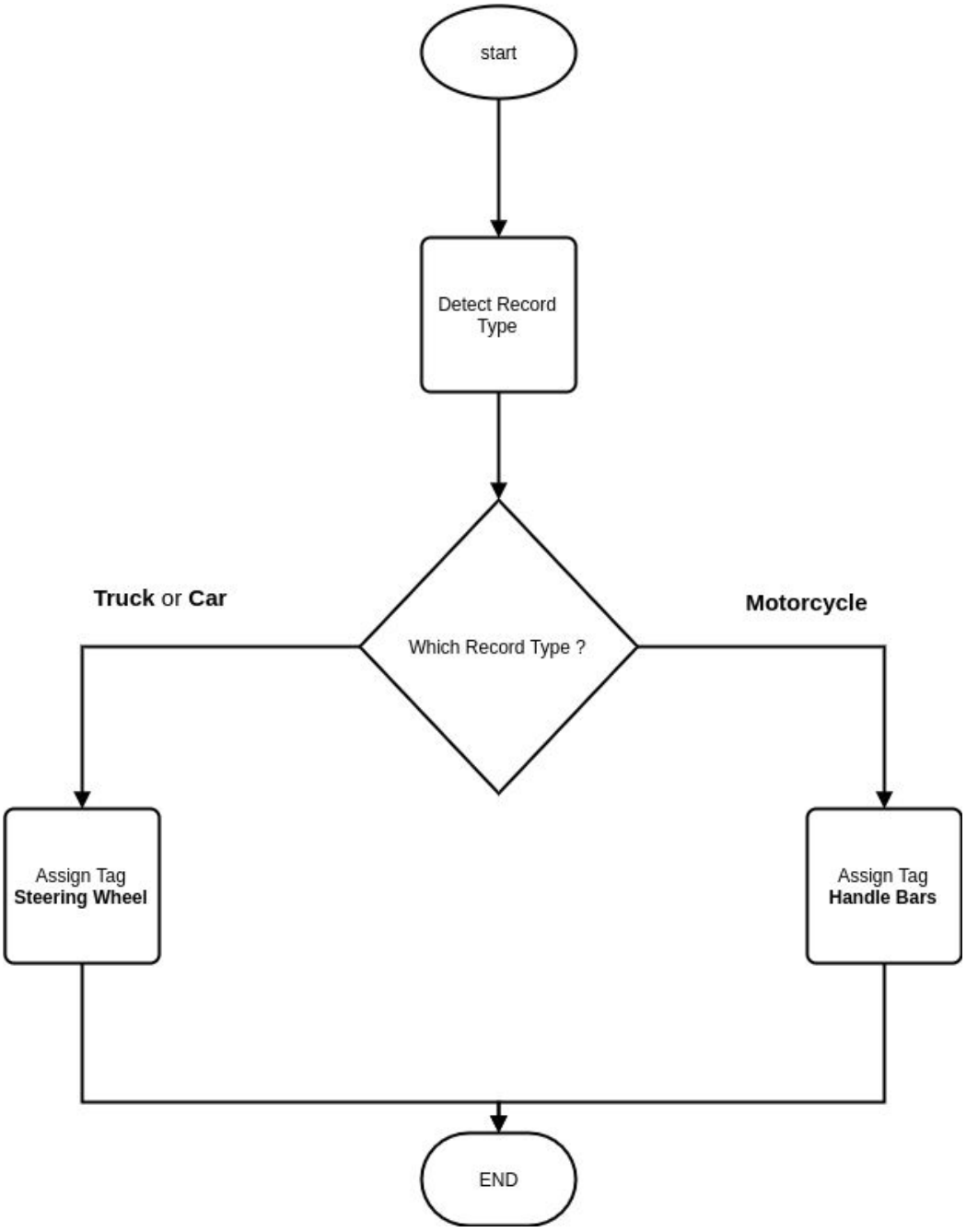
5. Repeat the above process for the Record Types: **Truck** and **Motorcycle**.

6. Create a record for **each** Record Type.



7. Visualforce Page Implementation.

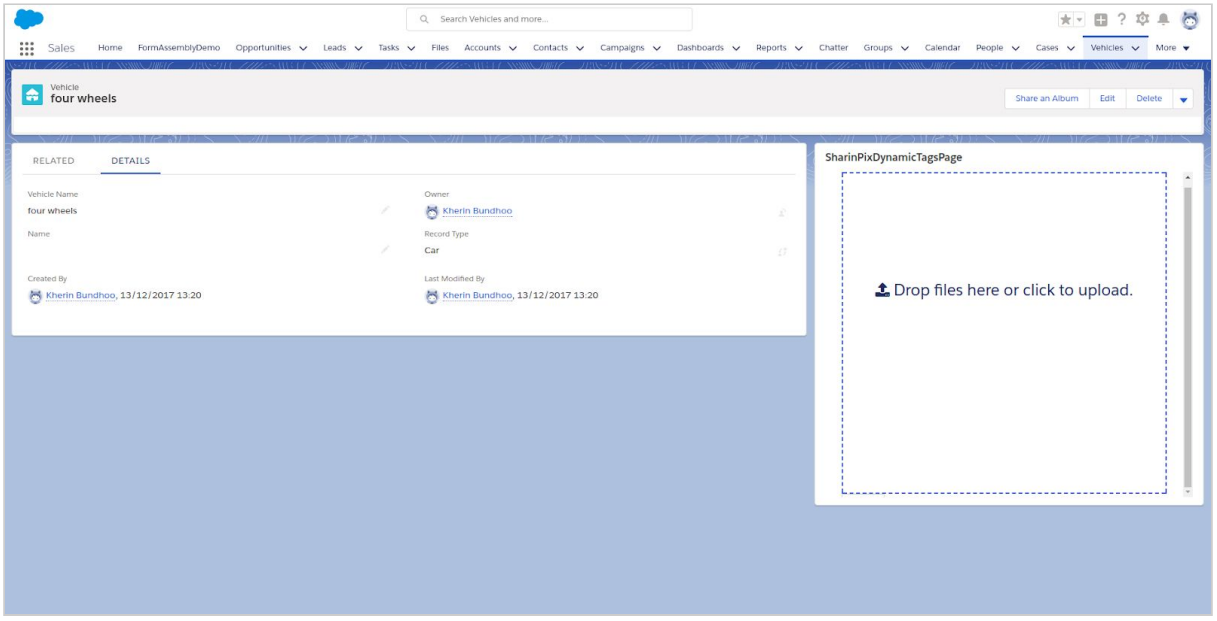
The diagram below describes the logic behind the Visualforce implementation and illustrates the steps for dynamically assigning tags depending on which record type the Visualforce Page is.



8. The code snippet below illustrates the implementation of the Apex controller for the Visualforce Page. The logic behind the code is similar to the one displayed in the above diagram.

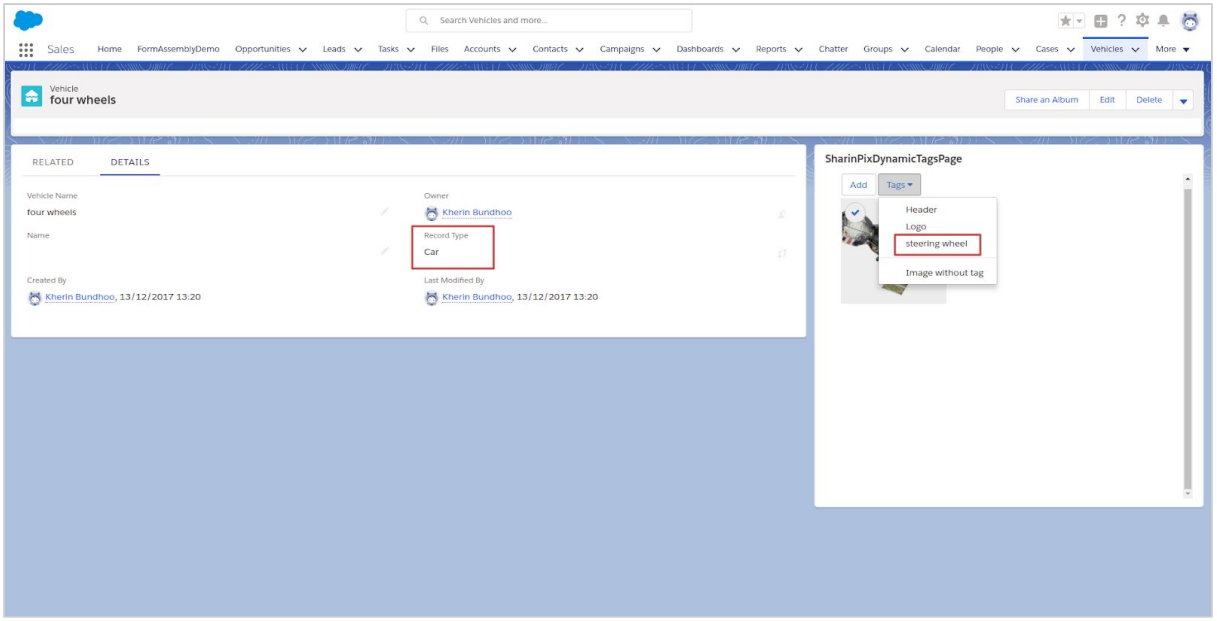
```
public class SharinPixDynamicTagsCtrl {
    public String parameters {get; set; }

    public SharinPixDynamicTagsCtrl(ApexPages.StandardController controller){
        Id vehicleId = controller.getId();
        Map<String, Object> steeringMap = new Map<String, Object>{
            'steering Wheel' => new Map<String, Object> {
                            'en' => 'Steering Wheel'
                        }
        };

        Map<String, Object> handleBarMap = new Map<String, Object>{
            'Handler Bars' => new Map<String, Object> {
                            'en' => 'Handle Bars'
                        }
        };

         Map<String, Object> vehicleMap;

        Vehicle__c v = [SELECT Id, RecordTypeId, RecordType.Name FROM Vehicle__c WHERE Id=:
vehicleId];
        if(v.RecordType.Name == 'Motorcycle'){
            vehicleMap = handleBarMap;
        }else if(v.RecordType.Name == 'Car' || v.RecordType.Name == 'Truck'){
            vehicleMap = steeringMap;
        }


        Map<String, Object> params = new Map<String, Object> {
            'abilities' => new Map<String, Object> {
                vehicleId => new Map<String, Object> {
                    'Access' => new Map<String, Object> {
                        'see' => true,
                        'image_list' => true,
                        'image_upload' => true,
                        'image_tag' => true,
                        'image_delete' => true
                    },
                    'Tags' => vehicleMap
                }
            }
        };
        parameters = JSON.serialize(params);

    }    }
```

9. The Visualforce Page **SharinPixDynamicTagsPage** is added to the pagelayout of the custom object **Vehicle.**

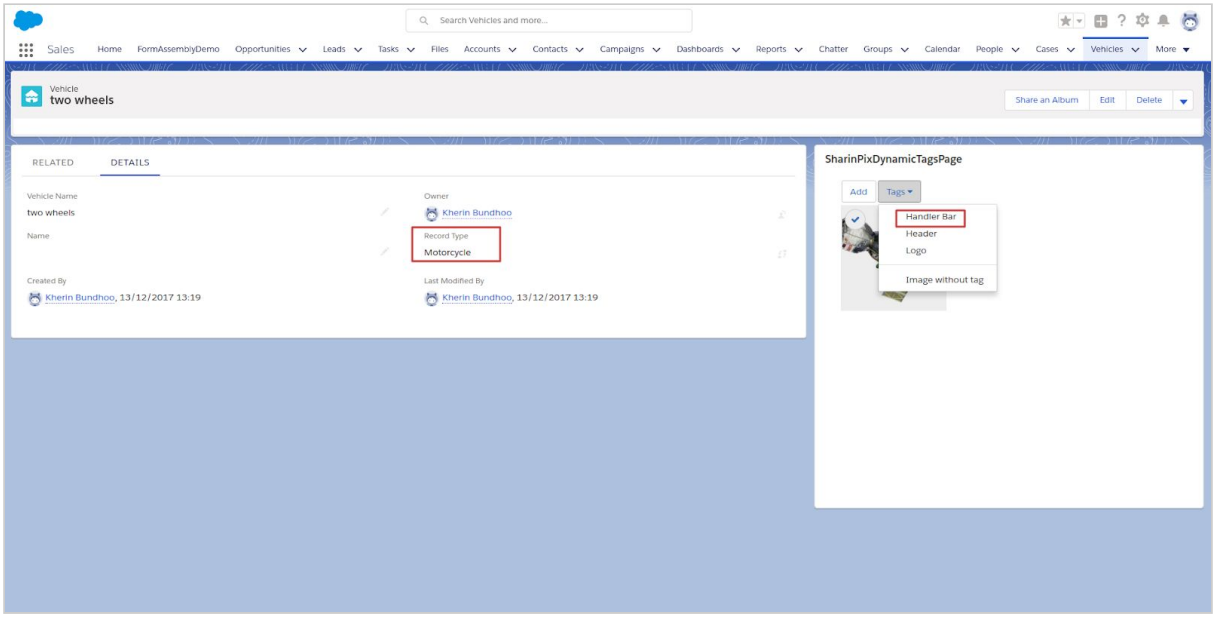


- Conclusion

10. The figure shows that the tag **steering wheel** is displayed on the Record Type **Car.**



11. The figure below shows that the tag **Handle Bar** is displayed on the Record Type **Motorcycle.**



Enrich Salesforce with image data through webhooks

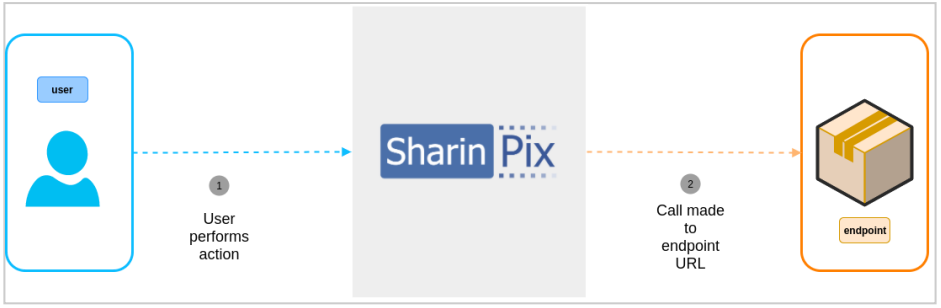
What is Webhook?

A WebHook is a callback request (HTTP POST) made to a specified URL when something happens.
In our case, we have these configurable Webhooks available:

- New image
- New publication
- New provider reply
- New tag image
- Delete image
- Delete tag image
- Upload done
- New einstein prediction

These callbacks can be configured on the admin page, in the Webhooks section. You can provide your URL on which you want the WebHook (callback) to occur.

1. Webhook

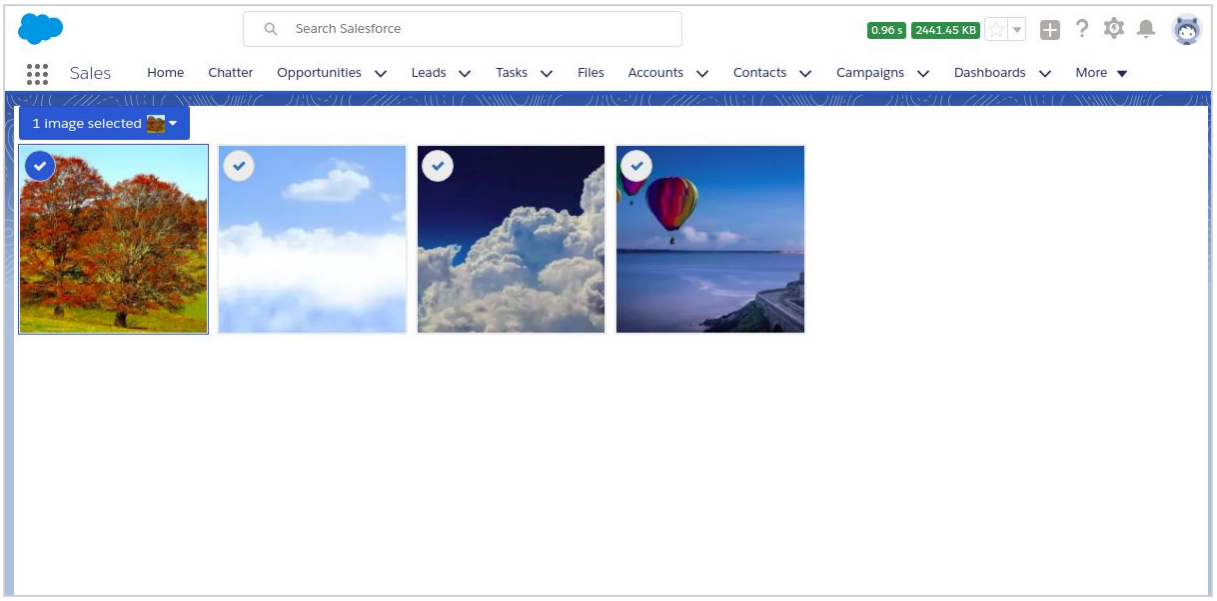


2. Events

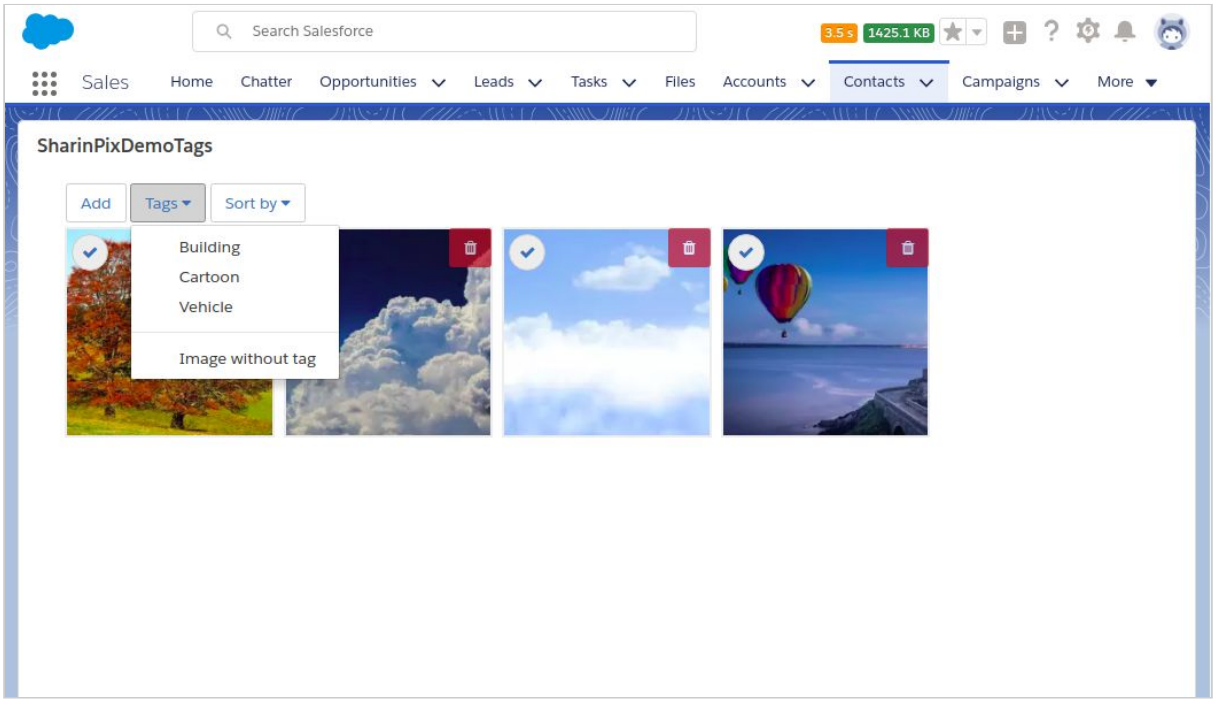
As mentioned above, a webhook is a callback request made to a specific endpoint URL when an event happens. In SharinPix, these events correspond to a particular action performed by the user within the application.

These actions are illustrated in the figures below:

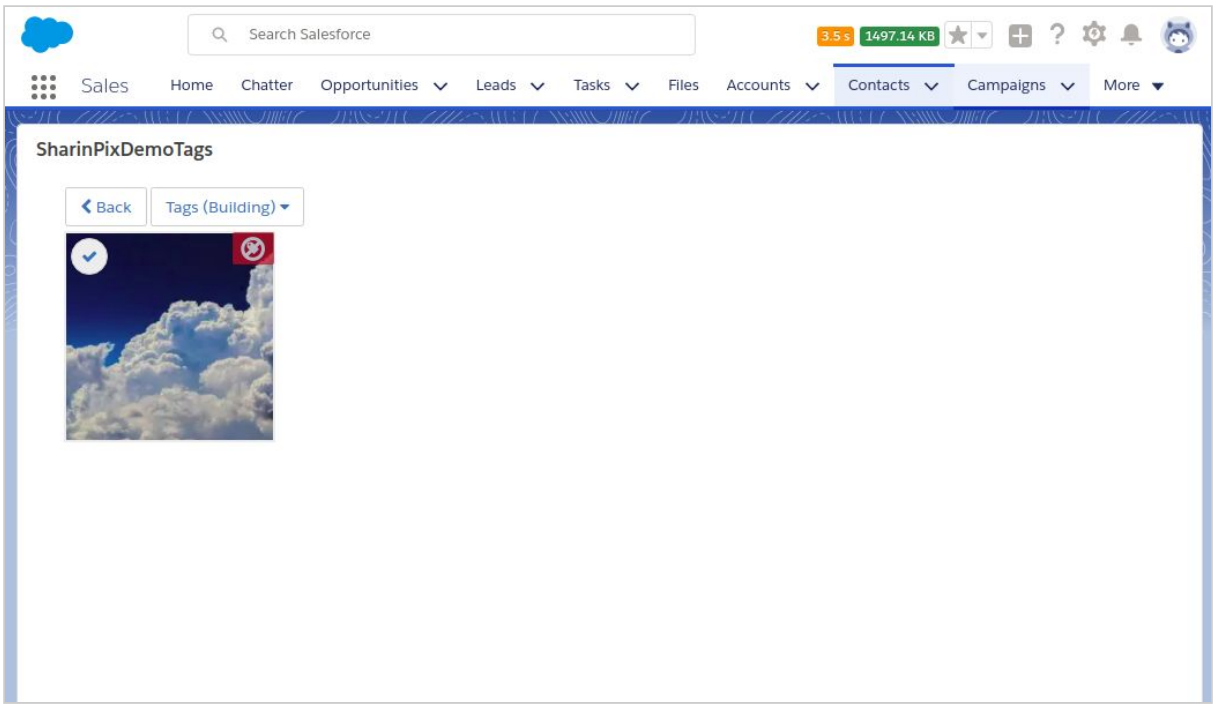
A “**image-new**” event triggered when an image is added/uploaded to SharinPix. More precisely when the user clicks on the ‘Add’ button as shown below.



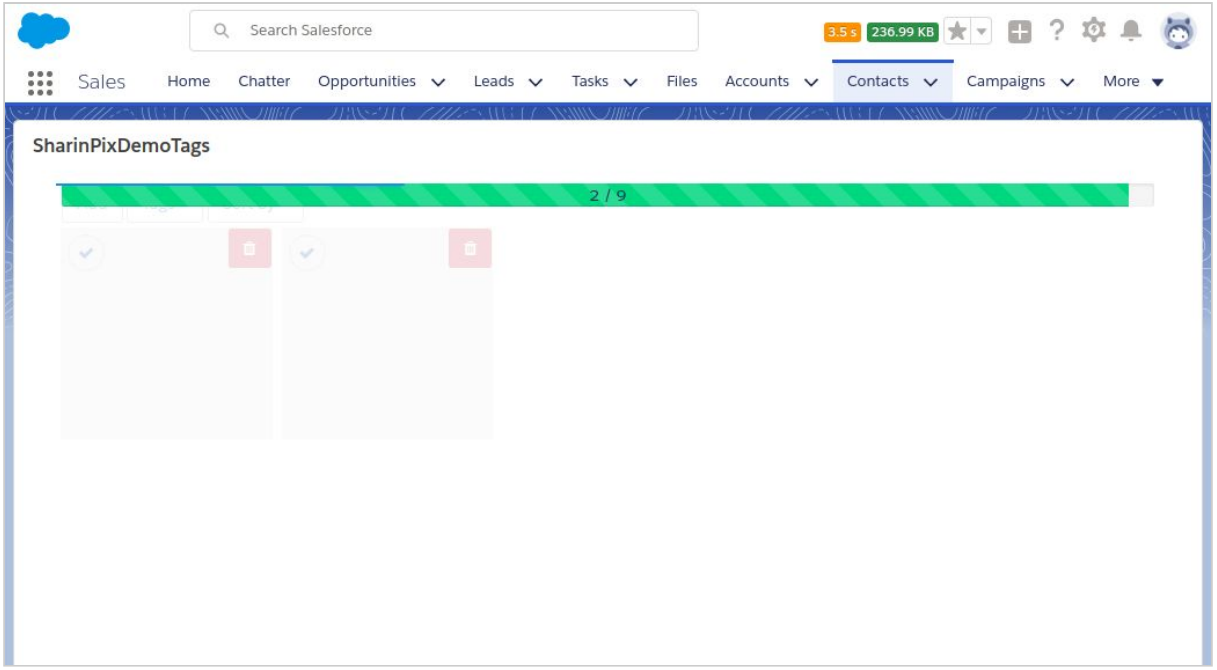
A “**tag-image-new**” event is triggered when the user tags an image as illustrated below.



A “**tag-image-deleted**” event is triggered when the user deletes a tag(‘building’ in this case) from an image. The operation is described in the figure below.



The “**upload-done**” event is triggered when an image has been uploaded into SharinPix. The figure below shows an image being uploaded. When the upload has reached its completion, the event “upload-done” is then triggered.



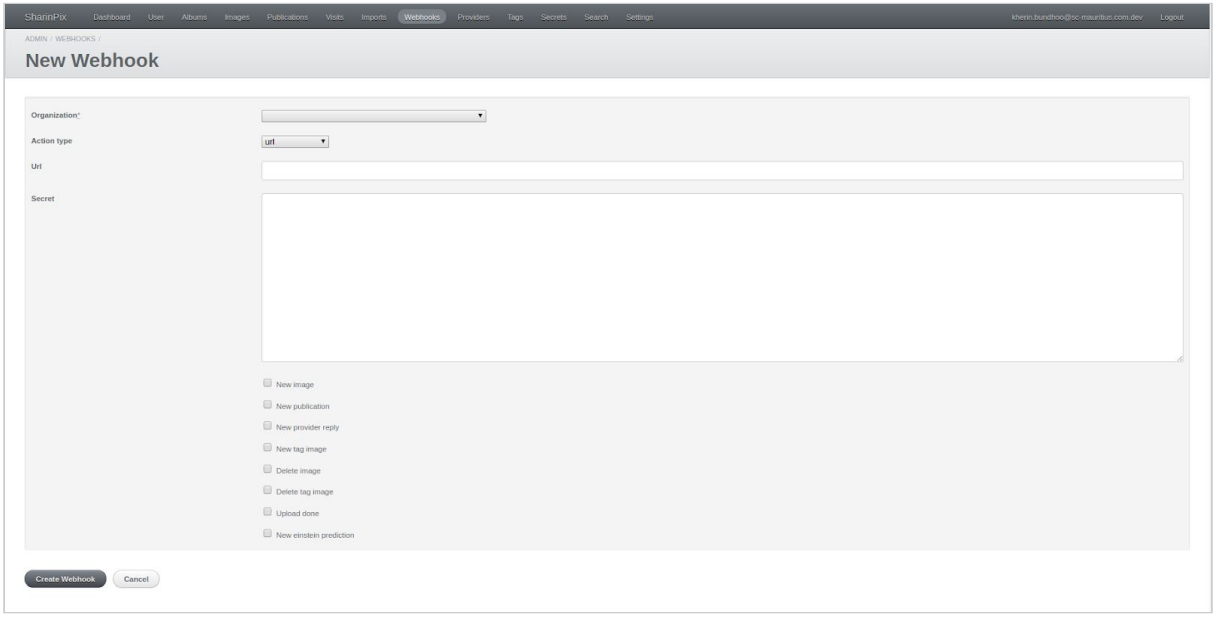
Types of webhooks

1. Call to URL with JSON payload using HTTP POST
2. Call to a static method in an Apex class

To configure a webhook along with its endpoint url, access the Admin Dashboard and follow the instructions below.

1. Call to URL with JSON payload using HTTP POST

- **Select the ‘Webhooks’ menu item from the navigation bar.**
- **Click on the ‘New Webhook’ button.**
- **Enter the relevant details for the webhook.**



2. Call to a static method in an Apex class

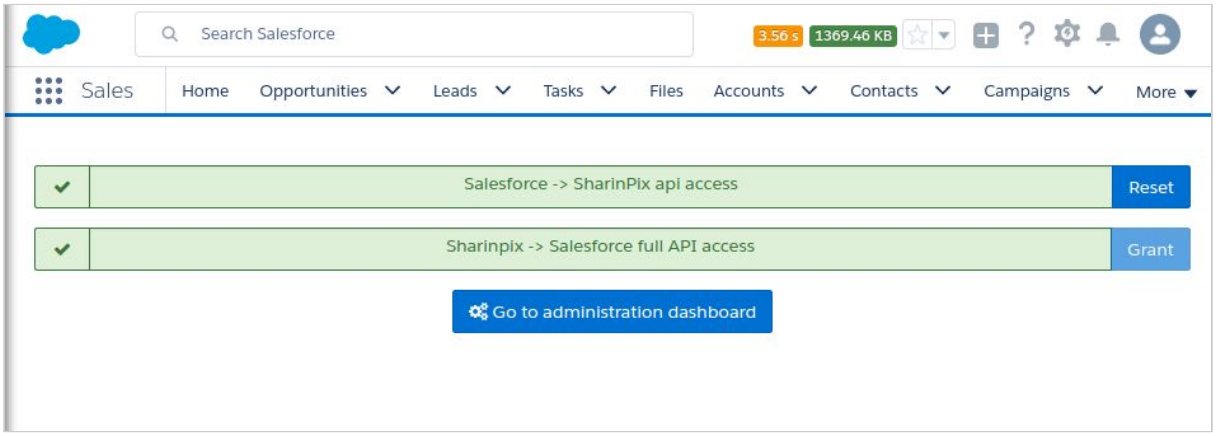
In this case, we will use another type of webhook which will execute a static method on your Salesforce environment.

Sample:

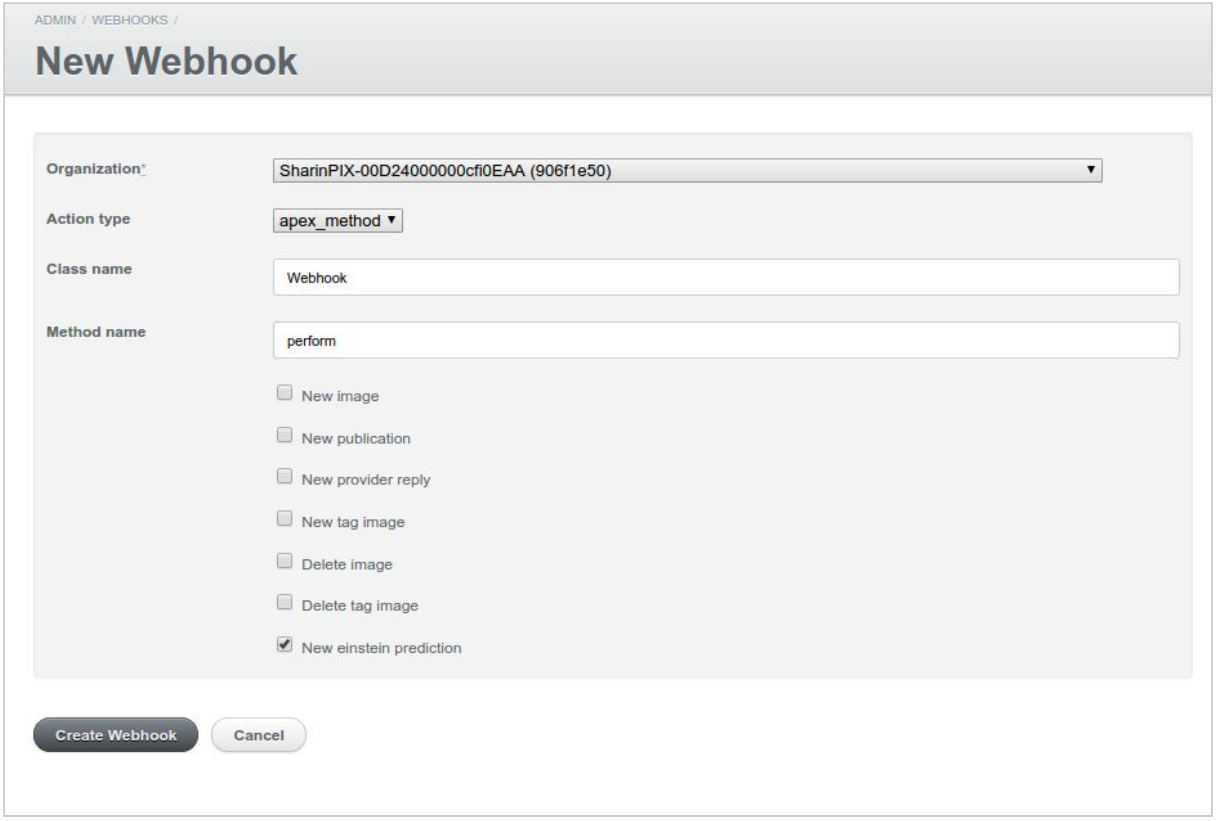
```
public class Webhook {
    public static void perform(string payload, string webhookInformation){
        // your business logic here
    }
}
```

payload is the json string containing the relevant actions.
webhookInformation contains information about the webhook such as the event type.

Now check whether you have already granted the Salesforce Full API access to SharinPix. To do so, go to the SharinPix Settings tab and check. If not, click on Grant to authenticate.



Next, go to the SharinPix Administration dashboard, click on the menu **Webhooks** and **New Webhook**. Select Action type **apex_method** and insert your class name and method name and select **New einstein prediction** as event type.



Click on **Create Webhook** to save.

Whenever a prediction call is made, this webhook will be executed as soon as a response is received from Einstein.

Find out how to set up and handle callbacks when events like adding a new image occurs here: https://github.com/SharinPix/demo-apex/tree/webhook

Image Syncing with the use of Apex methods in a webhook

It is possible to perform an image syncing, that is update the fields of a SharinPix Image object, through the use of Action Type: **apex_method** within a webhook.

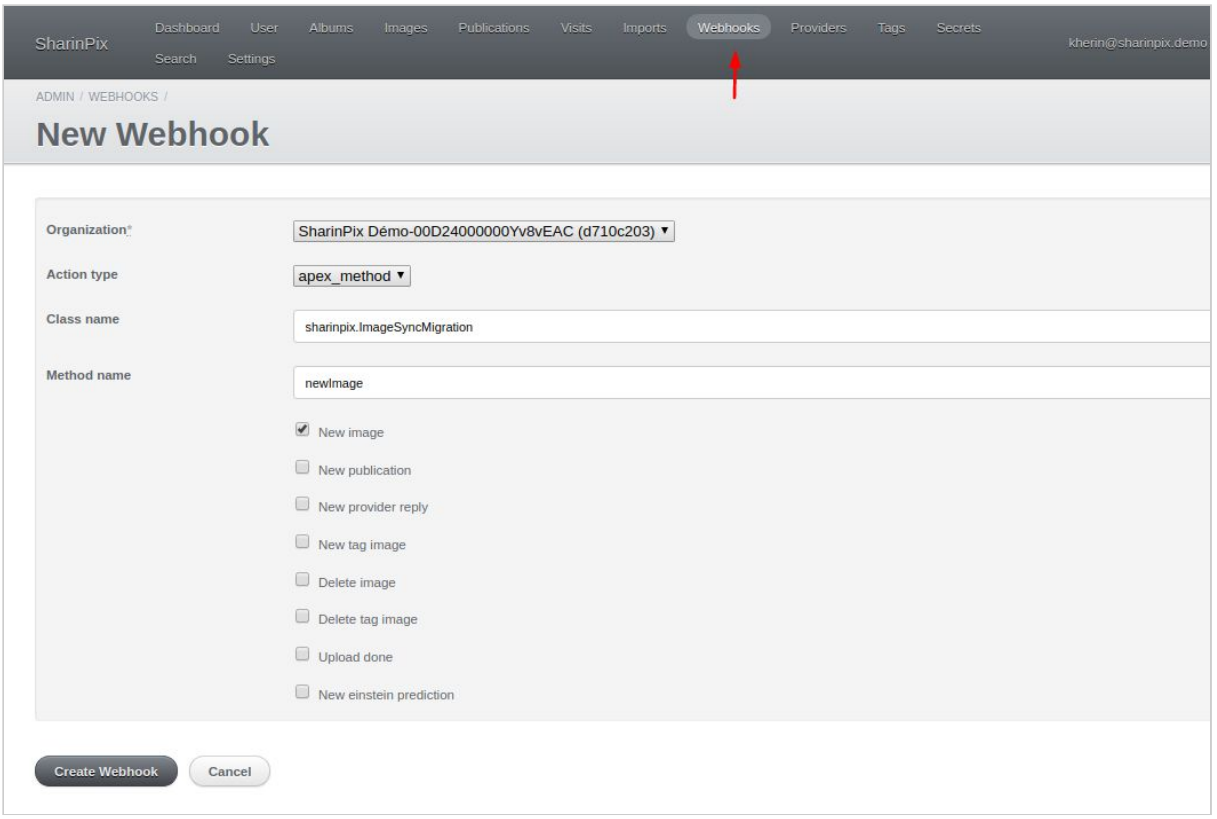
Assumptions:

- Ensure a SharinPix Album has been added to the page-layout / page of the Object of your choice.
- Ensure a SharinPix Image related list has been added to the page-layout of the chosen Object.
- Ensure that Image Sync has already been configured. To setup Image Sync, please refer to this section: Setup SharinPix Image Sync

Case Study: New Image

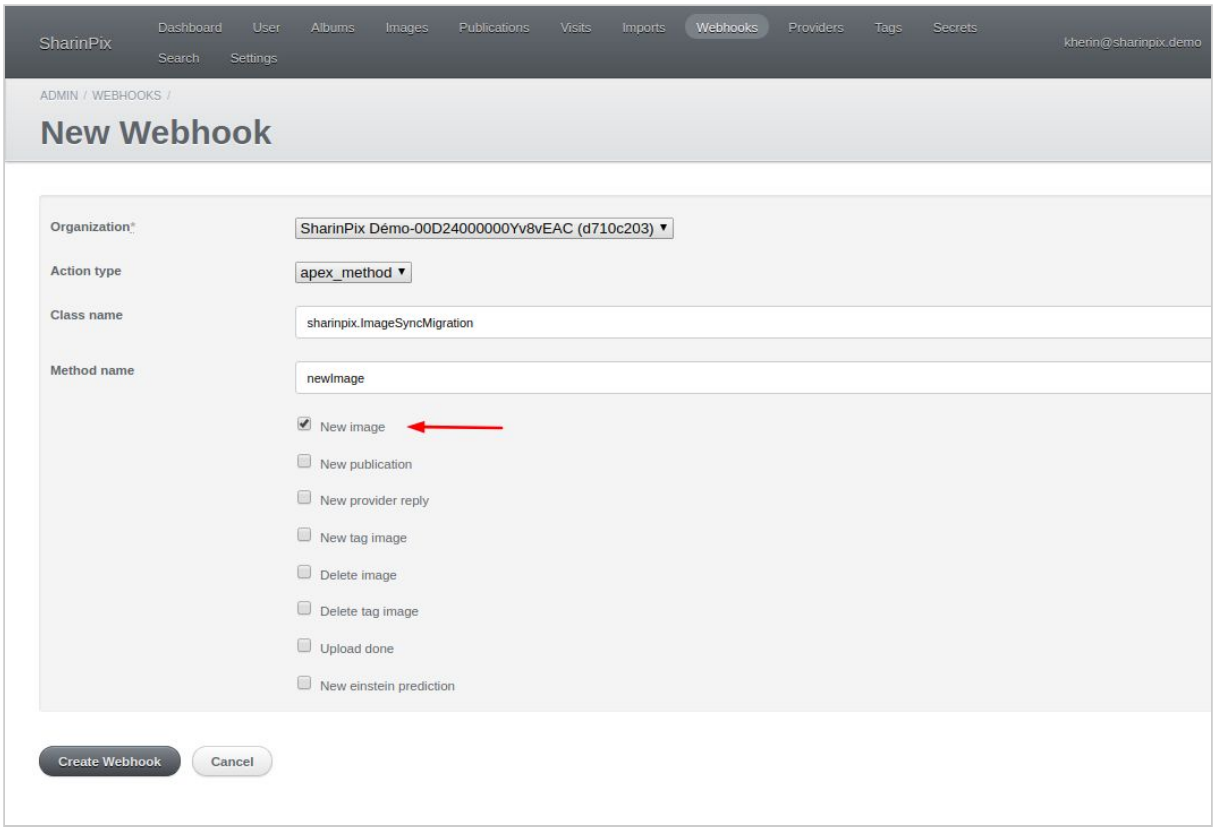
The objective of this case study is to resync an image that has been newly-added on a SharinPix album. In this context, resync means creating a new SharinPix Image record with its fields containing the corresponding details of the newly-added image in the SharinPix Album.

- Go to SharinPix Admin and navigate to the **Webhooks** section as shown in the image below. Create a new webhook or edit an existing one.

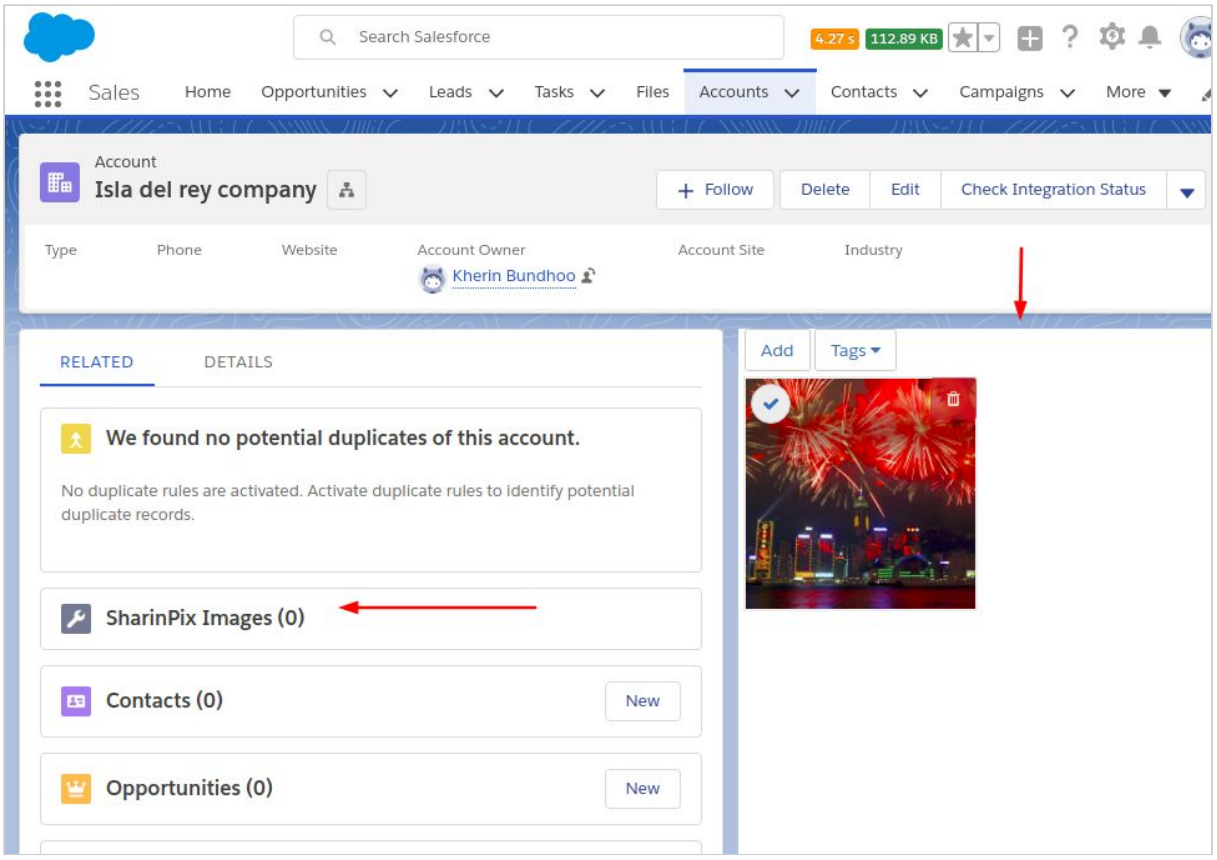


- At the **Organization** list, select the relevant Organization identifier.
- At the **Action Type** list, select **apex_method**.
- At the **Class name** field, enter **sharinpix.ImageSyncMigration**, which references a class in the SharinPix package.
- At the **Method name** field, enter **newImage**.

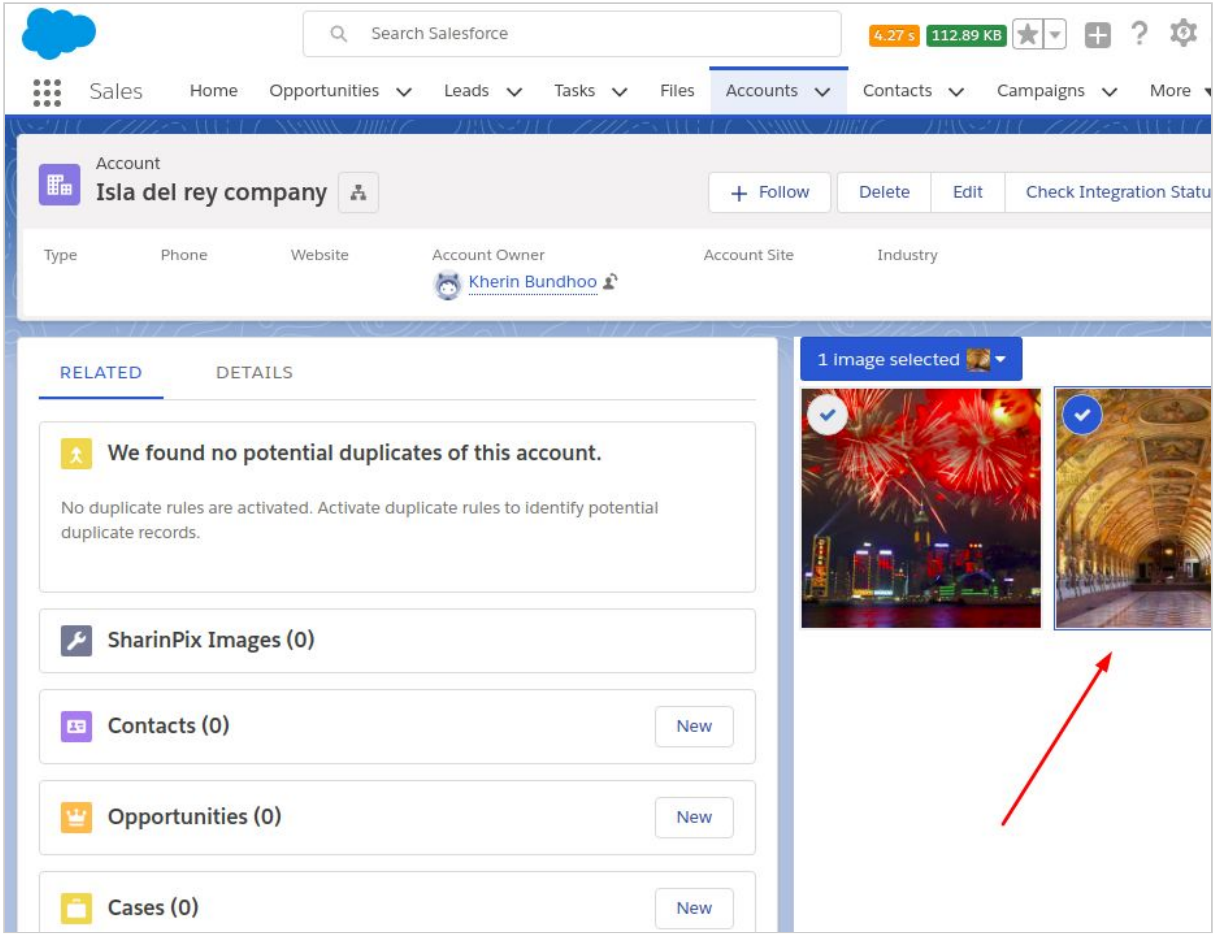
- Check the input checkbox besides the label **New image**.



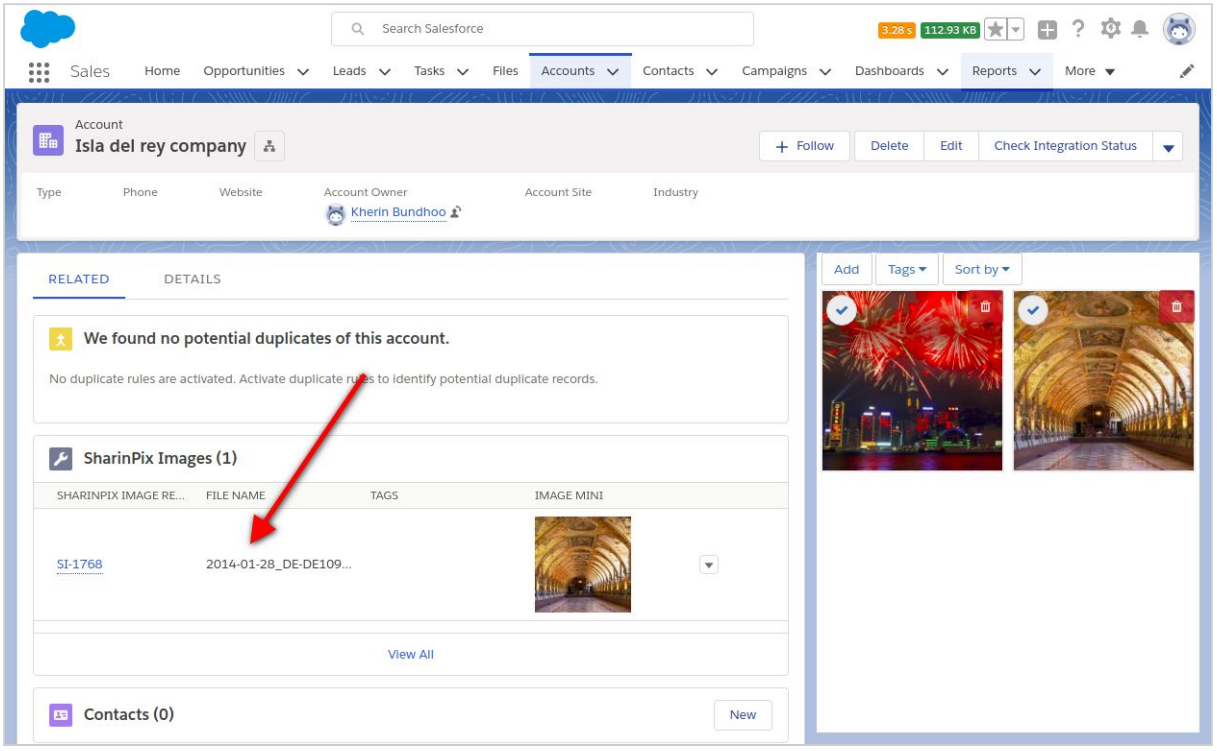
In this context, the related list of SharinPix Images and the SharinPix Album is present on the page-layout and page of the Account Object respectively.



- Add a new image on the SharinPix Album.



- Once the image has been uploaded, you need to refresh the current webpage. A new entry in the SharinPix Images related list has been added as shown in the image below.

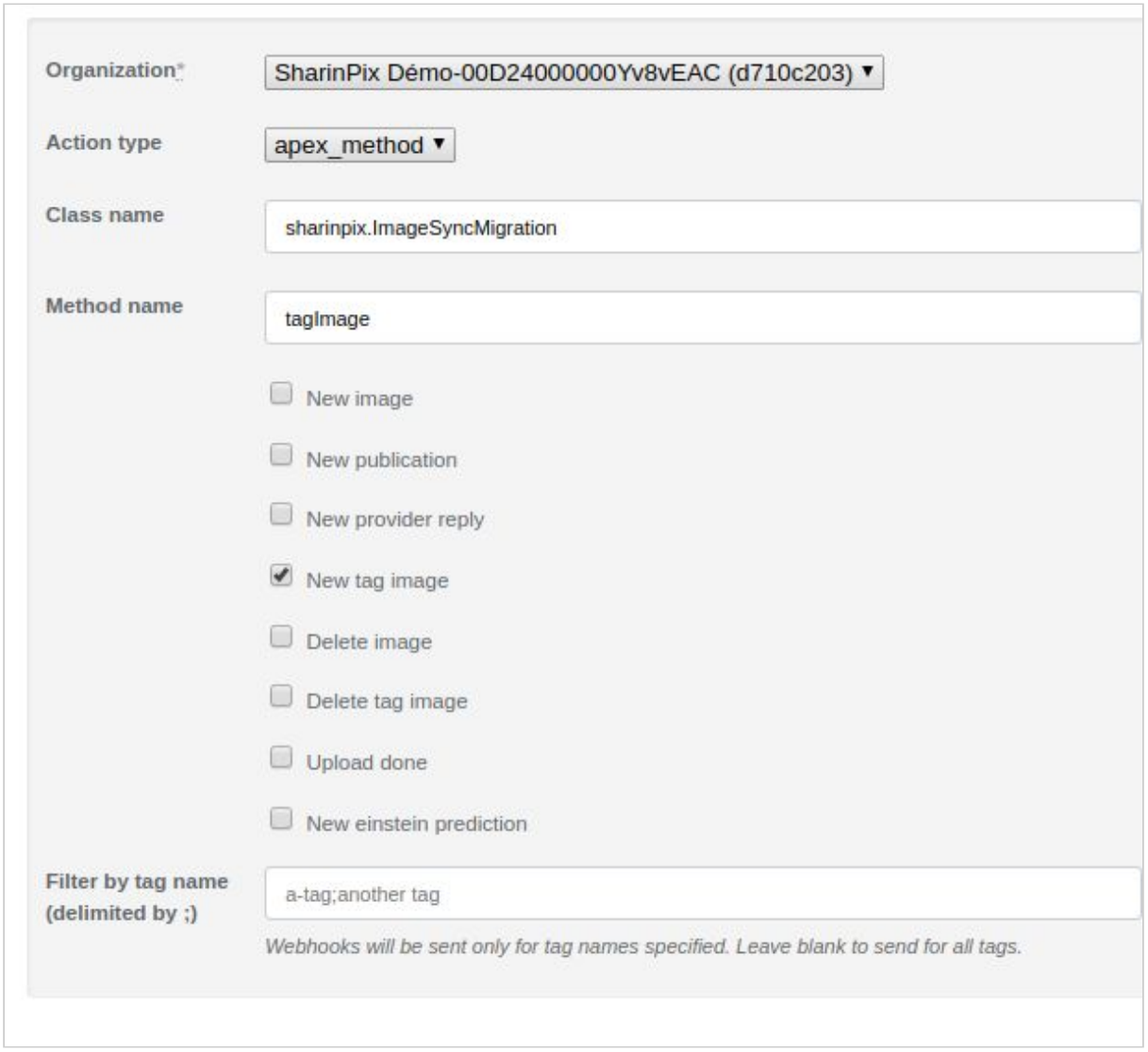


1. The new entry contains the filename of the newly-added image in the SharinPix Album and a reference to a SharinPix Image with fields containing the corresponding details of the newly-added image.

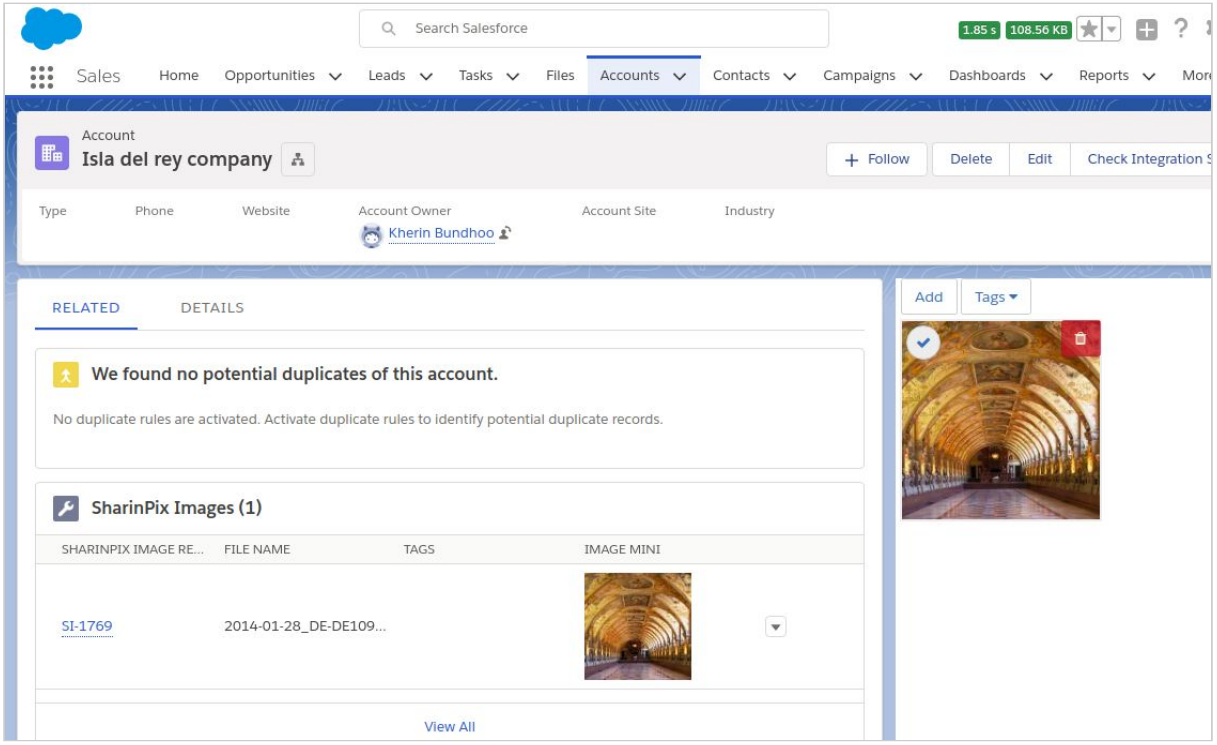
Case Study: New tag Image

The objective of this case study is to resync the SharinPix Album upon tagging of an image. In this context, when a image is tagged, this change is reflected upon the SharinPix Image record corresponding to the newly-tagged image .

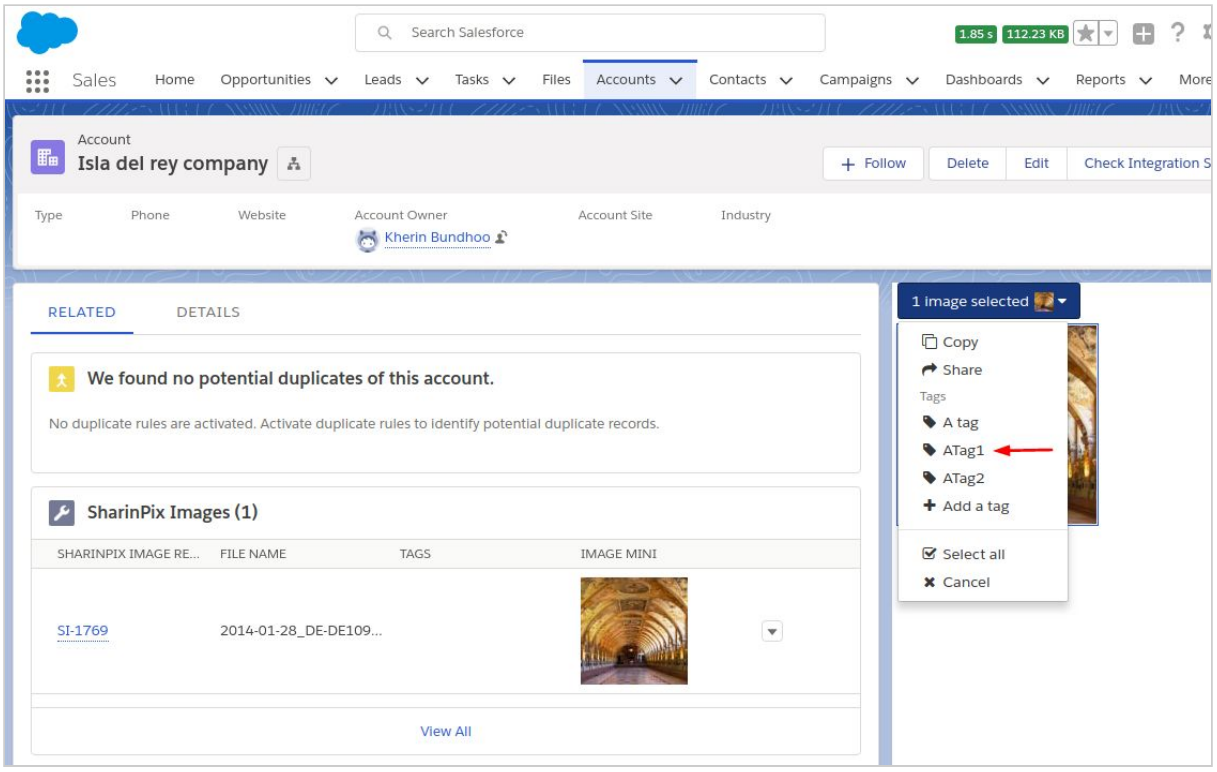
- Access the **Webhooks** section. Create a new webhook or edit an existing one.



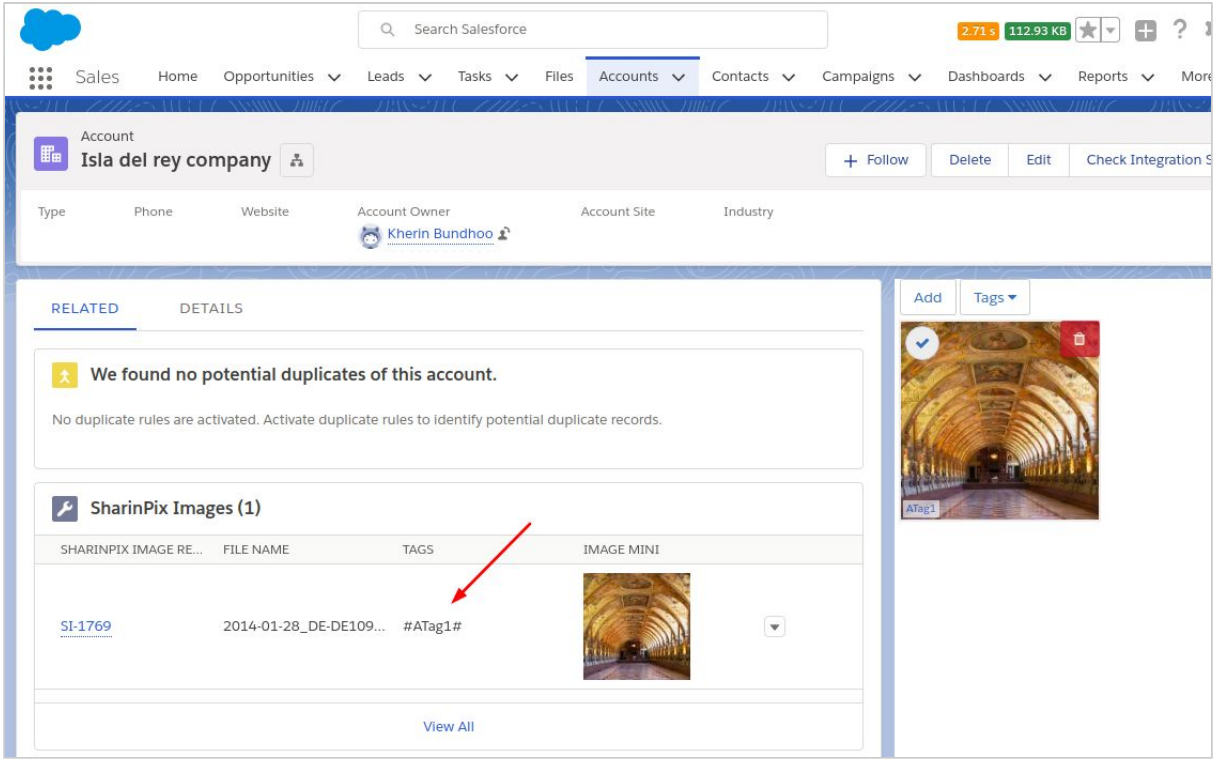
- At the **Organization** list, select the relevant Organization identifier.
- At the **Action Type** list, select **apex_method**.
- At the **Class name** field, enter **sharinpix.ImageSyncMigration**, which references a class in the SharinPix package.
- At the **Method name** field, enter **tagImage**.
- Select the **New tag image** checkbox.
- In the field **Filter by tag name (delimited by ;)**, you are able to specify which tags will trigger the activation of the webhook. You can leave this field blank if you need the webhook to be activated by all tags.
- In this context, the related list of SharinPix Images and the SharinPix Album is present on the page-layout and page of the Account Object respectively.



- Select an image from the SharinPix Album. Select a tag for the image. If no tag is present, create a new tag. In this case, the tag labelled **ATag1** has been selected.



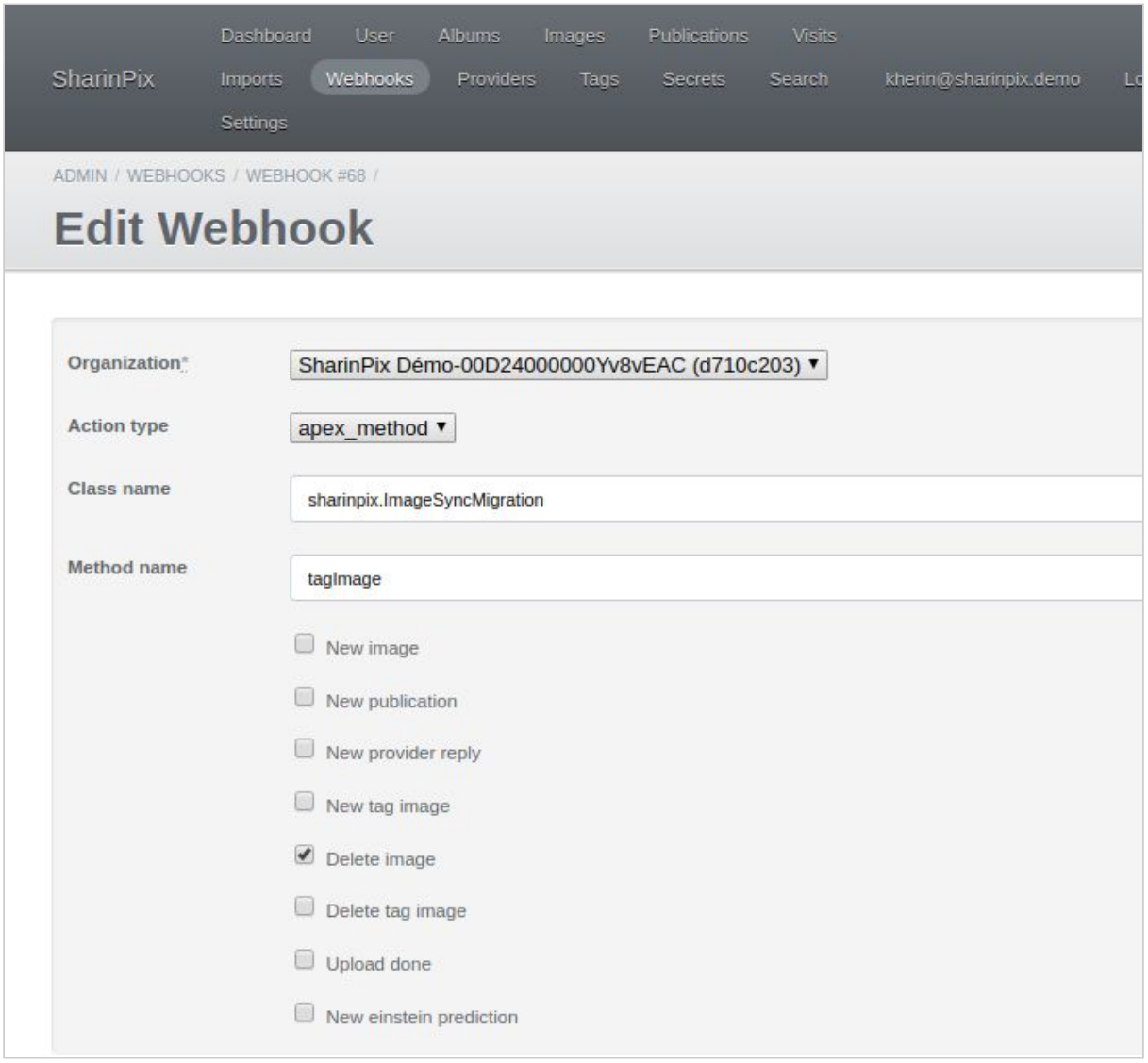
- Once the current webpage is refreshed, the tag is present in the **TAGS** field of the SharinPix Image record corresponding to the newly-tagged image.



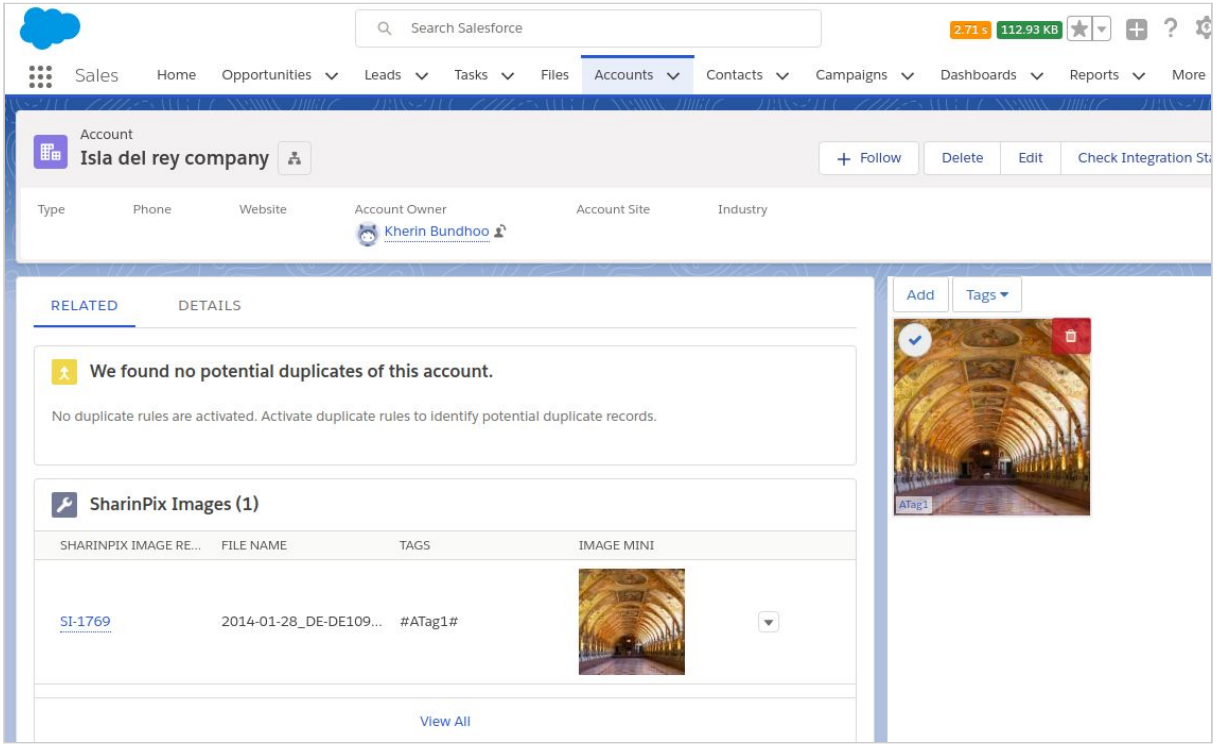
Case Study: Delete tag Image

The objective of this case study is to resync the SharinPix Album upon deletion of an existing tag of an image. In this context, when the tag of an image is deleted, this change is reflected upon the SharinPix Image record corresponding to the image .

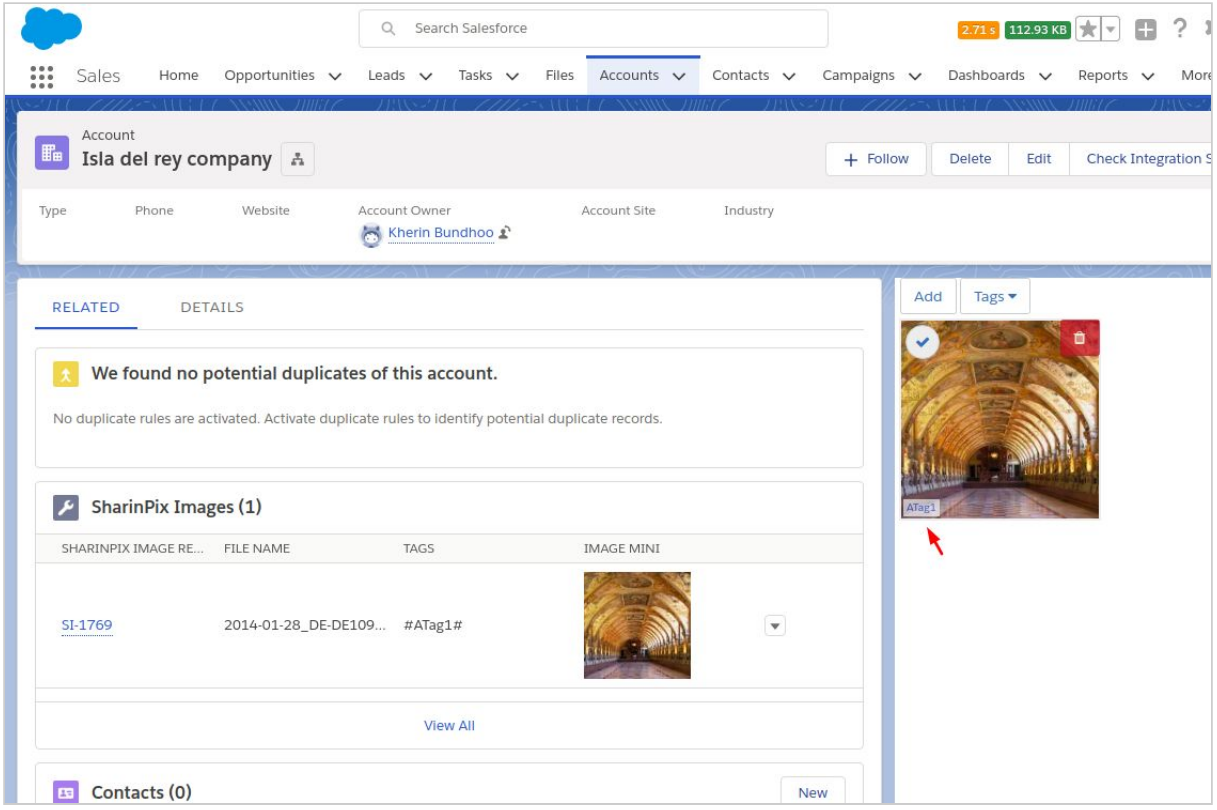
- Access the **Webhooks** section. Create a new webhook or edit an existing one.



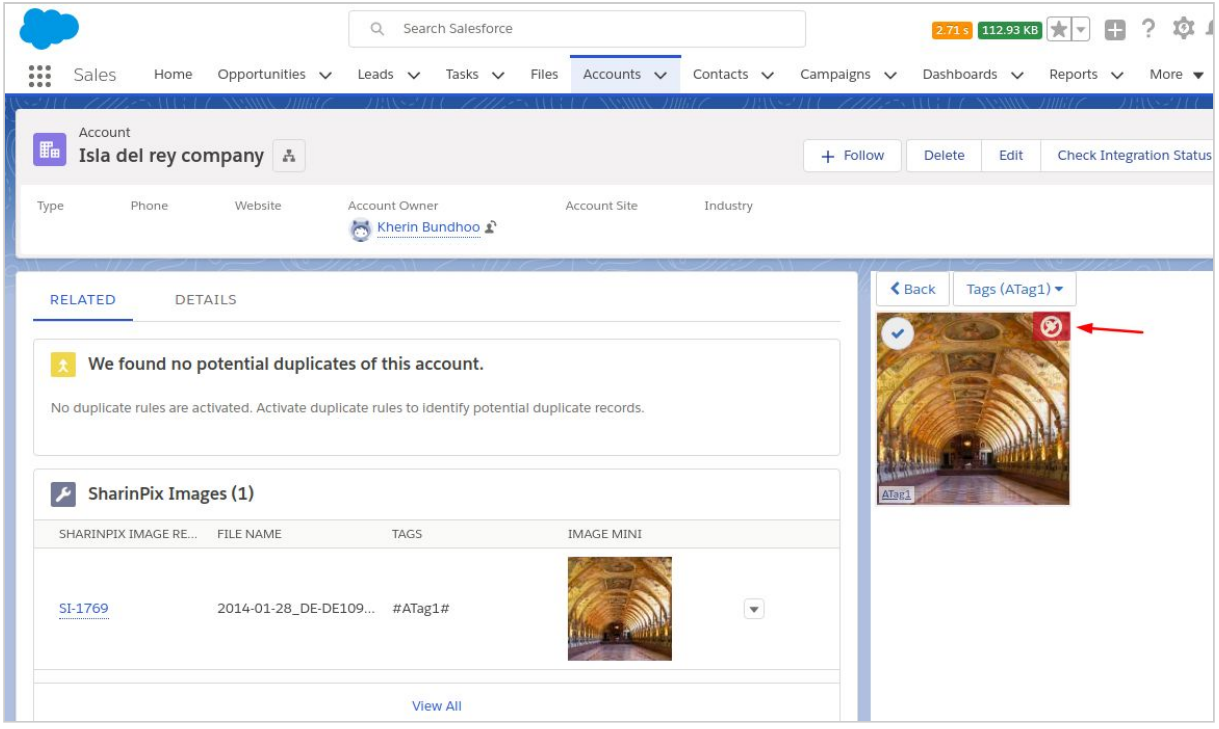
- At the **Organization** list, select the relevant Organization identifier.
- At the **Action Type** list, select **apex_method**.
- At the **Class name** field, enter **sharinpix.ImageSyncMigration**, which references a class in the SharinPix package.
- At the **Method name** field, enter **tagImage**.
- Select the **Delete tag image** checkbox.
- In the field **Filter by tag name (delimited by ;)**, you are able to specify which tags will trigger the activation of the webhook. You can leave this field blank if you need the webhook to be activated by all tags.
- In this context, the related list of SharinPix Images and the SharinPix Album is present on the page-layout and page of the Account Object respectively.
- Select an image that has an existing tag. In this case, the image illustrated in the figure below possesses the tag labelled **ATag1**. Note that the SharinPix Image record , within the related list of the corresponding image also contains the same tag in the **TAGS** field.



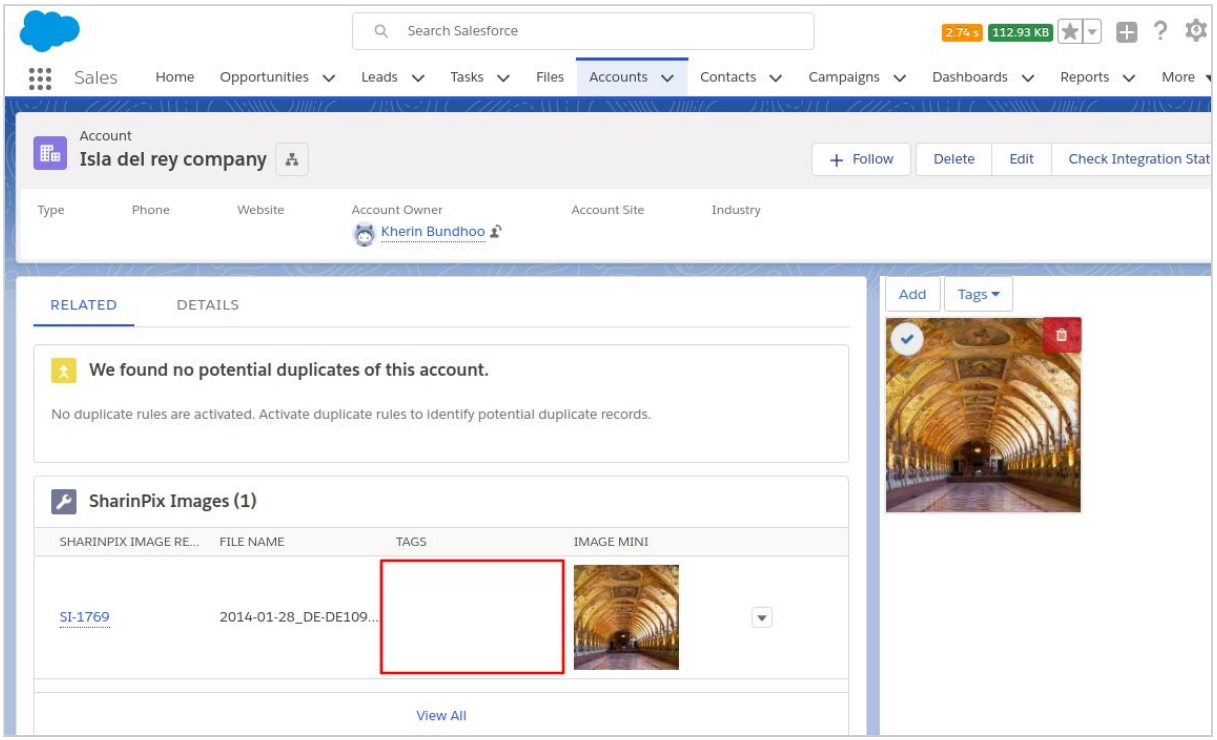
- Click on the tag label found at the foot of the image.



- Delete the tag and refresh the current webpage.



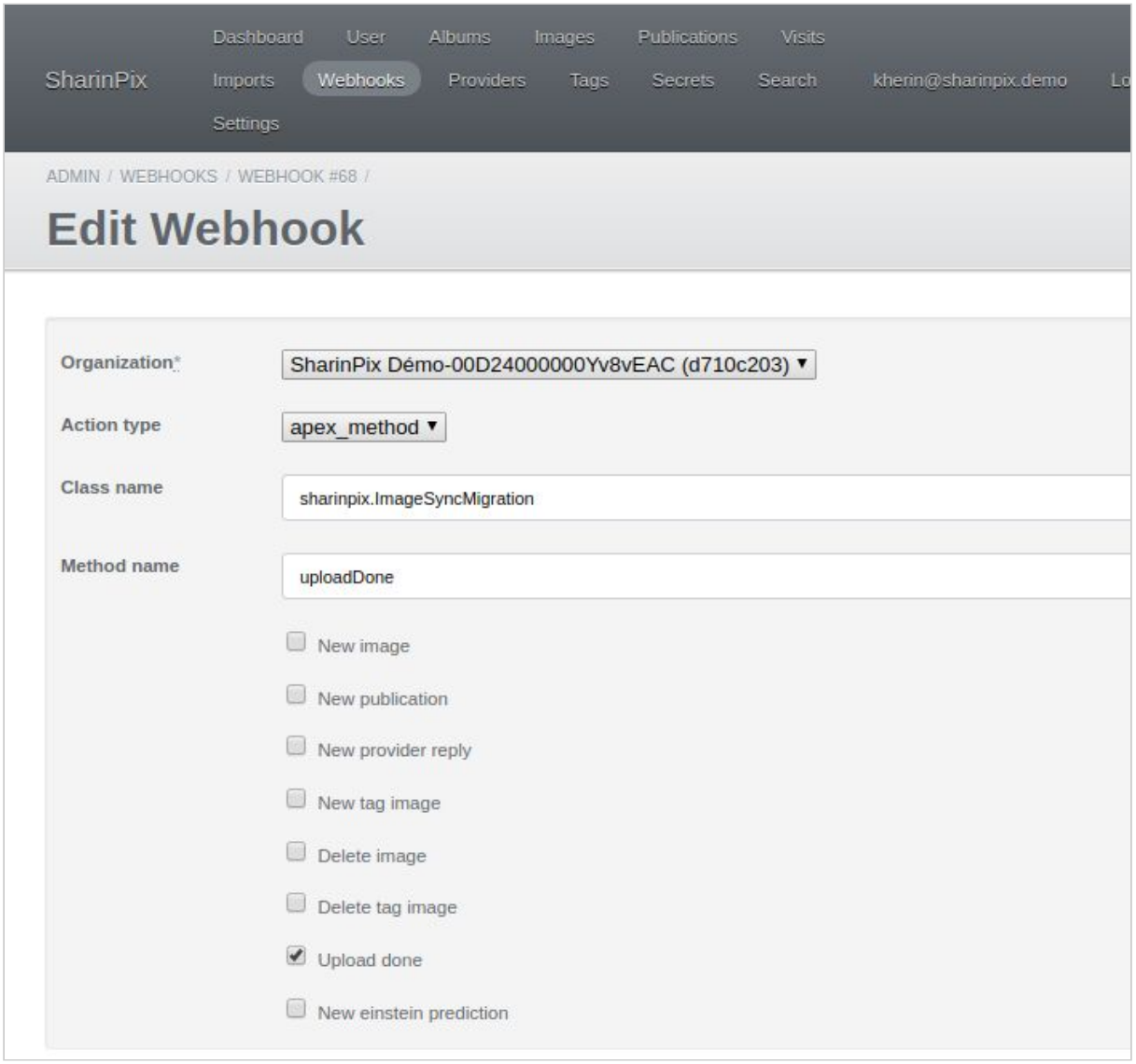
- The tag label is removed from the **TAGS** field in the corresponding SharinPix Image record.



Case Study: Upload done

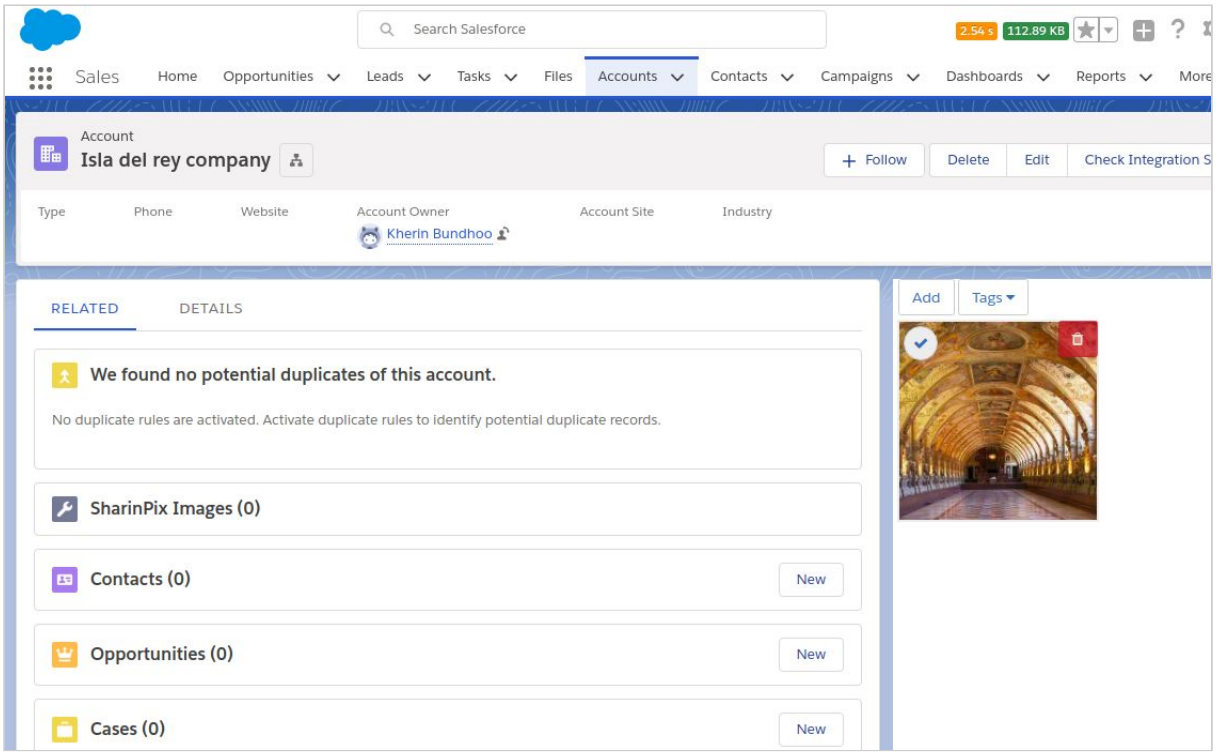
The objective of this case study is to resync the SharinPix Album upon uploading a new image. This webhook proves to be useful when already existing images on the SharinPix Album need to be resynced (Create a Sharinpix Image record corresponding to the relevant image).

- Access the **Webhooks** section. Create a new webhook or edit an existing one.

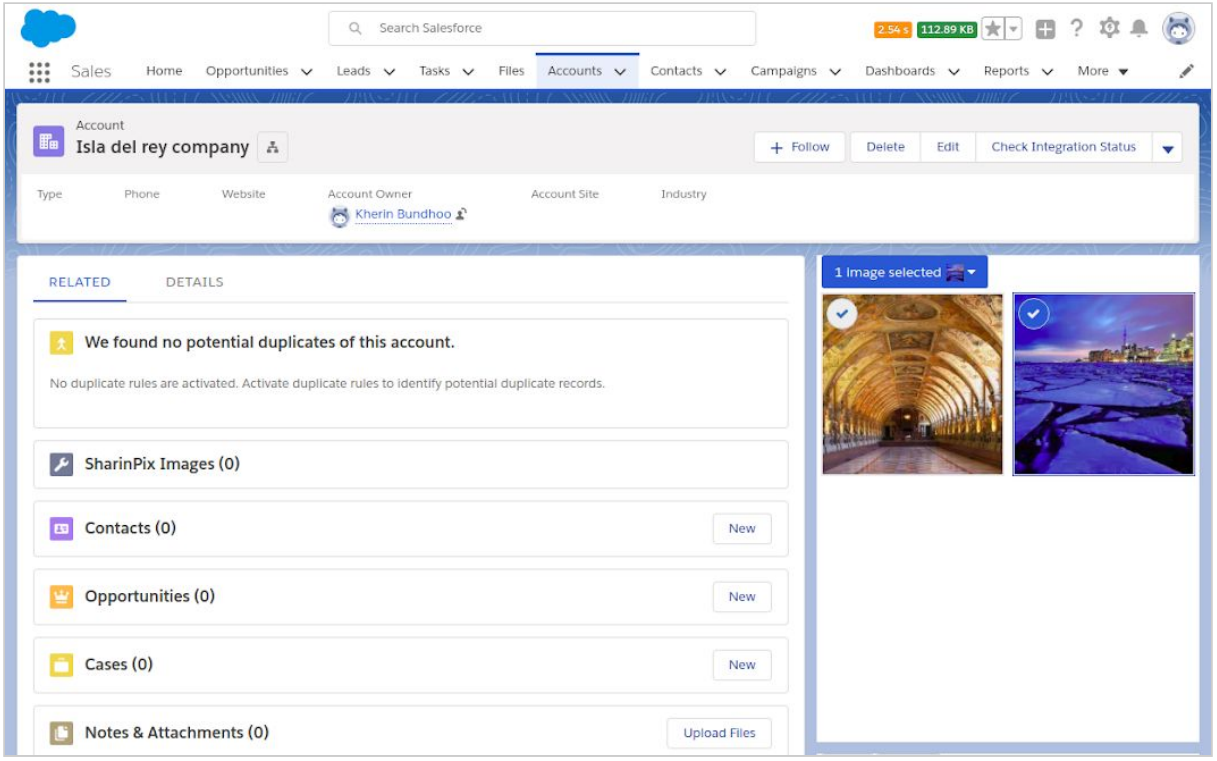


- At the **Organization** list, select the relevant Organization identifier.
- At the **Action Type** list, select **apex_method**.
- At the **Class name** field, enter **sharinpix.ImageSyncMigration**, which references a class in the SharinPix package.
- At the **Method name** field, enter **uploadDone**.
- Select the **Upload Done** checkbox.

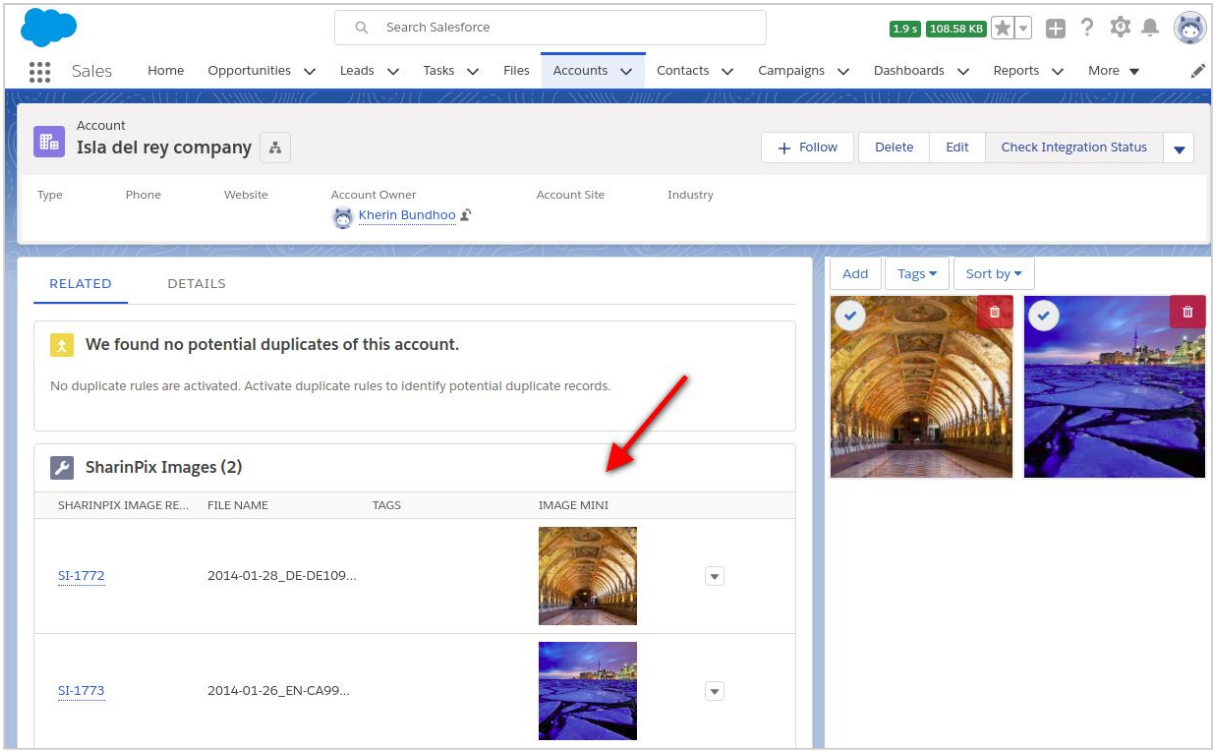
- In this context, the related list of SharinPix Images and the SharinPix Album is present on the page-layout and page of the Account Object respectively.
- From the image below, it can be seen that the SharinPix Album has 1 image already present. However, this image does not have an equivalent SharinPix Image record. To remediate this issue, another image needs to added to the SharinPix Image for a resync process to take place which will create corresponding SharinPix Image records for all images in the SharinPix Album.



- Upload a new image.



- Refresh the current webpage. It can be observed that new SharinPix Image records are present in the related list, including the corresponding record for the already present image shown previously.



Upload a Salesforce attachment as a SharinPix image

Once deployed, this demo will add a trigger to all newly created attachments on Salesforce cases. After that, set up Email-to-Case in your salesforce org if not already done. Ensure you have a SharinPix album in your case page.
You can now send and email with image (JPEG) attachments to see this demo in action.

Find out how to upload attachment received by email to an object’s related album here: https://github.com/SharinPix/demo-apex/tree/email2case_attachment

- Activate Email-to-Case.
- Save the Email Service Address.
- Send an email to the above Email Service Address along with image(s).
- Then, go to the newly created Case object. The images in the email attachment are present in the SharinPix album of the Case object.

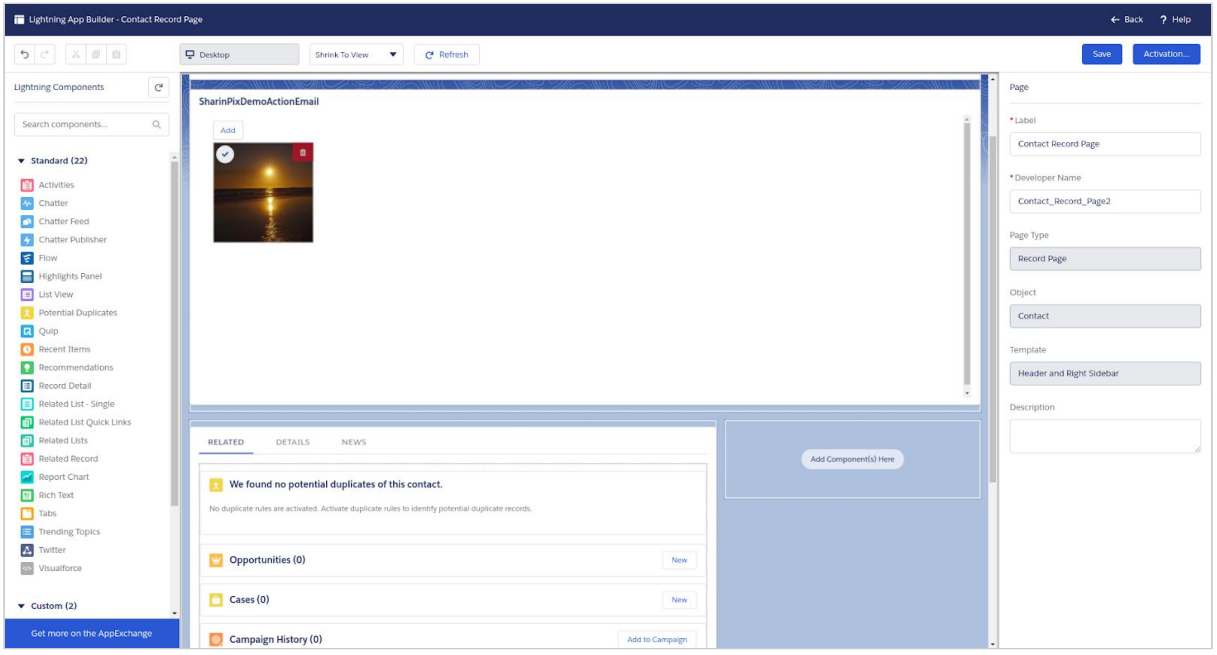
This usage requires to buy external packages usage from SharinPix, to have more information contact the SharinPix sales team info@sharinpix.com.

Send a Salesforce email with SharinPix images

Find out how to send emails containing SharinPix images with our demo here: https://github.com/SharinPix/demo-apex/tree/action_email

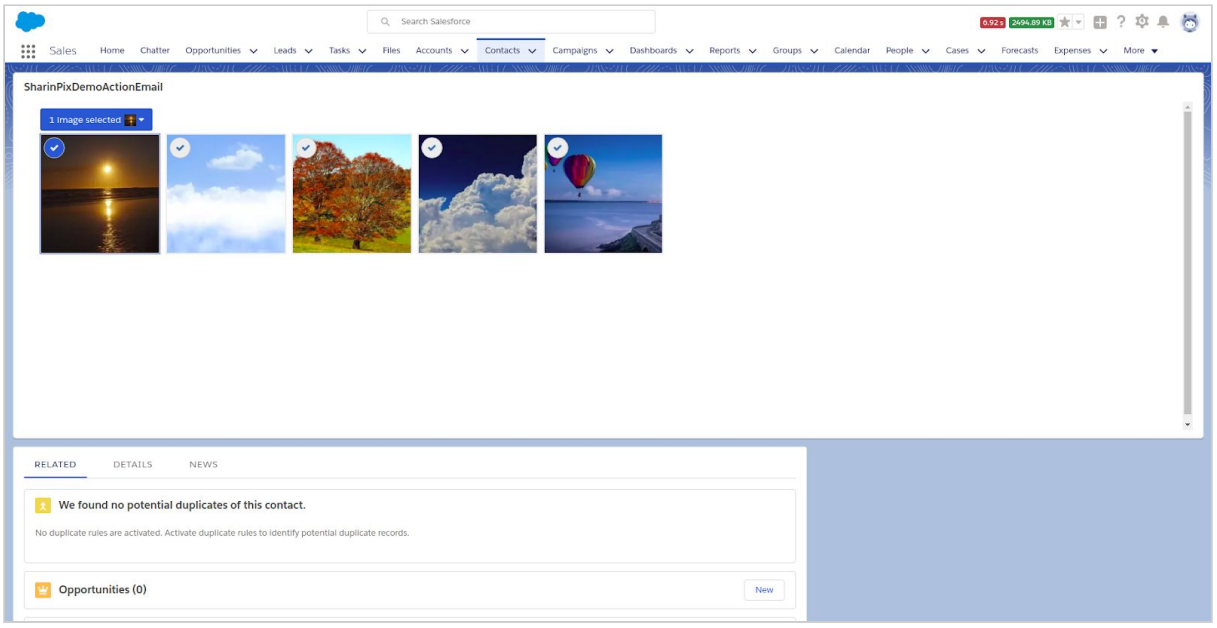
1. Add the Visualforce page to the Contact page-layout

Add the Visualforce page to the contact of your choice. Make sure the ‘Email’ field does in fact contain a correct email address to which the uploaded images will be sent to.
(Learn how to add the Visualforce Page: here)

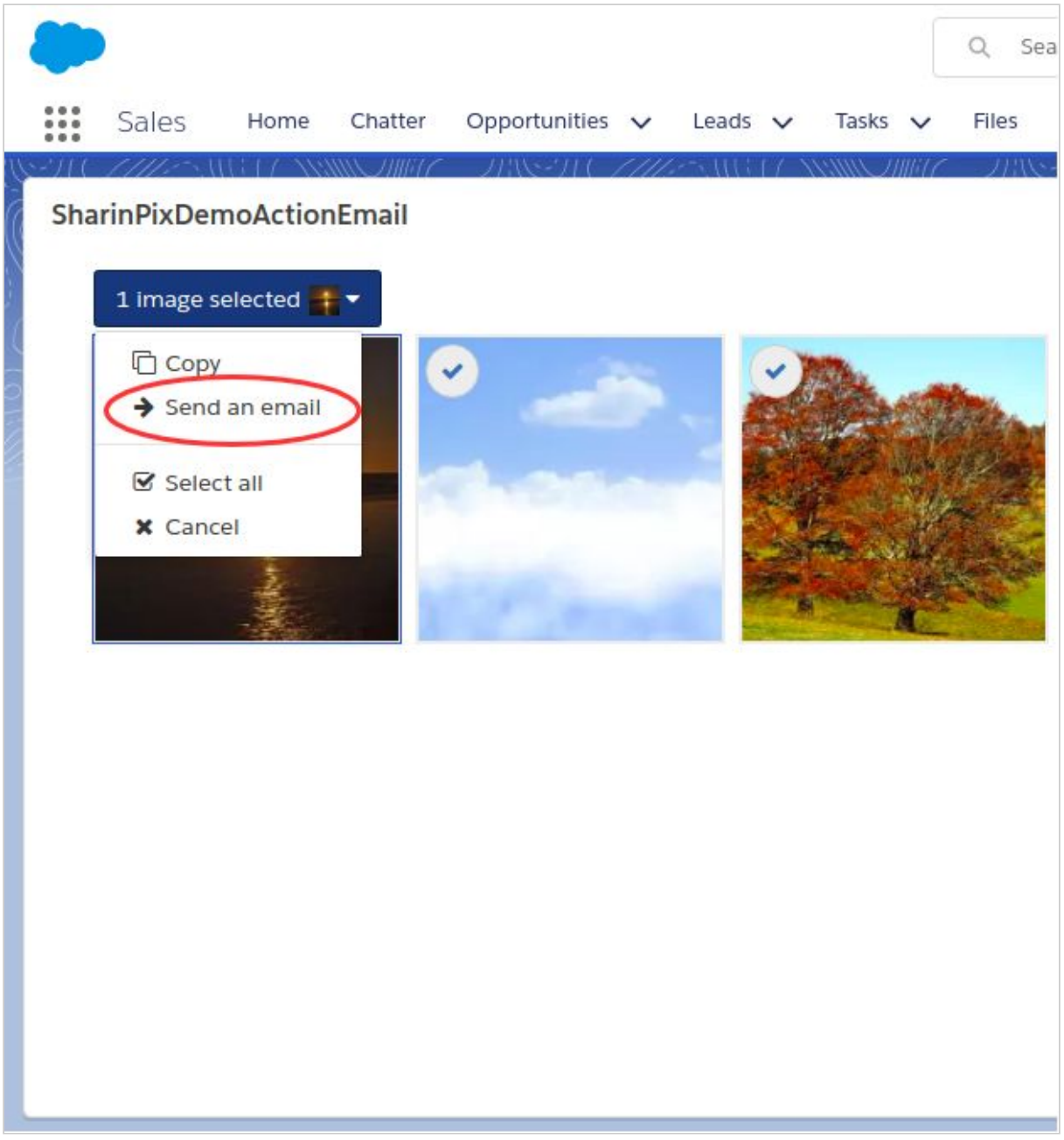


2. Upload an image

Upload image.

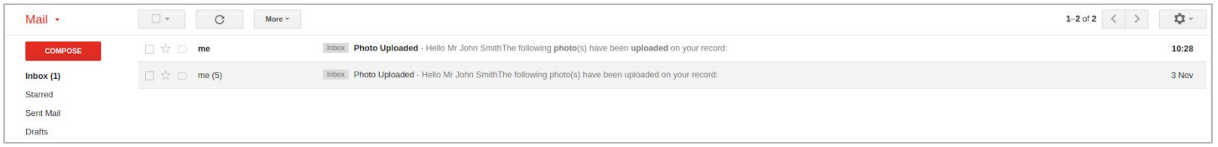


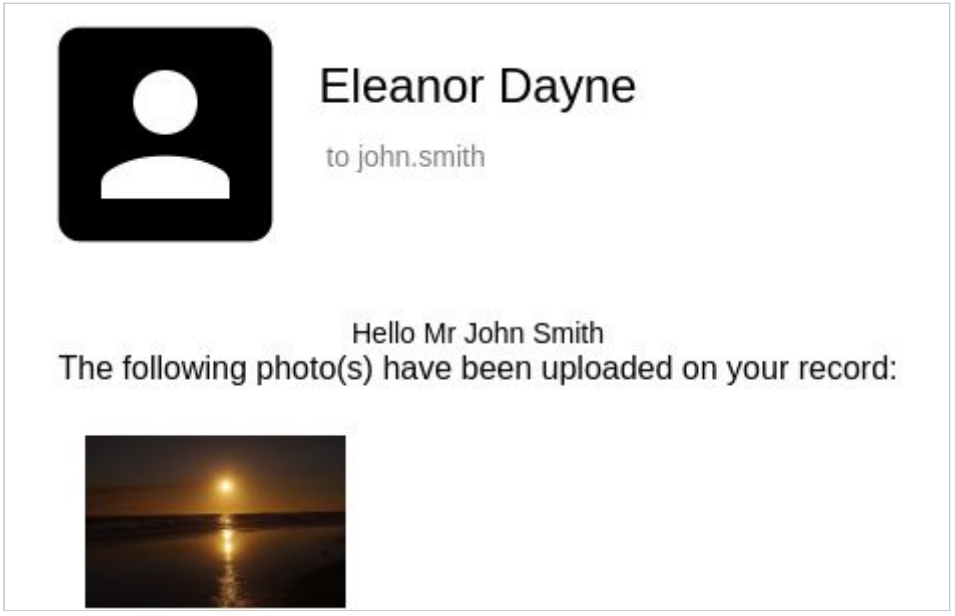
Click on the ‘Send an email’ option.



- To enable the action **Send an email**, add the action inside the apex class. Follow the link to find out how to add an action programmatically: here
- When the **Send an email** option is selected, a SharinPix event named **Send an email** is triggered and captured via Javascript. To find out how to capture more events follow this link: here

You will then receive the following email.

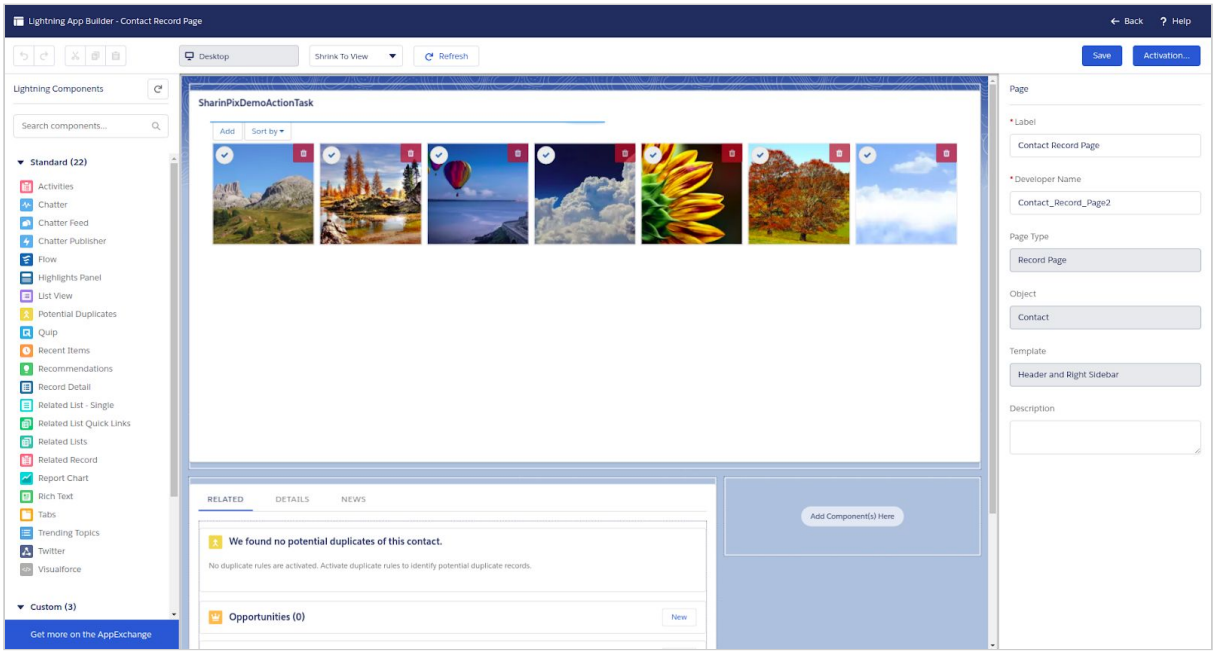




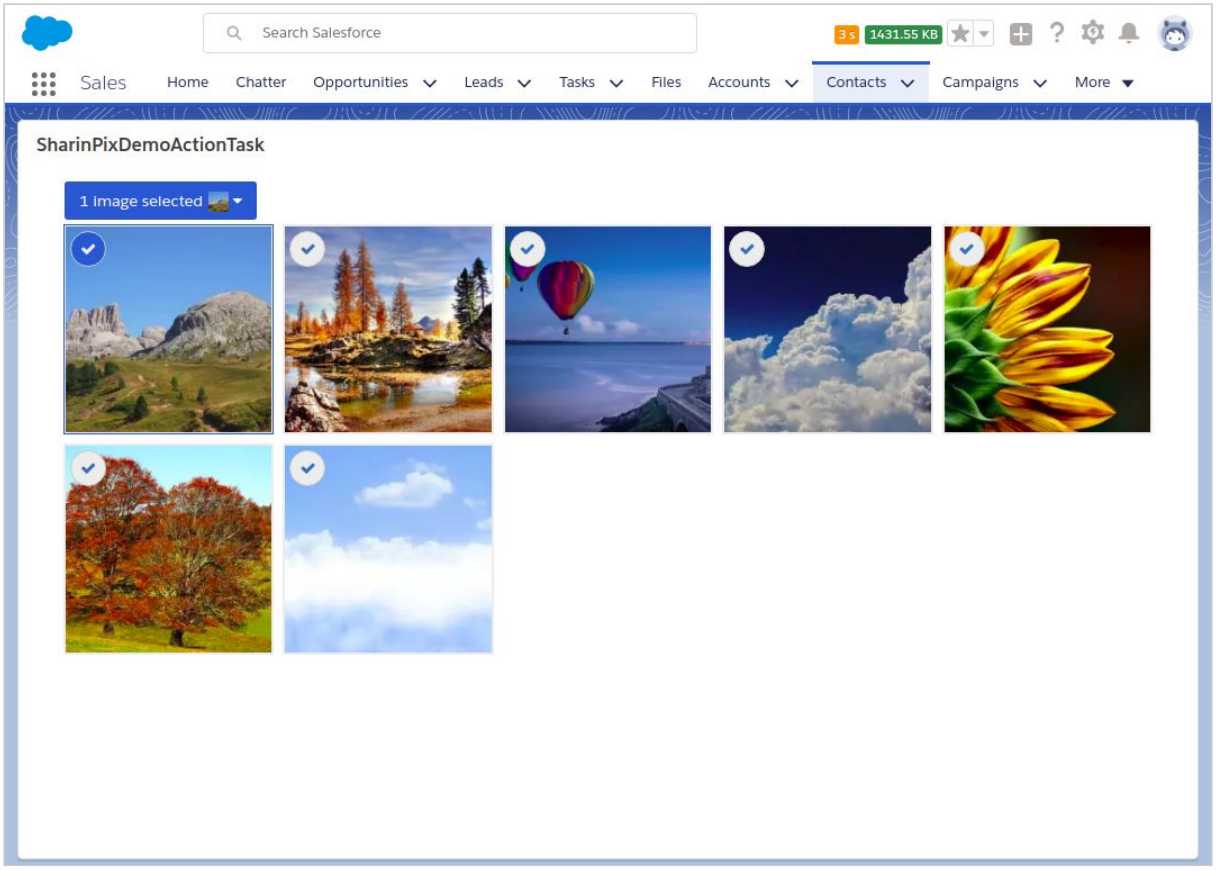
Generate a Salesforce task from SharinPix images

Find out how to create a case directly from a SharinPix album here:
https://github.com/SharinPix/demo-apex/tree/action_task
With a little manual setup as described on the link’s ReadMe, you can assign the selected images to the task being created.
(Learn how to add the Visualforce Page: here)

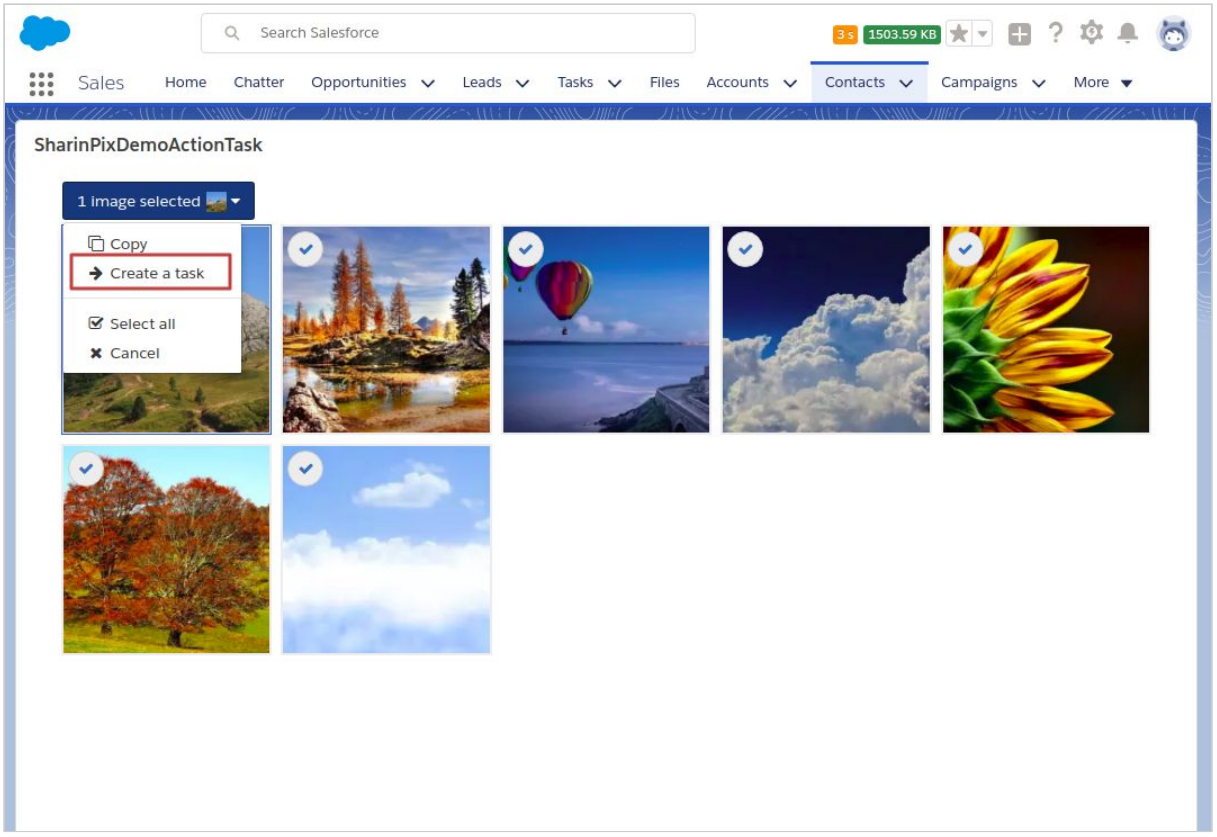
1. Add Visualforce page to page-layout of Contact object



2. Select image



3. Select ‘**Create a task**’

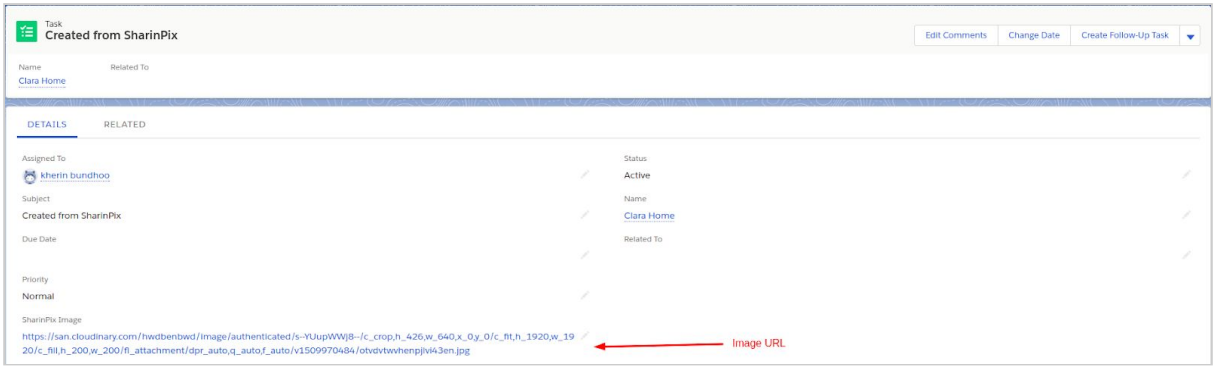


4. Attach image to task

- To enable the **Create a task** action, follow this link: here
- When the **Create a task** option is selected, a SharinPix event named **Create a task** is triggered and captured via Javascript. To find out how to capture more events follow this link: here
- Add a field with the name ‘**SharinPix_Image**’ and of type URL to the Task Object.
- In the Apex class, **SharinPixDemoActionTaskCtrl**, uncomment the following line(edit class in Developer Console):

```apex
global with sharing class SharinPixDemoActionTaskCtrl {
    global String parameters { get; set; }

    global SharinPixDemoActionTaskCtrl(ApexPages.StandardController stdCtrl) {
        Id contactId = stdCtrl.getId();
        Map<String, Object> params = new Map<String, Object> {
            'abilities' => new Map<String, Object> {
                contactId => new Map<String, Object> {
                    'Access' => new Map<String, Object> {
                        'see' => true,
                        'image_list' => true,
                        'image_upload' => true,
                        'image_delete' => true,
                        'image_crop' => true,
                        'image_rotate' => true
                    },
                    'Actions' => new List<String> {
                        'Create a task'
                    }
                }
            }
        };
        parameters = JSON.serialize(params);
    }

    @RemoteAction
    global static string createTask(String albumId, String imageUrl) {
        Task aTask = new Task();
        aTask.Subject = 'Created from SharinPix';
        aTask.Status = 'Active';
        aTask.Description = '';
        aTask.WhoId = albumId;
        // Uncomment below to add image to task. Create SharinPix_Image__c field first
        // aTask.SharinPix_Image__c = imageUrl;
        insert aTask;
        return aTask.Id;
    }
}
```

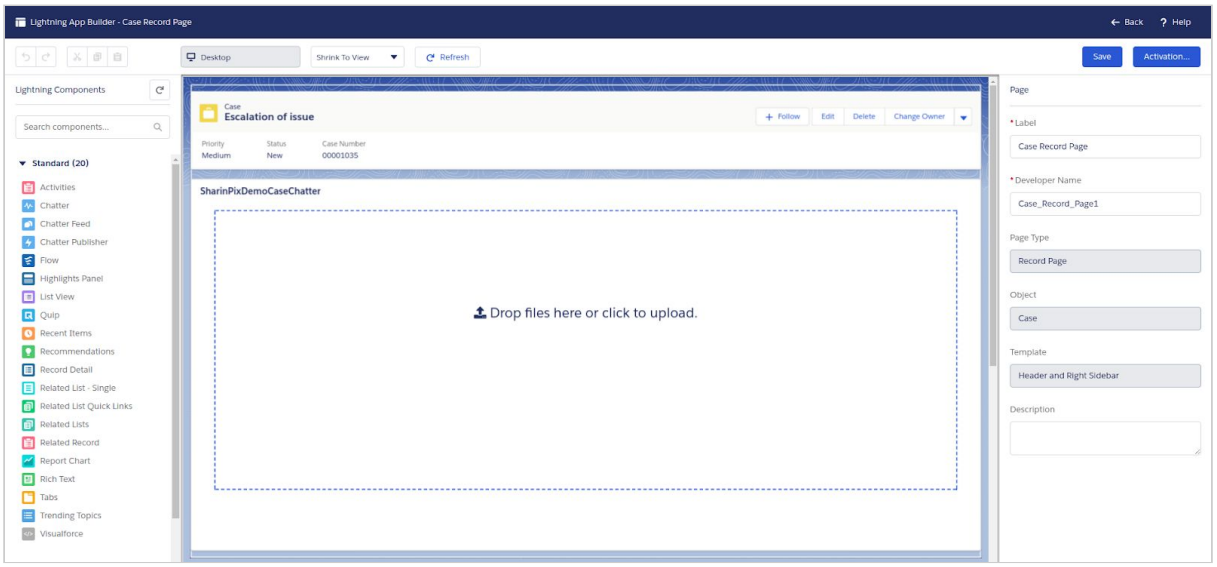
- A new task is created along with the image URL:



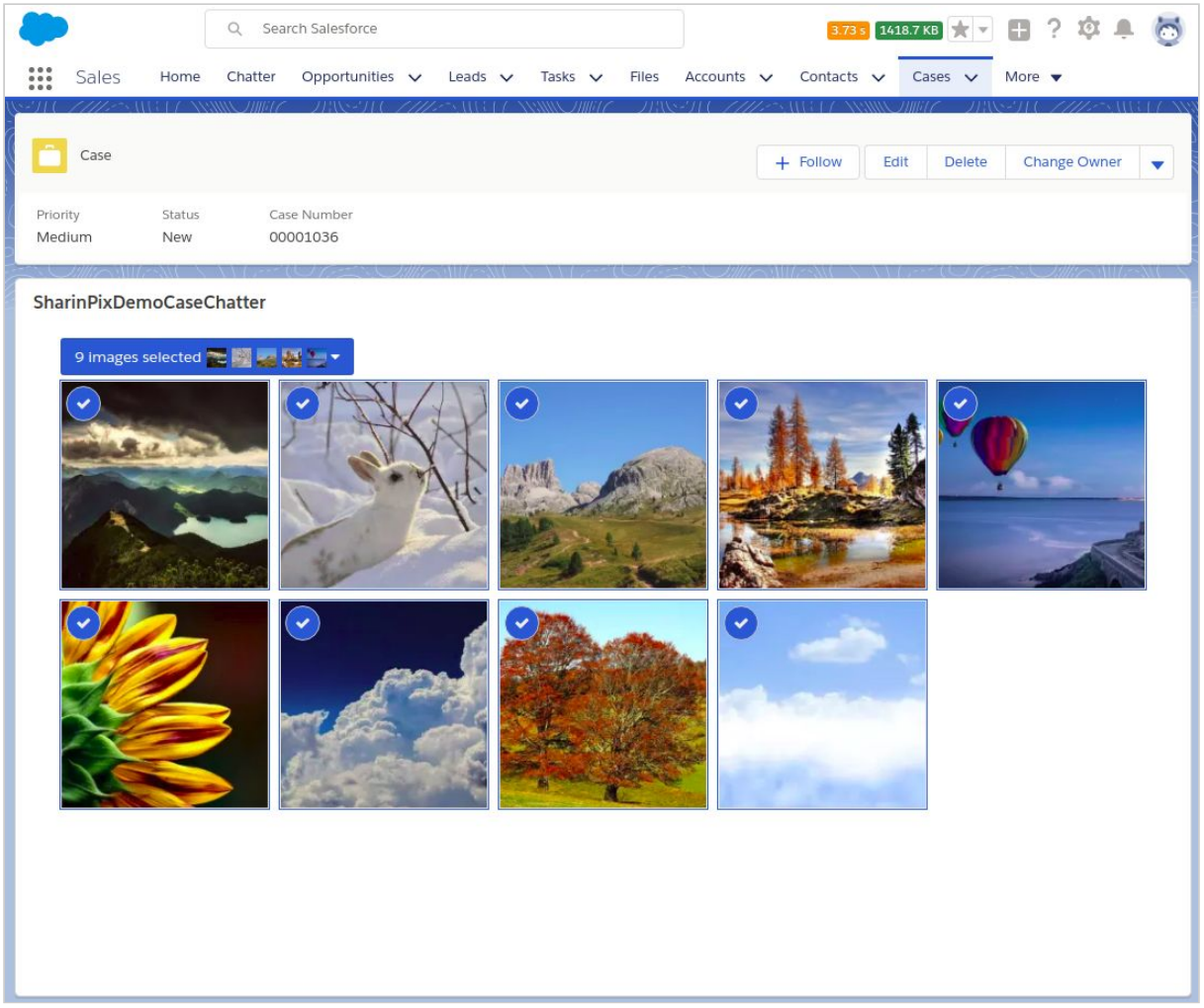
Generate a Salesforce Chatter post from images

Find out how to post your images to Chatter and make it more visual here:
https://github.com/SharinPix/demo-apex/tree/chatter_rich-text
(Learn how to add the Visualforce Page: here)

1. Add Visualforce page to page-layout of Case object

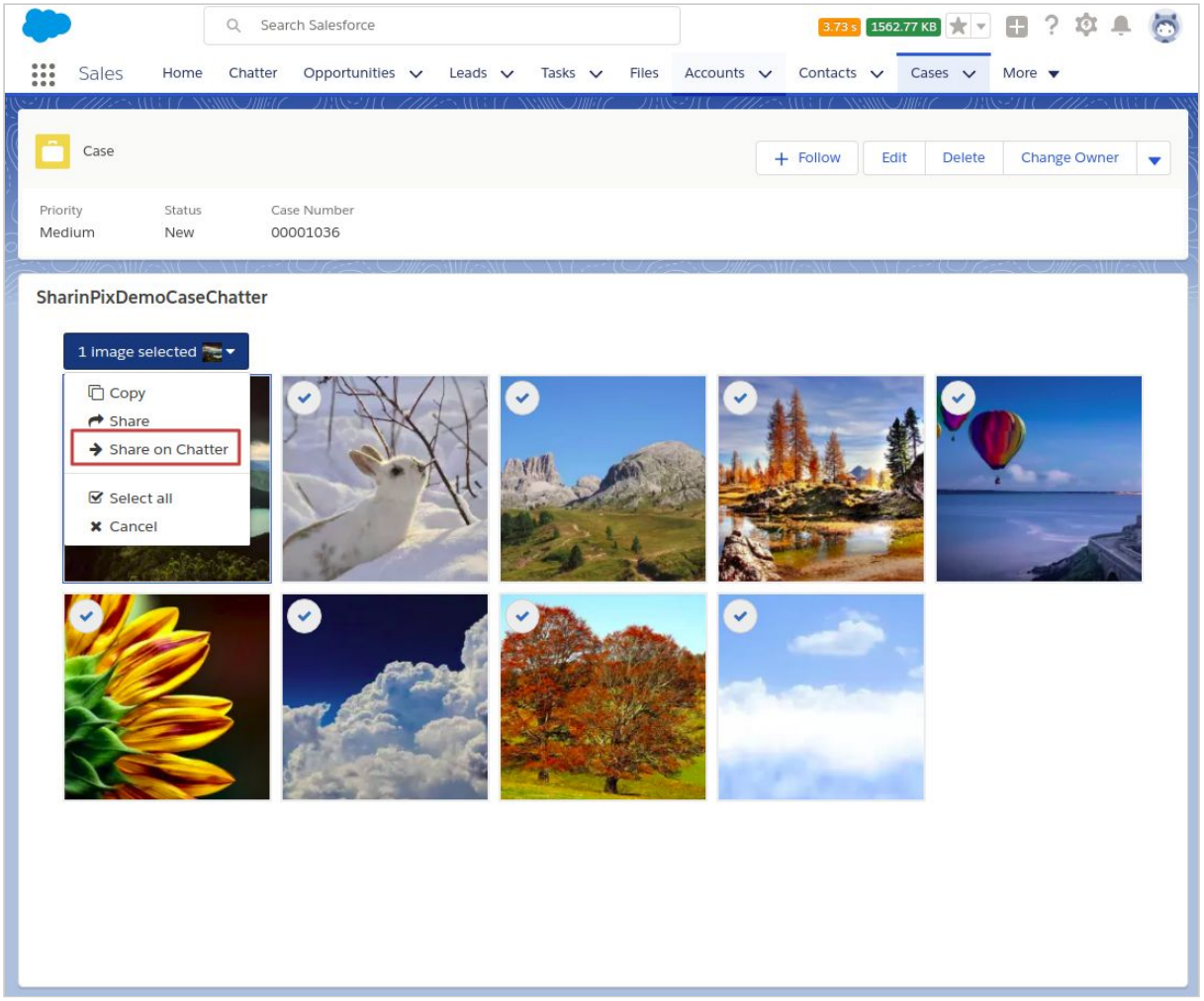


2. Add images

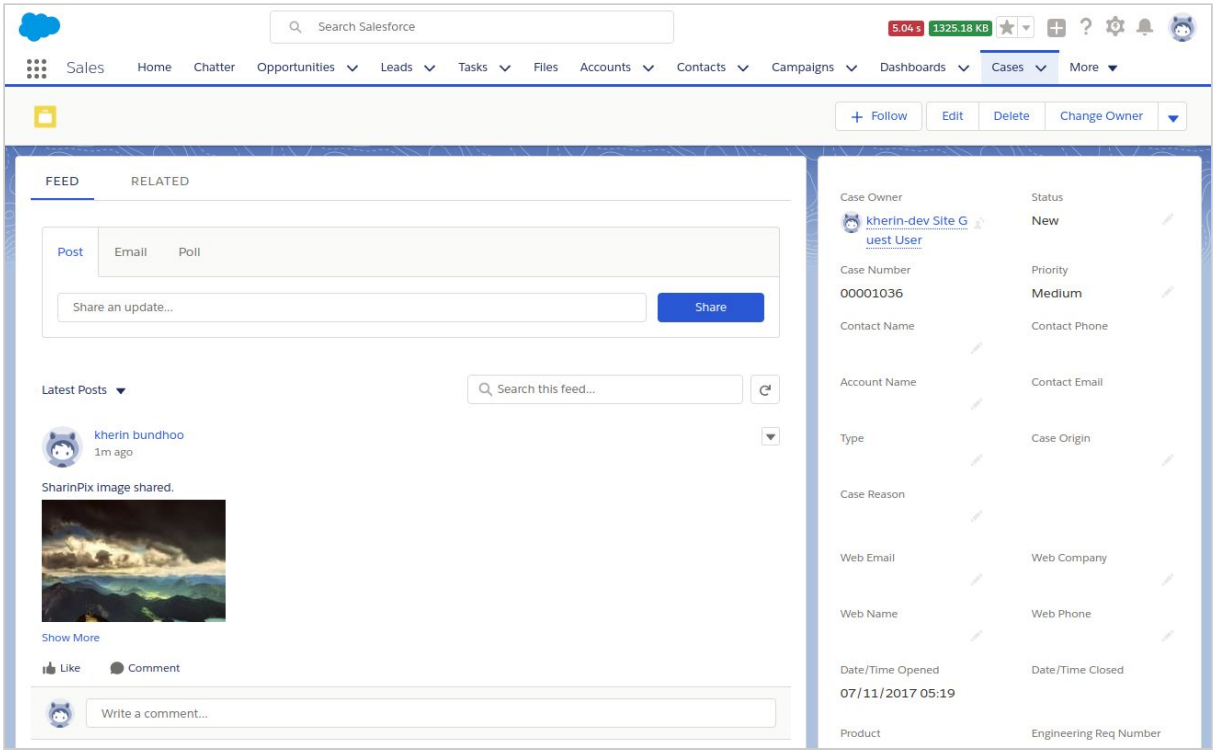


- To enable the **Share on Chatter** action, follow this link: here

3. Select ‘**Share on Chatter**’



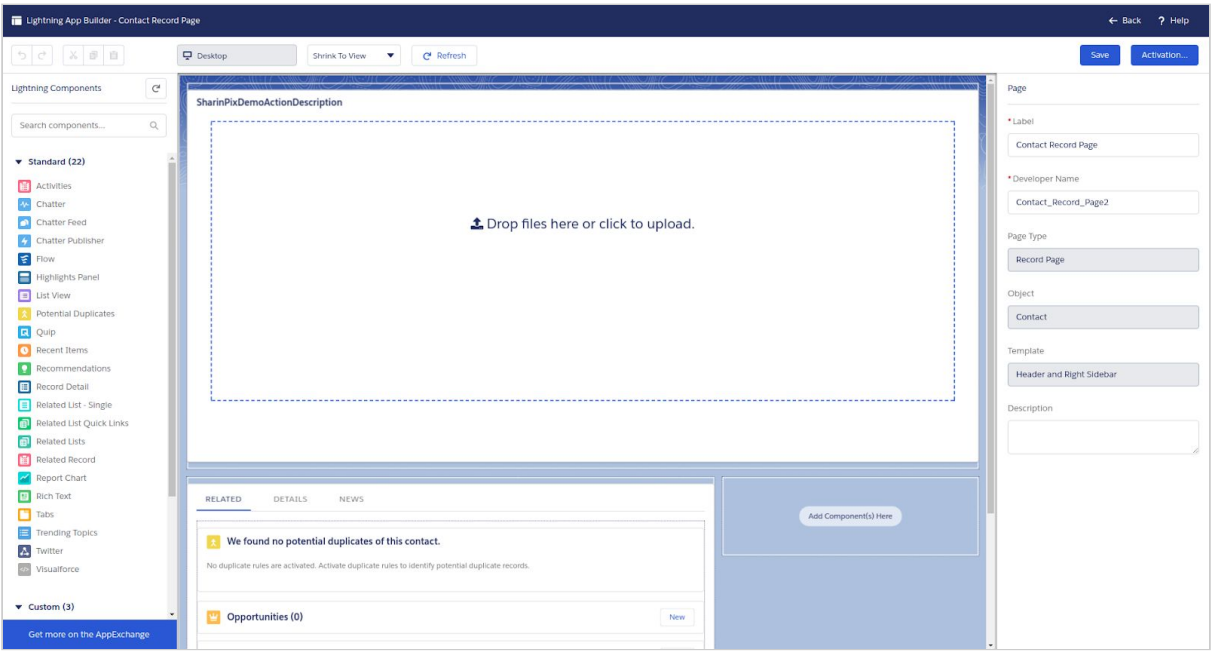
4. Image is shared on Chatter



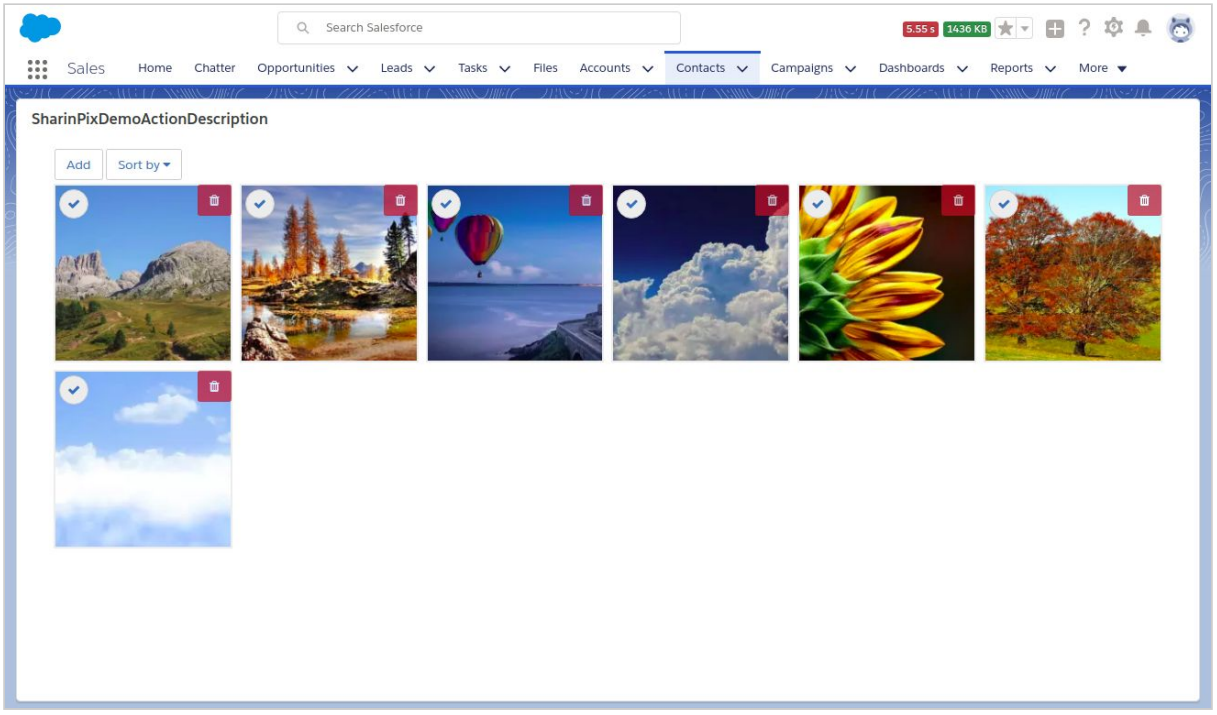
Use images inside Salesforce fields

Find out how to update fields on Salesforce directly from a SharinPix album here:
https://github.com/SharinPix/demo-apex/tree/action_description
(Learn how to add the Visualforce Page: here)

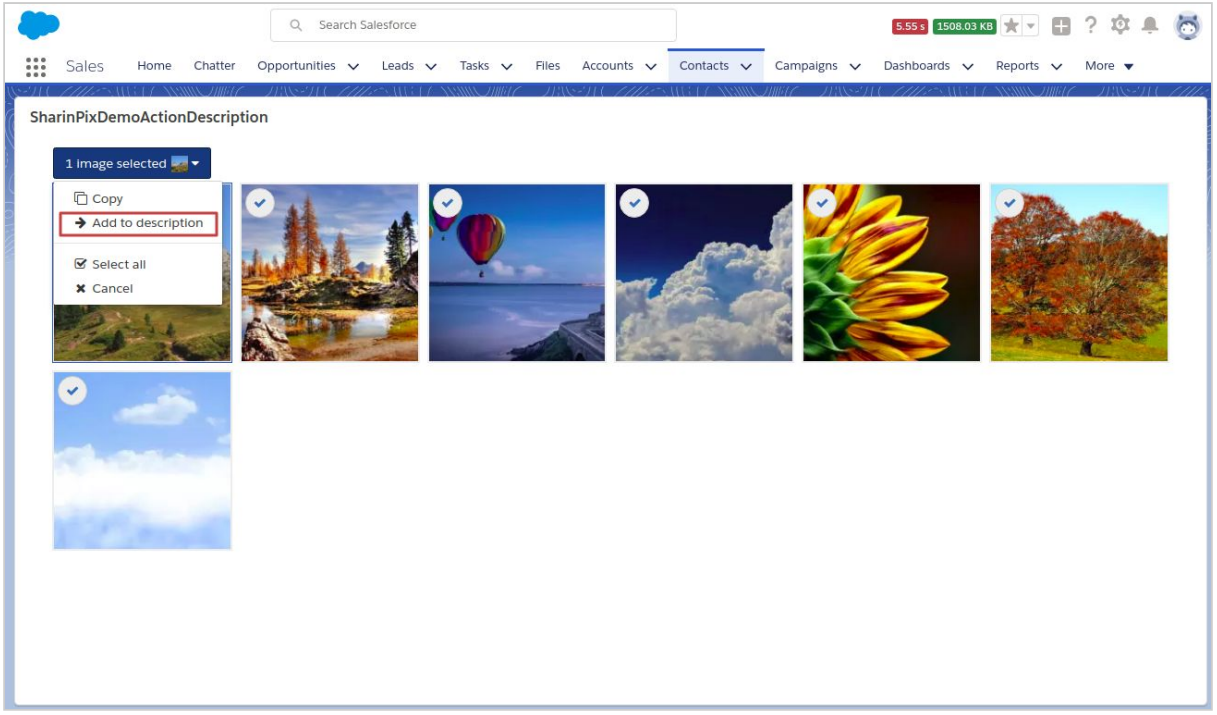
1. Add Visualforce page to page-layout of Contact object



2. Add images



3. Select '**Add to description**’



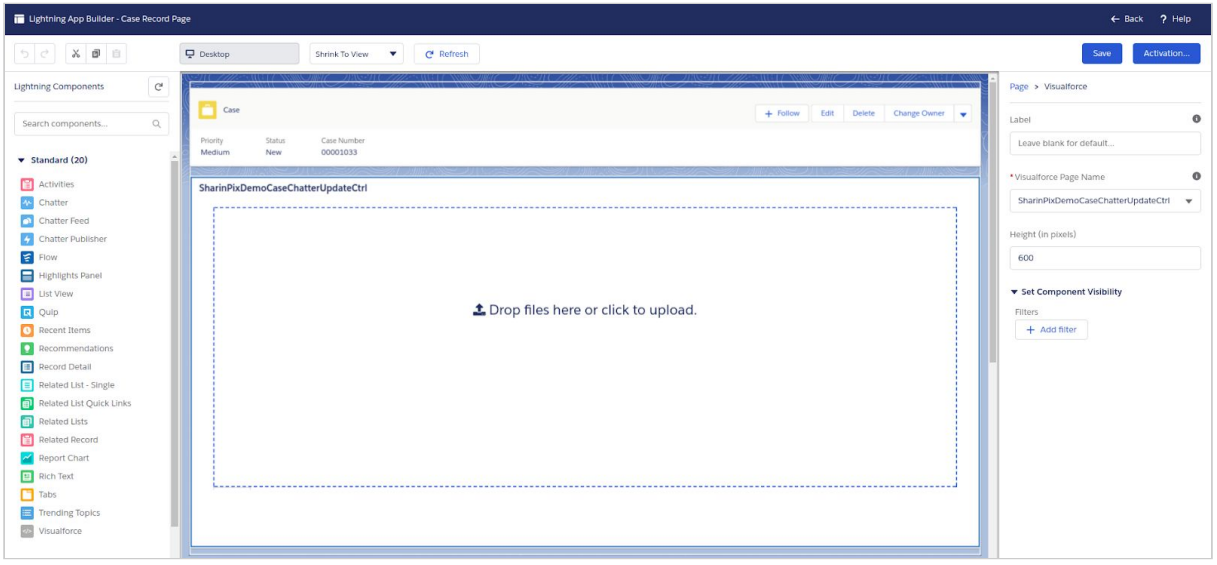
- To enable the **Add to description** action, follow this link: here

4. Image URL is added to the **Description** field

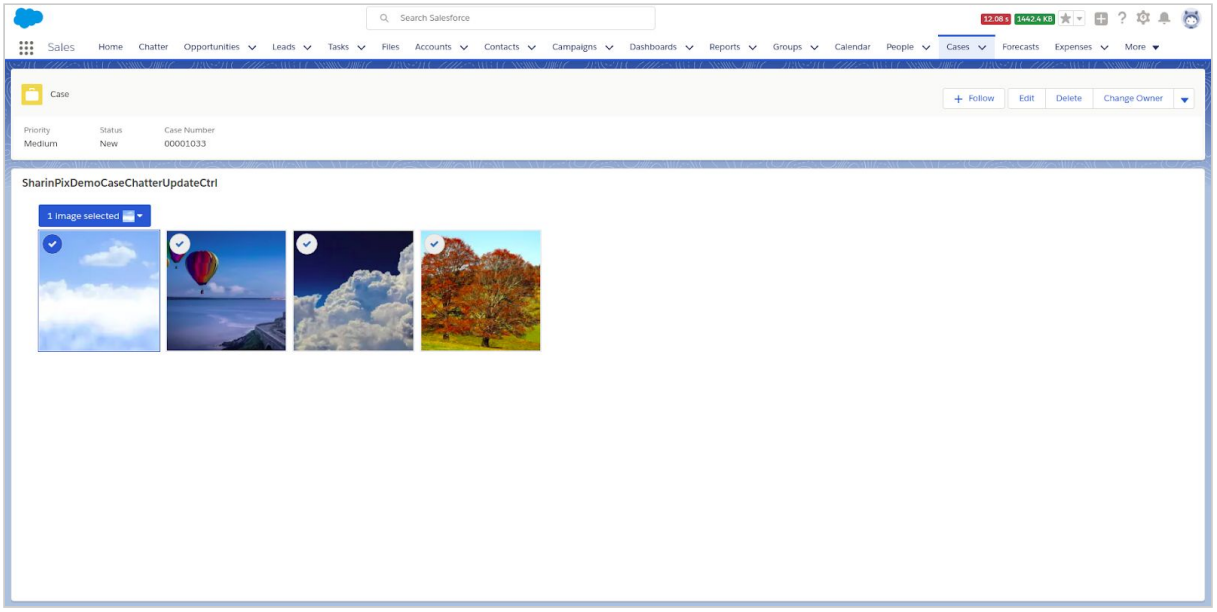
Post in Salesforce Chatter on SharinPix new image upload

Find out how to set up SharinPix to automatically share an update on Chatter whenever an image is added or tagged here: https://github.com/SharinPix/demo-apex/tree/chatter_update

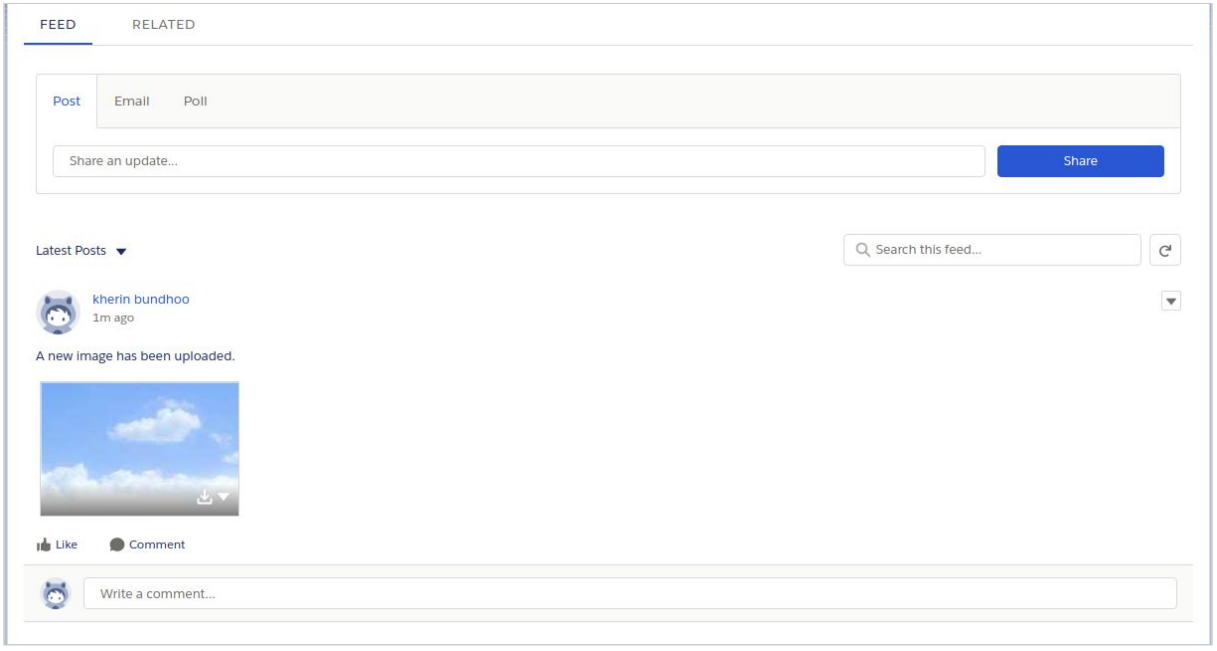
1. Add the Visualforce page to the page-layout of Cases



2. Upload an image



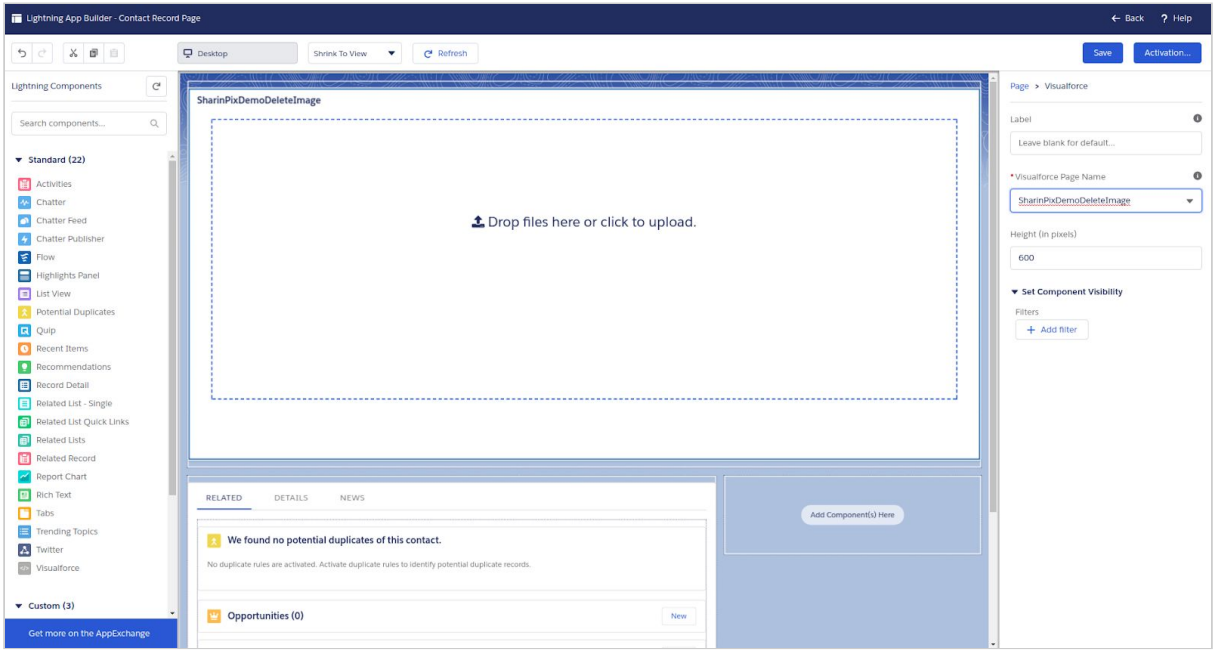
Once you refresh your current browser page, a post about the newly-uploaded image should appear inside the chatter feed of the current record.



Delete SharinPix images in bulk

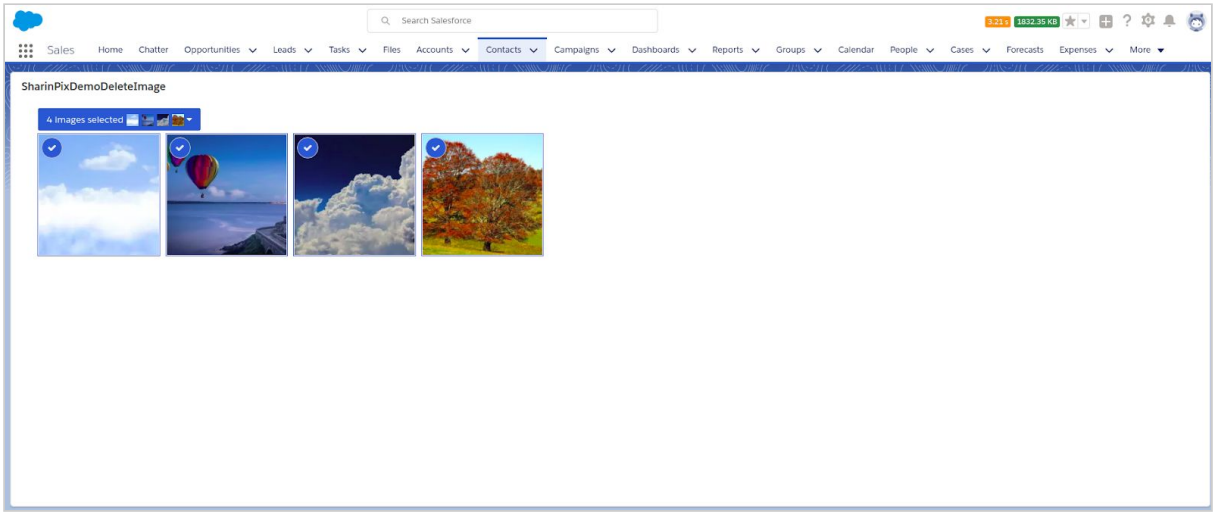
Find out how to enable deletion of images in bulk here:
https://github.com/SharinPix/demo-apex/tree/action_delete

1. Add Visualforce page to page-layout of Contact object

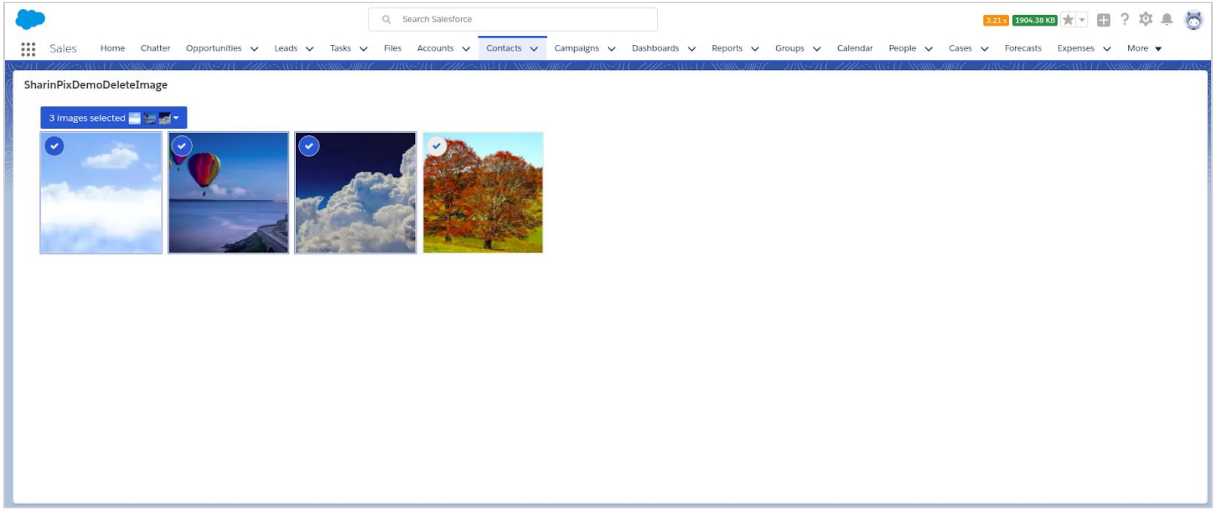


- To enable the **Delete images** action, follow this link: here
- When the **Delete images** option is selected, a SharinPix event named **Delete images** is triggered and captured via Javascript. To find out how to capture more events follow this link: here

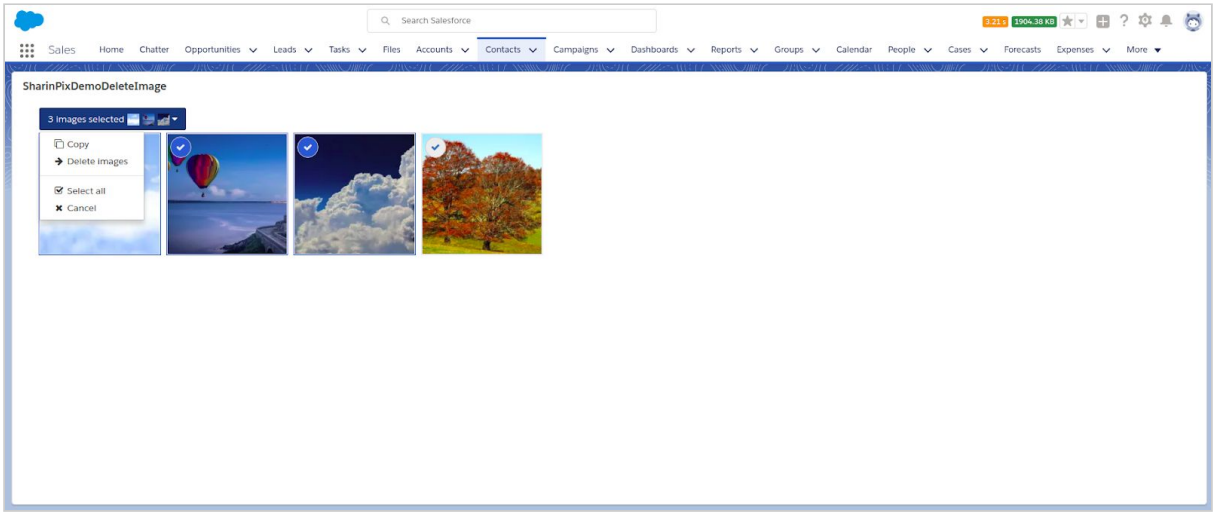
2. Upload images



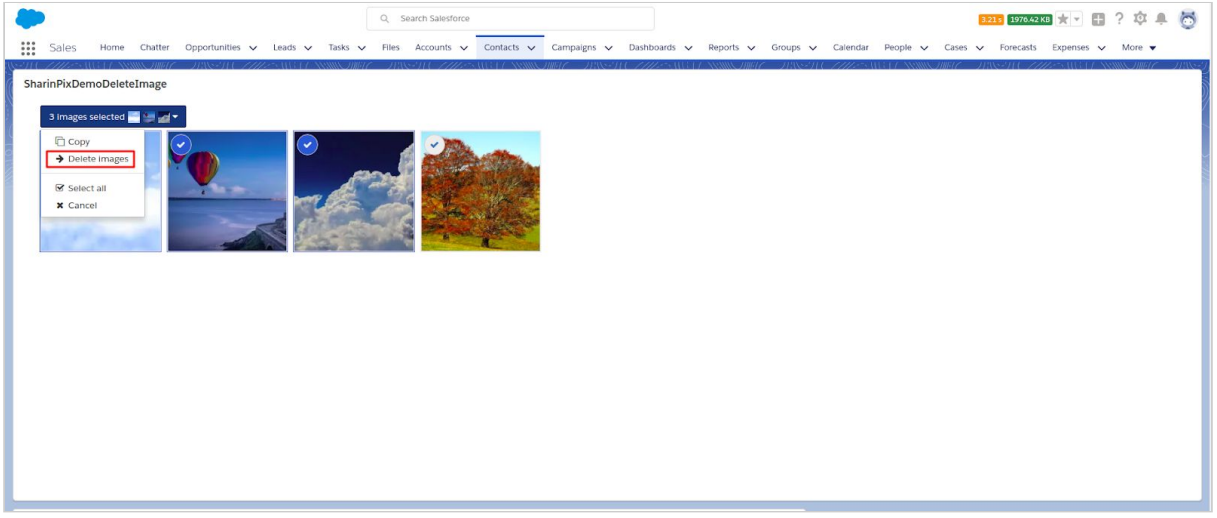
3. Select images to delete



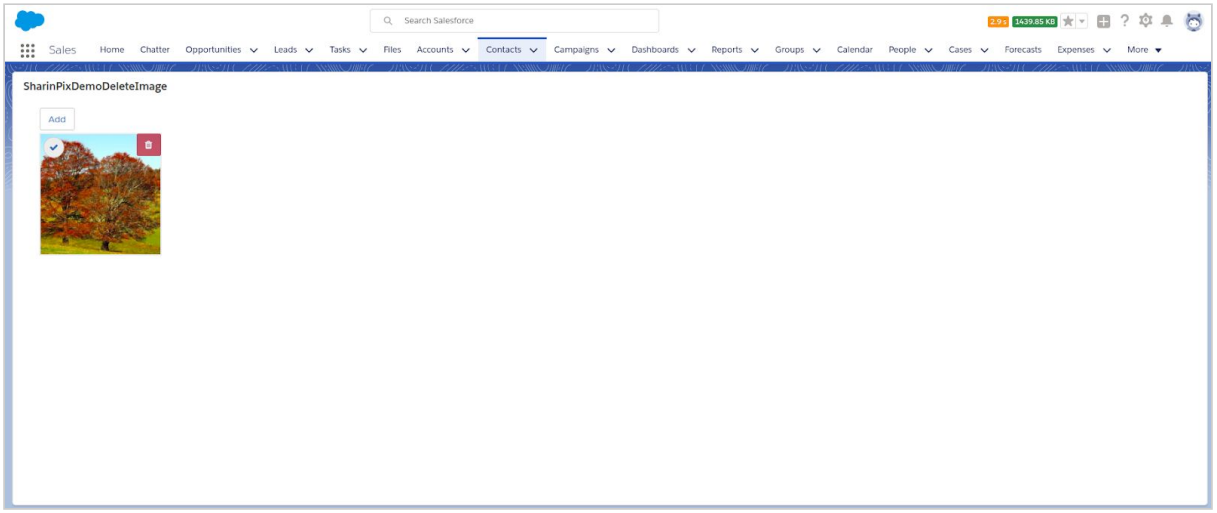
4. Click on the dropdown menu



5. Select ‘**Delete images**’



6. Reload the current page

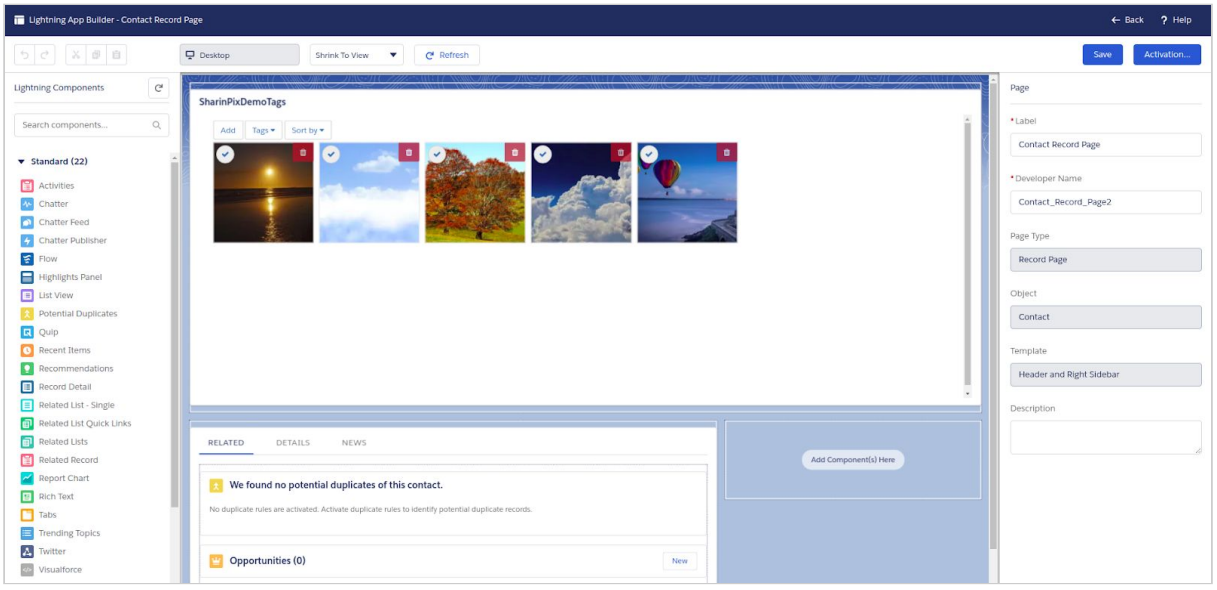


Add tags to your images

Find out how to set up tags to be displayed on an album here:
https://github.com/SharinPix/demo-apex/tree/tags

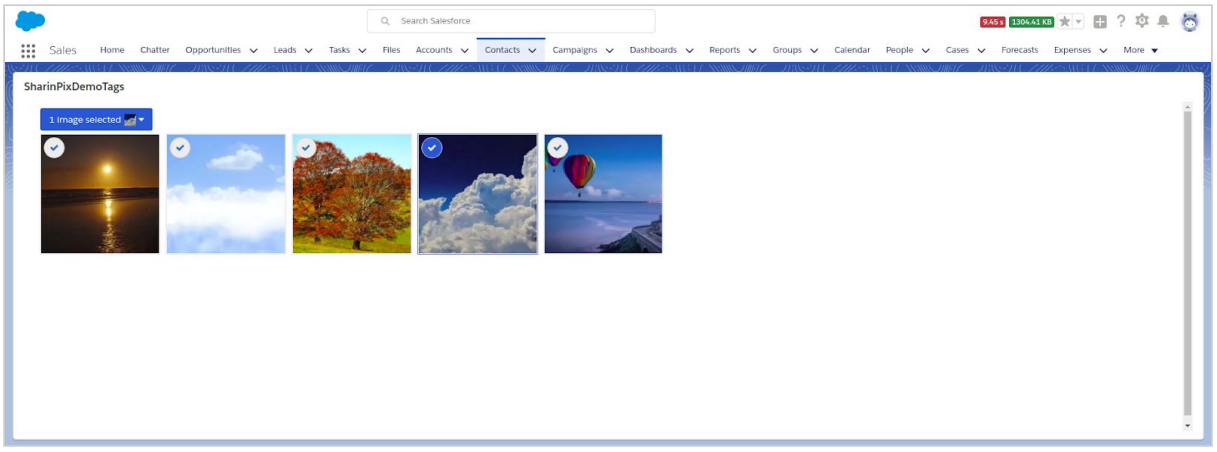
(Learn how to add the Visualforce Page: here)

1. Add the Visualforce page to the page-layout of the Contact Object

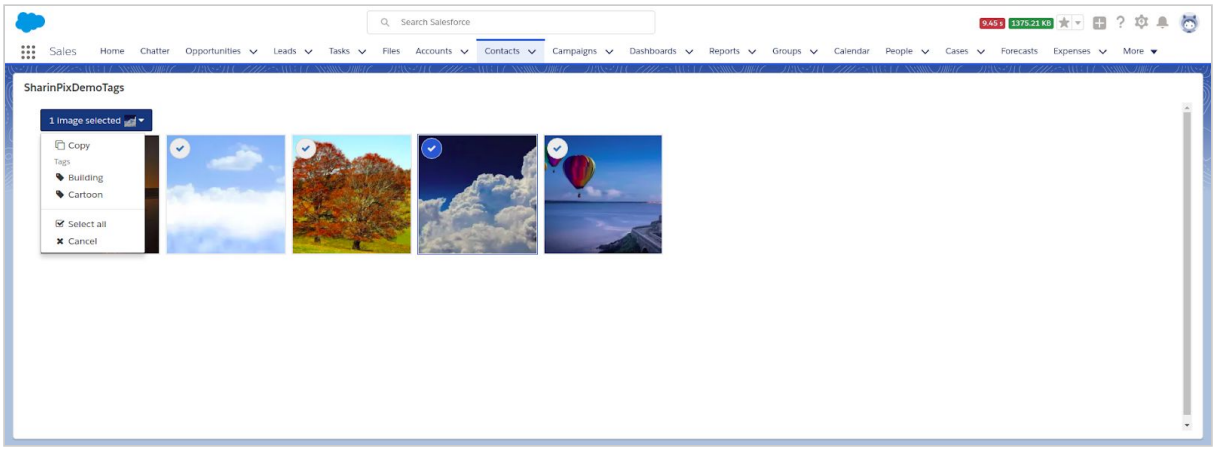


2. Tag the image(s)

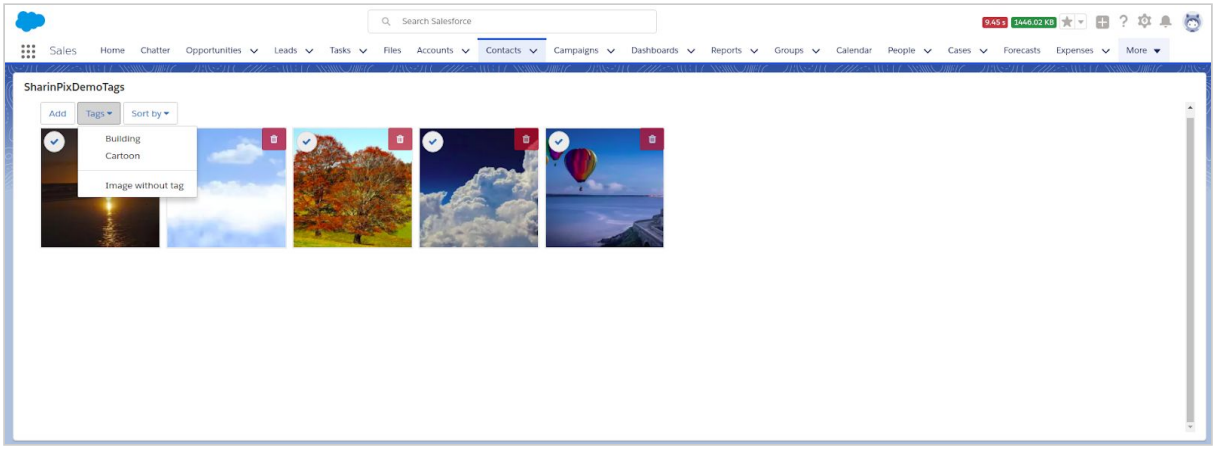
Select the image you want to tag.



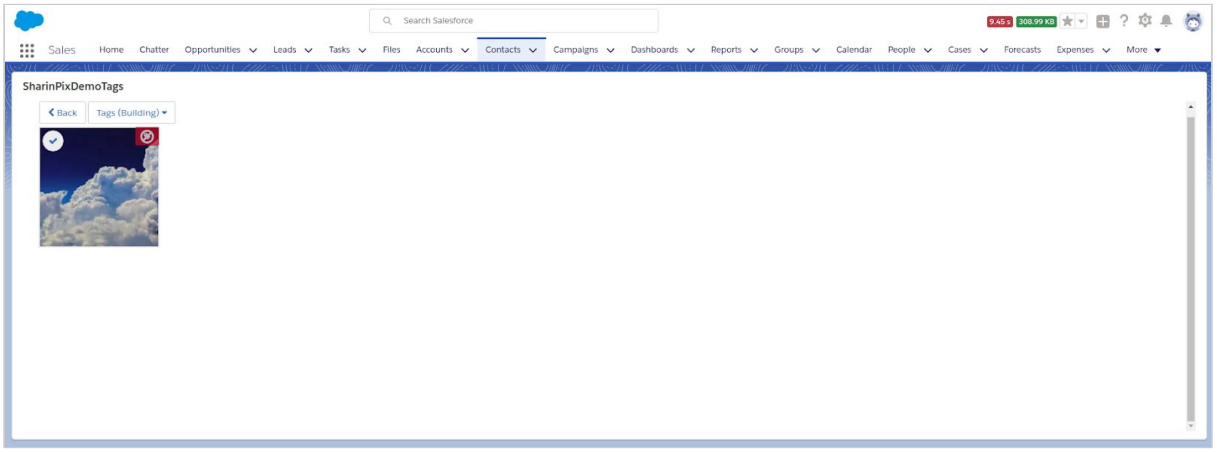
Click on the dropdown menu. Select a tag (in this case: ‘building’).



To only view images with the ‘building’ tag, select ‘Tags’.



Then, only image(s) with the ‘building’ tag are displayed.



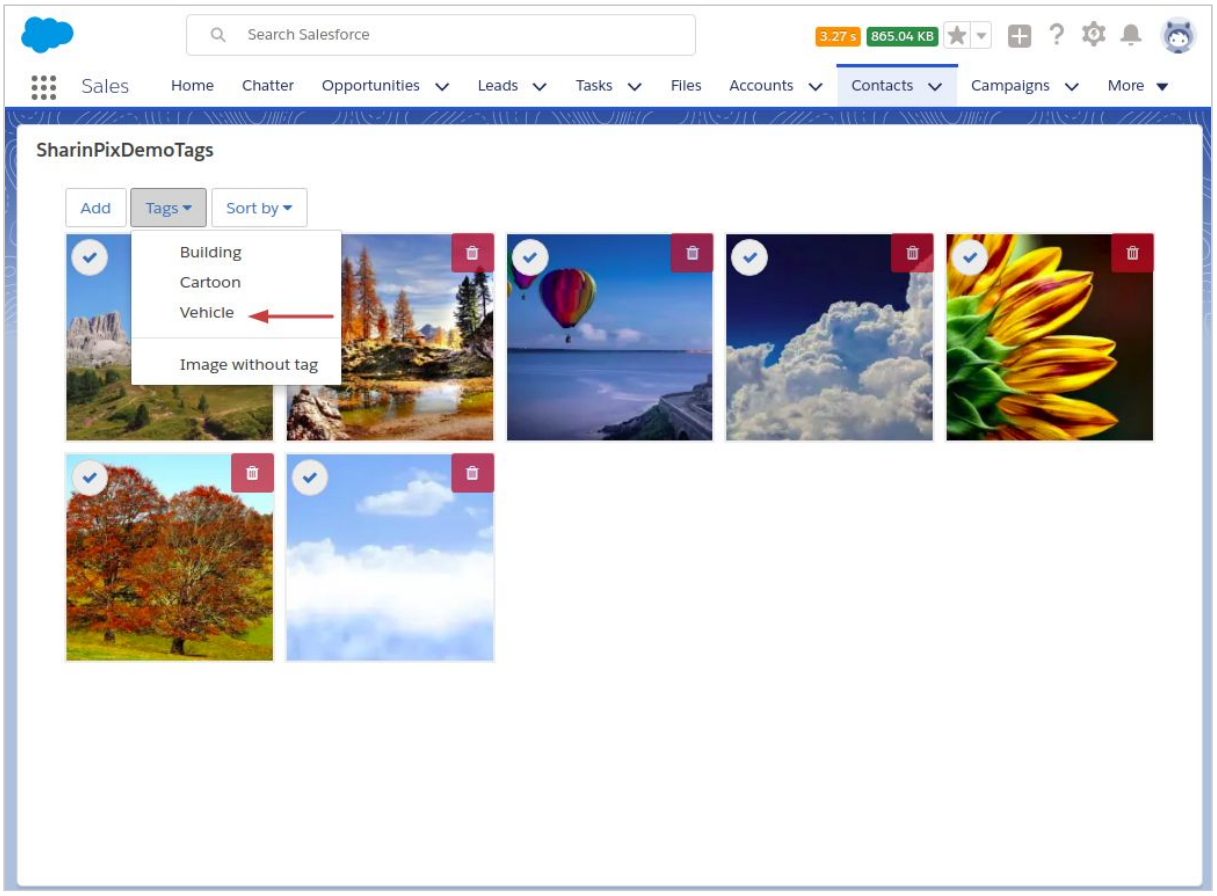
3. Create tags programmatically inside the apex class SharinPixDemoTagsCtrl.cls

```
'Tags' => new Map<String, Object> {
    'Cartoon' => new Map<String, Object> {
        'en' => 'Cartoon',
        'fr' => 'Déssin Animés'
    },
    'Building' => new Map<String, Object> {
        'en' => 'Building',
        'fr' => 'Bâtiment'
    }
}
```

- Add new tag inside code.

```
'Tags' => new Map<String, Object> {
    'Cartoon' => new Map<String, Object> {
        'en' => 'Cartoon',
        'fr' => 'Déssin Animés'
    },
    'Building' => new Map<String, Object> {
        'en' => 'Building',
        'fr' => 'Bâtiment'
    },
    'Vehicle' => new Map<String, Object>{
        'en' => 'Vehicle',
        'fr' => 'Véhicule'
    }
}
```

Refresh Visualforce page.



Update Salesforce with SharinPix image tags

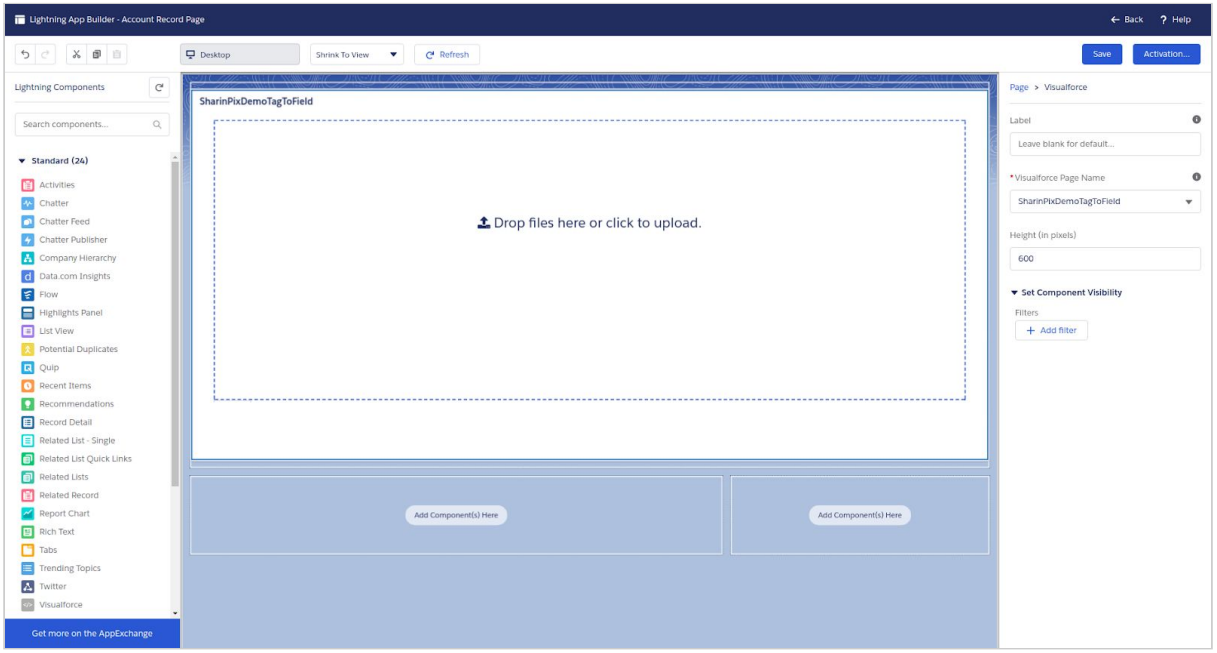
Find out how to update fields on Salesforce based on images being tagged here: https://github.com/SharinPix/demo-apex/tree/tag_to_field

Demo Description

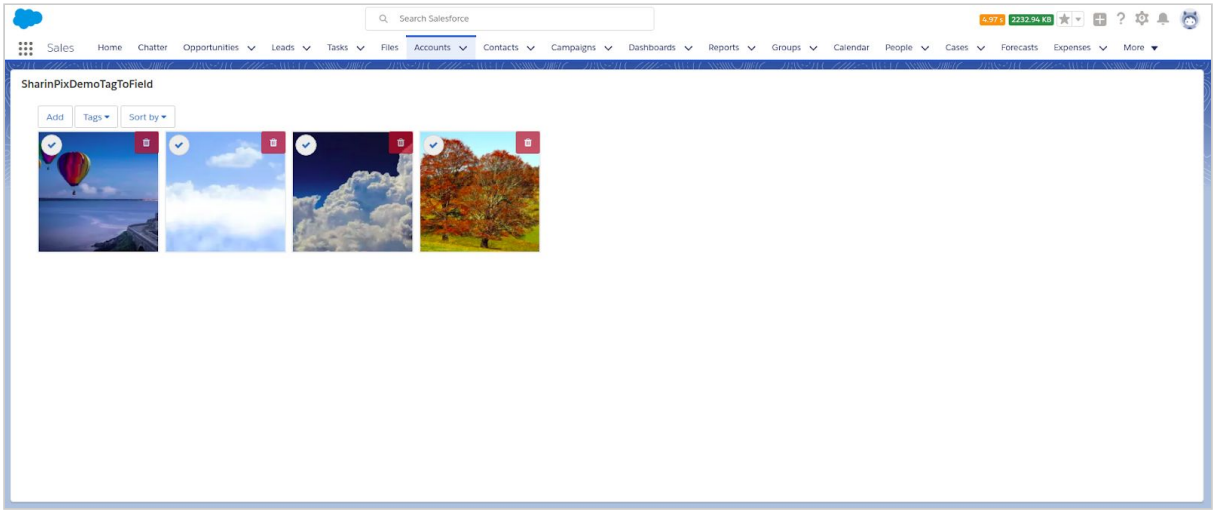
You can set up an automatic update on a Salesforce field based on tag(s) added to image(s). This demo, once deployed and added to the Account page-layout, will update the Description field with an image URL when an image is tagged with **tag_name**. Other data can also be retrieved and sent to Salesforce; these are available in the **e.data.payload** object.

(Learn how to add the Visualforce Page: here)

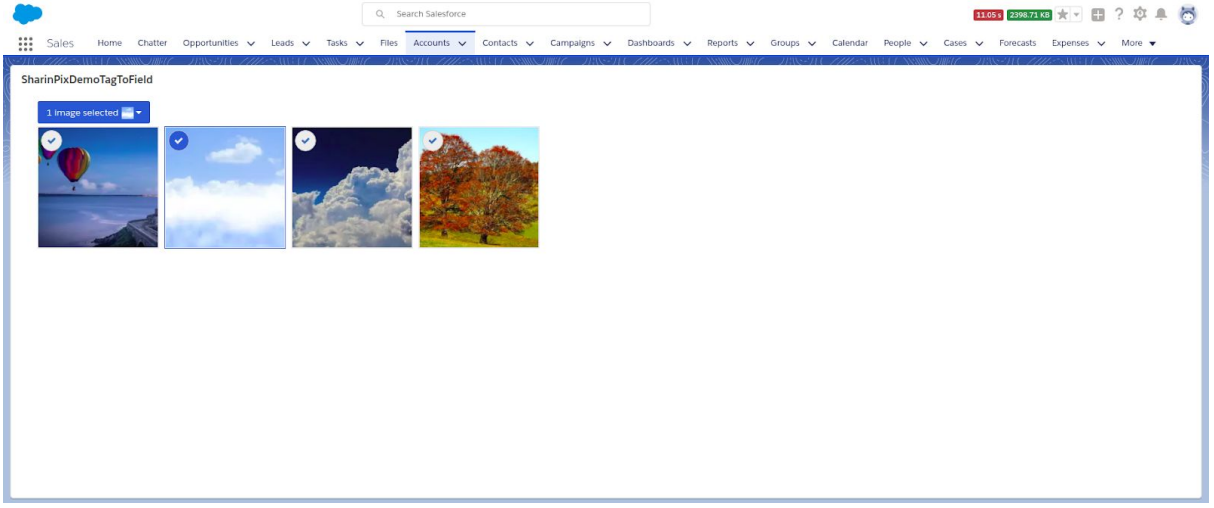
1. Add Visualforce page to Account page-layout



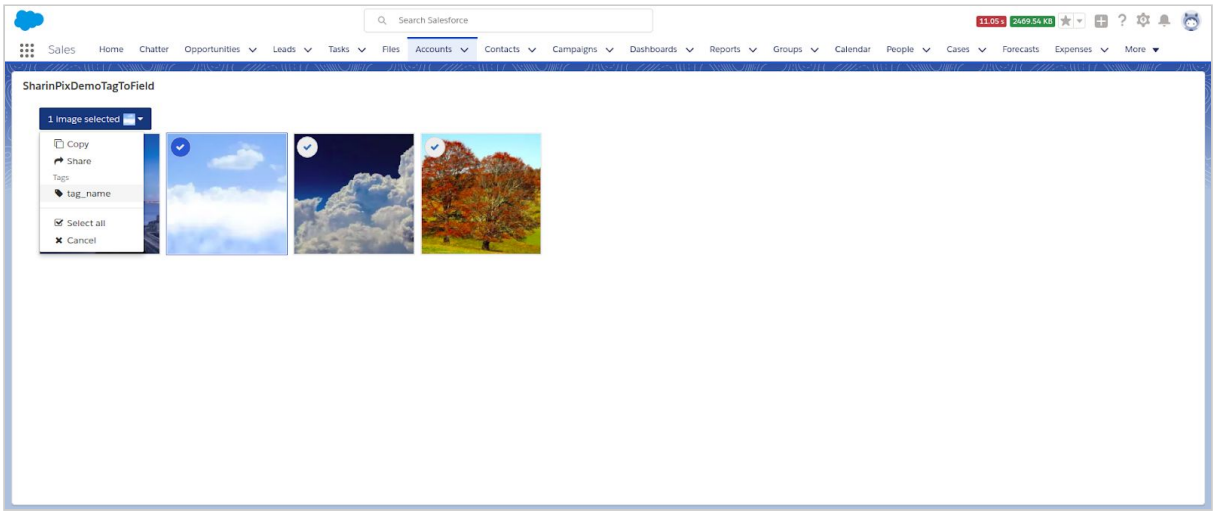
2. Add image(s)



3. Select image(s)



4. Select ‘**tag_name**’

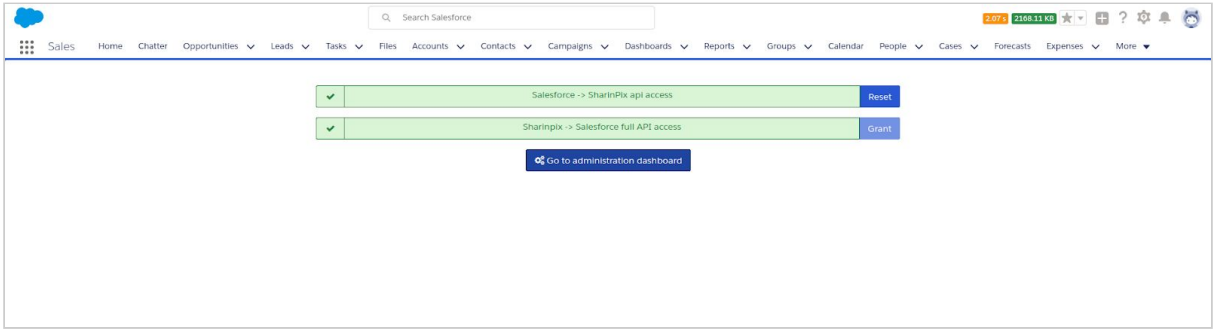


When you refresh your browser, in the **Description** field found in the record details, you should see a link.

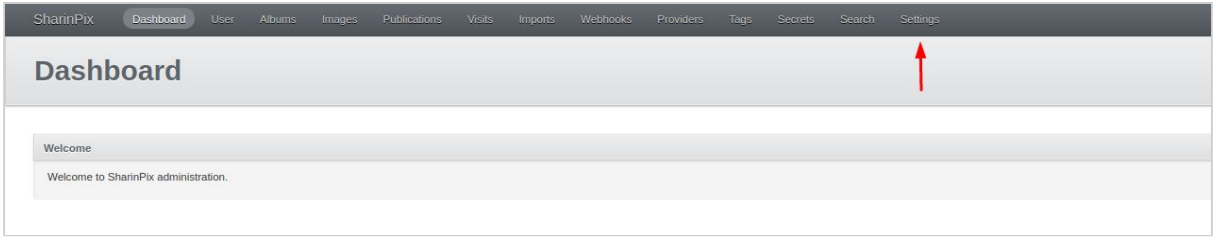
Adding tags to your uploaded images automatically

SharinPix allows you to add tags to your images as they are uploaded. These can be configured from the admin dashboard under organization settings.

To reach the administration dashboard, go to the SharinPix Settings tab on Salesforce and click on the “Go to administration dashboard” button.

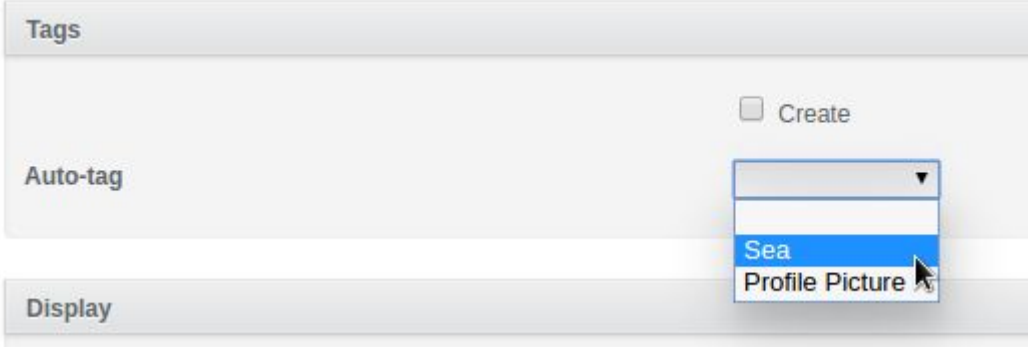


You will be taken to the SharinPix website; log in with your Salesforce account if you are not already. Head to the “Settings” page (select it from the top bar).



Click the “Edit Organization” button on the top-right.

Under the Tags section, for “Auto-tag” select the tag to be applied by default on all newly uploaded images. Please note that tags that have the “public” attribute set to true do not appear on this list as these are used for sharing images outside of Salesforce.



Finally, click on Update Organization to save your changes.

Using this, every single pictures uploaded in any album will be auto-tagged. If ever you need to add the Auto-tag feature in a single album (within a specific object as an example) you may have to use the Custom Permissions or a custom implementation as described in our demo at https://github.com/SharinPix/demo-apex/tree/auto-tag

Please note that if the auto_tag dropdown menu is empty, it may be because you have no tags available yet.

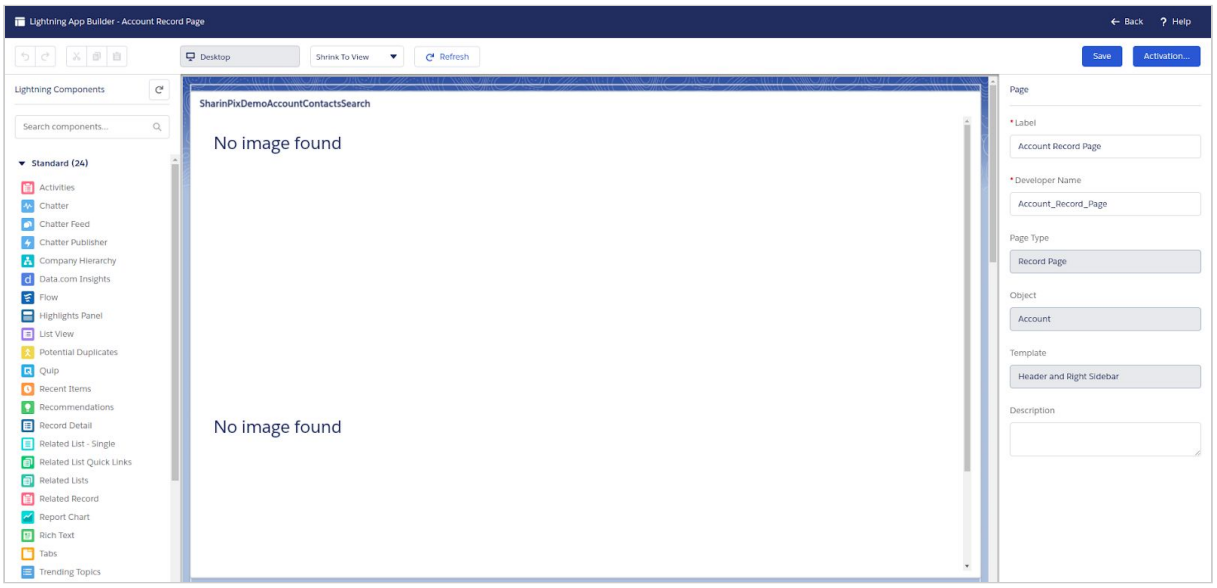
Display Contacts Images at Account level (Account Contacts Search)

Find out how you can create an album that shows all contacts’ pictures on their respective account here: https://github.com/SharinPix/demo-apex/tree/account_contacts_search
This is actually a search that is being performed and the demo’s ReadMe contains details on how you can customize this search for other uses.

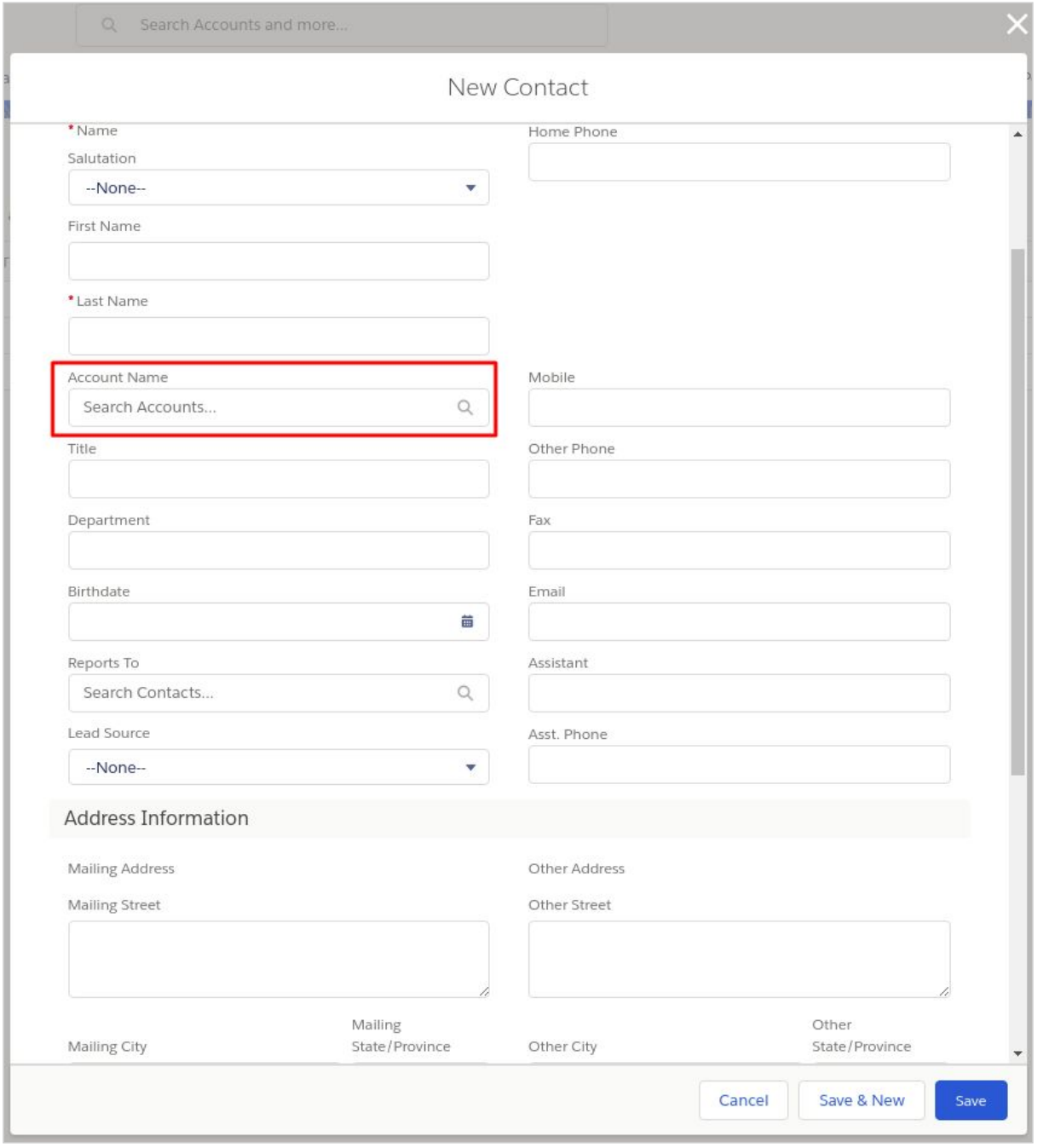
(Learn how to add the Visualforce Page: here)

1. Add Visualforce page to page-layout of Account object

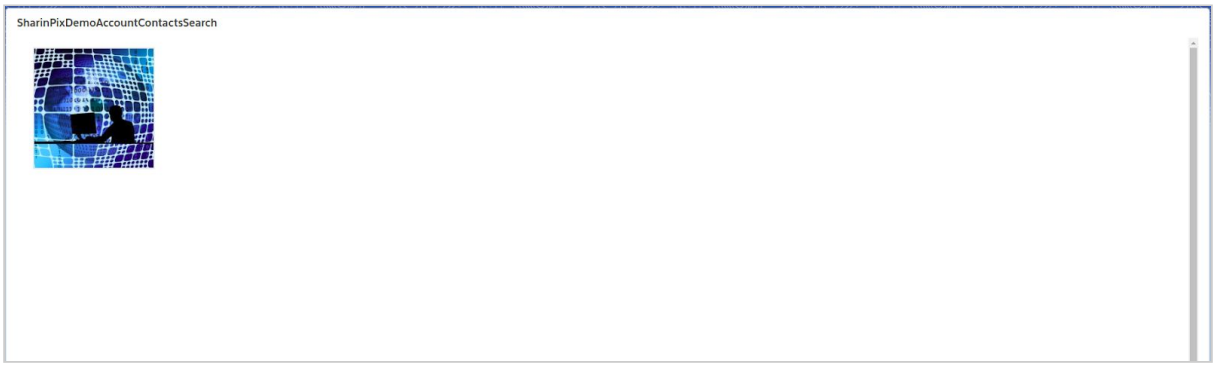
Add the Visualforce Page **SharinPixDemoAccountContactSearch** to the page-layout of the Account Object as shown in the figure below.



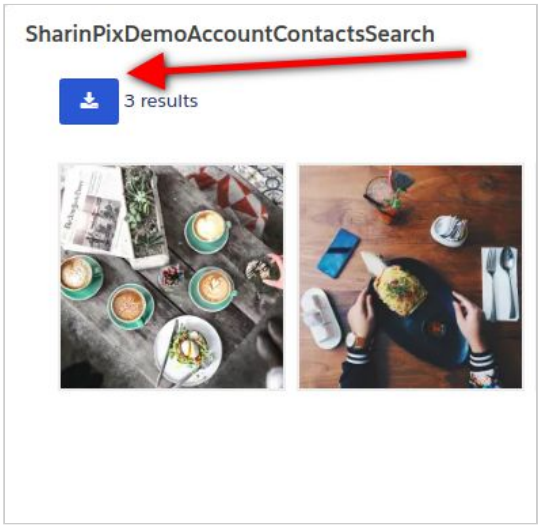
2. Create a Contact object and associate it with the relevant Account object



3. The images from the contact’s SharinPix album are displayed onto the Visualforce page found in the Account object as mentioned previously.



4. Enable the download and zipping of the search image results



It is possible to download the search image results into zipped folders through the use of the following parameters.

- **download** - (true or false) enable or disable the download feature.
- **download_filename** - specify the name of the downloaded image file (only 1 image)
- **download_filenames** - specify the name format of the downloaded image files (multiple images)

In order to change the values of these parameters, access the Apex Controller class **SharinPixDemoAccountContactsSearchCtrl**.

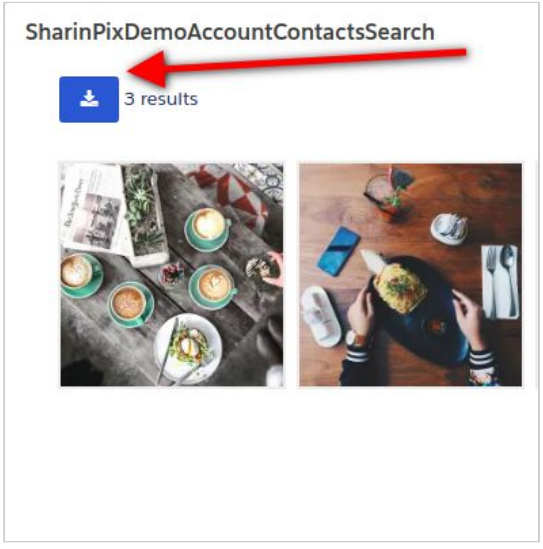
When assigning the value to the params variable, pass in values corresponding to the correct parameters as shown below

```
params = new Map<String, String> { 'path' => '/search?search_bar=false&q=' +
clientInstance.token(query)
+'&download=true&download_filename=my_zip_filename&download_filenames=inside_the_zip-00001'
};
```

Replace **my_zip_filename** and **inside_the_zip-00001** with the filename(s) of your choice.

5. Download and zipping in action

After following the steps as mentioned above, a download button will be visible on the Visualforce Page, just as shown in the figure below.



You will need to click upon the button **twice**, for the image download to start.Once the download process is initiated, the image file or files will be compressed into a zipped folder(with the **.zip** extension).

Webform with image upload

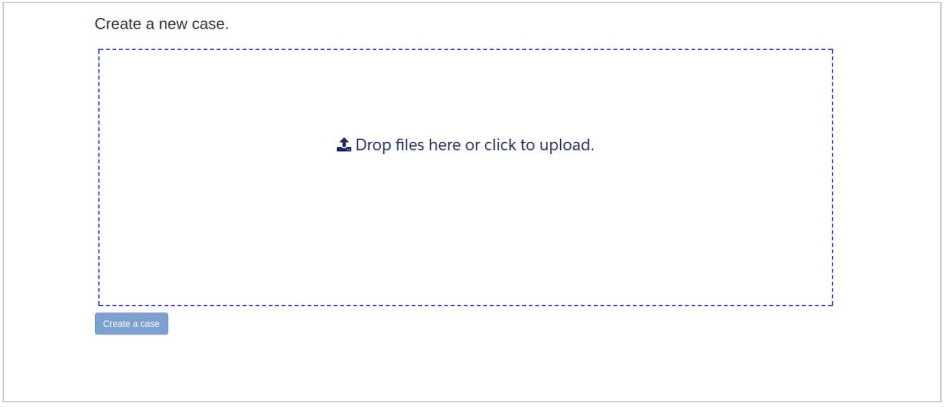
You can add albums on Visualforce sites that you create and then grant your guests public access to them. Find out how here:
https://github.com/SharinPix/demo-apex/tree/case_webform

1. Create new site

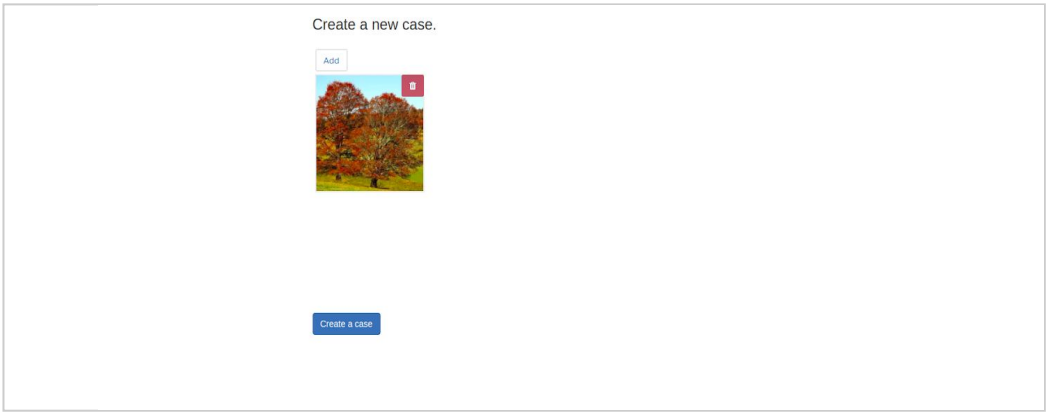
- Go to **Setup**.
- Type ‘**Sites**’ inside the Quick Find box.
- Create a new Site.
- Fill in the required fields with the relevant details.
- In the **Active Site Home Page**, select **SharinPixDemoWebform.**
- Click on **Save**.

2. Visit newly-created site

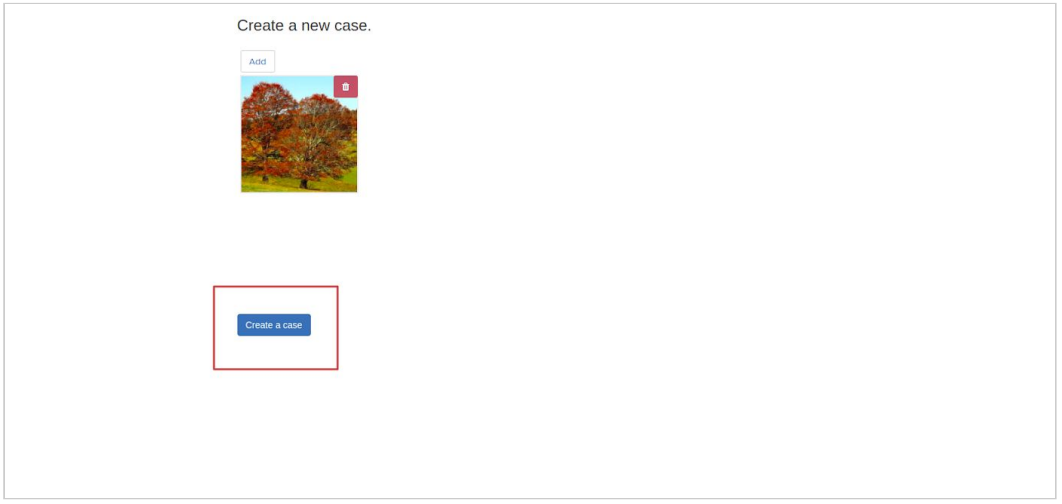
Type inside the address bar of your browser, the site url.



3. Upload image



4. Click on the ‘**Create a case**’ button



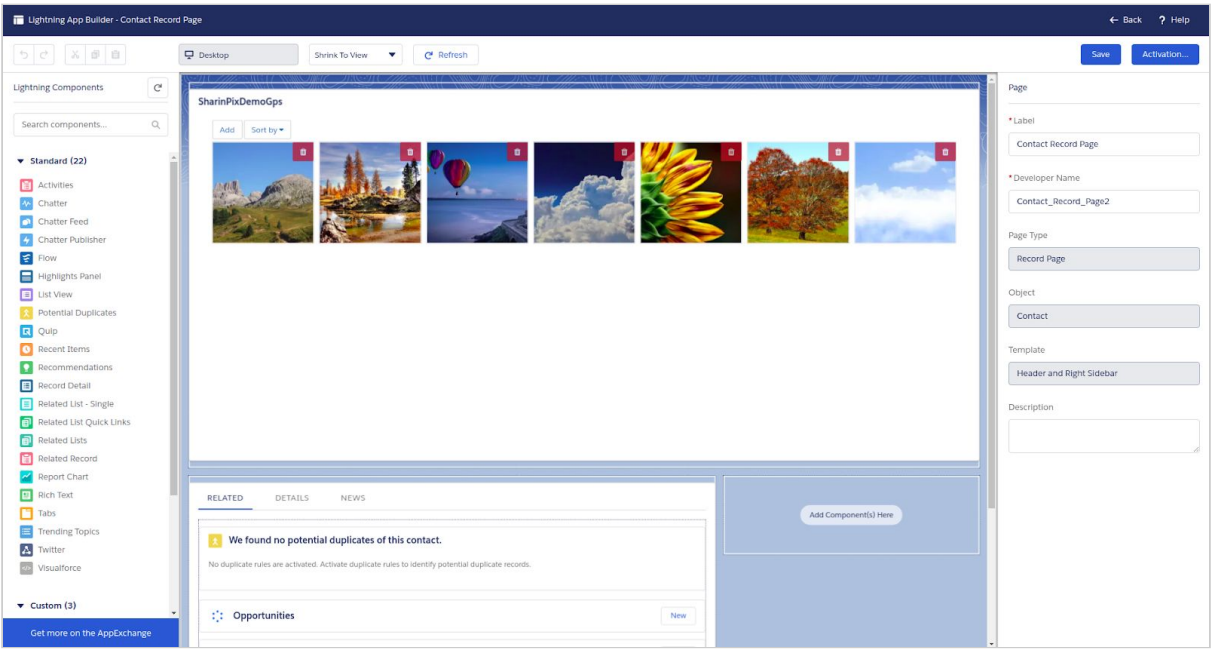
5. A case has been created along the uploaded image

GPS

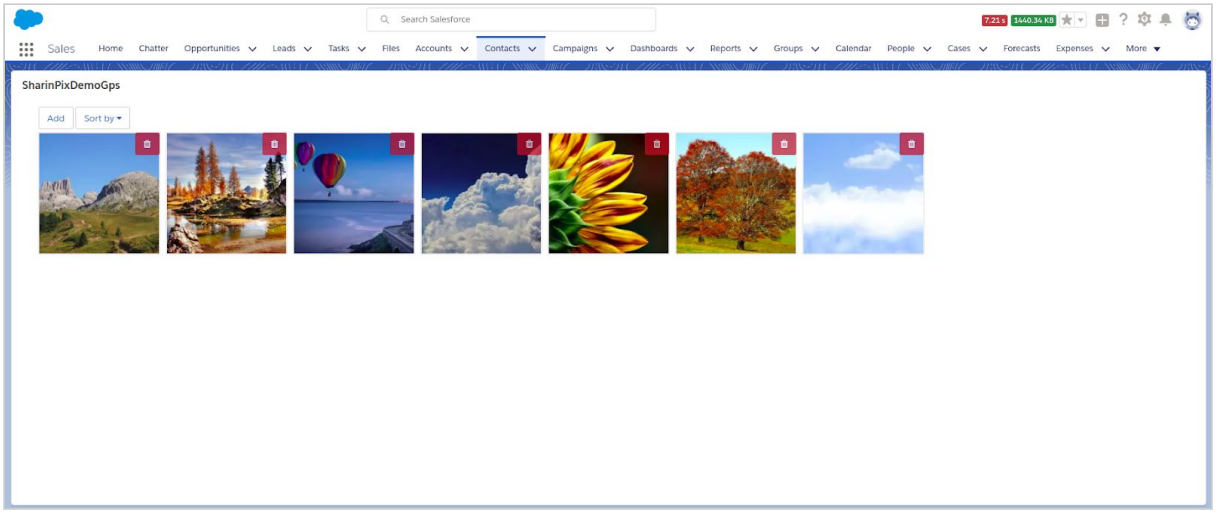
Find out how to extract GPS coordinates from each image that viewed here: https://github.com/SharinPix/demo-apex/tree/gps

(Learn how to add the Visualforce Page: here)

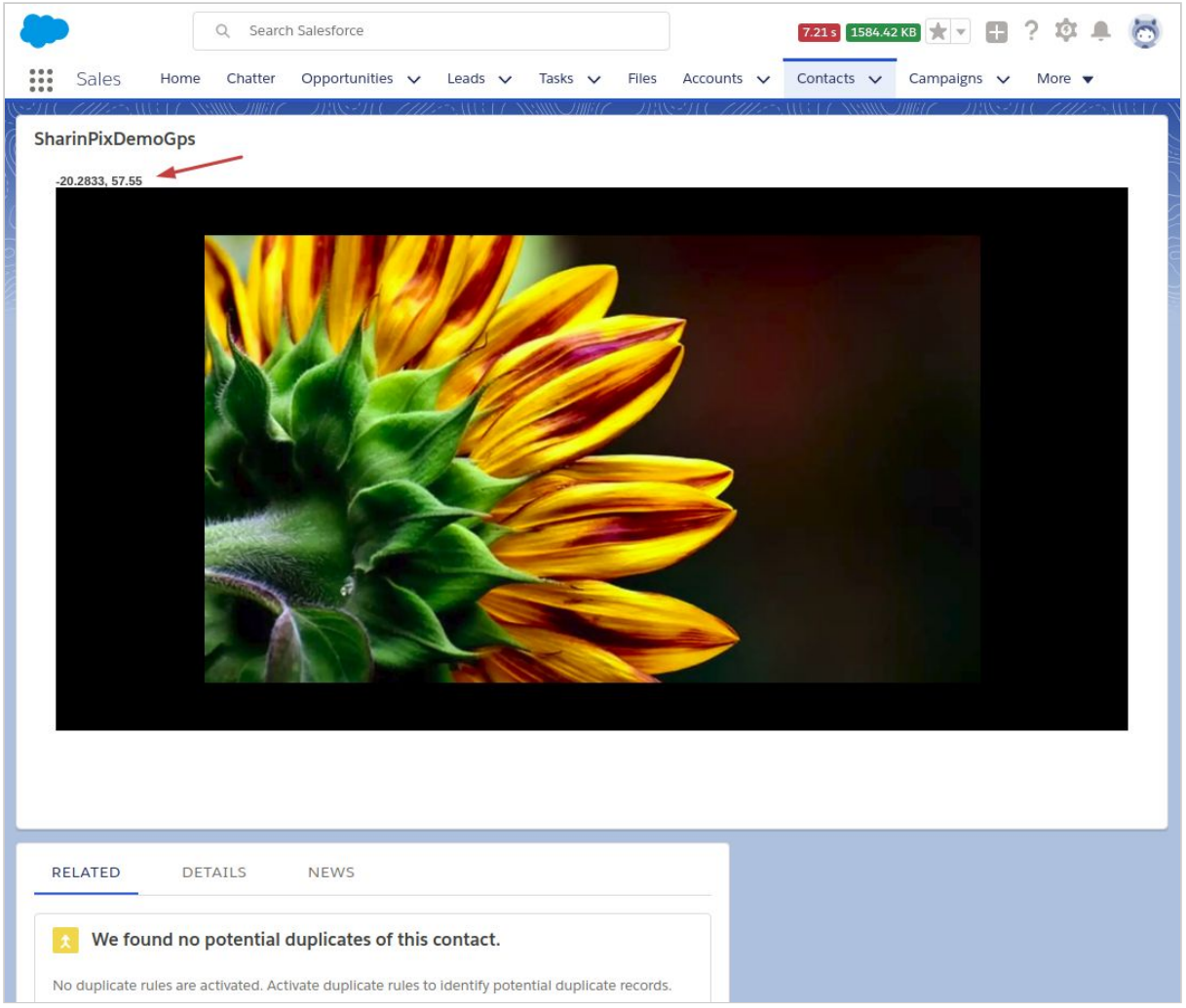
1. Add Visualforce page to page-layout of Contact



2. Click on image



3. GPS coordinates are displayed



Open SharinPix Album with Lightning Action

The present demo will illustrate how it is possible to launch a SharinPix Album that will be able to do the following:

- Provide access to an object-specific SharinPix Album along with its corresponding images.
- Can be used as a custom action in Lightning Experience and the Salesforce app.

Referencing the SharinPix Lightning Component

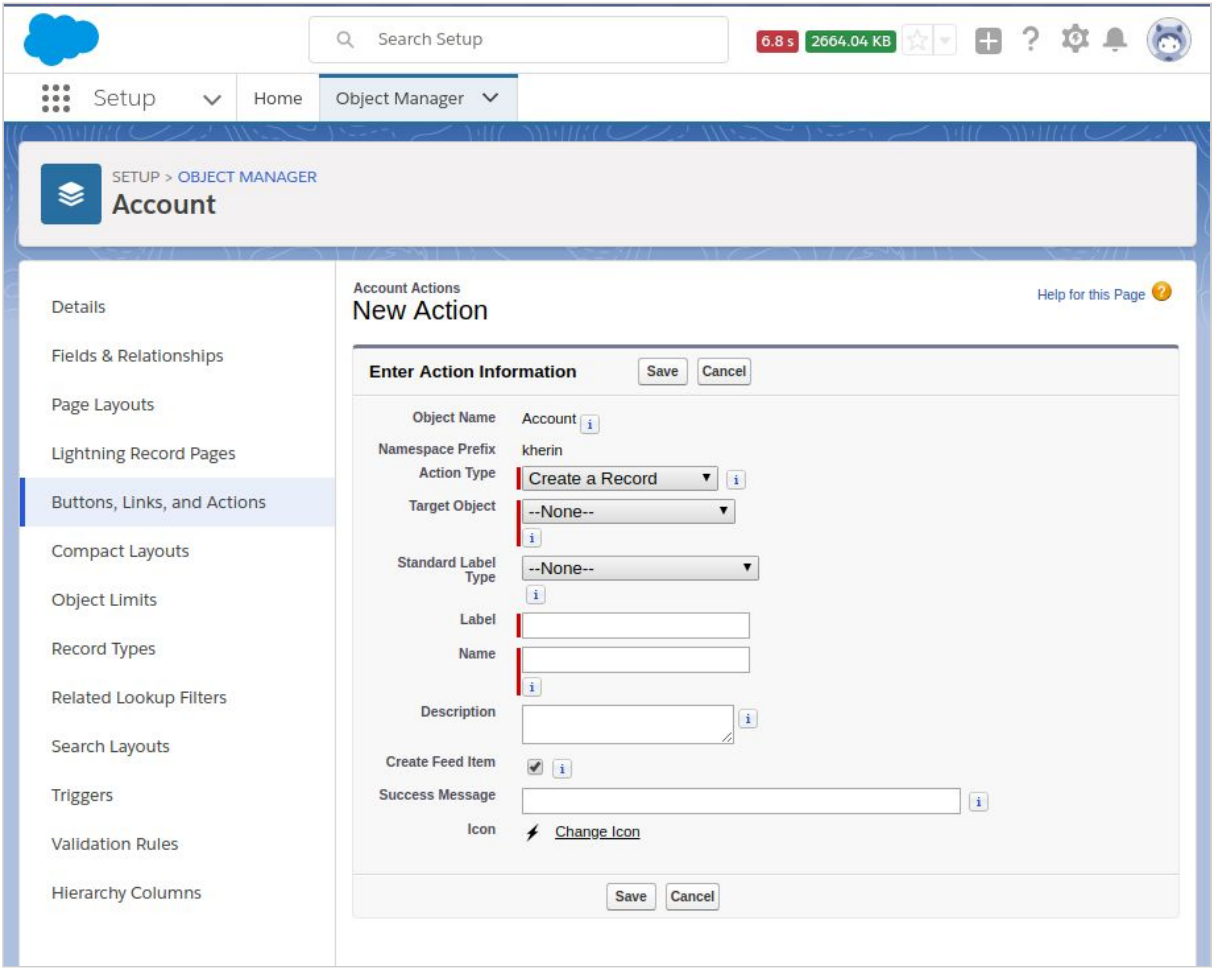
SharinPix Album in Lightning Experience

The content below will highlight the steps to use the context-aware Lightning component as a custom action within Lightning Experience.

Assumptions:

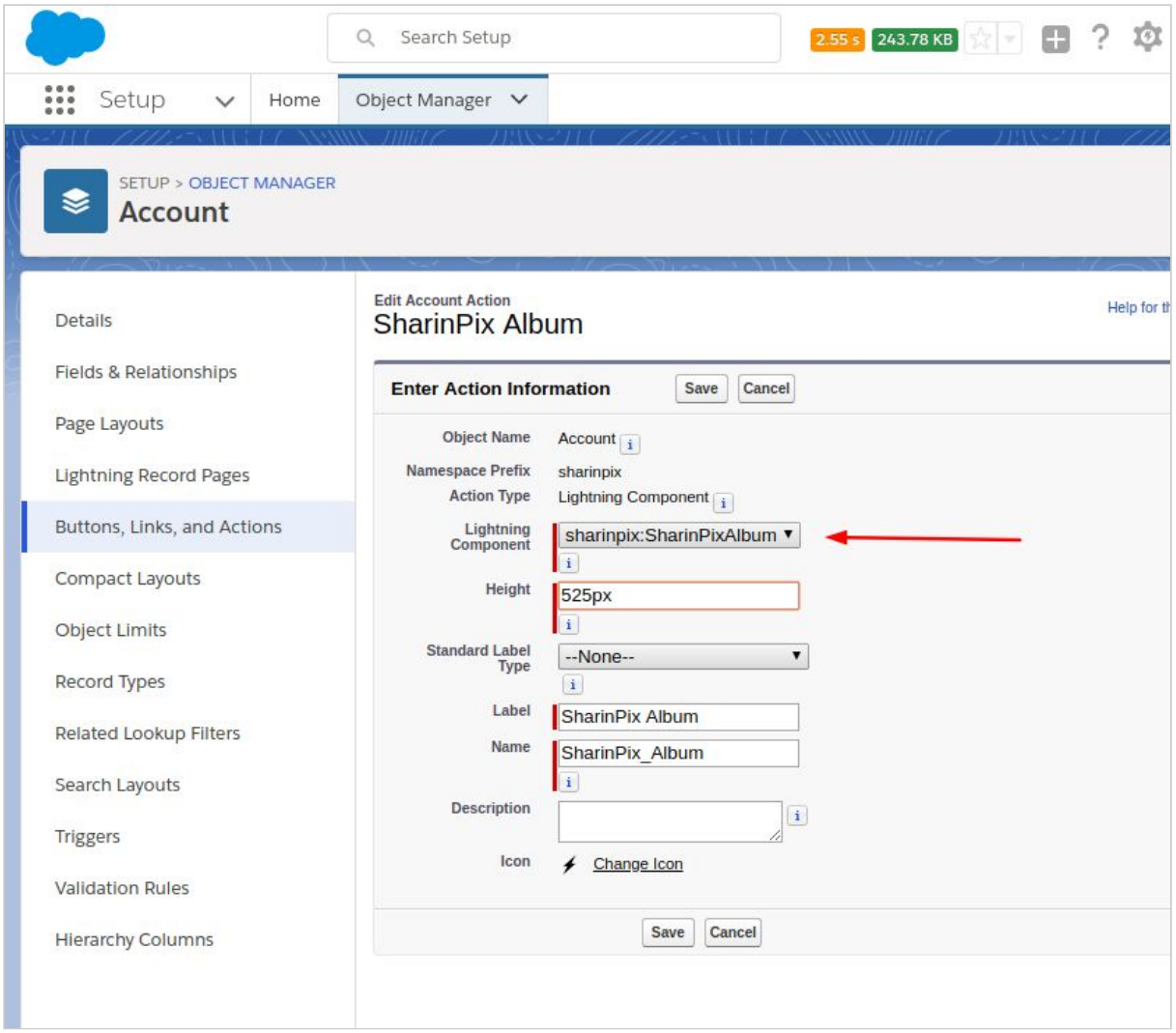
- The SharinPix Package is already installed on the current Salesforce Org.
- The user has access to the developer console.

1. Go to **Setup** -> **Object Manager.** Select the Object Type on which you intend to add the custom action. In the present case, it will be added on the Account Object.
2. On the left-hand-side of the screen, select the **Button, Links and Actions** item.



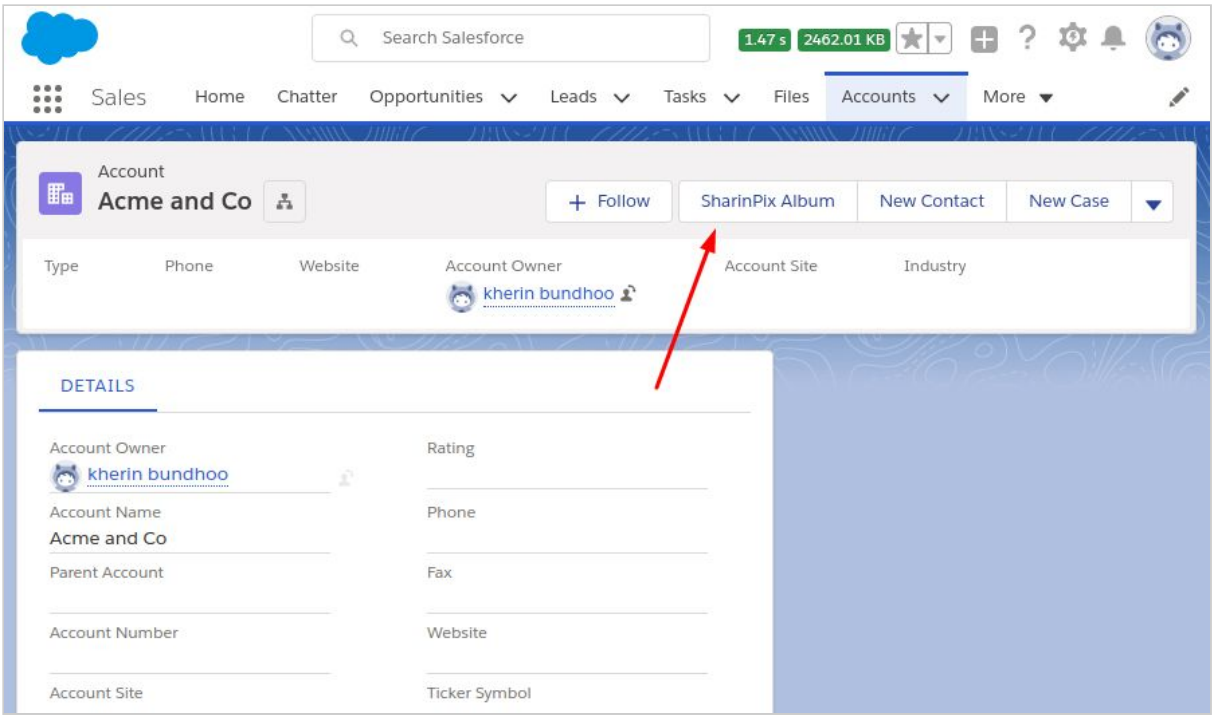
3. For the **Action Type** picklist, select **Lightning Component.**

4. For the **Lightning Component** field, select <sharinpix>**:SharinPixAlbum**. (as shown in the figure below).

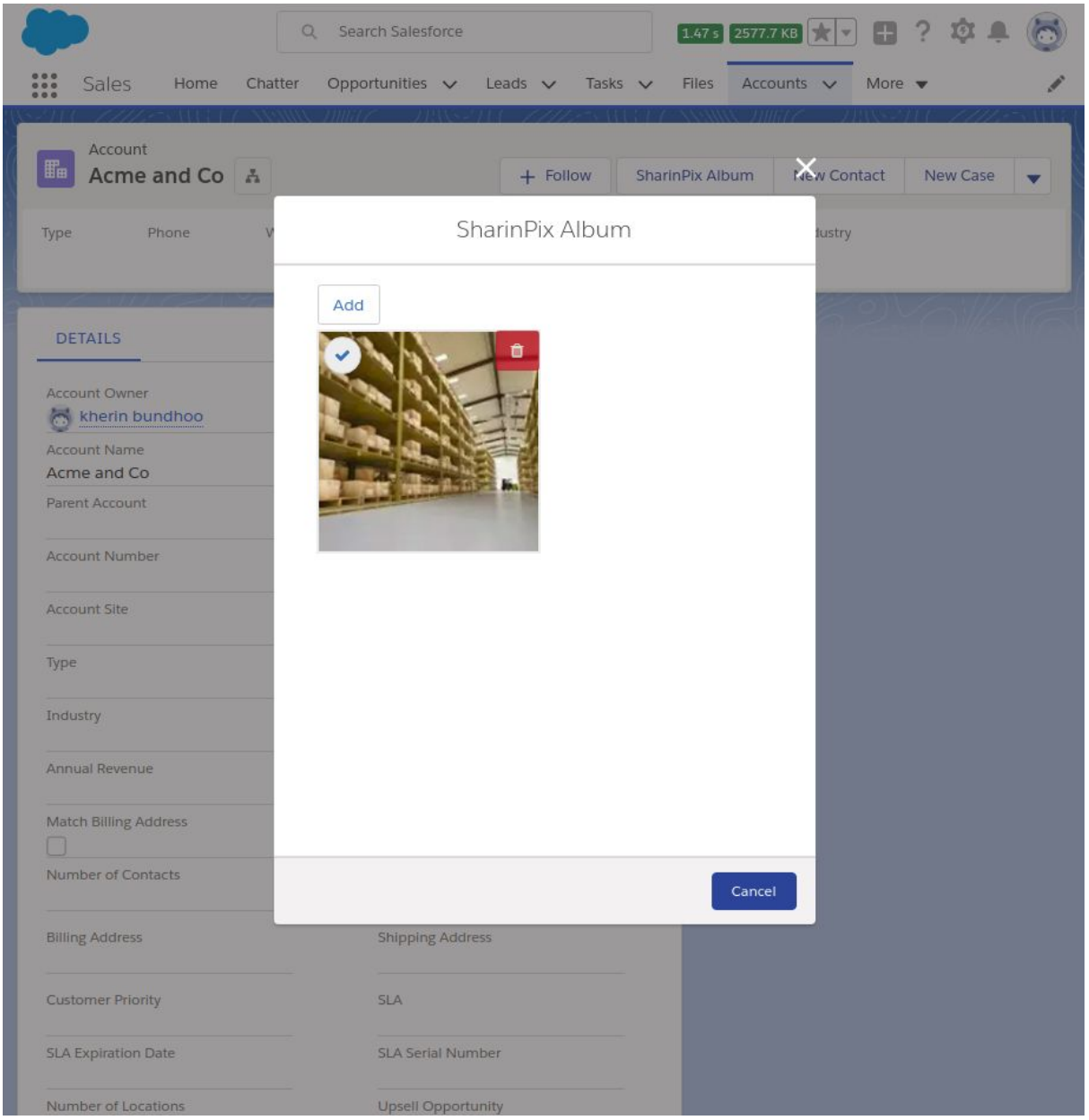


5. Adjust the **Height** to a minimum of **525px**.
6. Select a **Standard Label Type.**
7. Label: **SharinPix Album**
8. Name: **SharinPix_Album**
9. Click on **save**.
10. The next step is now to add the Custom Action to the Account Page Layout.
11. Head over to the Account Page Layout most relevant to your case.
12. From **Mobile & Lightning Actions**, drag and drop the **SharinPix Album** action inside the **Salesforce Mobile and Lightning Experience Actions** section. Click on **save**.

13. Access an account record. The newly-created custom action should appear on the page-layout.



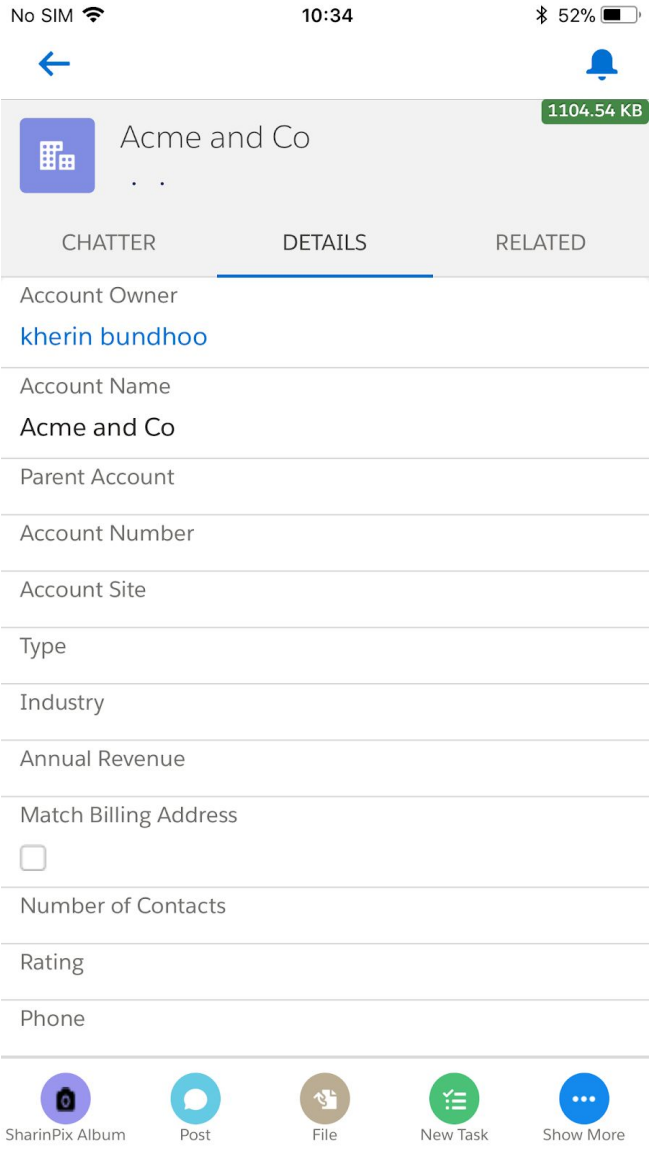
14. When the action is selected, the **SharinPix Album** is launched as shown in the image below.



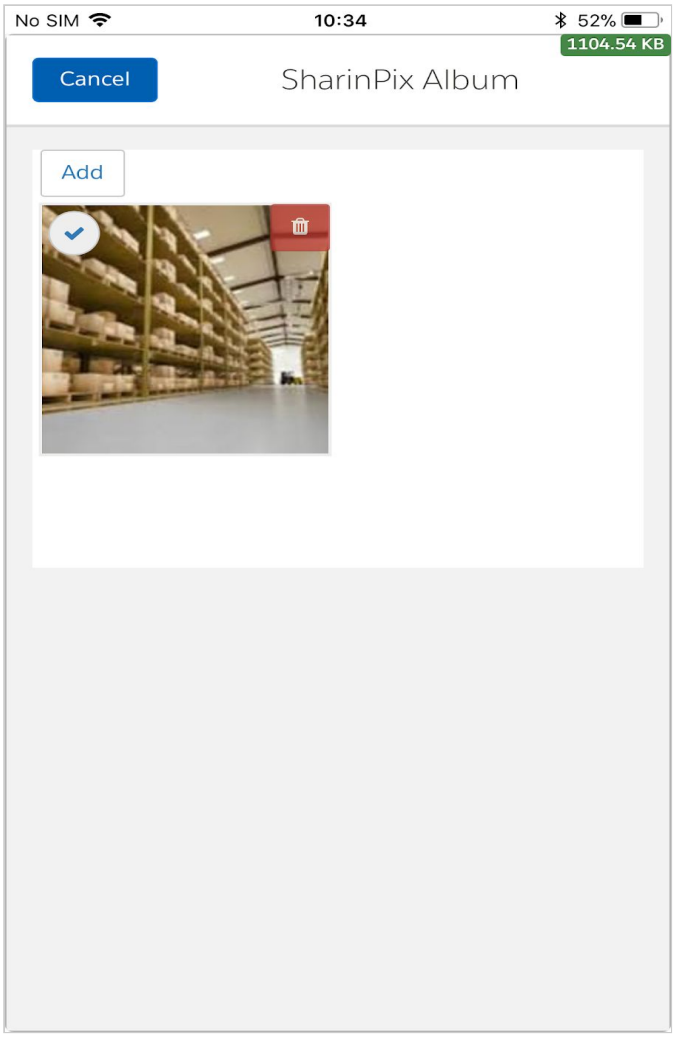
SharinPix Album in Salesforce App

The **SharinPix Album** custom action can also be accessed from the Salesforce mobile application as shown in the images below.

1. Access the **SharinPix Album** action from the **action bar**.



2. Open the **SharinPix Album**.

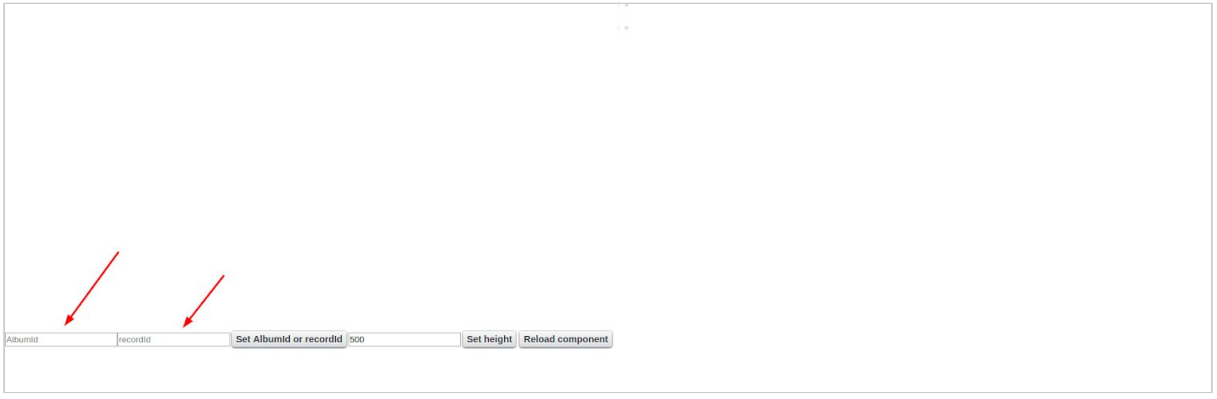


Lightning Component Wrapper

Find a demo on how to use the SharinPix Lightning Component on your code here: https://github.com/SharinPix/demo-apex/tree/lightning_cmp_form

This demo provides an example of how to use the SharinPix Lightning component.

- Preview the SharinPixDemoWrapperApp from the developer console to open the application.
- You can change the album ID or record ID passed to the component on the fly and the SharinPix Lightning Component will update itself.



Resize/Crop Images for download

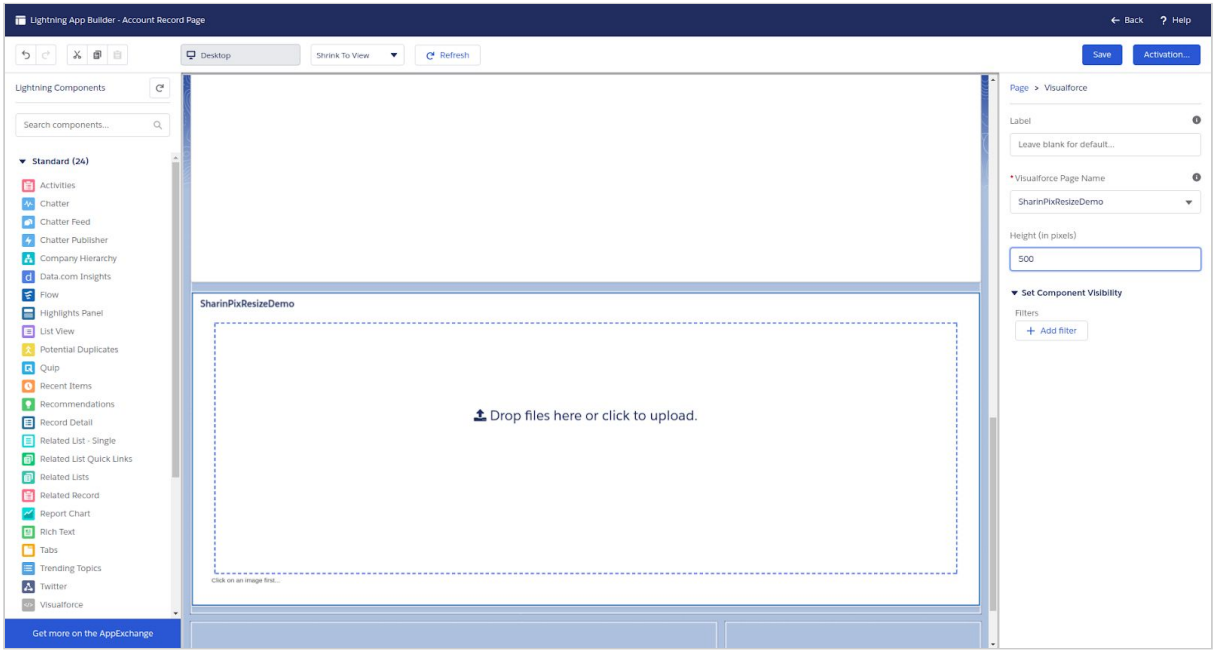
This demo explains how to Crop/Resize an image from a SharinPix album. The Visualforce page in this demo will open the resized/cropped image in a new tab.
The Apex controller for this demo has a standalone(static) method that takes 4 parameters and returns the URL of the new image. The parameters are:

- The ID of the image that needs to be resized,
- The new width,
- The new height,
- And the crop type.

Find out how to resize images with different crop parameters explained here:
https://github.com/SharinPix/demo-apex/tree/image_crop_resize

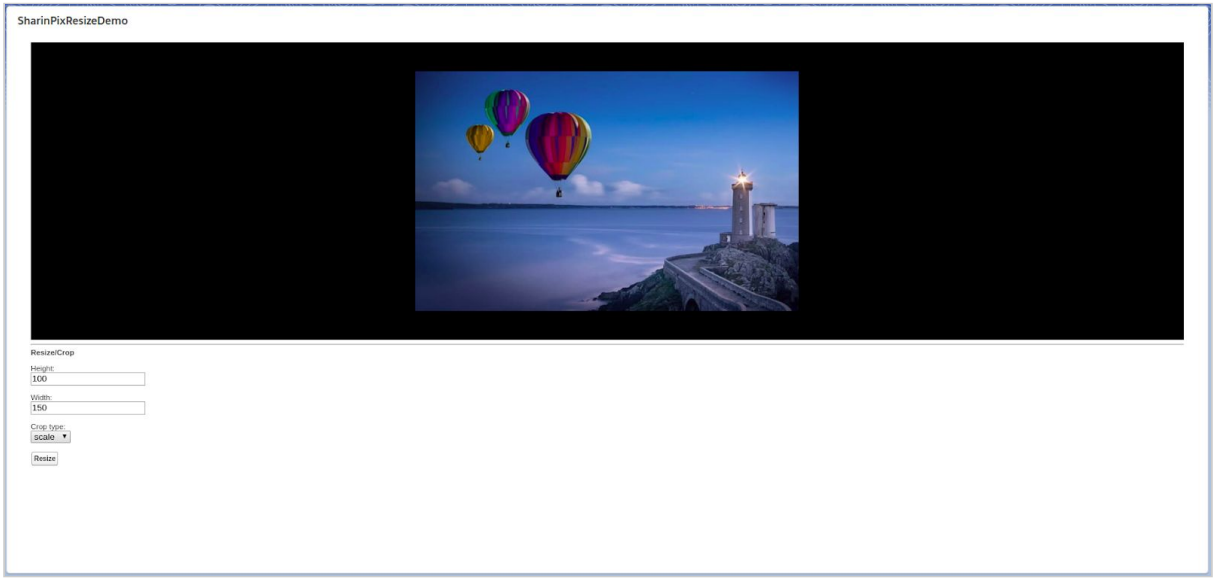
(Learn how to add the Visualforce Page: here)

1. Add Visualforce page to page-layout of the object



2. Add an image

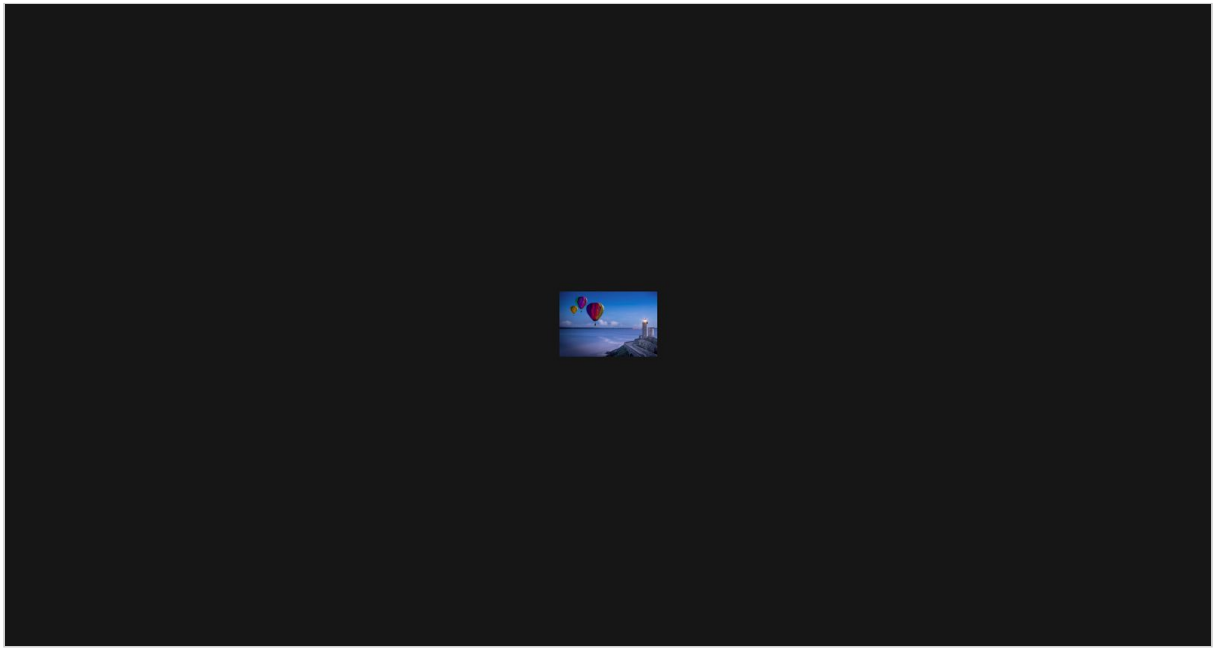
3. Click on image



4. Adjust values for Height, Width and Crop type

5. Click on ‘**Resize**’

Another tab, displaying the image transformation, is opened.



Making image Collage/Mashup

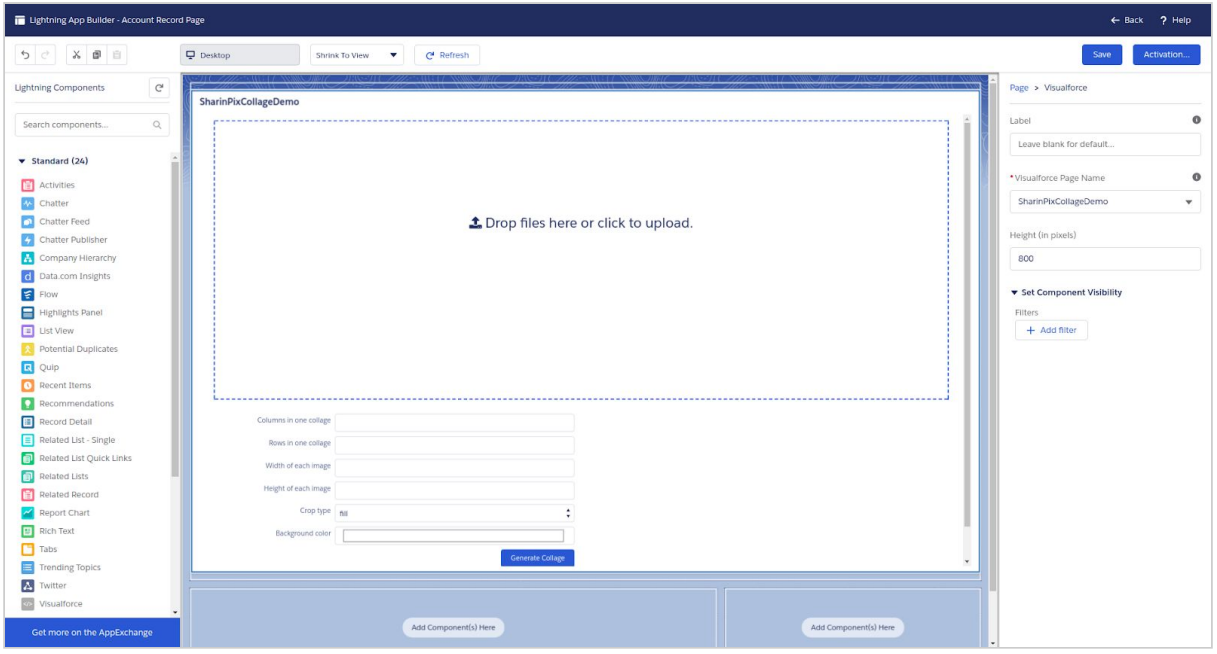
This demo explains how to make a collage/mashup out of all the images in an album.
The number of rows and columns in the collage can be specified.
The Apex controller of this demo has a standalone(static) method that takes 7 parameters to generate a collage and returns a list of image URLs. The parameters are:

- The album ID that will be used for the collage,
- The number of columns that a collage will have
- The number of rows that a collage will have,
- The width of each image (in pixels) in the collage,
- The height of each image (in pixels) in the collage,
- The crop type that will be used for the collage, (See demo above for more detail on crop types),
- The background color for the collage.

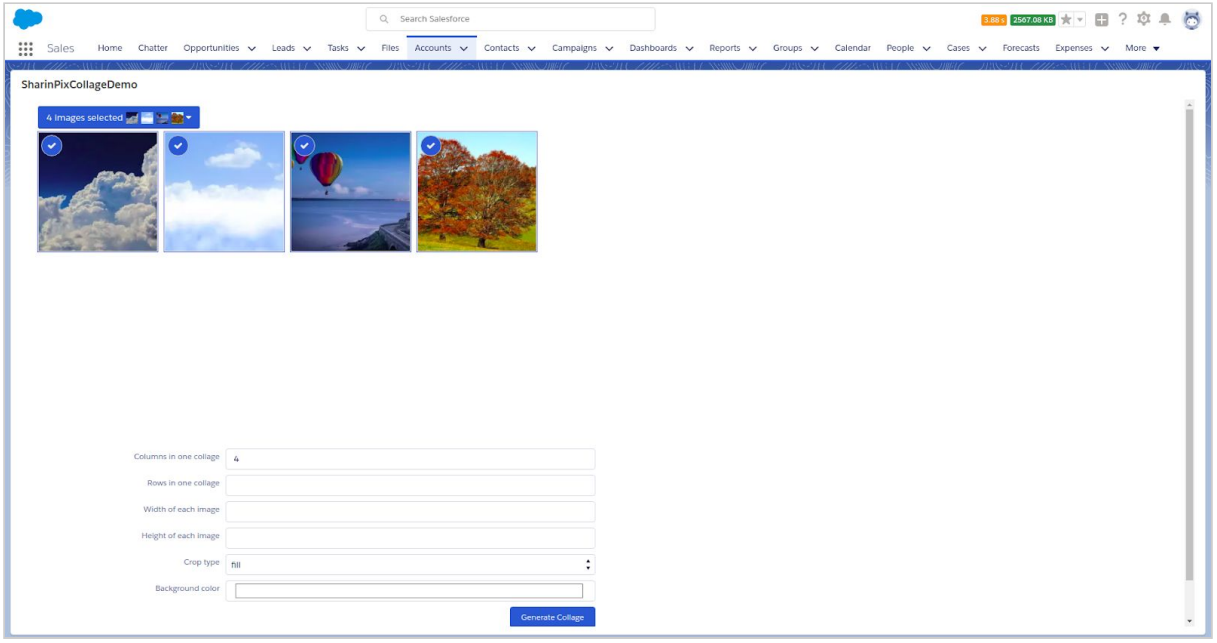
You can also customize the this demo to only take some images from an album and make a collage.
Find out how to make a collage/mashup here:
https://github.com/SharinPix/demo-apex/tree/image_collage_mashup

(Learn how to add the Visualforce Page: here)

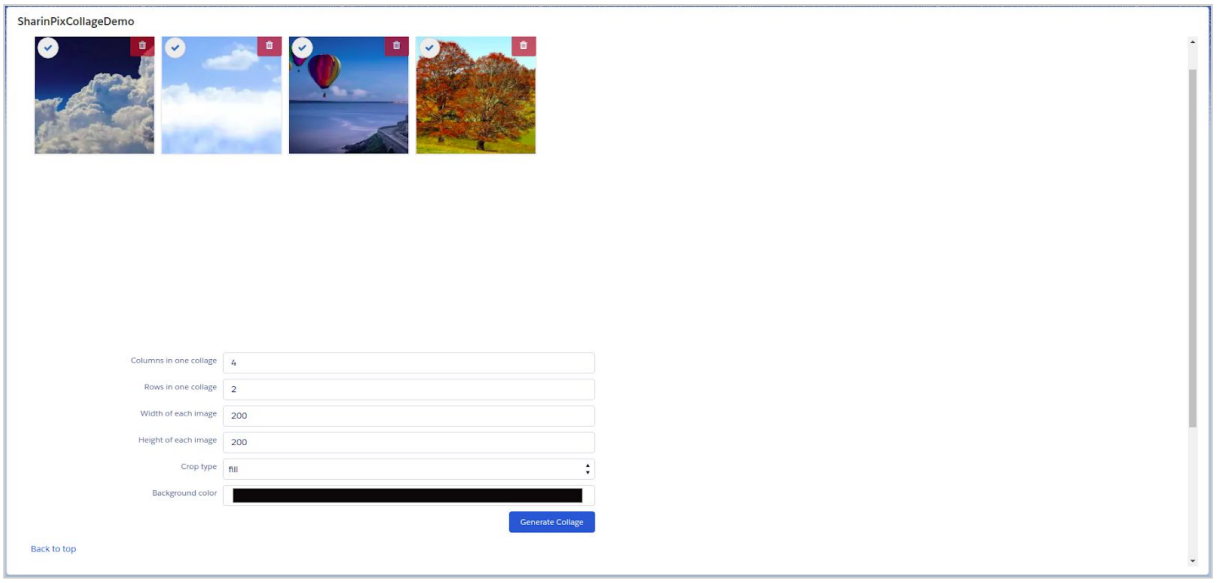
1. Add Visualforce page to page-layout of object



2. Add images

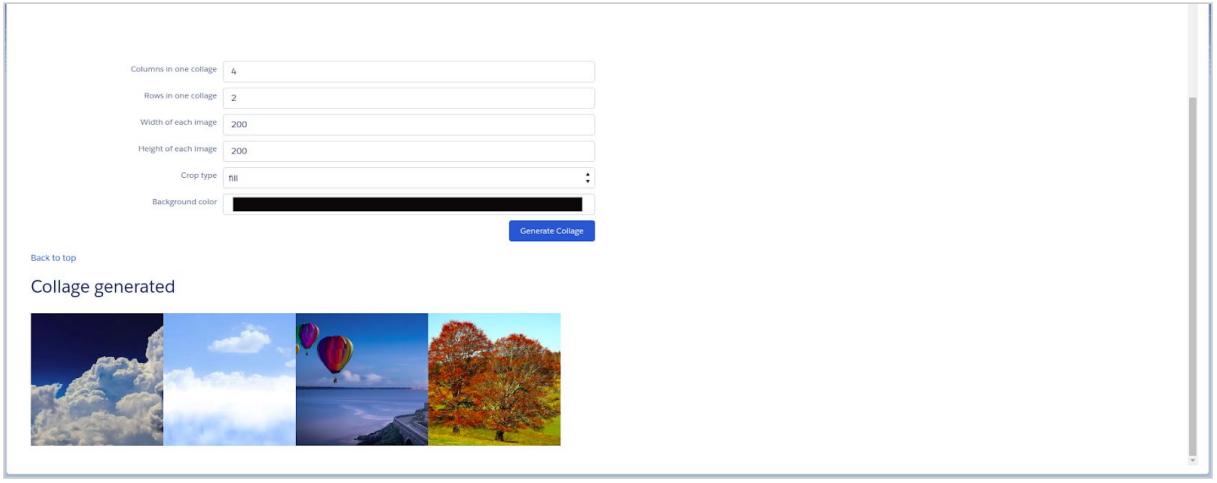


3. Adjust values for collage transformation parameters



4. Click on **‘Generate Collage’**

A collage is generated and displayed at the bottom.



Upload using URL

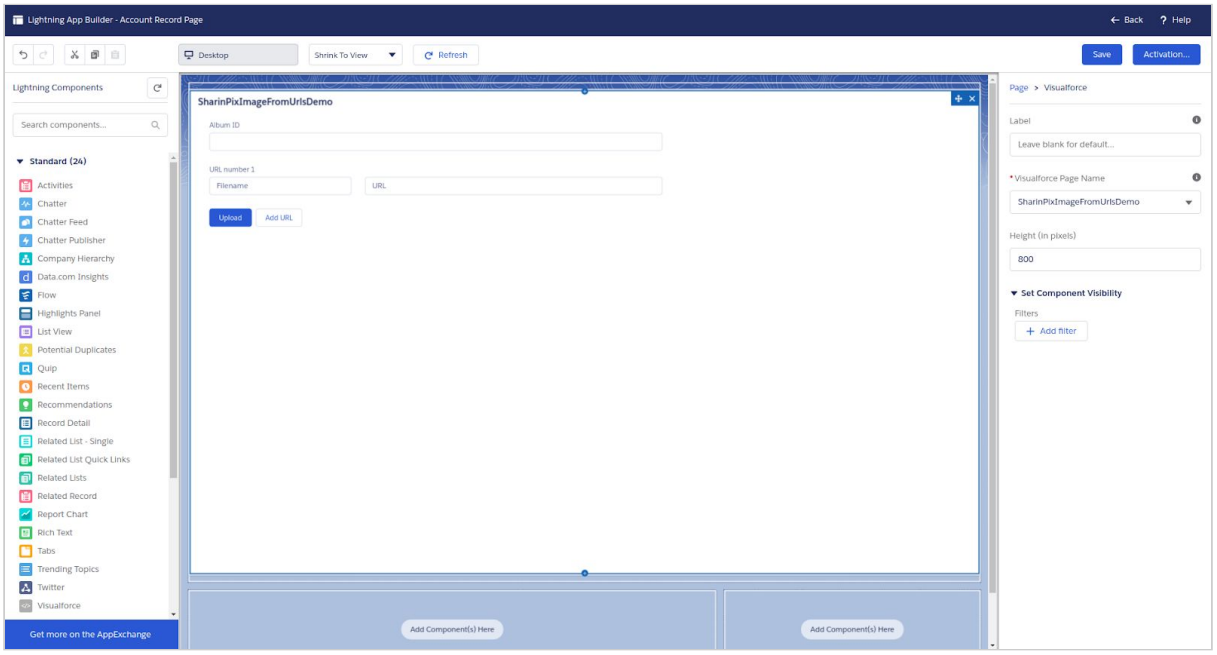
You can send images for import using URL to SharinPix.
In this demo, you can send up to 10 URL for upload in a specific album.
The Apex controller for this demo have a standalone(static) method that takes 3 parameters in order to perform a URL import (upload) to SharinPix. The parameters are:

- The album ID in which you want to upload the file,
- The URL of the file,
- The filename of the file.

Find the resources here: https://github.com/SharinPix/demo-apex/tree/upload_by_url

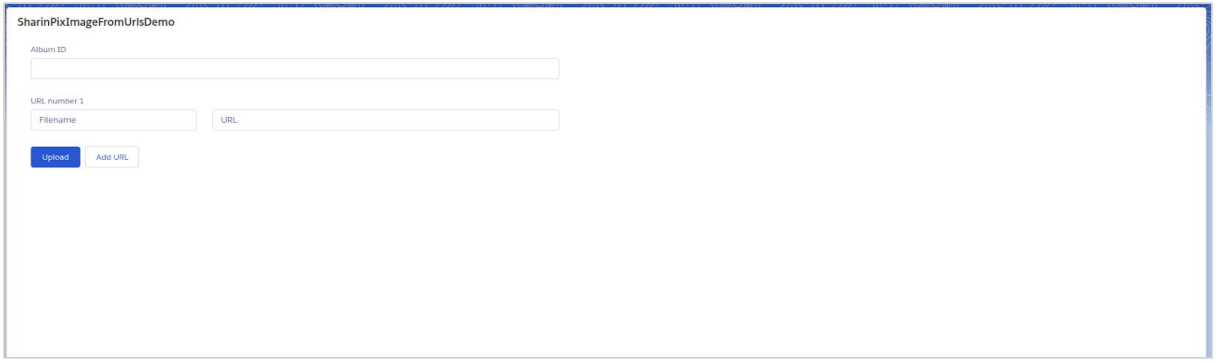
(Learn how to add the Visualforce Page: here)

1. Add Visualforce page to page-layout of object



2. Fill in relevant parameter values

- The album ID in which you want to upload the file,
- The URL of the file,
- The filename of the file.



Click on ‘**Add url**’ so as to add another image URL or click **‘Upload’** in order to upload the image to the SharinPix Album with the corresponding Album ID.

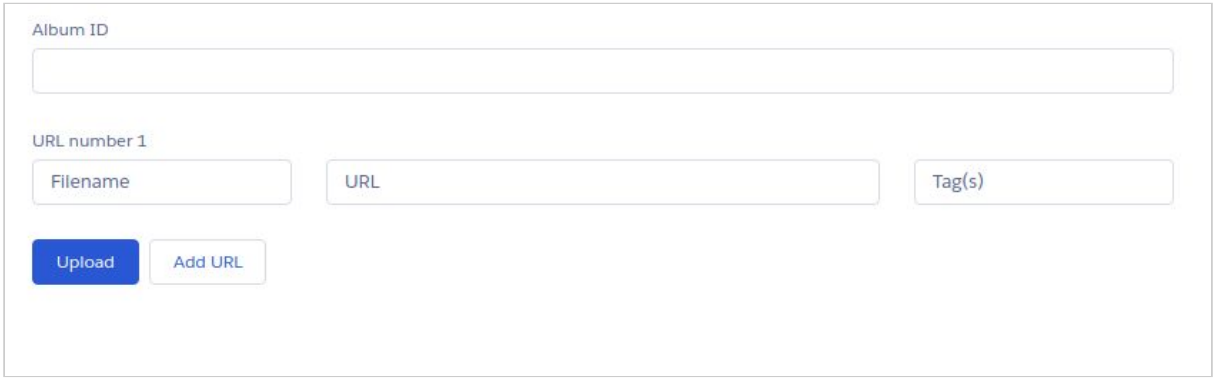
Upload image(s) with tag(s) using URL

In contrast to the above demo, the current one enables the addition of tags to an image .The Apex controller for this demo have a standalone(static) method that takes 4 parameters in order to perform a URL import (upload) to SharinPix. The parameters are:

- The album ID in which you want to upload the file,
- The URL of the file,
- The filename of the file.
- The tags to be applied on an image.

Find the resources here: https://github.com/SharinPix/demo-apex/tree/upload_with_tags

1. Preview the Visualforce Page



2. Fill in relevant parameter values

- The album ID in which you want to upload the file,
- The URL of the file,
- The filename of the file.
- The tag or tags to be applied on the image.

Note: To apply multiple tags to the same image, the tags must be separated by the ‘**#**' symbol inside the text field.

For example:

- **Single tag:** bar
- **Multiple tags:** foo#bar

Click on ‘**Add url’** so as to add another image URL or click **‘Upload’** in order to upload the image to the SharinPix Album with the corresponding Album ID.

Move image to another album

The following code snippet illustrates the way to move an image to another album with the use of two piece of information:

- The **public id** of the image to be moved.
- The **id** of the destination album.

```
sharinpix.Client spClient = sharinpix.Client.getInstance();
Id albumId = '<<album-id>>';
String imagePublicId = '<<image-public-id>>';
Map<String, Object> claims = new Map<String, Object> {
        'admin' => true
};
Map<String, Object> body = new Map<String, Object>{
    'move_operation' => new Map<String, Object>{
        imagePublicId => albumId
    }
};
String token = spClient.token(claims);
List<Object> resultPublicId = (List<Object>) spClient.post('/images/move',
Blob.valueOf(JSON.serialize(body)), claims);
```

NOTE:

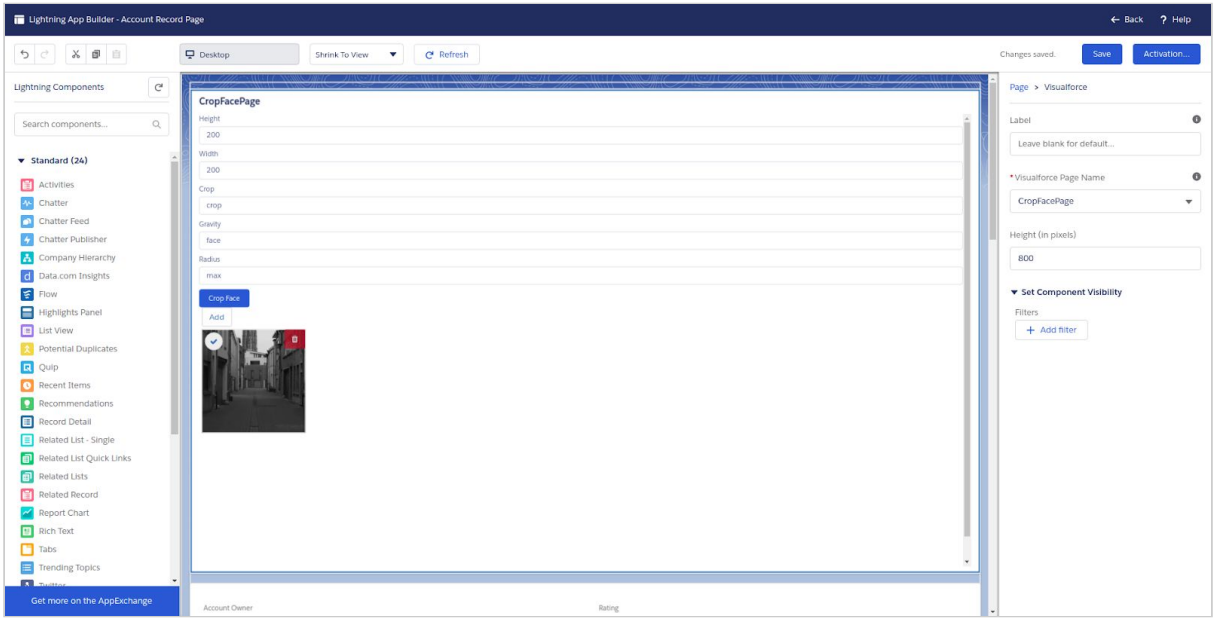
- `'<<image-public-id>>'` should be replaced by the **public Id** value of the image to be moved
- `'<<album-id>>'` should be replaced by the **id** of the destination album.
- The list **resultPublicId** contains the public id of the image that has been moved, which therefore indicates a successful move operation.

Crop image to face

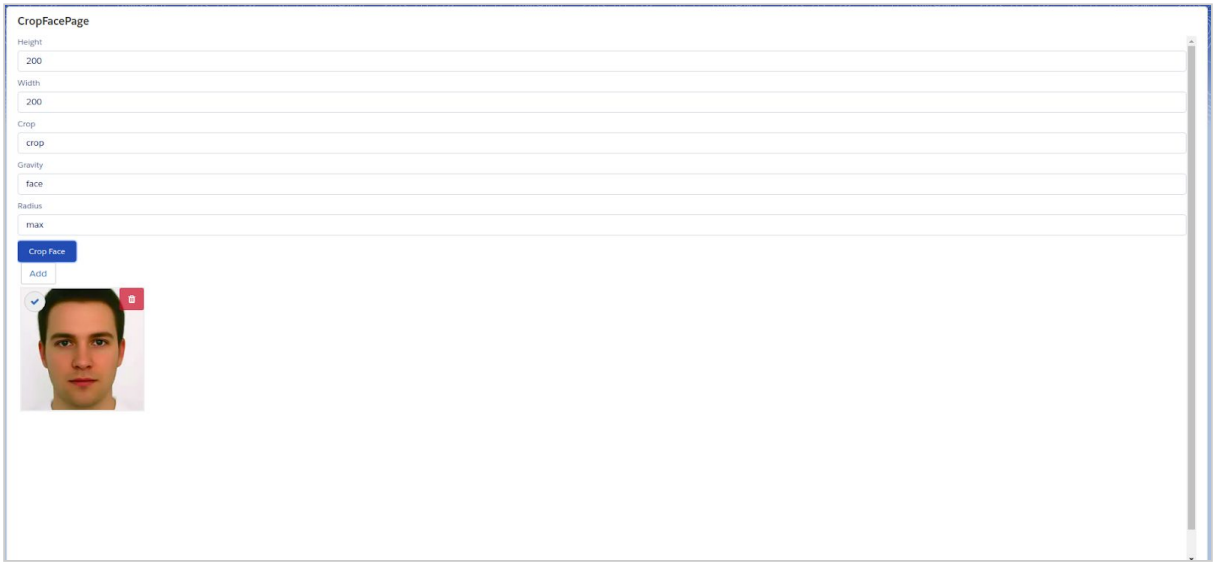
In this demo, you will be shown how to use SharinPix and crop an image to a face when detected. This demo contains a class named “ImageFaceCrop”. This class contains “cropFace” method which can be used to crop images to face. The method takes an image ID and returns a URL of the image cropped to face.
Find out the resources here:
https://github.com/SharinPix/demo-apex/tree/crop_face_image

(Learn how to add the Visualforce Page: here)

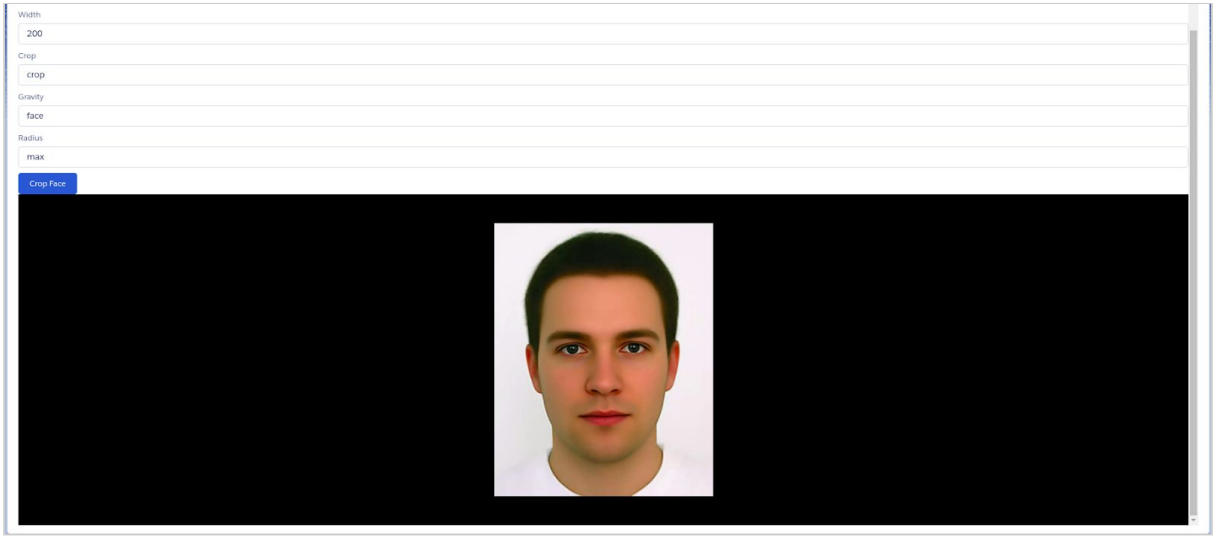
1. Add Visualforce page to page-layout of Account object



2. Enter parameter values to crop face

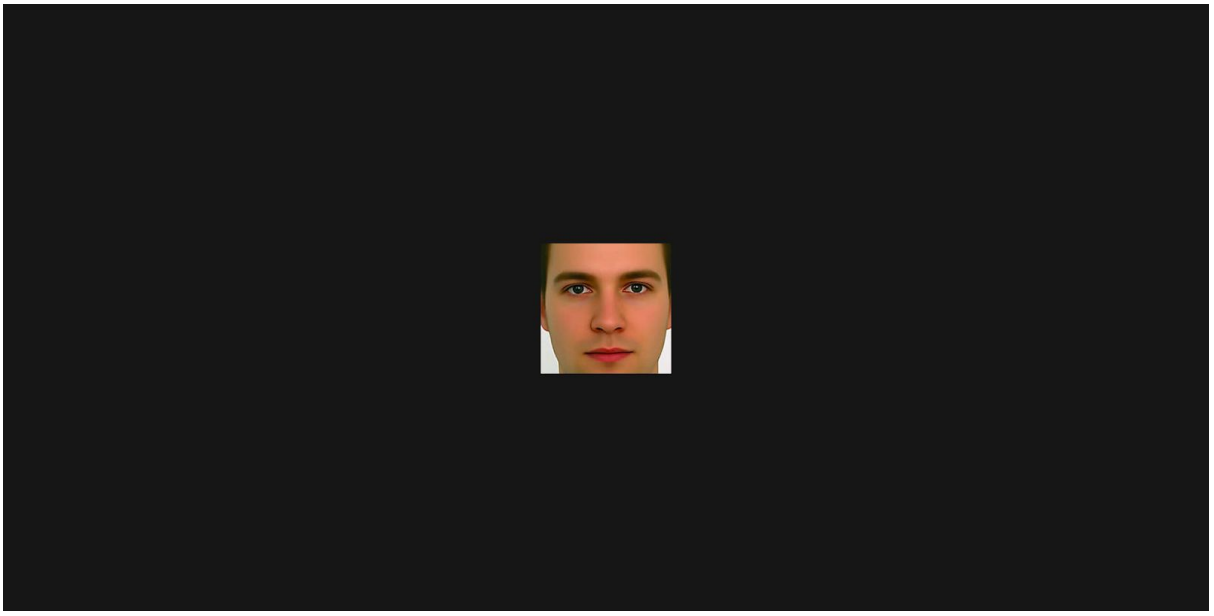


3. Click on image



4. Click on ‘**Crop face**’

Another tab, displaying the image of the cropped face, is opened.



Inserting text overlays to image

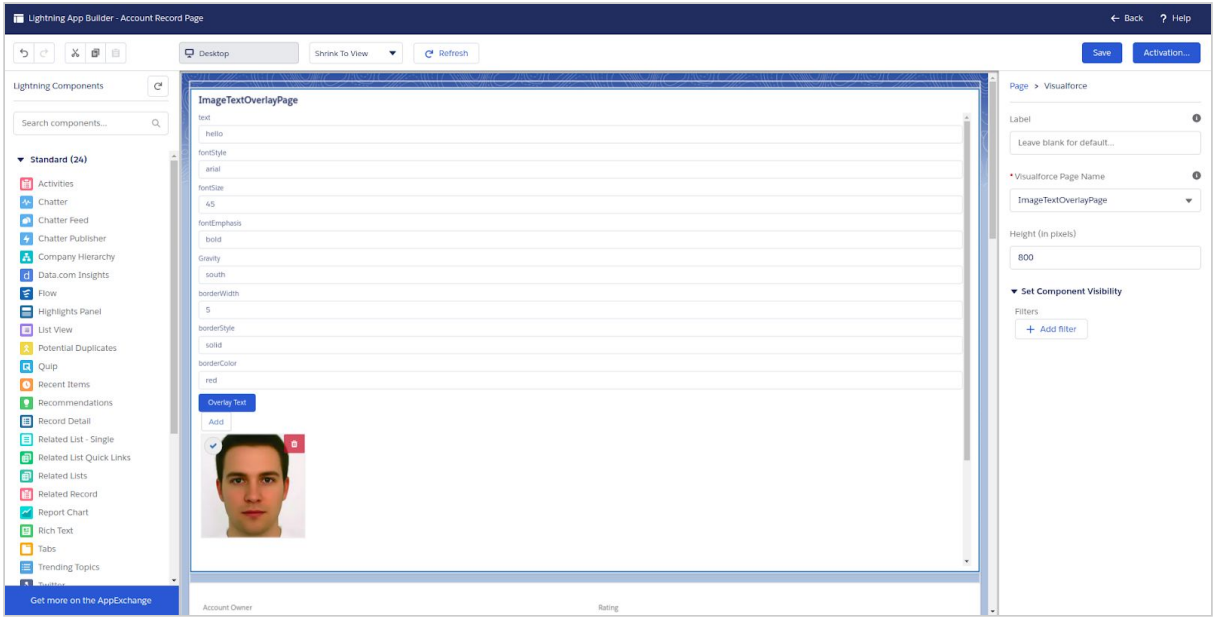
In this demo, you will be shown how to add a text overlay to an image.
This demo contains a class named “ImageTextOverlay”. This class contains an “overlayText” method that takes in 9 configurable parameters to return an image with the text overlay.
The parameters are:

- Image ID, the image on which you want to add a text,
- The text that you want to add,
- The font family for the text,
- The font size,
- The font emphasis (bold, or italics),
- The gravity (north_east, north, north_west, west, south_west, south, south_east, east, center, or face),
- The border width (in pixels),
- The border style (solid, dashed, or dotted),
- The border color.

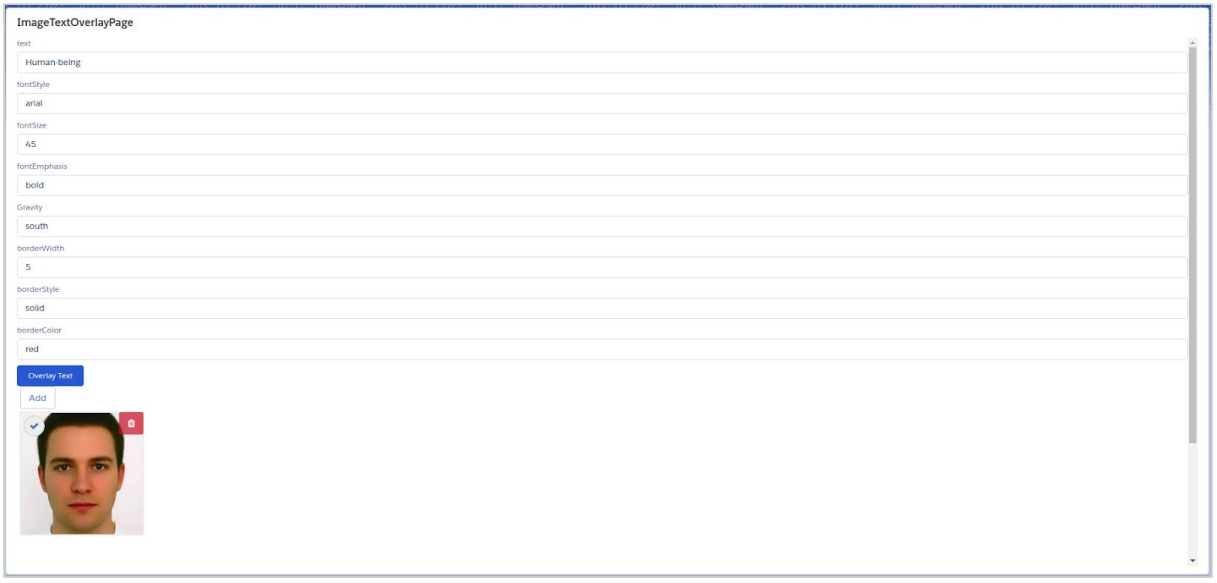
Find out the resources here:
https://github.com/SharinPix/demo-apex/tree/image_text_overlay

(Learn how to add the Visualforce Page: here)

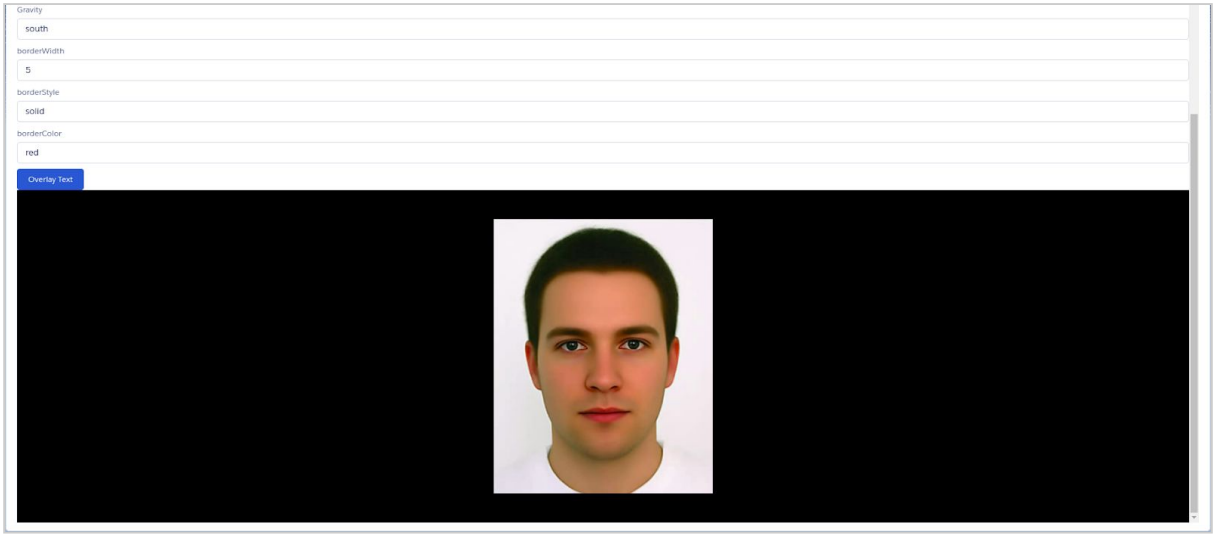
1. Add Visualforce page to page-layout of Account object



2. Fill in parameter values to overlay text on image

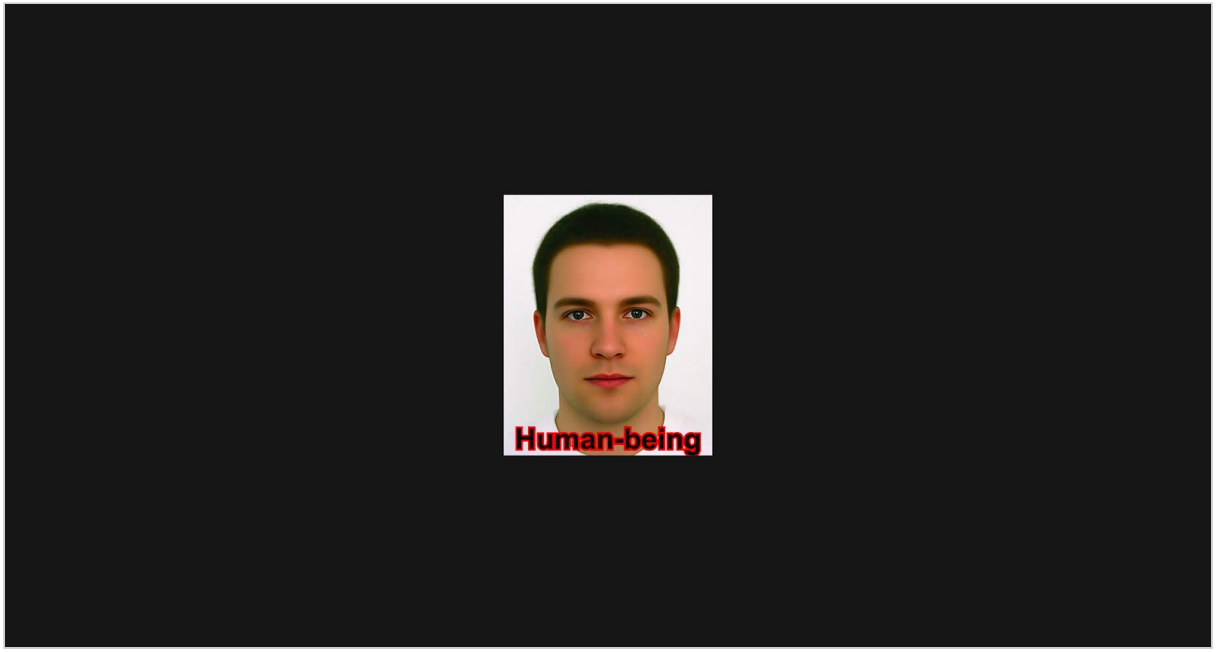


3. Click on image



4. Click on ‘**Overlay Text**’

Another tab opens, displaying image with its text-overlay transformation.



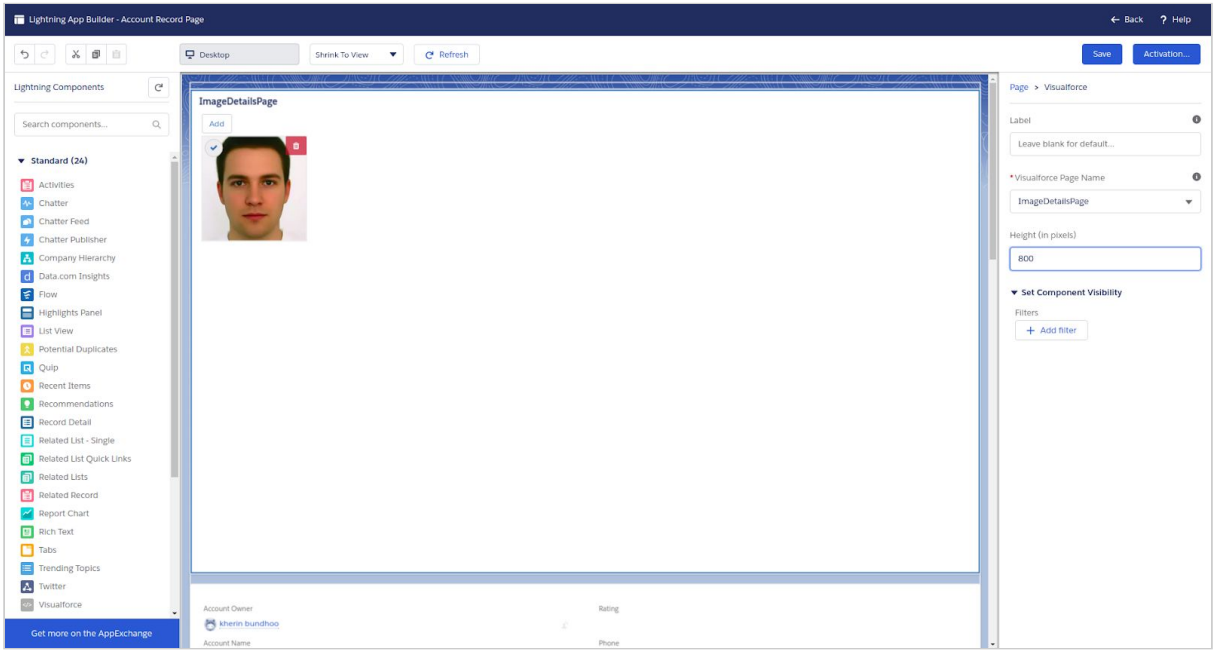
Capturing SharinPix events using Javascript

In this demo, you will get an insight on how to capture Sharinpix events, more specifically the event named ***viewer-image-viewed***. After the latter process, the demo will also describe how useful data (filename, format, created date, image url) can be extracted from the payload provided by SharinPix. Those data are then displayed upon their corresponding fields.

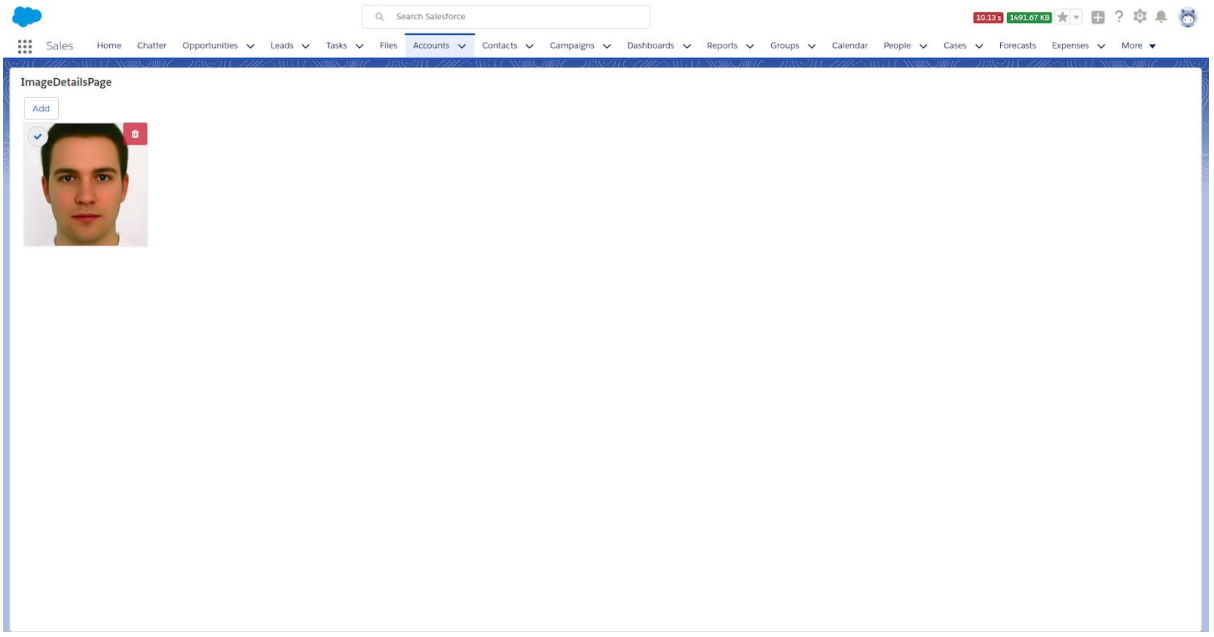
Access the resources here:
https://github.com/SharinPix/demo-apex/tree/js-events

(Learn how to add the Visualforce Page: here)

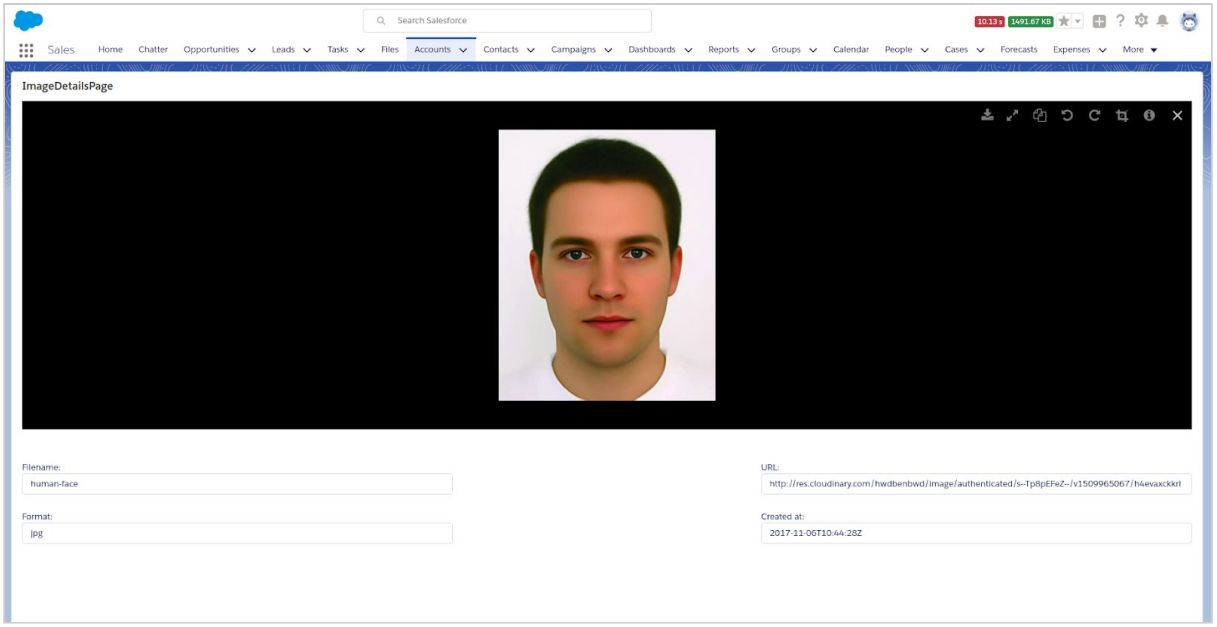
1. Add Visualforce page to page-layout of Account object



2. Click on image



Details about the image’s filename, URL, file format and Created Date are displayed inside relevant fields.



Download images from an album into a Zip archive

The following demo shows how to download a zip archive containing an album’s original images. The zipping process is done using JavaScript libraries, hence you will need to use a modern browser for the demo to run adequately.

Find below the JavaScript libraries used and their corresponding support across browser versions:

(**Note: The following data are snapshots at the time this document has been written. Visit the links below to obtain the up-to-date values).**

1. JSZIP (v3.1.3)

Support

Opera	Firefox	Safari	Chrome	Internet Explorer	Node.js
Yes	Yes	Yes	Yes	Yes	Yes
Tested with the latest version	Tested with 3.0 / 3.6 / latest version	Tested with the latest version	Tested with the latest version	Tested with IE 6 / 7 / 8 / 9 / 10	Tested with node.js 0.10 / latest version

Source: https://stuk.github.io/jszip/

2. FileSaver (v1.3.3)

Supported browsers

Browser	Constructs as	Filenames	Max Blob Size	Dependencies
Firefox 20+	Blob	Yes	800 MiB	None
Firefox < 20	data: URI	No	n/a	Blob.js
Chrome	Blob	Yes	500 MiB	None
Chrome for Android	Blob	Yes	500 MiB	None
Edge	Blob	Yes	?	None
IE 10+	Blob	Yes	600 MiB	None
Opera 15+	Blob	Yes	500 MiB	None
Opera < 15	data: URI	No	n/a	Blob.js
Safari 6.1+*	Blob	No	?	None
Safari < 6	data: URI	No	n/a	Blob.js
Safari 10.1+	Blob	Yes	n/a	None

Source: https://github.com/eligrey/FileSaver.js/

3. Jquery (v3.1.1)

Current Active Support

Desktop

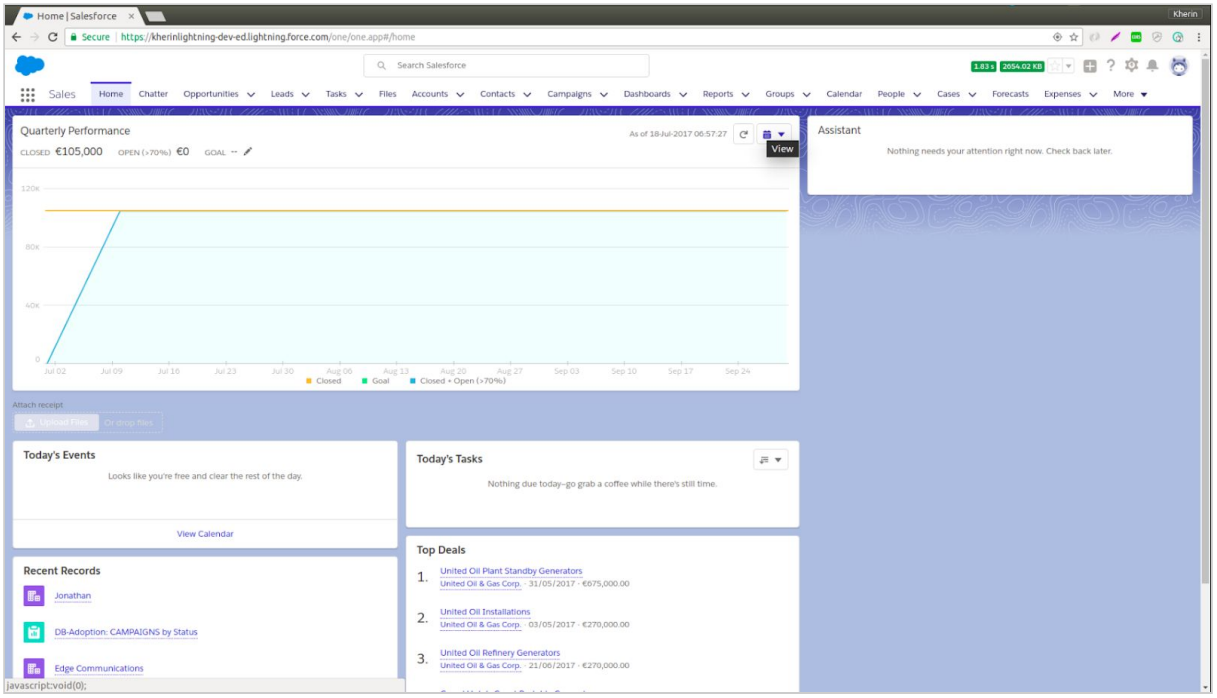
- Chrome: (Current - 1) and Current
- Edge: (Current - 1) and Current
- Firefox: (Current - 1) and Current
- Internet Explorer: 9+
- Safari: (Current - 1) and Current
- Opera: Current

Source: https://jquery.com/browser-support/

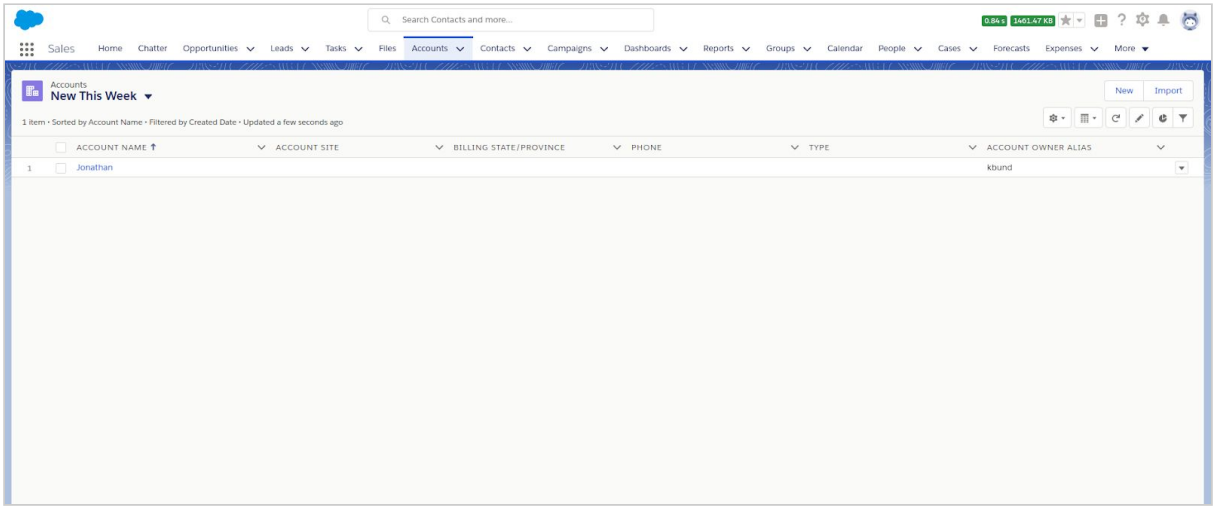
Access the code and other resources here:
https://github.com/SharinPix/demo-apex/tree/js-zip-download

After deploying the code (as explained in the README file), you will need to follow the steps below:

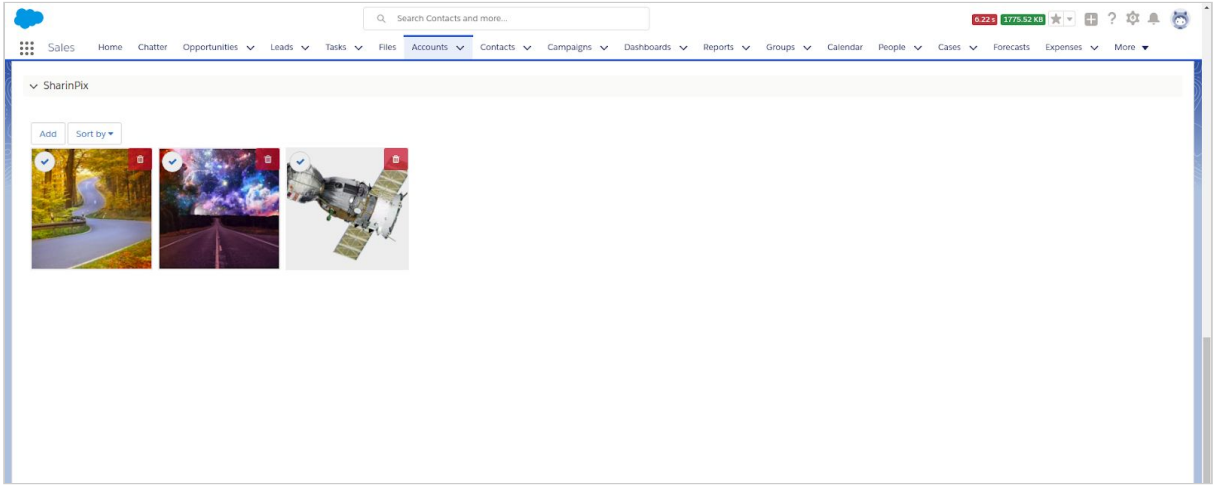
1. Open up your browser and go to your Salesforce environment.



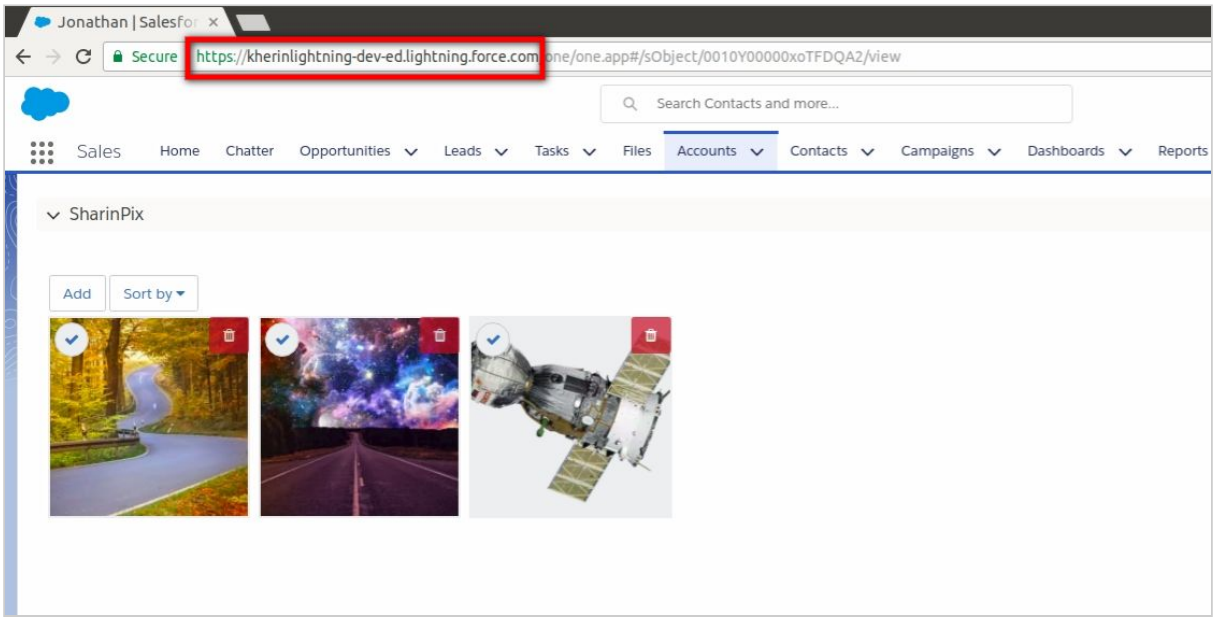
2. Go to any object that has a SharinPix album with images. In this demo, the Account object will serve as an example.



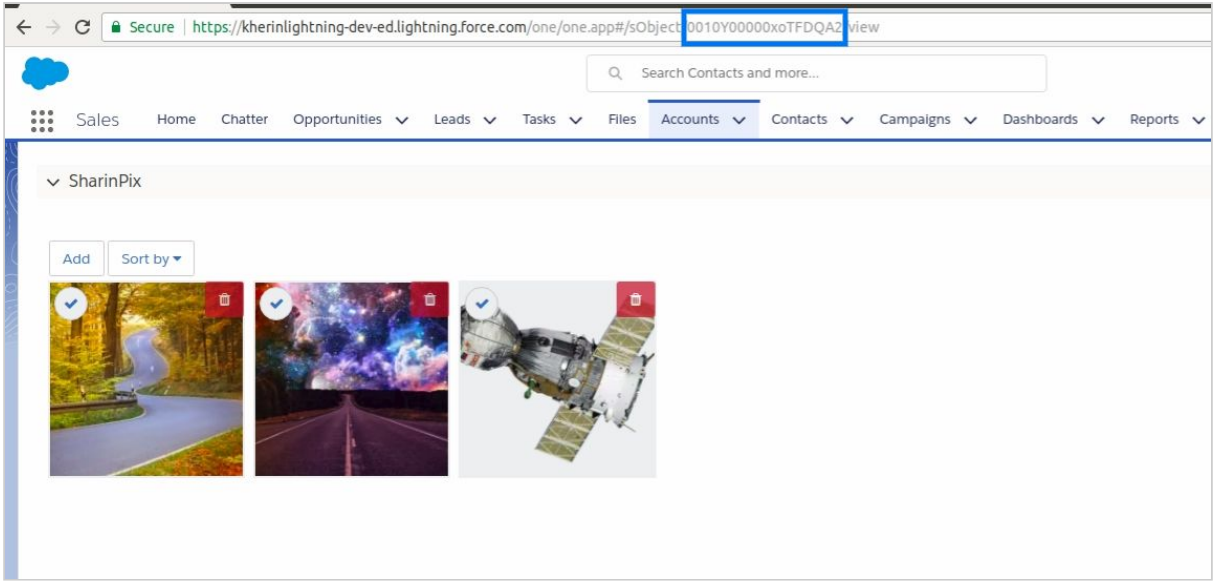
3. Select a record.



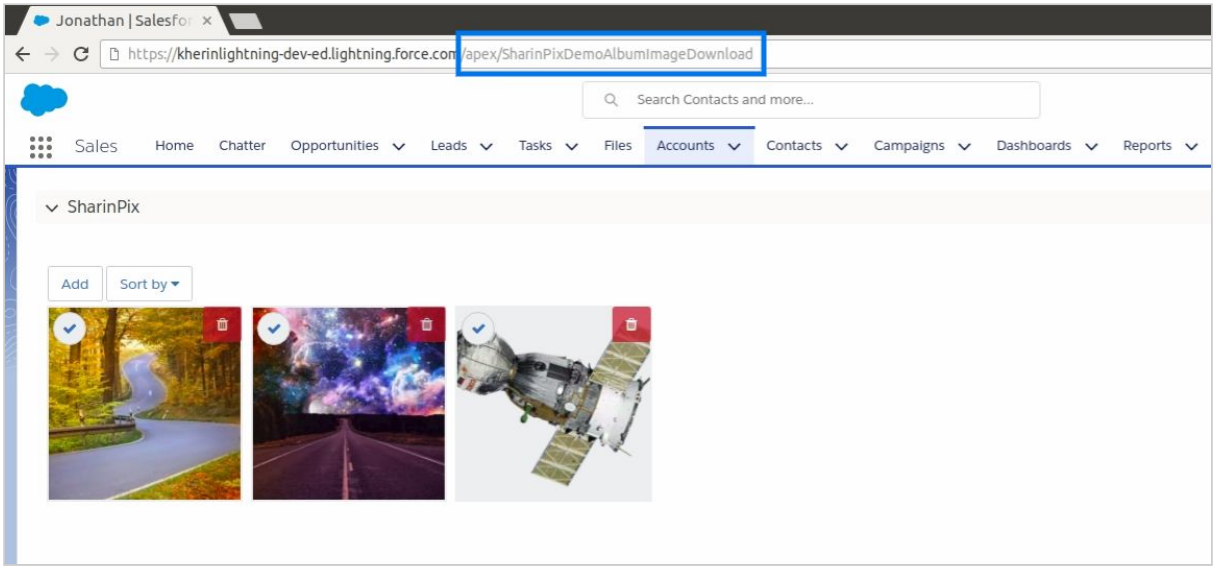
4. Modify the URL inside the address bar of the browser you are using.
The environment URL of your Salesforce environment should be at the same position as highlighted in the figure below.
(**Note: The environment URL of your salesforce environment will differ from that shown in the figure below.)**



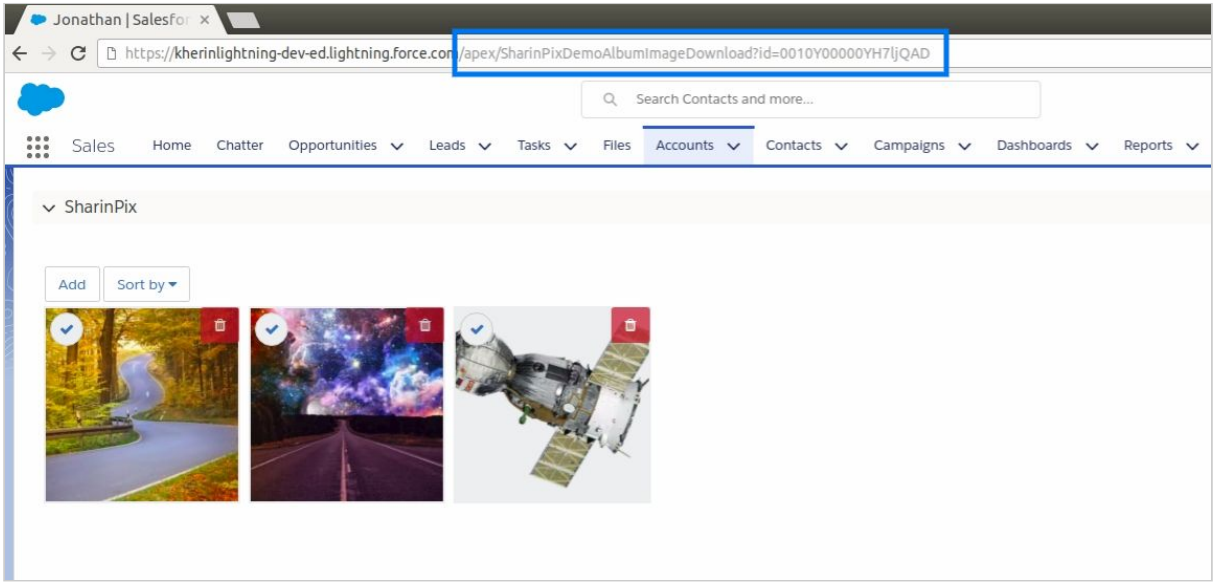
5. The Album Id for the current record is shown in the figure below. (Copy the Id value as it will be used in the next steps).



6. Append the Environment URL with **/apex/SharinPixDemoAlbumImageDownload**, where **SharinPixDemoAlbumImageDownload** corresponds to the Visualforce Page used in this demo.

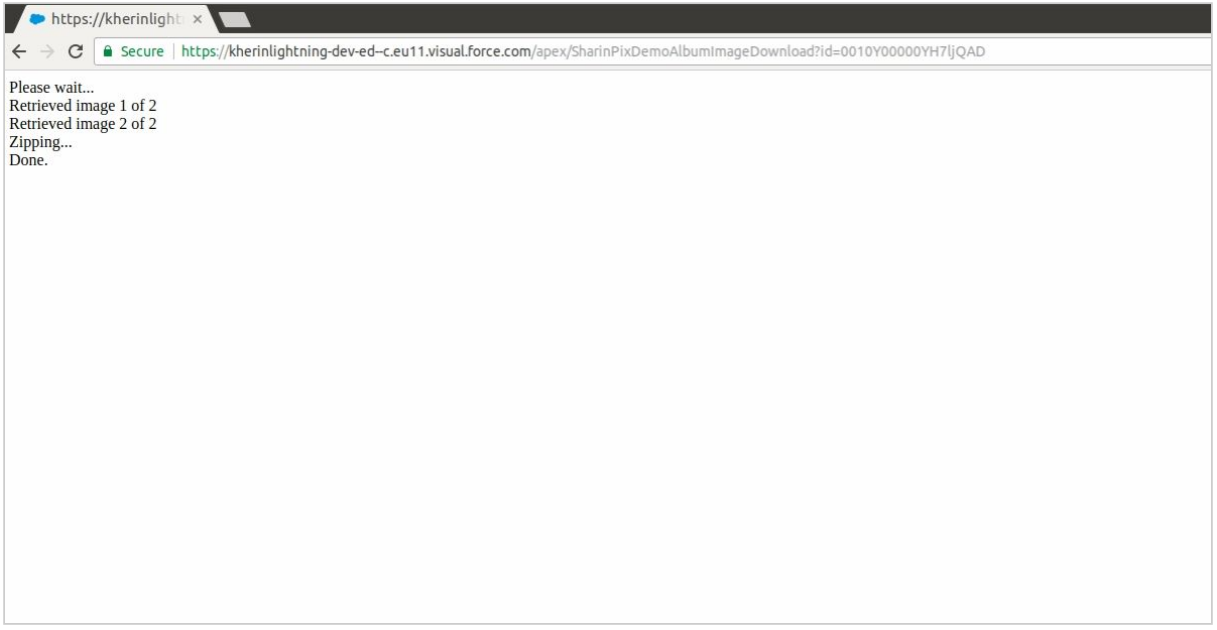


7. Append the parameter **Id** and its corresponding value(the Album Id copied before).

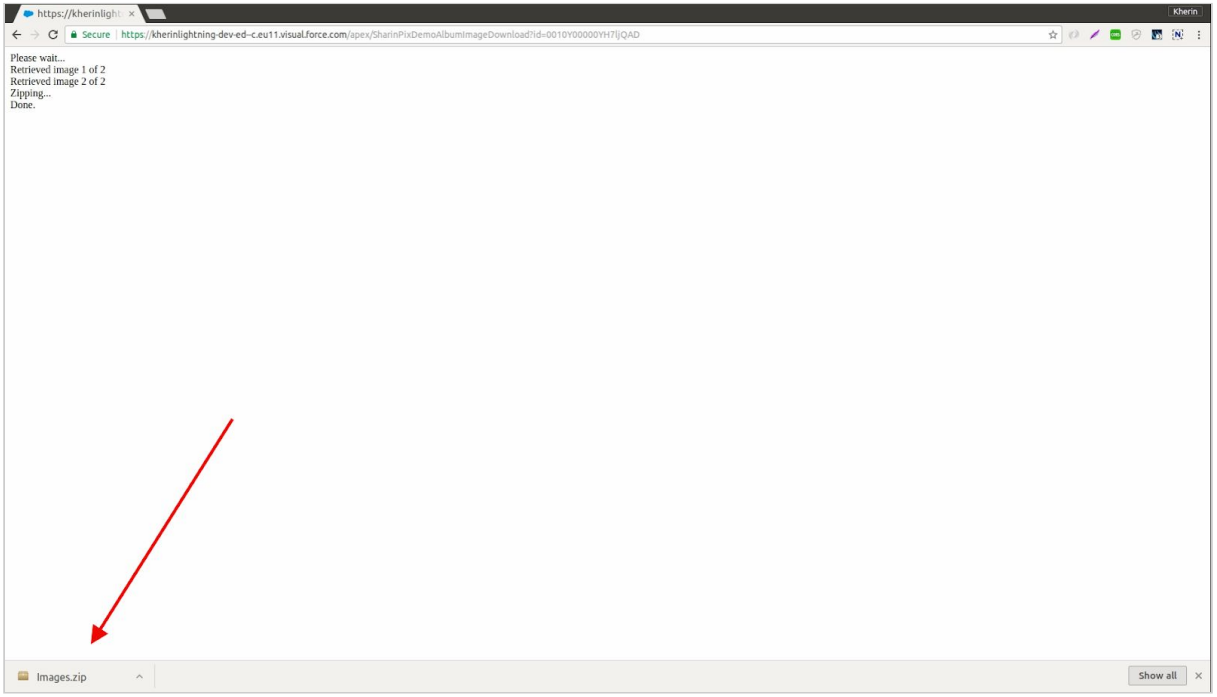


8. After you have followed the above steps and entered the values that are relevant to you, press **Enter** to be directed towards the Visualforce Page that will zip the images.

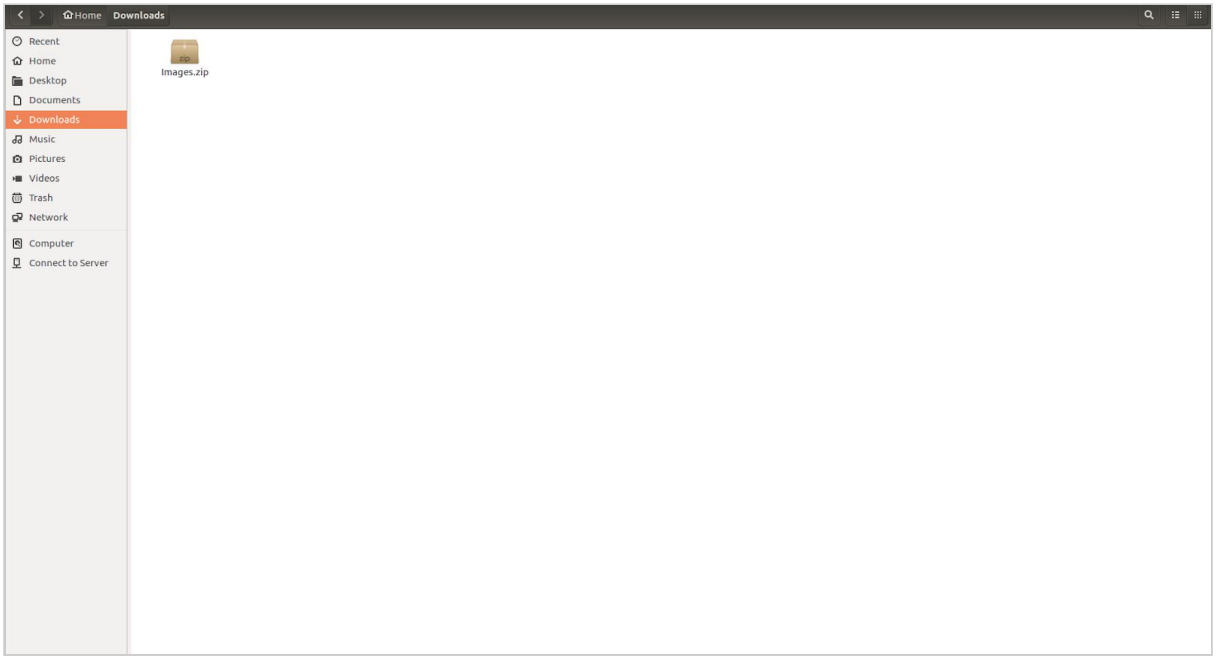
The Visualforce Page will display the number of images in the album being zipped and will log its progress.



9. Once the zipping process is done, the zipped file containing all the images in the album will start inside the browser window. The downloaded archive file is labelled “Images” and is of format “zip”.



10. Once you access the directory where downloaded files are saved (defined by your browser), you will be able to extract the image files from the archive.



11. Extract the image files from the archive. The extracted files will correspond to the images present in the SharinPix Album of the record mentioned above.

12. Another parameter can be appended to the Environment URL, that gives you the possibility to specify the name of the archive to be downloaded. (In the figure below, the value is *HELLO)*

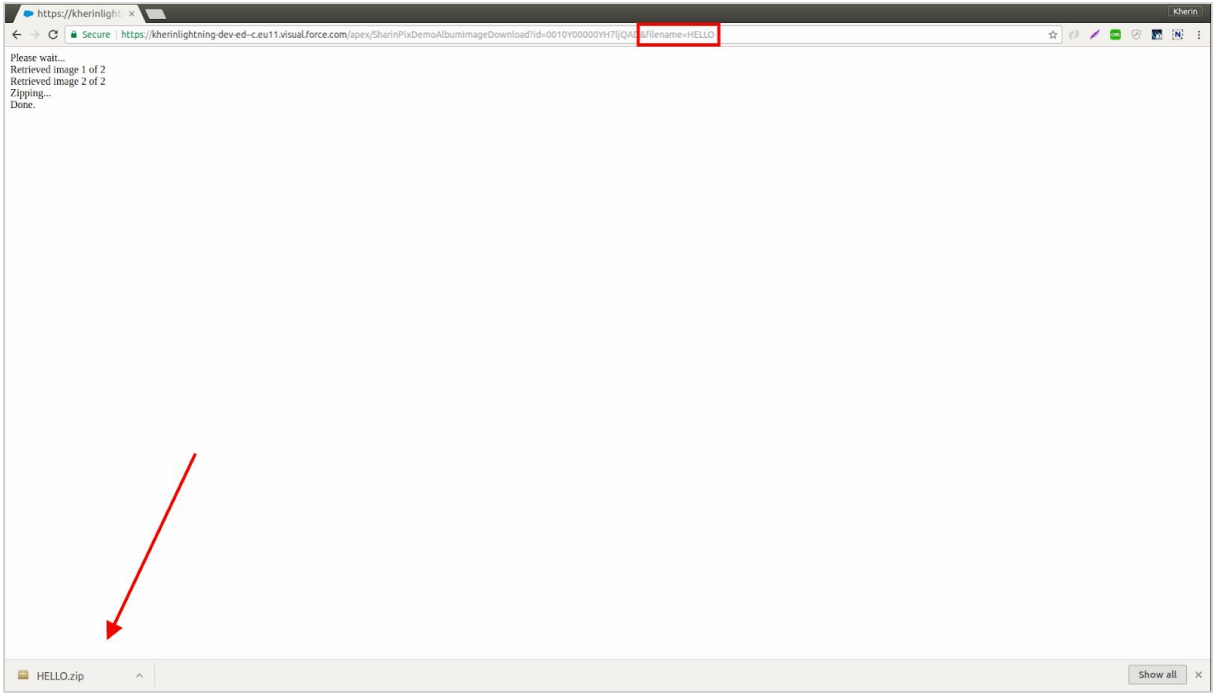


Image downloads and zipping from multiple albums

While the example above demonstrates a use case wherein images are downloaded from a single album, the present scenario will delve into the download and zipping of multiple images across **many** albums.

The difference between the present example and the above one are as follows:

- The present use case allows the download and zipping of images originating from multiple albums.
- The present use case allows for a greater number of callouts to be made, referring to Salesforce callout limits, than the previous example.

The resources used in this example:

- Apex Controller: SharinPixDemoAlbumImageDownloadV2Ctrl.cls
- Visualforce Page: SharinPixDemoAlbumImageDownloadV2Page.page

These resources can be found on: GitHub.

Overview of the Apex Controller

When you access the **SharinPixDemoAlbumImageDownloadV2Ctrl** apex class (found here), you will see the code as displayed below.

```
public class SharinPixDemoAlbumImageDownloadV2Ctrl {
  public List<Id> albumIdList = new List<Id> { 'album-id-1', 'album-id-2' };
  private sharinpix.Client clientInstance { get; set; }

  public SharinPixDemoAlbumImageDownloadV2Ctrl (ApexPages.standardController ctrl) {
    clientInstance = sharinpix.Client.getInstance();
  }

  public String getAlbums() {
    List<Map<String, Object>> albumList = new List<Map<String, Object>>();
    for(Id albumId : albumIdList){
      albumList.add(new Map<String, String> {
        'id' => albumId,
        'token' => fetchSharinPixToken(albumId)
      });
    }
    return JSON.serialize(albumList);
  }


public String fetchSharinPixToken(Id albumId) {
 return clientInstance.token(
      new Map<String, Object> {
        'exp' => DateTime.now().addHours(1).getTime(),
        'abilities' => new Map<String, Object> {
        albumId => new Map<String, Object> {
          'Access' => new Map<String, Boolean> {
            'see' => true,
            'image_list' => true
          }
        }
      }
    });
  }
}
```

1. The line of code, as shown in the snippet below, illustrates the declaration of a **List** collection with the **album Ids** from which the images will be downloaded and zipped.**(Note: the values ‘album-id-1’ and ‘album-id-2’ are only placeholder values, you should replace them with those relevant to your actual context)**

```
public List<Id> albumIdList = new List<Id> { 'album-id-1', 'album-id-2' };
```

Overview of the Visualforce Page

When you access the **SharinPixDemoAlbumImageDownloadV2Page** visualforce page (found here), you will see the code as displayed below.(**Note: the JavaScript libraries have been omitted for expressiveness).**

```
<apex:page standardController="Account" extensions="SharinPixDemoAlbumImageDownloadV2Ctrl"
showHeader="false" sidebar="false" standardStylesheets="false">
<!-- OMITTED JAVASCRIPT LIBRARIES -->
    <script>
        var albums = JSON.parse('{! albums }');
        var images = [];

        function loadAlbums() {
            var albumData = [];
            $.each(albums, function(index, album) {
                albumData.push(loadAlbumImages(album, 1));
            });

            $.when.apply($, albumData).then(function() {
                downloadAndZip();
            });
        }

        function loadAlbumImages(album, page) {
            url = "https://app.sharinpix.com/api/v1/albums/" + album.id + "/images?token="
+ album.token + "&page=" + page;
            return $.get(url).then(function(response) {
  if (response.length) {
                    response.forEach(function(image) {
                        images.push({ filename: image.filename, url: image.original_url,
format: image.infos.format, data: null });
                    });
                    return loadAlbumImages(album, ++page);
                }
            });
        }

        function downloadAndZip() {
            downloadImages(function() {
                if (images.length) {
                    var zip = new JSZip();
                    deferreds = [];
                    images.forEach(function(image) {
                        var filename = image.filename + '.' + image.format;
                        deferreds.push(zip.file(filename, image.data, { binary: true }));
                    });
```

```
                $.when.apply($, deferreds).then(function() {
                    zip.generateAsync({ type: "blob" }).then(function(content) {
                        saveAs(content, 'sample'.replace(/\+/g, ' ') + ".zip");
                        $("body").append('Download complete.');
                    });
                });
            }
        });
    }

    function downloadImages(callback) {
        var deferred = $.Deferred();
        if (images.length === 0) {
            $("body").append('No images in album.');
            deferred.resolve(images);
        }
        var numImagesFetched = 0;
        images.forEach(function(image) {
            var req = new XMLHttpRequest();
            req.open("GET", image.url, true);
            req.responseType = "blob";
            req.onload = function(event) {
                image.data = req.response;
                numImagesFetched++;
                if (numImagesFetched === images.length) {
                    deferred.resolve(images);
                }
            };
            req.send();
        });
        deferred.done(callback);
    }
loadAlbums();
</script>
Starting download...<br/>
</apex:page>
```

Referring to the above code snippet, the function **loadAlbums()**, kickstarts the whole process for image downloads and eventual zipping of all these images.

IMPORTANT NOTE: The above code can be integrated with any user interface depending on your particular use case. To be able to launch the image download and zipping process, you only have to call the function loadAlbums**.**

If you want to see the image download and zipping process in action, follow the next steps:

- Transfer the Apex class **SharinPixDemoAlbumImageDownloadV2Ctrl** and Visualforce Page **SharinPixDemoAlbumImageDownloadV2Page** to your Salesforce environment.

- **Replace the album Ids.**
- Preview the Visualforce Page **SharinPixDemoAlbumImageDownloadV2Page** from *Setup* or the *Developer Console.*
- After a few moments(depending on the number of images found in the album(s)), you should see the images have been downloaded and zipped.

NOTICE

You will found in the Visualforce Page (SharinPixDemoAlbumImageDownloadV2Page), the following code snippet:

```
image.filename + Math.random().toString(36).replace(/[^a-z]+/g, '').substr(0, 5)
```

The use for the code above is to generate and append a random value at the end of each image filename. The reason behind its usage is to distinguish between images with identical filenames. Without this piece of code, the JS Libraries used in this example will overwrite (replace the initial image with the next one) the image files with duplicate filenames. In the event you are required to preserve the original filename, you must ensure the original filenames are unique and you will need to remove the following code:

```
+ Math.random().toString(36).replace(/[^a-z]+/g, '').substr(0, 5)
```

SharinPix Offline Mobile App

What is SharinPix Offline Mobile App ?

The SharinPix Offline mobile app is a native mobile application available on both platforms IOS/Android. The fundamental role of SharinPix is to evidently allow users to upload and store images inside SharinPix Albums.

In order to ensure a smooth and frictionless user experience on mobile platforms, the SharinPix Offline App serves two main purposes:

- Upload images in the background without making the user wait for the upload to reach completion.
- In the event, the device has no internet access, the SharinPix Offline Mobile App, as its name suggests, saves the images taken while the device was offline, and uploads them to their respective SharinPix Album when an internet connection has been established.
- In the event, the device loses connection in the middle of an upload, the SharinPix Offline App has the ability to save the upload progress and start the upload again right off where it reached when an internet connection has been re-established.

Where can i find the SharinPix Offline Mobile App ?

Get it on Google Play:
https://play.google.com/store/apps/details?id=com.sharinpix.SharinPix

Download on the App Store: https://itunes.apple.com/us/app/sharinpix/id1199668730?mt=8

What is SharinPix Launcher URL ?

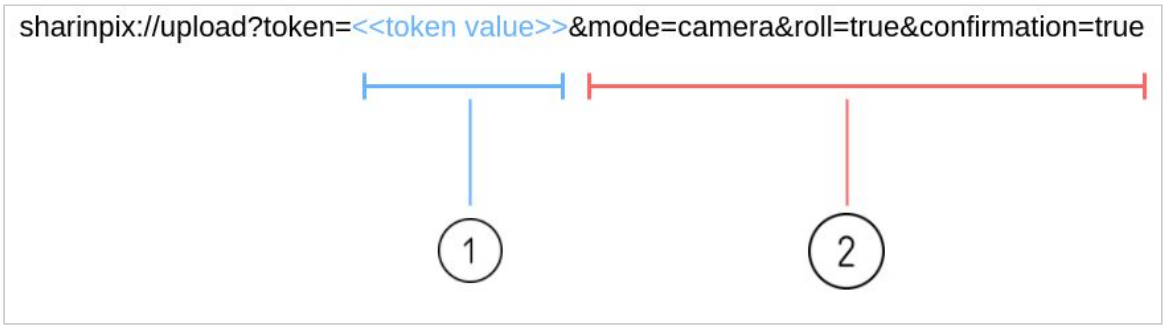
The SharinPix Launcher URL is a special URL that uses the concept of mobile deep linking to trigger the SharinPix Offline Mobile App from another app (e.g Salesforce App, Field Service Lightning App).

This proves to be useful for example in the following context:

Photos are to be taken in an area with no network connection. The SharinPix Launcher is URL used to trigger the SharinPix Offline Mobile App. The photos are shot via the latter, and when the device gains an internet access, the SharinPix Offline Mobile App uploads the images on the SharinPix Album of the correct record. The SharinPix Offline Mobile App is able to identify to which record the images are to be uploaded to with the help of the SharinPix Launcher URL which contains the Id of the record.

Structure of the SharinPix Launcher URL

Find below an explanation on the different elements that make up the SharinPix Launcher URL.



1. The **token value** refers to a long string hash of characters that contains the different specifications of an album including its id, abilities, Access permissions, actions, etc…

Creation of the token value

Token values can be created with the use of a **Trigger** that will generate a token value with the help of the SharinPix Package whenever a new record is inserted. The **Trigger** will then update a designated field with the generated token value.

The code snippet implements the WorkOrder Trigger. The whole purpose of the Trigger is to generate a SharinPix Token Value (when a new WorkOrder record is inserted) that will match the correct SharinPix Album to the newly-inserted record and insert the token value inside the **SharinPix_Token__c** custom field.

```
trigger SharinPixWorkOrderTrigger on WorkOrder (after insert, before update) {
    sharinpix.Client clientInstance = sharinpix.Client.getInstance();
    String token;
    List<WorkOrder> updatedWOrders = new List<WorkOrder>();
    for (WorkOrder wOrder : Trigger.new) {
        if (String.isBlank(wOrder.SharinPix_Token__c)) {
            token = sharinpix.Client.getInstance().token(
                new Map<String, Object> {
                    'album_id' => wOrder.Id,
                    'exp' => 0,
                    'path' => '/pagelayout/' + wOrder.Id,
                    'native_app_options' => new Map<String, String> {},
                    'abilities' => new Map<String, Object> {
                        wOrder.Id => new Map<String, Object> {
                            'Access' => new Map<String, Boolean> {
                                'see' => true,
                                'image_list' => true,
                                'image_upload' => true,
                                'image_delete' => true,
                                'image_annotate' => true
                            },
                            'Display' => new Map<String, Object> {
                                'tags'=> true
                            }
                        }
                    }
                }
            );
            if (Trigger.isInsert) {
                updatedWOrders.add(new WorkOrder(
                    Id = wOrder.Id,
                    SharinPix_Token__c = token
                ));
            } else {
                wOrder.SharinPix_Token__c = token;
            }
        }
    }
```

```
        if (Trigger.isInsert) { update updatedWOrders; }
    }
```

Referencing a token value from a Salesforce Field

Using a merge-field syntax, it is possible to reference a token value from a record field in This feature proves to be essential when launching the SharinPix Offline Mobile App as presented in the next sections.

Custom Field to be added

To construct the URL App Launcher which will be wielded to launch the SharinPix Mobile Application, a SharinPix token need to be stored before being referenced. In the present context, the custom field **SharinPix_Token__c** is used. Its properties are listed below:

i. Field Properties

Property	Value
Data Type	**Long Text Area**
Length	**32, 768**

Syntax for IOS platforms:

```
sharinpix://upload?token={!$ WorkOrder.SharinPix_Token__c }
```

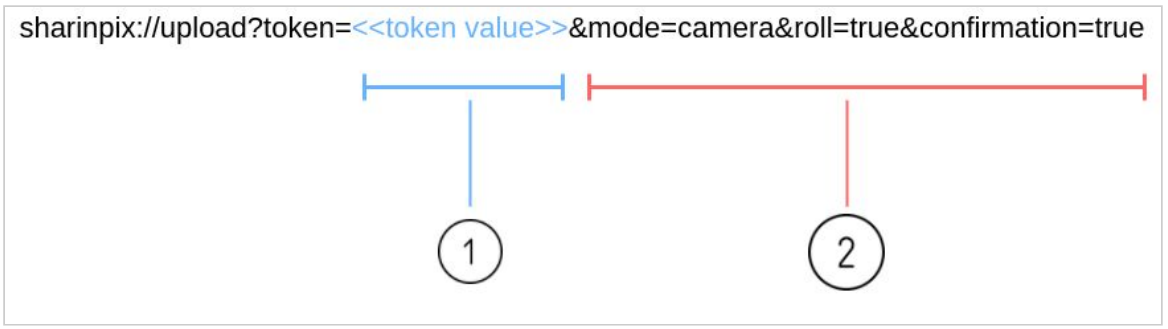
Syntax for Android platforms:

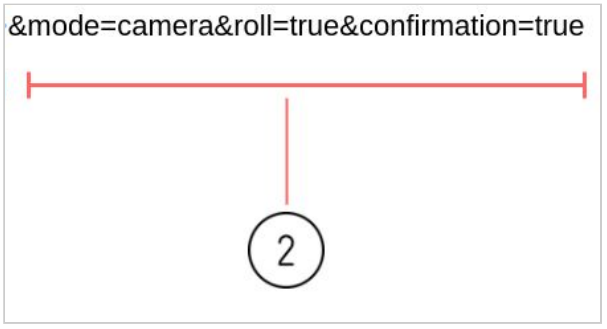
```
sharinpix://upload?token={! WorkOrder.SharinPix_Token__c }
```

Note: Pay attention to the “$” sign after the exclamation point in the IOS syntax.

In both code snippet, the custom field **SharinPix_Token__c** is referenced from the Standard Object **WorkOrder**.

2. The set of **parameters** defines how the SharinPix Mobile application will be launched.





To create a SharinPix URL that launches the SharinPix mobile application both the **token value** and the set of **parameters** are appended to the end of the following partial URL:

sharinpix://upload

These parameters are enumerated as follows:

1. **mode**

   The parameter **mode** takes two values:

   - **camera** - this mode will open the camera for the user once the application is launched.
   - **roll** - this mode will open the roll/gallery of the mobile device once the application is launched.

2. **roll**

   The parameter **roll** takes two values:

   - **true** - this value allows the user to choose from the roll/gallery of the mobile device.
   - **false** - this value DOES NOT allow the user to choose from the roll/gallery of the mobile device.

3. **Confirmation**

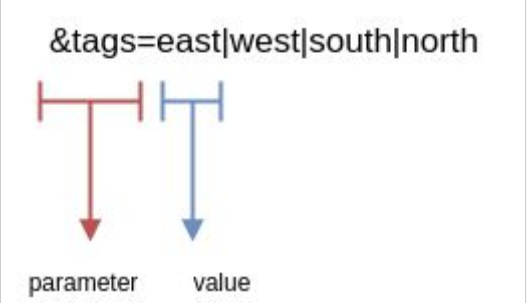
The parameter **confirmation** takes two values:

- **true -** this value previews the image on the display before uploading it.
- **false** - this value DOES NOT preview an image on the display before uploading it.

4. Tagging

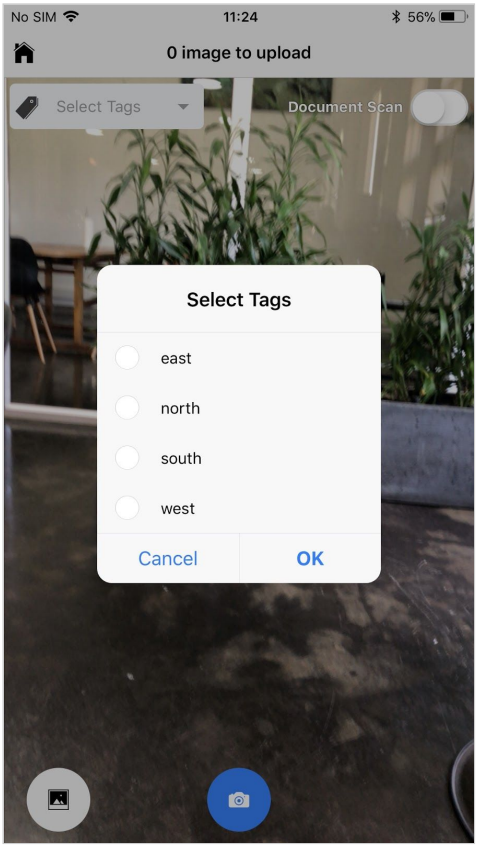
NOTE (DEPRECATION): the parameter ***auto_tag*** has been deprecated and replaced by ***auto_tags.*** It is strongly advised to stop using ***auto_tag*** as it will be rendered non-operational in the near future.

a. The parameter **tags** takes as value(s) a label value that will be available for the user to select while tagging an image when the SharinPix Url Launcher is used. The figure below shows the structure of the **tags** parameter:
b. The figure below shows the structure of the **tags** parameter:



i. The values are separated by the vertical pipe symbol “**|**”**.** If **tags** takes only one value, it is not required to use the symbol.

c. The figure below illustrates the result while accessing the **Select Tags** feature inside the SharinPix App.



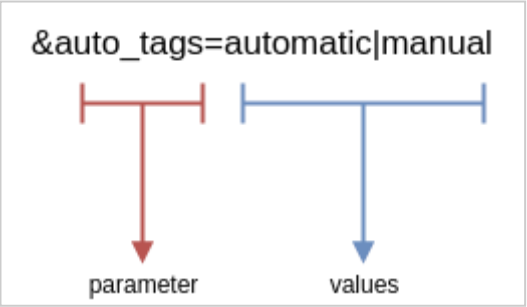
Characteristics of parameter **tags:**

- Un-selected by default
- Can be selected or deselected by the user.

5. Auto Tagging

d. The parameter **auto_tags** takes as value(s) a label value that will be automatically **assigned** to all images in the SharinPix Album when the SharinPix Url Launcher is used. The figure below shows the structure of the **auto_tags** parameter:

e. The figure below shows the structure of the **auto_tags** parameter:



i. The values are separated by the vertical pipe symbol “**|**”**.** If **auto_tags** takes only one value, it is not required to use the symbol.

IMPORTANT: *If the values contain special characters (i.e characters that are not on this list: https://www.w3schools.com/charsets/ref_html_ascii.asp), you need to* ***URL Encode*** *those special characters.* ***URL encoding*** *means converting special characters into a format that can be transmitted over the Internet.*

Example: if you want to use ***éssai*** as a tag value, you need to **url encode** it first as it contains a special character(**é**). There is a handy tool you could use to do that: https://www.urlencoder.org/ .

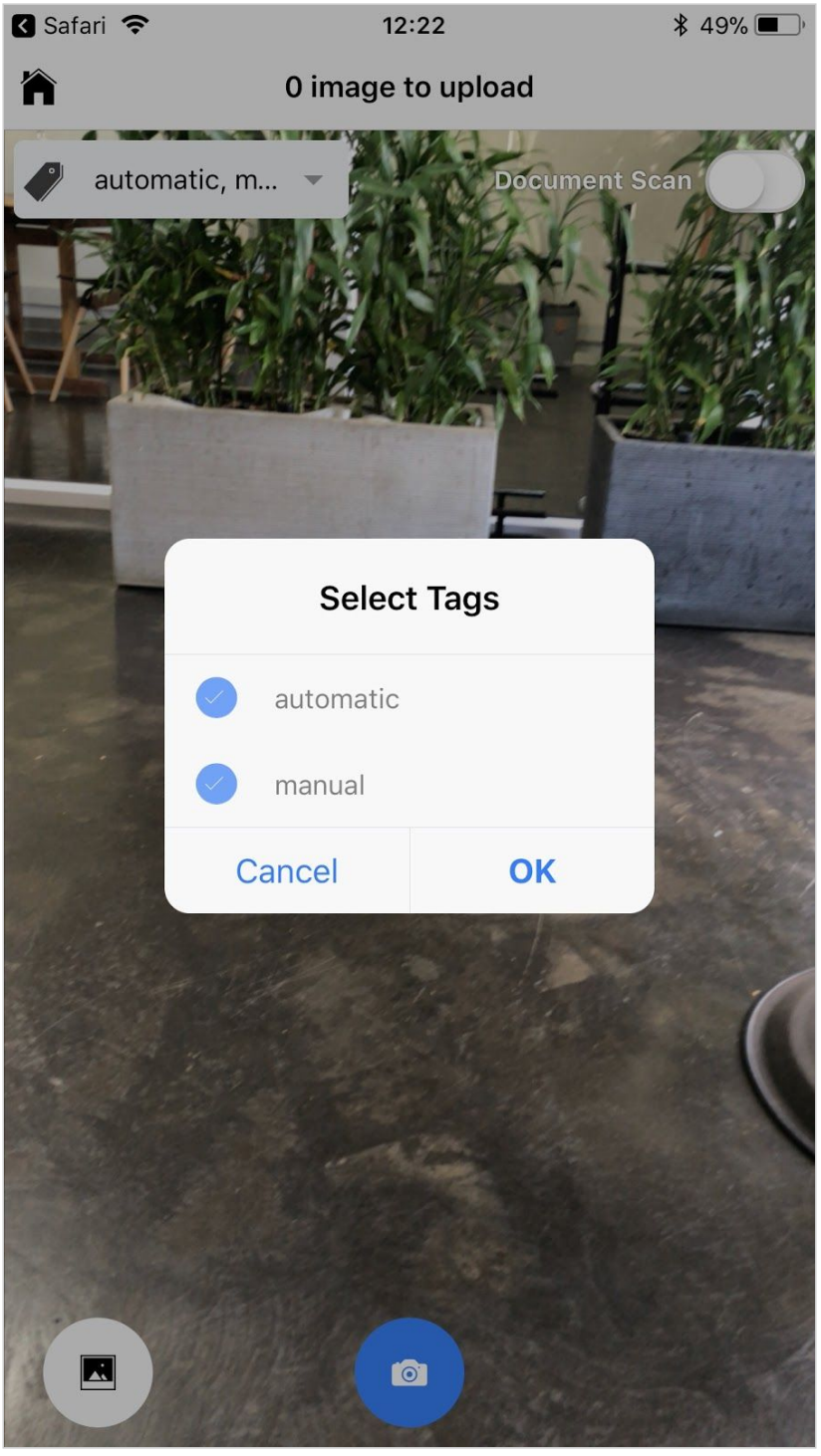
Before URL encoding**:**

éssai

After URL encoding**:**

%C3%A9ssai

f. The figure below illustrates the result while accessing the **Select Tags** feature inside the SharinPix App.



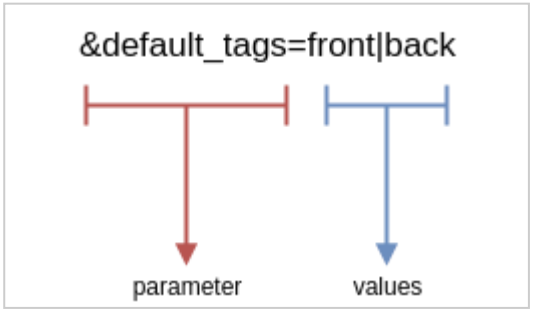
As it can be seen in the above figure, the tags **automatic** and **manual** are automatically selected and cannot be deselected by the user.

Characteristics of parameter **auto_tags:**

- Selected automatically by default
- **Cannot** be deselected by the user.

6. Default Tagging

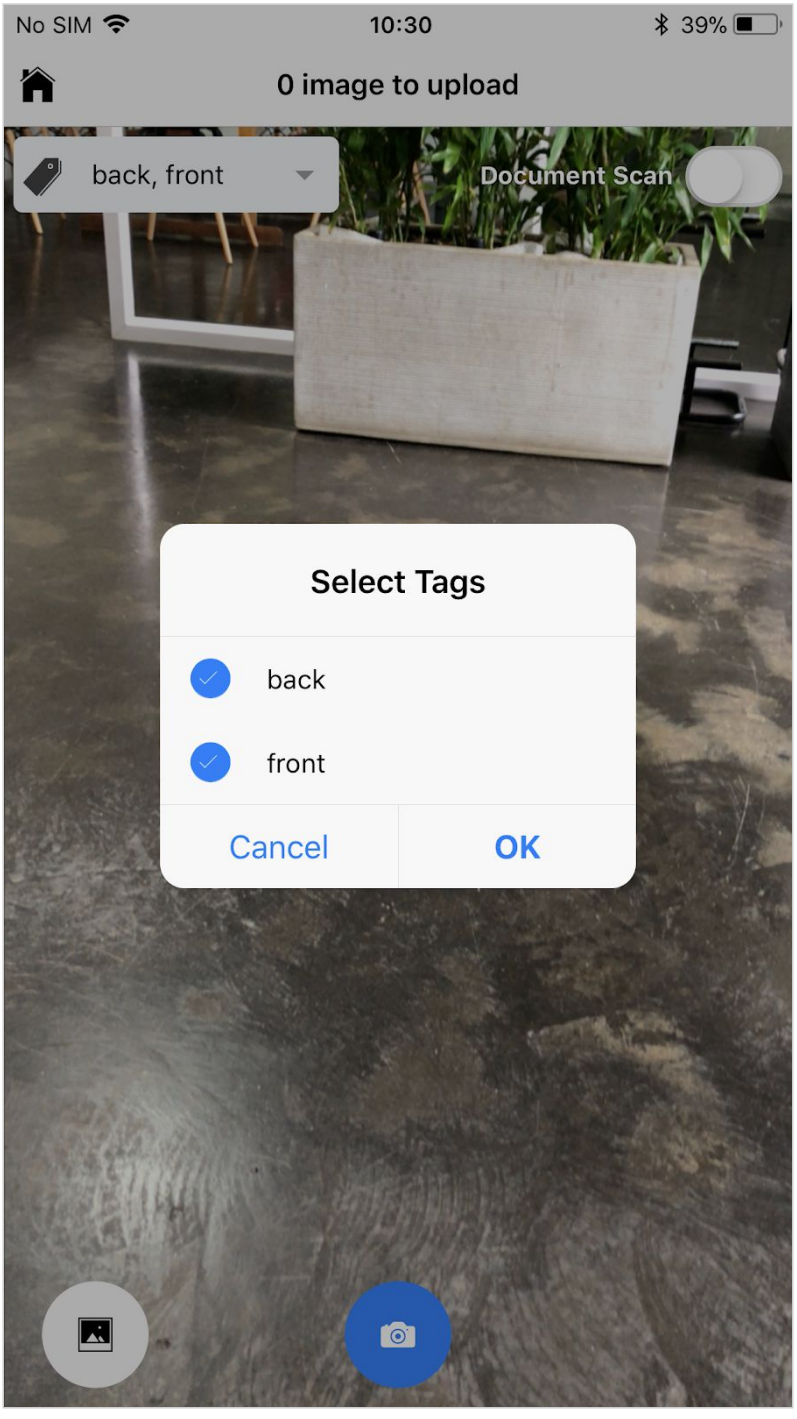
The parameter **default_tags** takes as value(s) a label value that will be automatically **selected** when the SharinPix Url Launcher is used. The figure below shows the structure of the **default_tags** parameter:



The figure below shows the structure of the **default_tags** parameter:

The values are separated by the vertical pipe symbol “**|**”**.** If **default_tags** takes only one value, it is not required to use the symbol.

The figure below illustrates the result while accessing the **Select Tags** feature inside the SharinPix App.

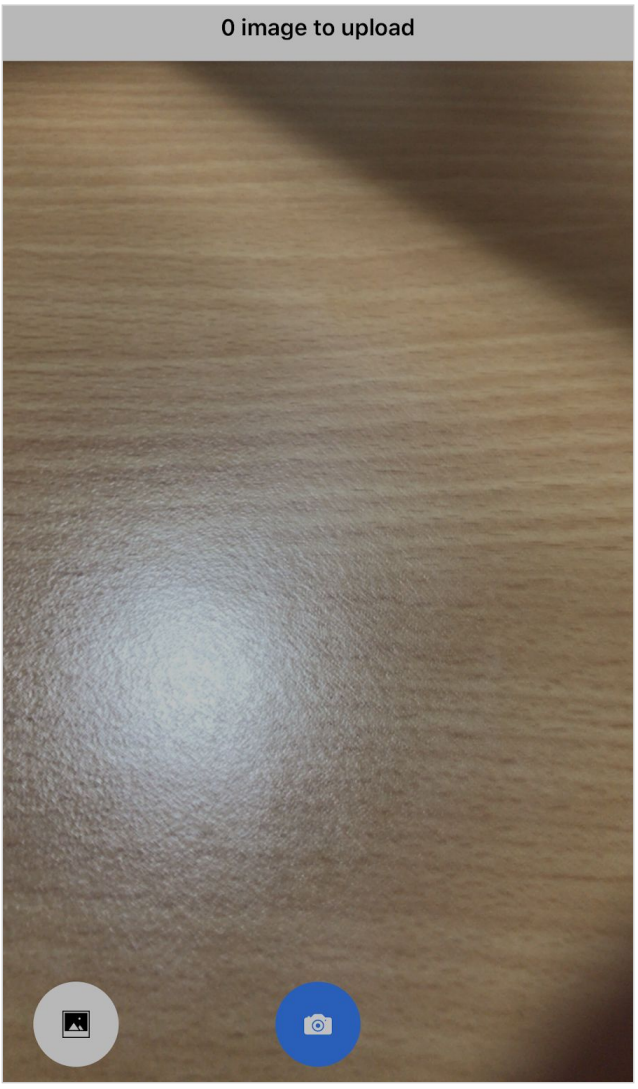
g. 

As it can be seen in the above figure, the tags **front** and **back** are automatically selected.

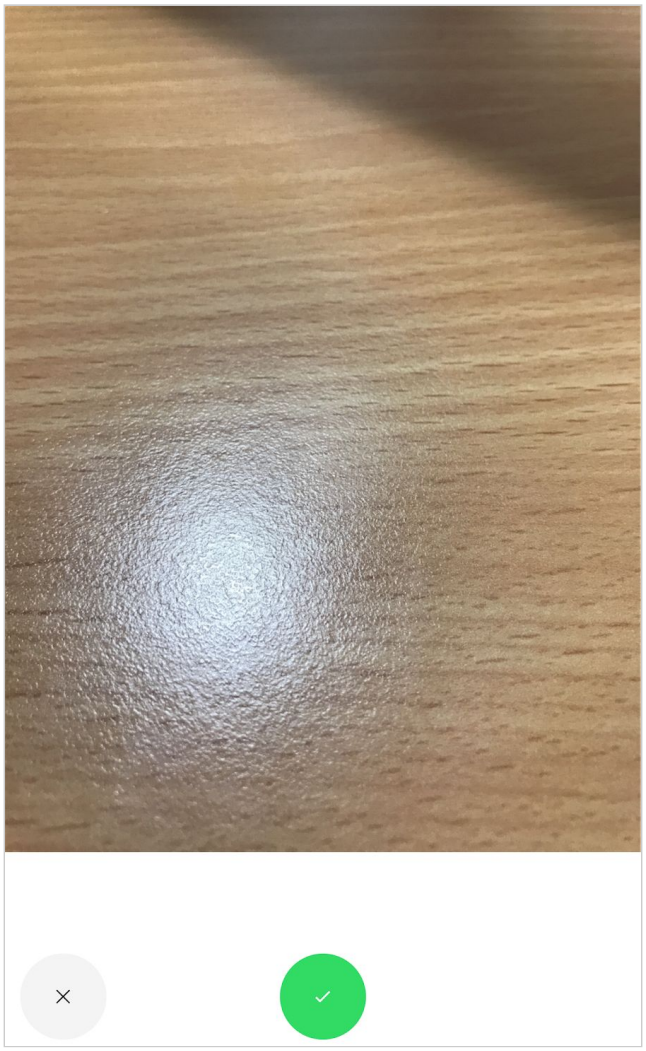
Characteristics of parameter **default_tags:**

- Selected automatically by default
- Can be selected/deselected by the user.

The figure below shows the camera behaviour when the **mode** parameter is set to **camera.**



After snapping a picture, a preview will be displayed if the parameter **confirmation** is set to **true**.



Integrating SharinPix with other Apps

1. Field Service Lightning Mobile App
    a. The steps on how to launch the SharinPix Offline Mobile App from the FSL Mobile App can be found in the section: Integrating SharinPix with FSL Mobile App
2. Salesforce Mobile App
    a. The steps on how to launch the SharinPix Offline Mobile App from the Salesforce Mobile App can be found in this section Launching SharinPix Offline Mobile App from Salesforce Mobile App
3. Integration with 3rd-party apps
    a. SharinPix offers the possibility to extend its image-gathering and image support features as it is able integrate itself with other Salesforce Appexchange products. The core principle of this integration revolves around the same concept covered in the two sections mentioned above. This concept can be reduced as such:
        i. The application needs to retrieve the SharinPix Mobile Launcher URL from a custom field.
        ii. Display the Launcher URL within a clickable UI component.

    Don’t hesitate to contact us (info@sharinpix.com) to ask for feedback on any app integration that you may have in mind.

Launching SharinPix Offline Mobile App from Salesforce Mobile App

The present section will enumerate the steps required in order for the SharinPix application to be launched from the **Salesforce Mobile App**.

The objective of the following demo is to launch the SharinPix Mobile App through the use of an **Custom Action** from a **Case** object.

1. Implementation of the Visualforce Page

The **Action Type** of the **Custom Action** will be **Custom Visualforce,** within which a link will be used to launch the SharinPix Mobile App.

The code snippet implementing the Visualforce Page is shown below.

```
<apex:page standardController="Case" showHeader="false" sidebar="false"
applyBodyTag="false">
    <a href="sharinpix://upload?token={! Case.SharinPix_Token__c}">Open SharinPix App</a>
</apex:page>
```

As seen above, the link contains the following content:

```
<a href="sharinpix://upload?token={! Case.SharinPix_Token__c}">Open SharinPix App</a>
```

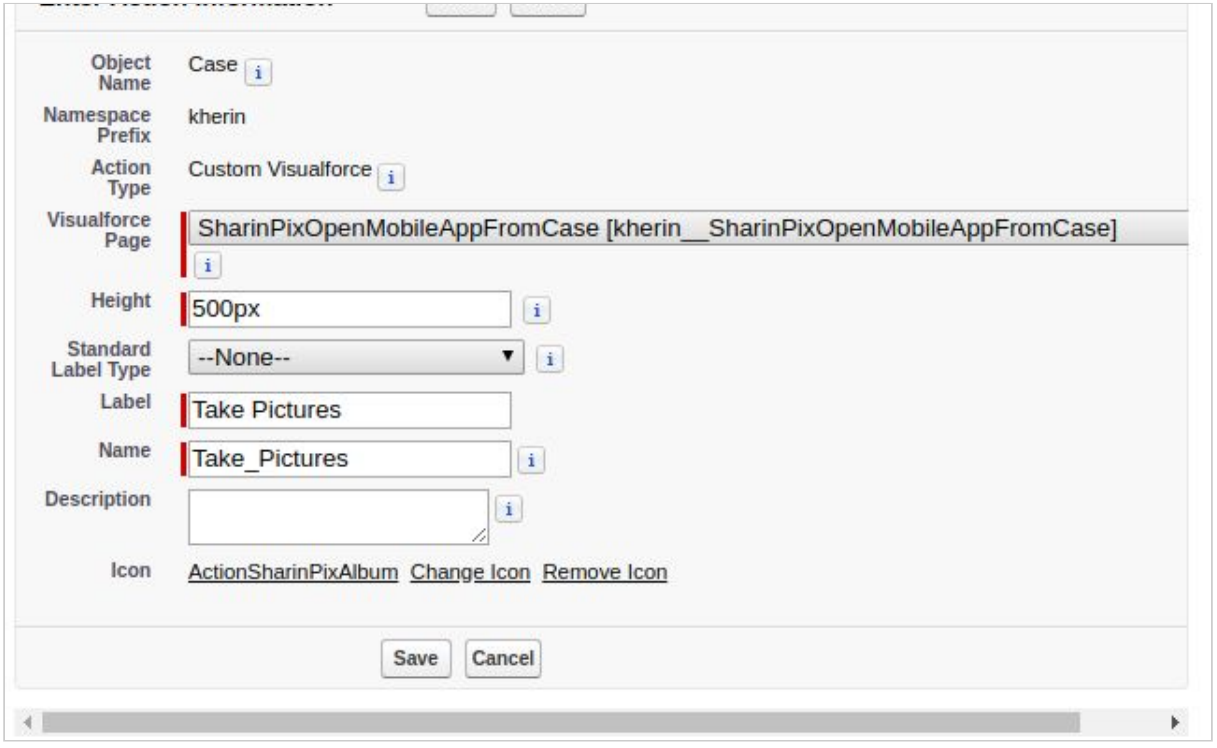
The merge-field {! Case.SharinPix_Token__c} references the SharinPix Token field containing the token value present on the current **Case** object.

Note: If you want this Visualforce Page to work on any Object, you just have to replace Case with the API Name of your Object.

Save the Visualforce Page as **SharinPixOpenMobileAppFromCase**.

2. Add action

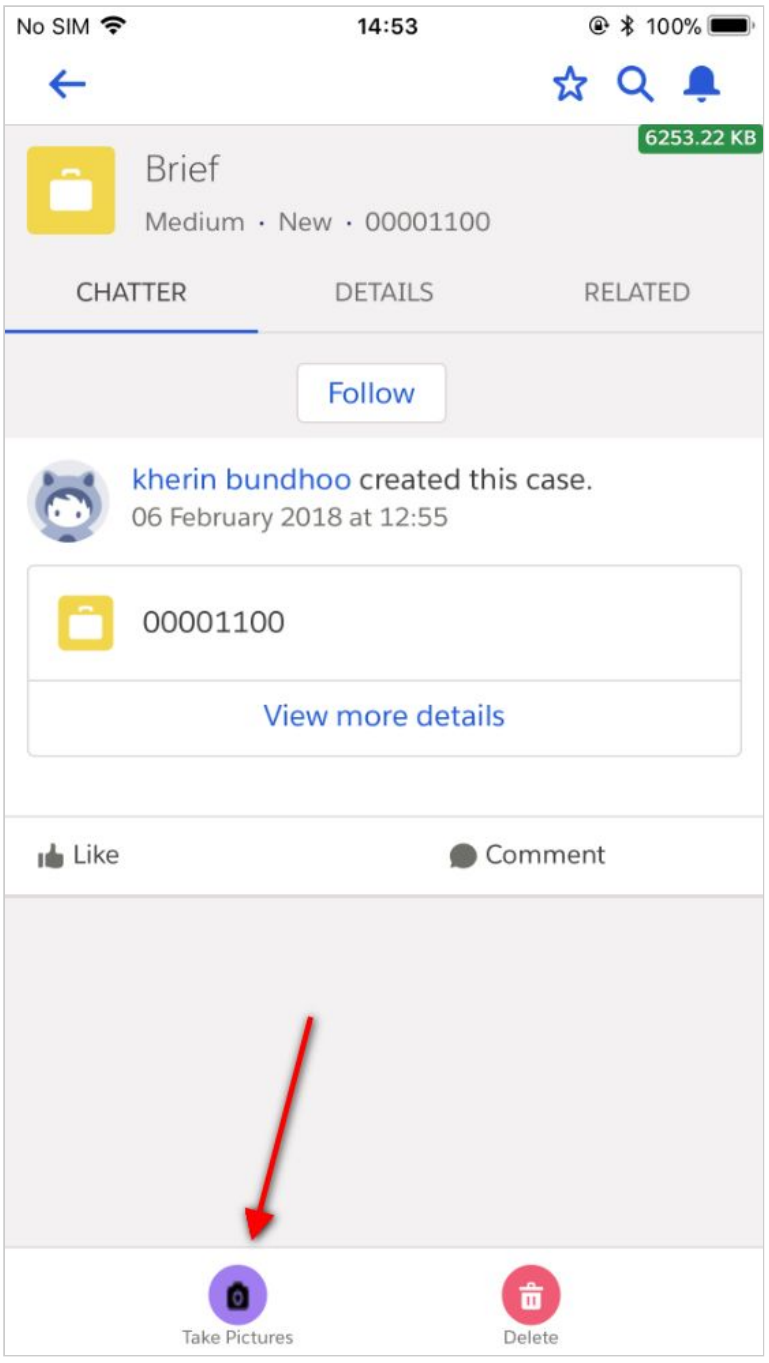
- Go to Setup -> Object Manager and access the Case Object.
- Select the **Buttons, Links, and Actions** section.
- Click on **New Action**.
- In the picklist **Action Type**, select **Custom Visualforce**.
- In the picklist **Visualforce Page,** select the Visualforce Page created above.
- In the field **Height**, enter 500px.
- In the field **Label,** enter **Take Pictures**.
- In the field **Icon,** select **ActionSharinPixAlbum** icon (which comes with the installed SharinPix Package).



- Click on **Save** when done.

3. Add action to page-layout of Case.

- Go to the **Case Page Layouts** section.
- Select the layout relevant to your context.
- From **Mobile & Lightning Actions**, drag and drop **Open SharinPix** inside the **Salesforce mobile and Lightning Experience Actions** section.
- Click on **Save** when done.
- Access a **Case** record on the **Salesforce Mobile App.** The custom action **Take Pictures** can be found on the **Action Bar** as shown in the image below.

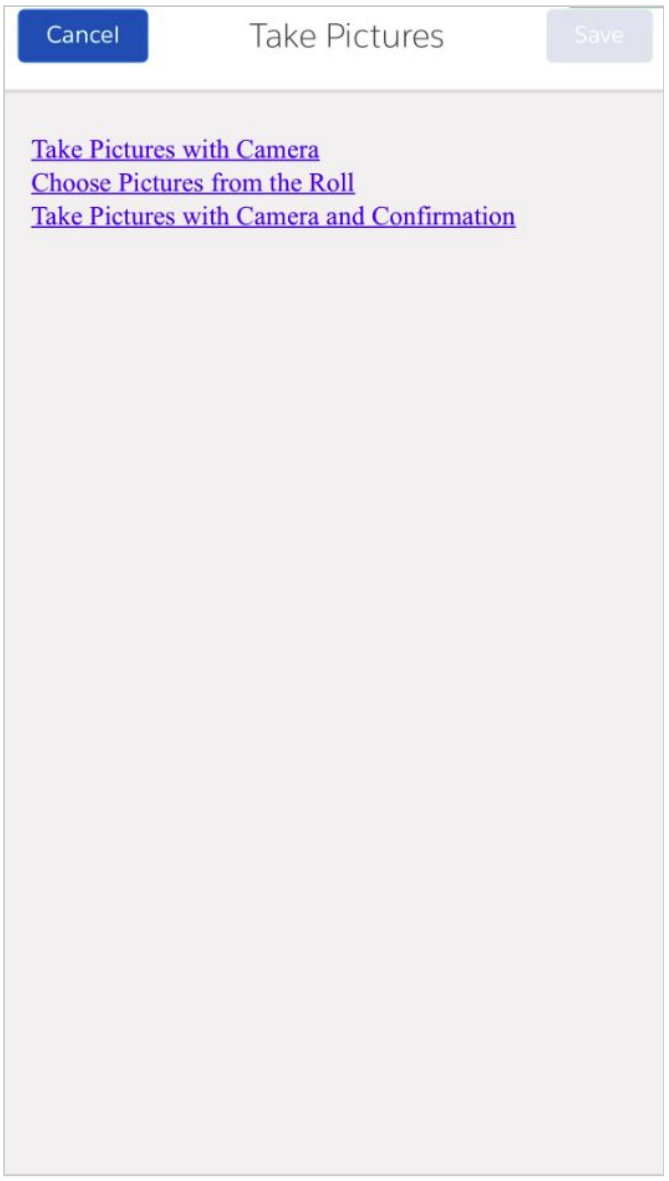


- Tap on Take Pictures. The Visualforce Page **SharinPixOpenMobileAppFromCase**, as implemented previously,is displayed.

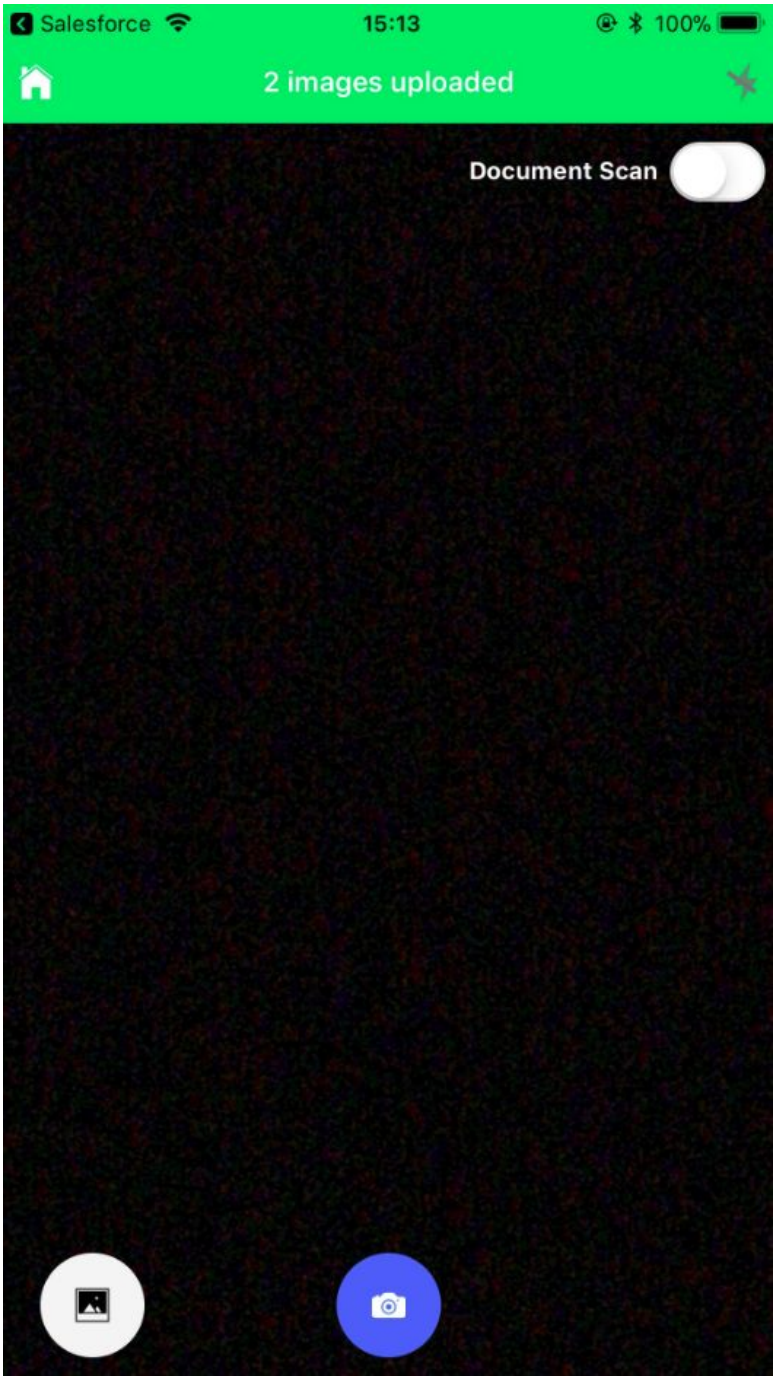
Each link corresponds to a SharinPix URL that will launch a particular camera mode:

- Launch Camera only
- Launch Image Roll only
- Launch Camera and enable confirmation prompt.

Refer to the Launch SharinPix Offline Mobile App section for a more in-depth view into the different camera modes available in the SharinPix Offline Mobile App.


Cancel
Take Pictures
Save
Take Pictures with Camera
Choose Pictures from the Roll
Take Pictures with Camera and Confirmation

- If you tap on the **Take Pictures with Camera** link, the SharinPix Mobile App will launch and the camera view will open by default.



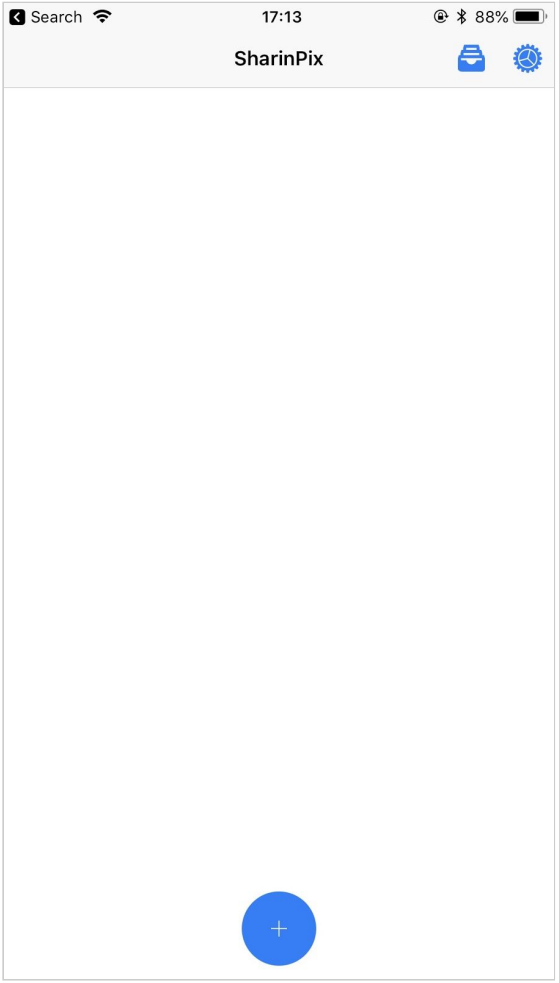
Using Jobs in SharinPix Mobile App

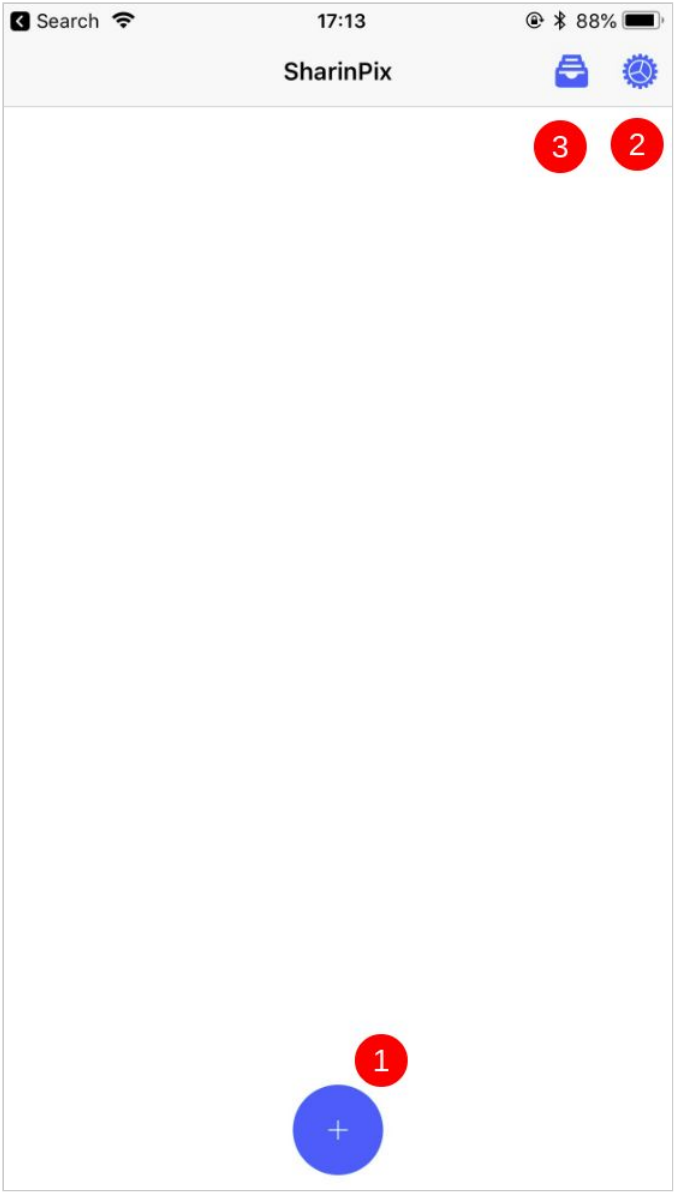
1. What is a Job ?

The meaning of a **Job** in the context of the SharinPix Mobile Application is simply the equivalent of an album that groups images together.

2. Create a Job manually

- Launch the SharinPix Mobile Application
- If there are no Jobs already created, the Home menu of the application should show as below.





1. Create a new Job
2. View Settings
3. View Archived Jobs

Create new Jobs

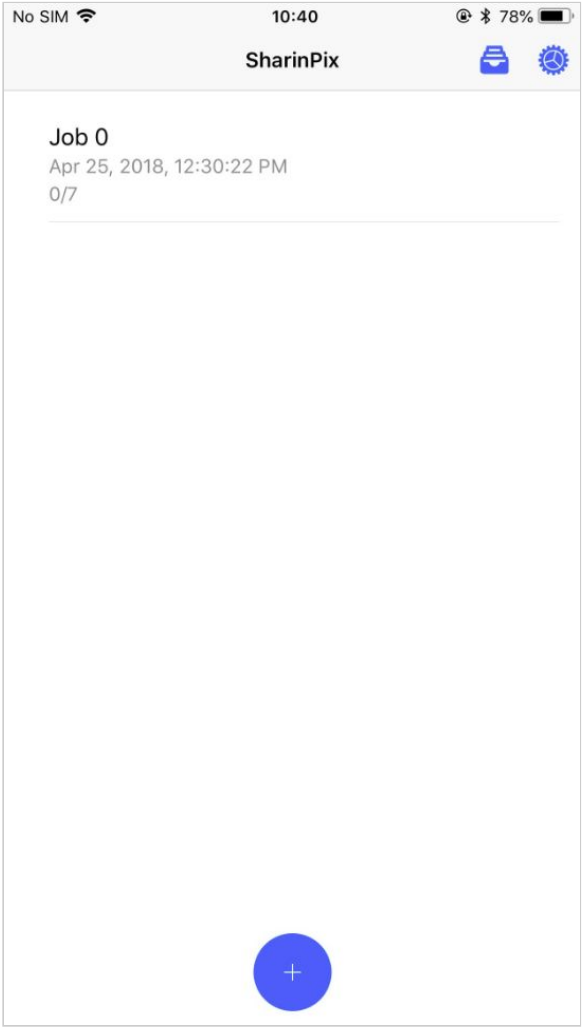
- In the event, that the SharinPix Mobile Application has no immediate internet access, it is still possible to create a Job manually and add images to it.
- Once the internet access is re-established, the images found in the job will be uploaded in the background.
- You are also able to access the images uploaded and view the status of a Job by following the steps below.
- You will still be able to add more images to the Job.

3. Archiving a Job

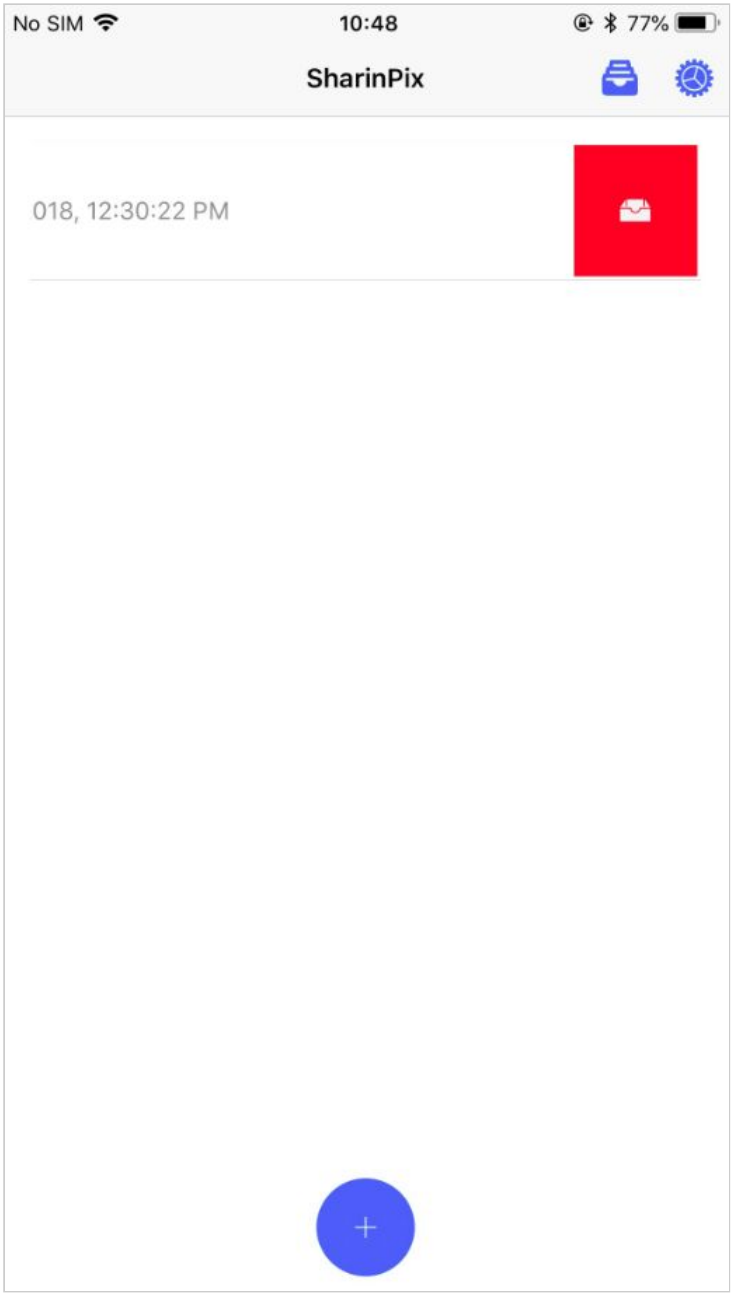
- What is “archiving a job” actually mean ?

Archiving a job simply means to remove the display of ongoing jobs from the main menu.

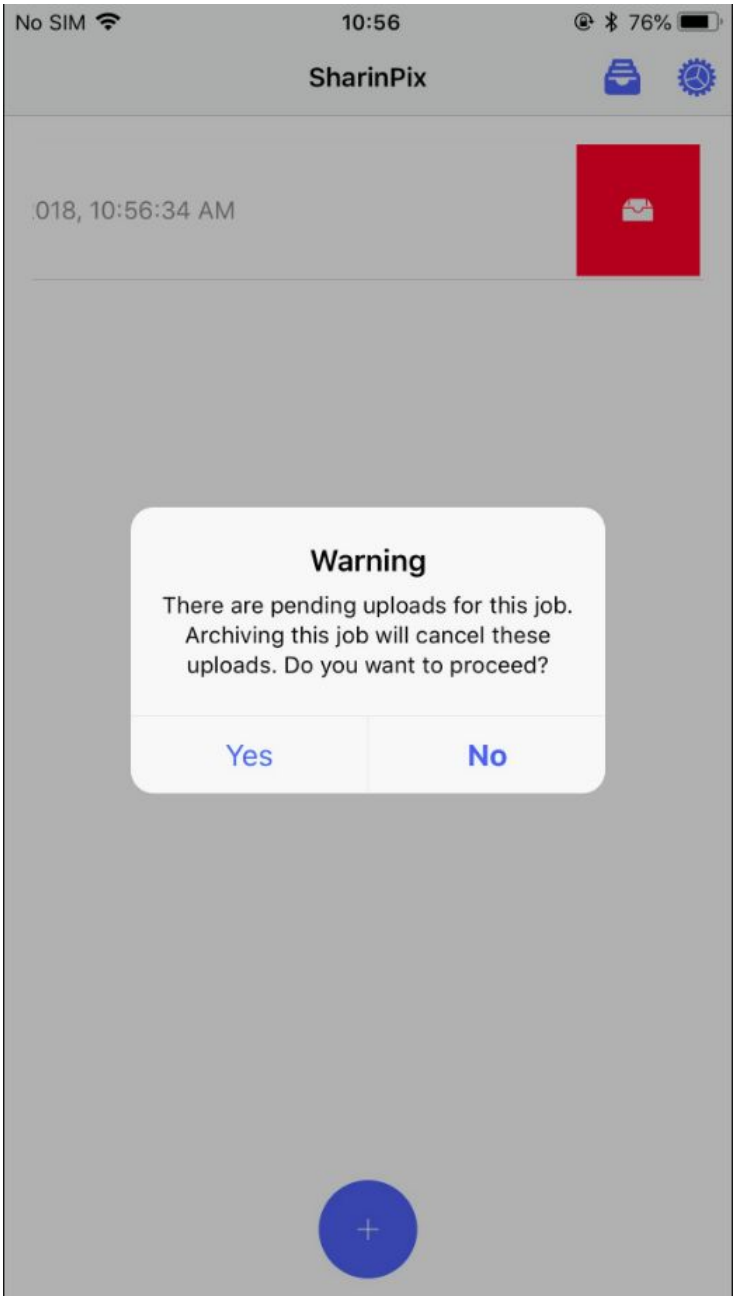
The screenshot below shows the **Home Screen** of the SharinPix Mobile Application, displaying ongoing jobs.



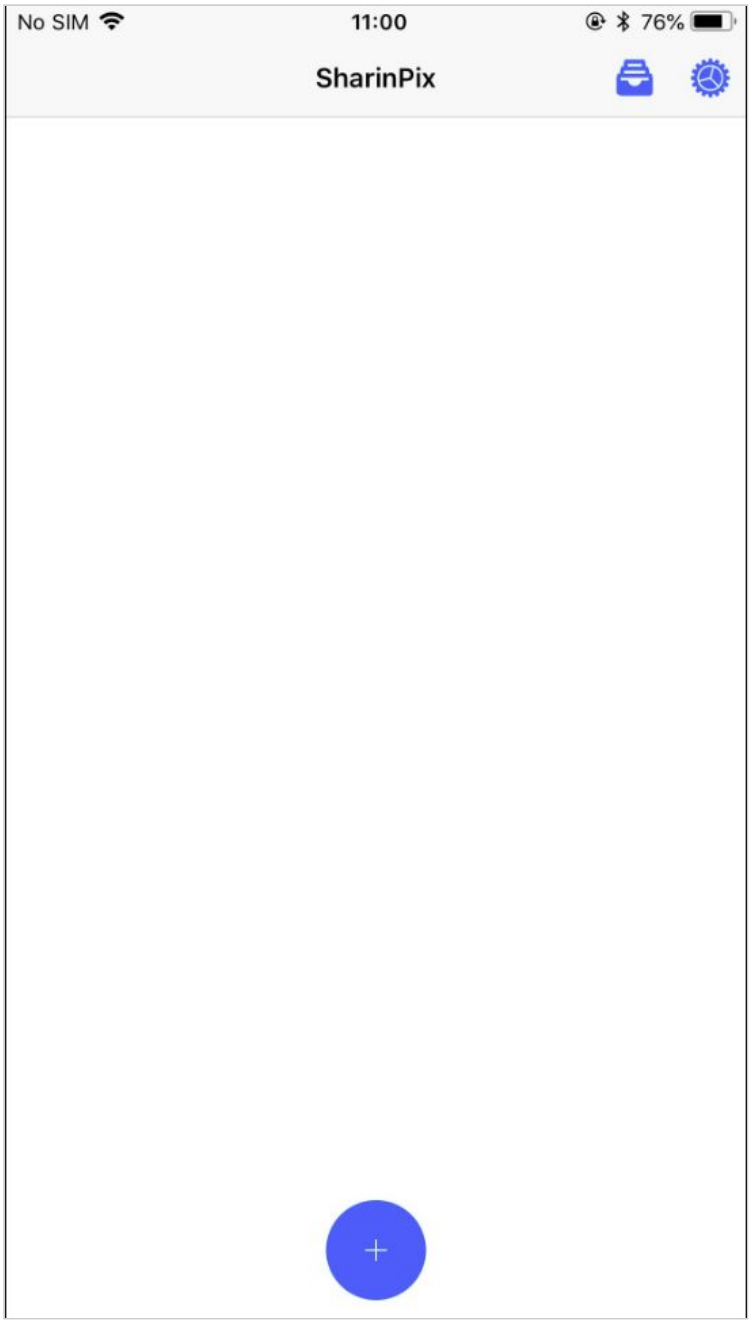
When the Job Item is **swiped to the left**, you will be able to select the **archive job** feature.



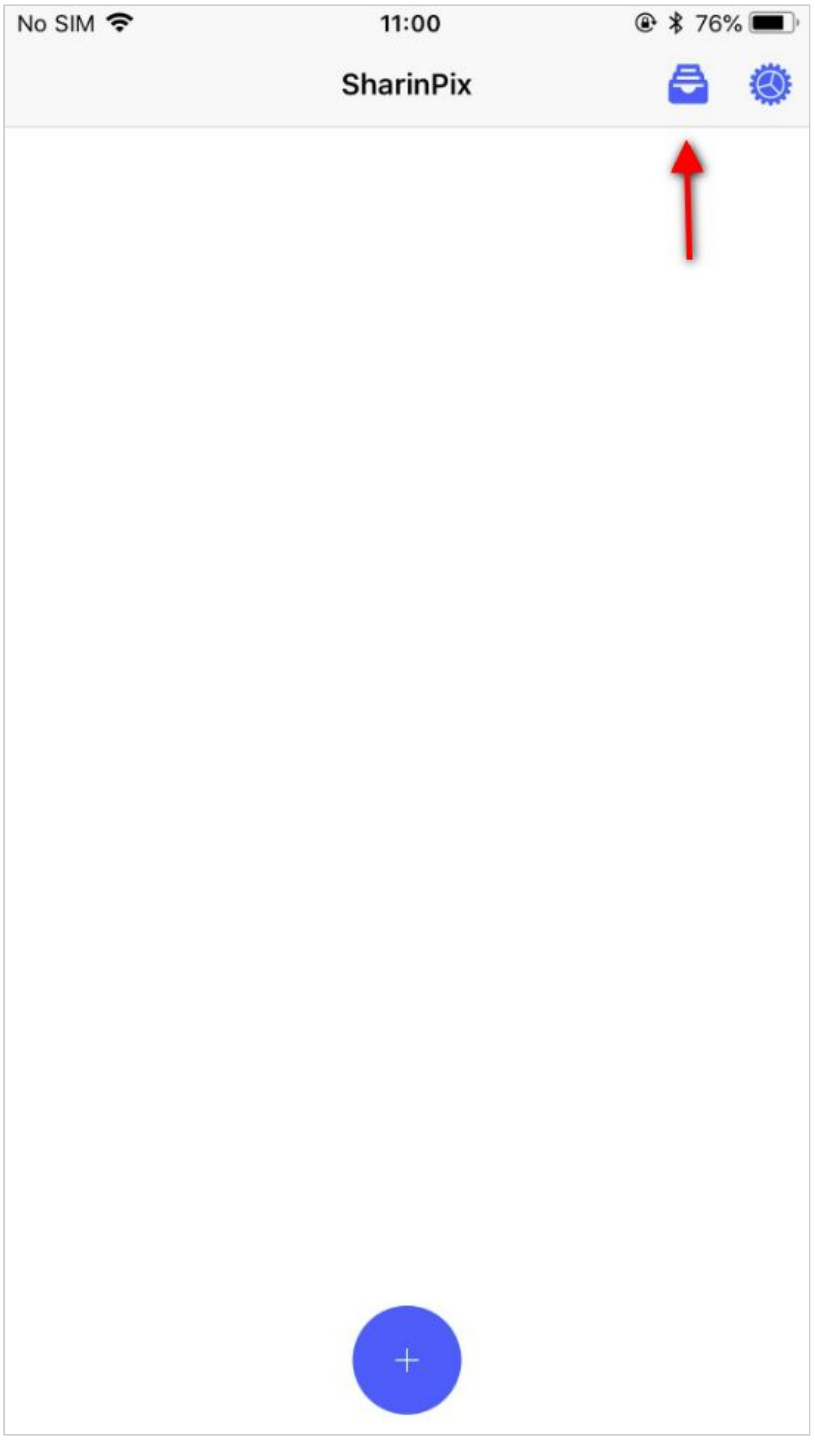
You will then receive a warning message prompting for your confirmation before proceeding to archiving the job. Click on **Yes** to proceed.



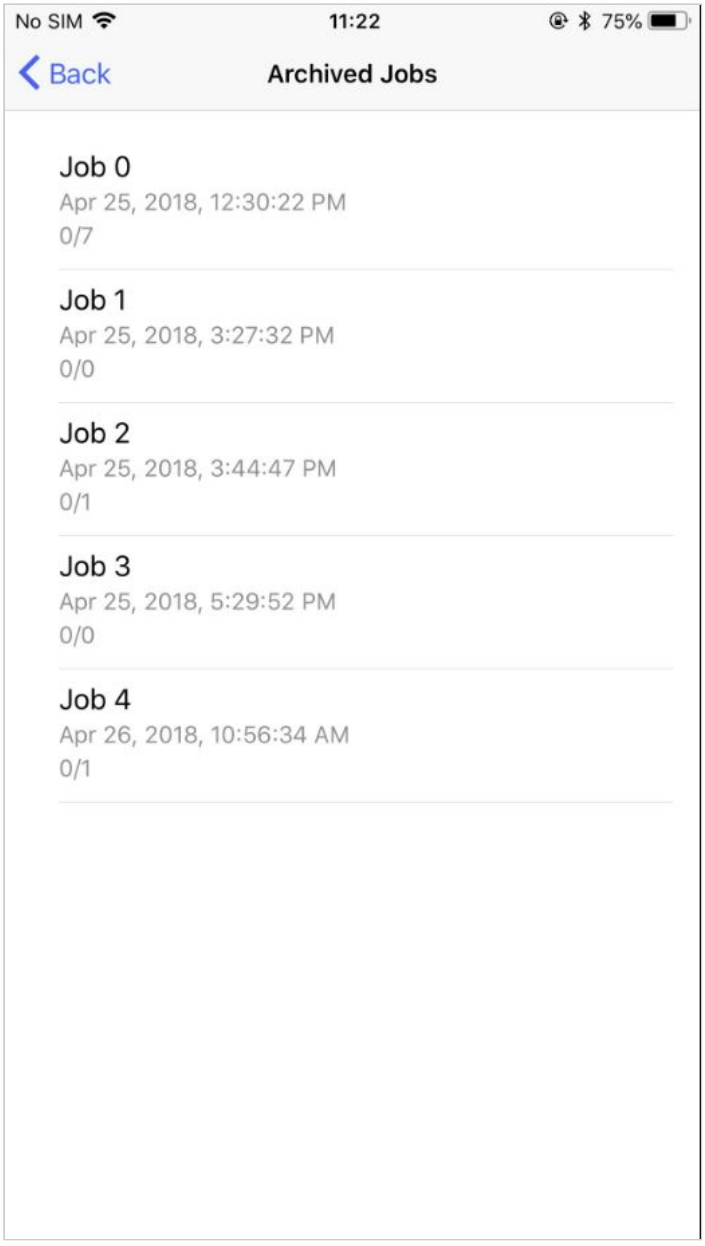
After the archiving the job, the **Home Screen** is cleared as shown below.



It is possible to access the archived job(s), by tapping on the **archived jobs** icon as shown below.

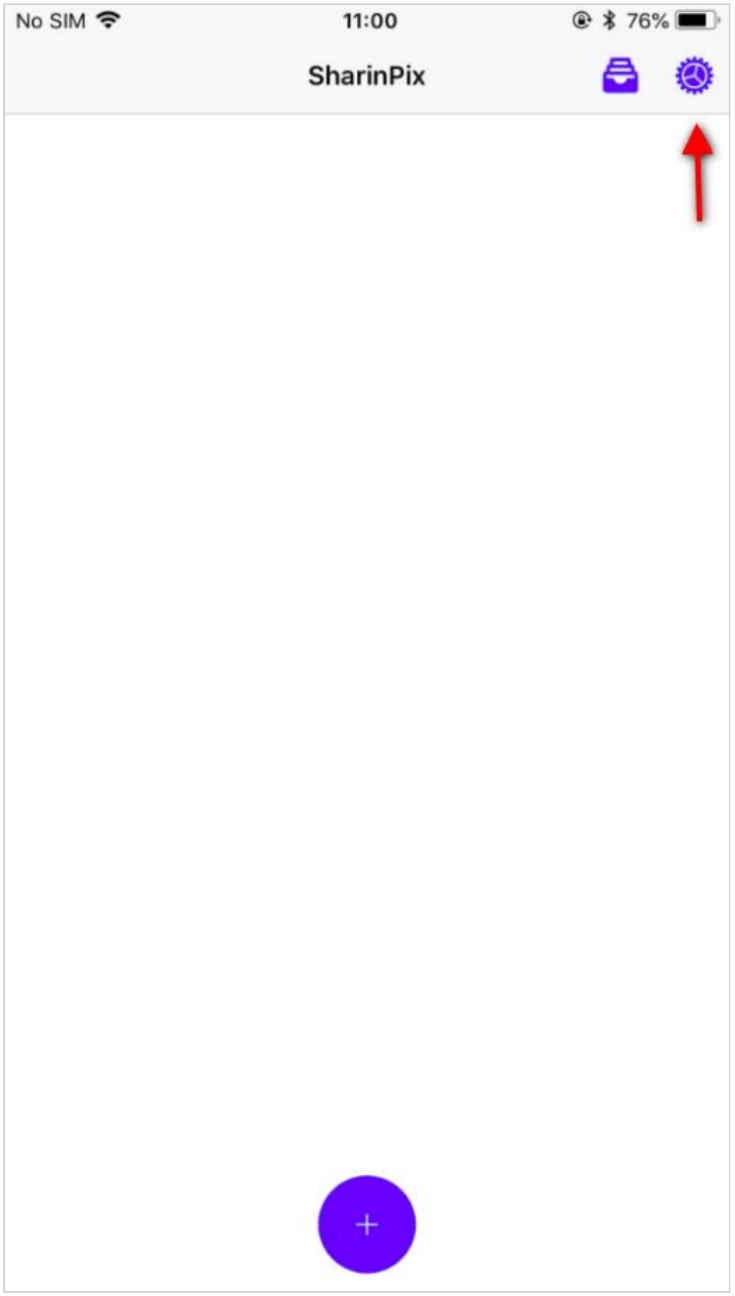


The **archived jobs screen** will then appear, listing all the archived jobs as shown below.

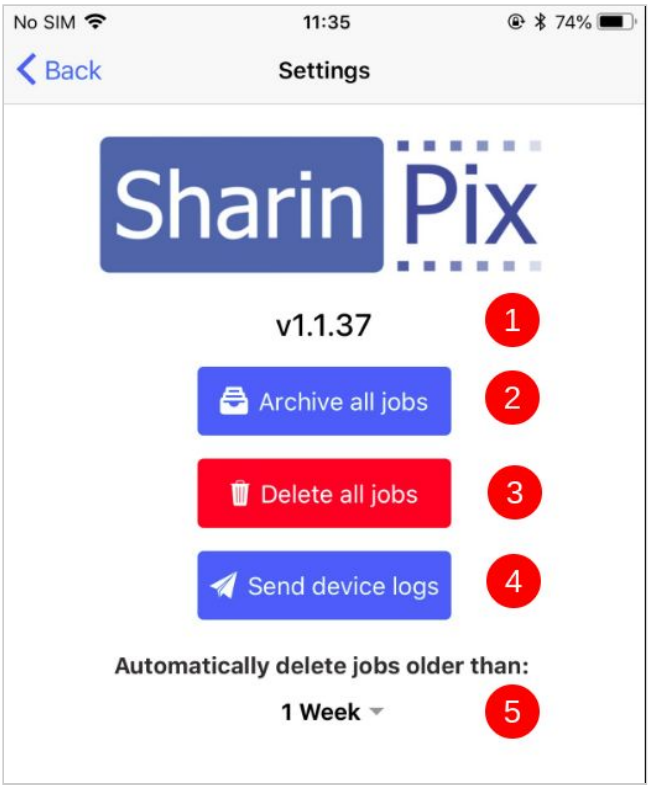


4. Accessing the Settings of the SharinPix Mobile Application

To access the **Settings screen** of the SharinPix Mobile Application, click on the **Settings icon** as shown in the screenshot below.



You will then get access to the **Settings screen** containing the following elements:



1. **SharinPix Mobile App Version Number**
    a. This simply displays the current version number of the SharinPix Mobile Application.
2. **Archive all jobs**
    a. This feature allows you to archive all ongoing jobs.
3. **Delete all jobs**
    a. This feature allows you to delete all **jobs**.
4. **Send device logs**
    a. This feature allows you to send report whenever you encounter a bug/issue/problem with the use of the SharinPix Mobile Application.
5. **Schedule automatic deletion of jobs**
    a. This feature allows you to specify the amount of time after the created date of the job before deleting it from the **archived list.**
    The available time ranges are:
        - 1 week
        - 2 weeks
        - 3 weeks
        - 4 weeks

Integrating SharinPix with FSL Mobile App

The present section will present how using SharinPix alongside the FSL Mobile App enables a frictionless experience for the mobile workforce facing situations where they are required to perform their field service work even without a web access!

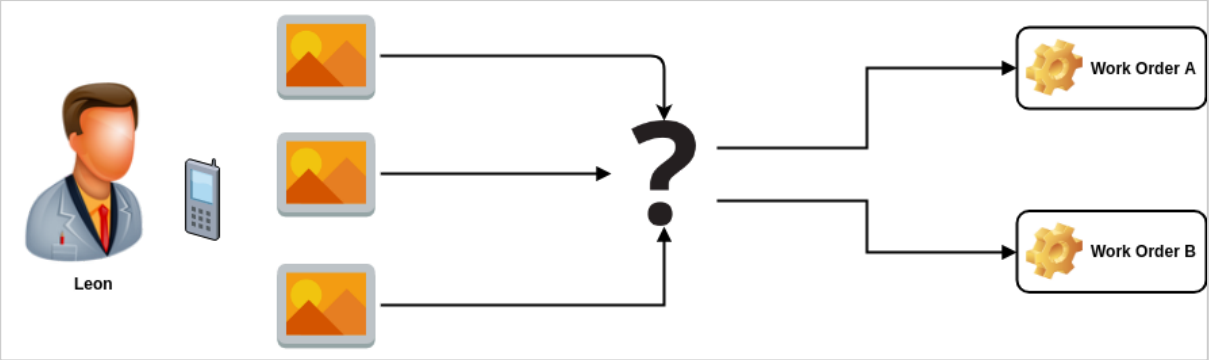
To better grasp how SharinPix can enhance your Field Service Management system, let’s take a look at a case study where SharinPix takes center stage.

Case Study: **CloudDeco**

Leon is a mobile technician working for CloudDeco, a company specialised in professional home decoration. On a daily basis, he is dispatched to the home’s of potential clients where is required to take pictures of the rooms where the clients wish to apply CloudDeco’s service.
These pictures will then help the decorators and designers back in CloudDeco’s office to propose a first sketch to the clients.

In order to coordinate field operations which include scheduling service appointments, dispatching mobile technicians and tracking appointment status, CloudDeco decided to make use of **Salesforce Field Service Lightning.** The impact on their field service management system was immediately beneficial. Each visit to a client’s home corresponded to **WorkOrder** record. However, Leon still faced some difficulties while performing his duty when using the FSL Mobile application. Those difficulties were as follows:

- When taking pictures on his phone, he was having a hard time referencing the correct **WorkOrder record** to the images he took. This lead to confusion as to which images belonged to which WorkOrder.



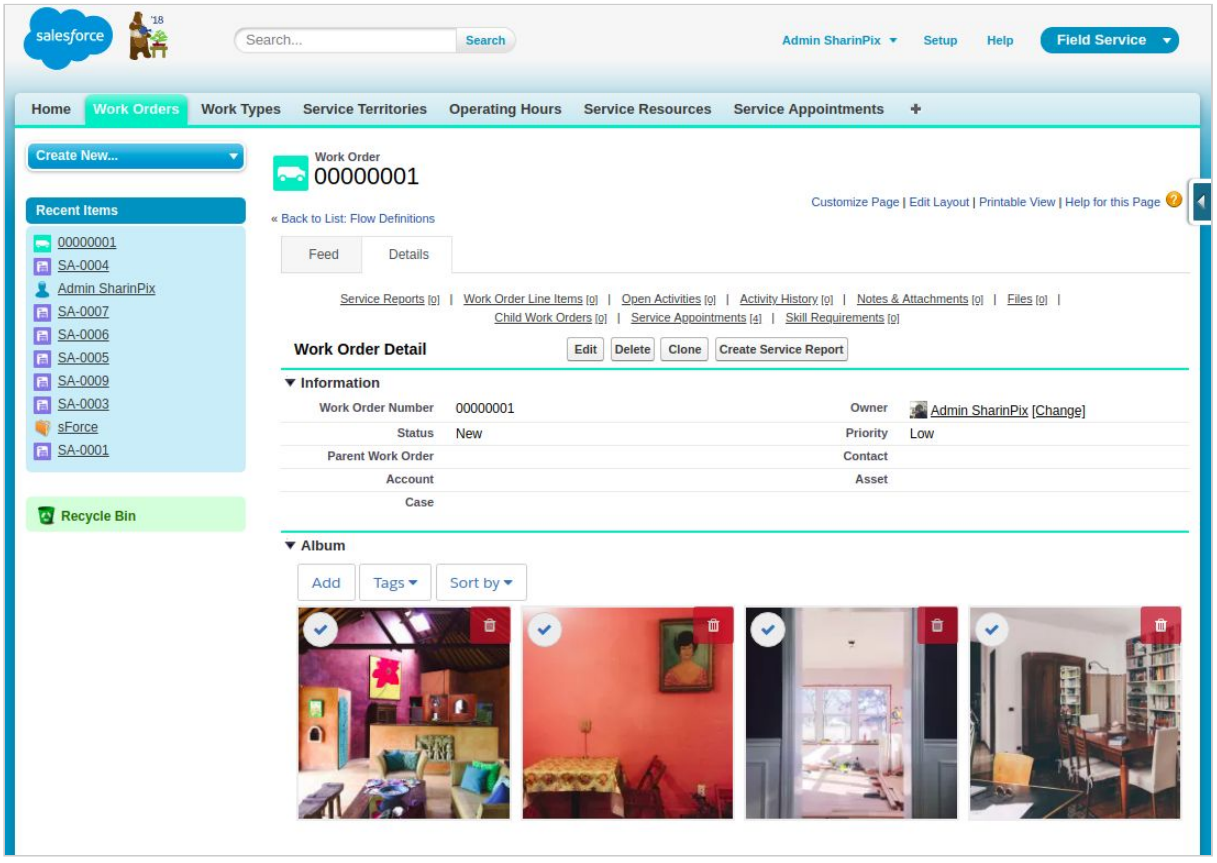
- Leon did not have any means to identify and label a picture. For example, this proved to be difficult for the designers to determine whether a room was a family room or another room.

How can SharinPix solves those problems?

By making use of the SharinPix Mobile application, Leon is able to take images for a given related Work Order record even without web access.

What are the benefits of using the SharinPix Mobile Application?

- **Take pictures. Reference WorkOrder record.**
  - The SharinPix app eases Leon’s workload as the pictures are automatically synced to the correct WorkOrder record.



- **Taking pictures have never been that fast**.
  - Leon is able to take pictures when there no internet connection. And the SharinPix app will upload them automatically in the background when an internet connection is established again.

This is how Leon is able to this:

- Whenever he needs photos for a corresponding Work Order, he launches the SharinPix App from this same record and take the pictures.
- If he is in a situation with no internet connectivity, the SharinPix app saves the photos and synchronize with FSL when the device comes online again.

To be able to launch SharinPix app from the Field Service Mobile application, just like Leon did, it is possible to make use of two methods:

- **App Extension**
- **Visual Flow**

Implementation using App extensions

Establishing a connection between the FSL mobile app and SharinPix requires the use of **App extensions.**

App extensions collectively represent an easy and convenient method that allows the users to pass data from the **FSL mobile app** to another app (in this context: **SharinPix**).

The subsequent sections will show a demo on how to create an app extension that will launch the **SharinPix application** from the **FSL mobile application**.

Assumptions

- Field Service Lightning has already been set up.
- Field Service Lightning Mobile Settings has already been configured.

If any prerequisites are missing, access the resources below:

- Enable Field Service Lightning.
- Configure FSL Mobile Settings.

Objectives of the demo

- Retrieve the **SharinPix token** value from a custom field on a **Work Order** record.
- Add a new **App Extension** in **Field Service Lightning Mobile Settings**.
- Launch **SharinPix** from the **FSL Mobile Application.**

1. Retrieve a SharinPix token

b. Add a custom field **SharinPix Token** on the object WorkOrder.

i. Field Properties

Property	Value
Field Label	**SharinPix Token**
Field Name	**SharinPix_Token**
Data Type	**Long Text Area**
Length	**32, 768**

c. Implement the **WorkOrder Trigger** using the code snippet found below. The whole purpose of the Trigger is to generate a SharinPix Token Value (when a **new** WorkOrder record is inserted) that will match the correct SharinPix Album to the newly-inserted record.

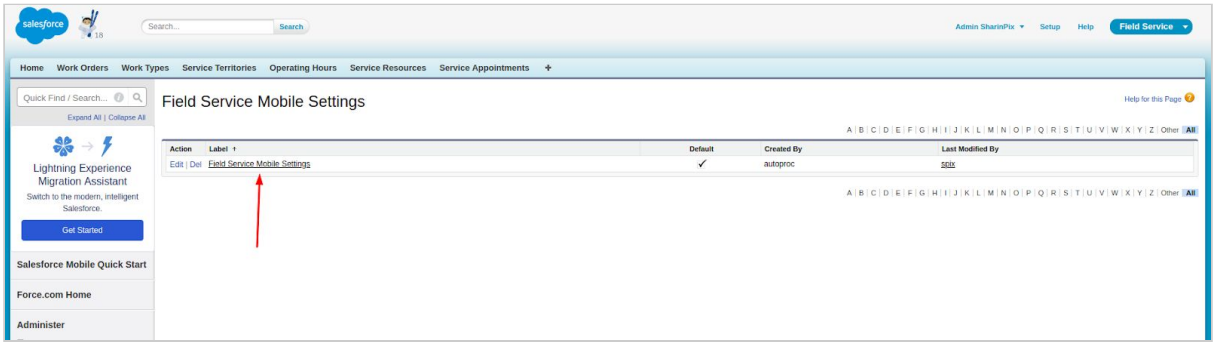
```
trigger SharinPixWorkOrderTrigger on WorkOrder (after insert, before update) {
    sharinpix.Client clientInstance = sharinpix.Client.getInstance();
    String token;
    List<WorkOrder> updatedWOrders = new List<WorkOrder>();
    for (WorkOrder wOrder : Trigger.new) {
        if (String.isBlank(wOrder.SharinPix_Token__c) {
            token = sharinpix.Client.getInstance().token(
                new Map<String, Object> {
                    'album_id' => wOrder.Id,
                    'exp' => 0,
                    'path' => '/pagelayout/' + wOrder.Id,
                    'native_app_options' => new Map<String, String> {},
                    'abilities' => new Map<String, Object> {
                        wOrder.Id => new Map<String, Object> {
                            'Access' => new Map<String, Boolean> {
                                'see' => true,
                                'image_list' => true,
                                'image_upload' => true,
                                'image_delete' => true,
                                'image_annotate' => true
                            }
                        },
                        'Display' => new Map<String, Object> {
                            'tags'=> true
                        }
                    }
                }
            );
            if (Trigger.isInsert) {
                updatedWOrders.add(new WorkOrder(
```

```
                    Id = wOrder.Id,
                    SharinPix_Token__c = token
                ));
            } else {
                wOrder.SharinPix_Token__c = token;
            }
        }
    }
    if (Trigger.isInsert) { update updatedWOrders; }
}
```

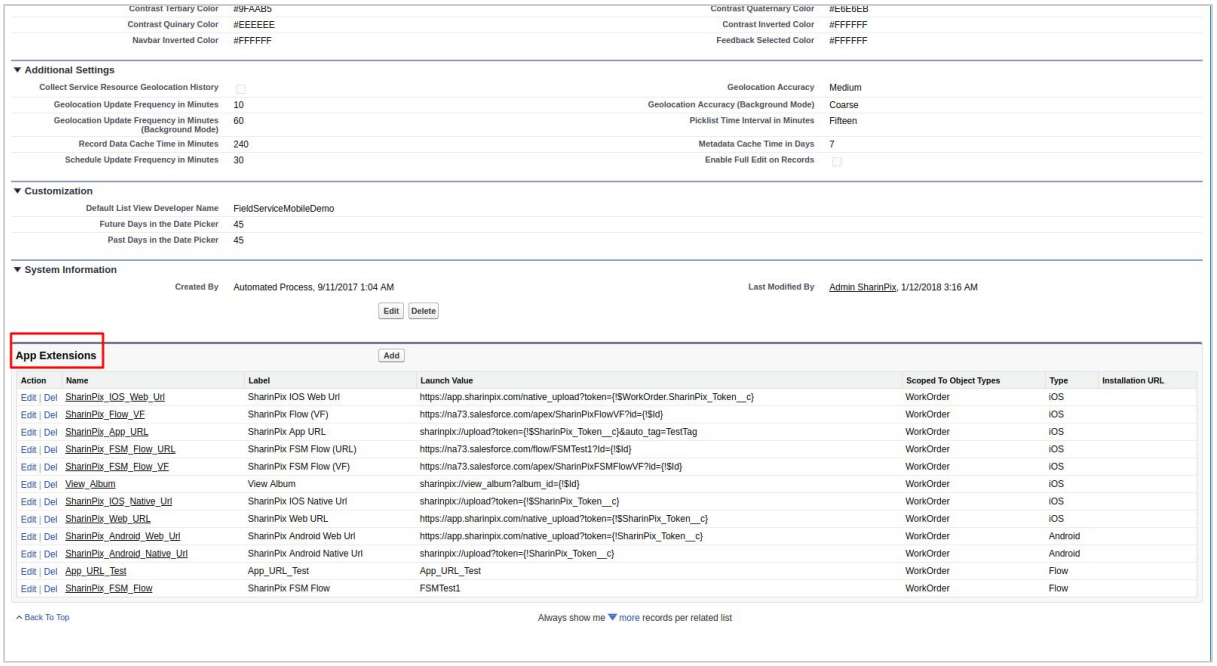
d. Create a new WorkOrder record. After insertion, ensure that the **SharinPix Token** field on the newly-inserted WorkOrder record contains the token value (which takes the form of a long string of characters).

2. Add a new App Extension

- Go to **Setup.** In the **Quick Find** box, type **Field Service Mobile Settings.** Access the corresponding search item result.
- Click on the **Field Service Mobile Settings** item. (Or any settings relevant to Organization.)

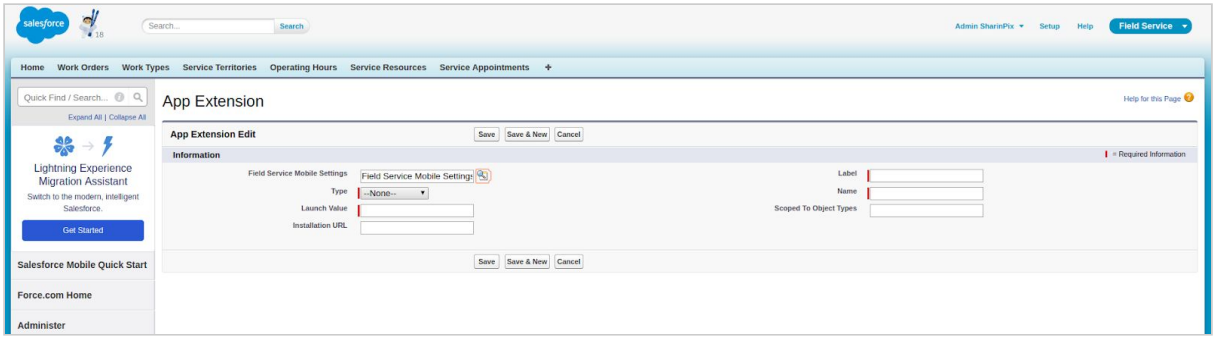


- Scroll towards to the **App Extensions** Section.



- Click on **Add**.

- You will be prompted to the screen as shown below.



- For the field **Field Service Mobile Settings**, select the record relevant to your Organization. In our case, the selected item is **Field Service Mobile Settings**.

- For the field **Type**, select the platform from which SharinPix will be launched. In our case, it will be **IOS**.
- For the field **Launch Value**, enter the value (SharinPix URL):
    - For **IOS platforms:** sharinpix://upload?token={!$SharinPix_Token__c}
    - For **Android platforms**: sharinpix://upload?token={!SharinPix_Token__c}

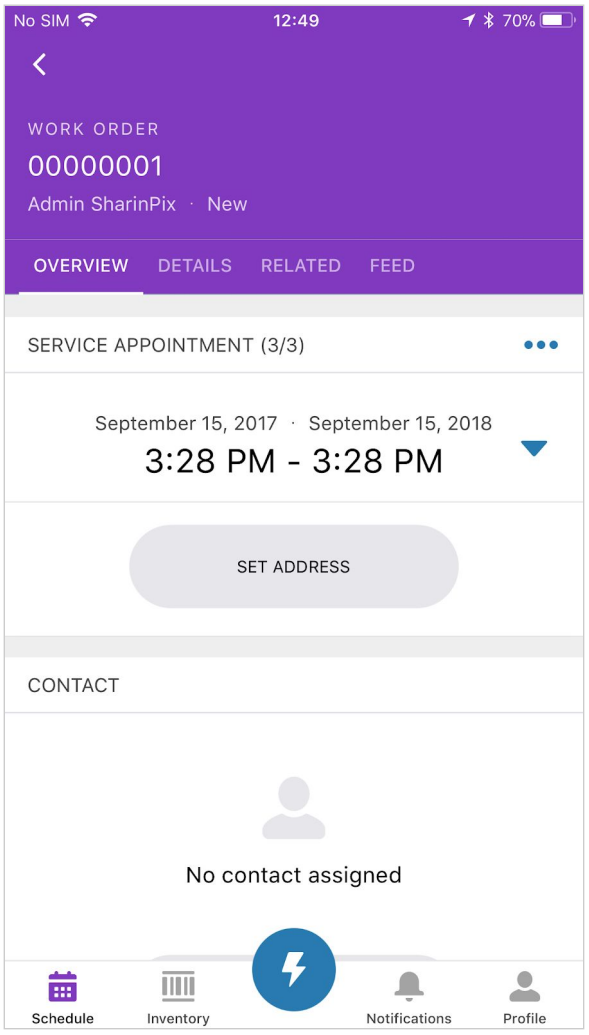
  NOTE: sharinpix://upload refers to the URL that will launch the SharinPix Mobile App. **SharinPix_Token__c** refers to the custom field containing the SharinPix token value defined in the previous sections.

  You can also use the **sharinpix://view** to launch SharinPix Mobile App in order to view all the pictures already taken by this device and check the status of the upload. Make sure you are using the right url for the right action you want to achieve. **sharinpix://upload** is meant to upload images while **sharinpix://view** is only meant to view the images of an album.
  For an overview of the structure of the SharinPix URL, find it here.

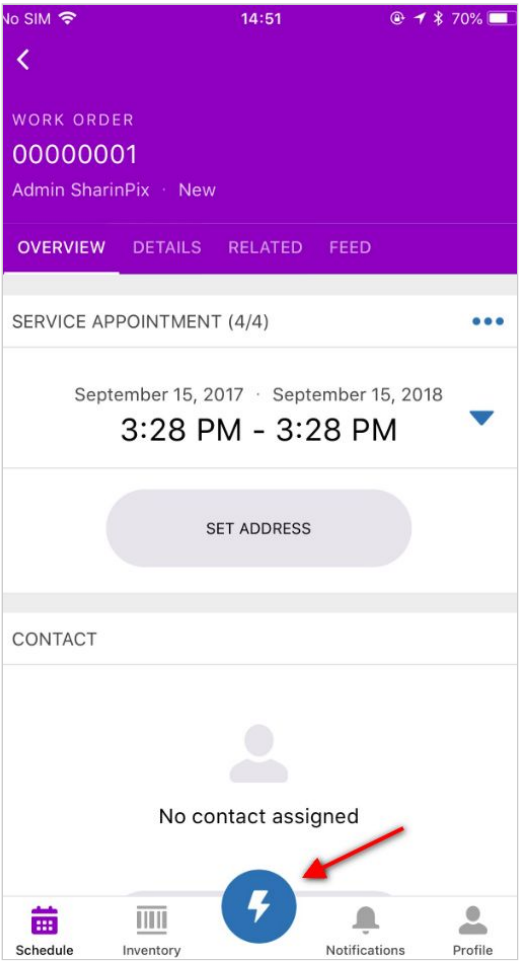
- For the field **Label**, enter value: **SharinPix IOS Native Url**
- For the field **Scoped To Object Types**, enter value: **WorkOrder**.

3. Launching SharinPix from the FSL Mobile Application

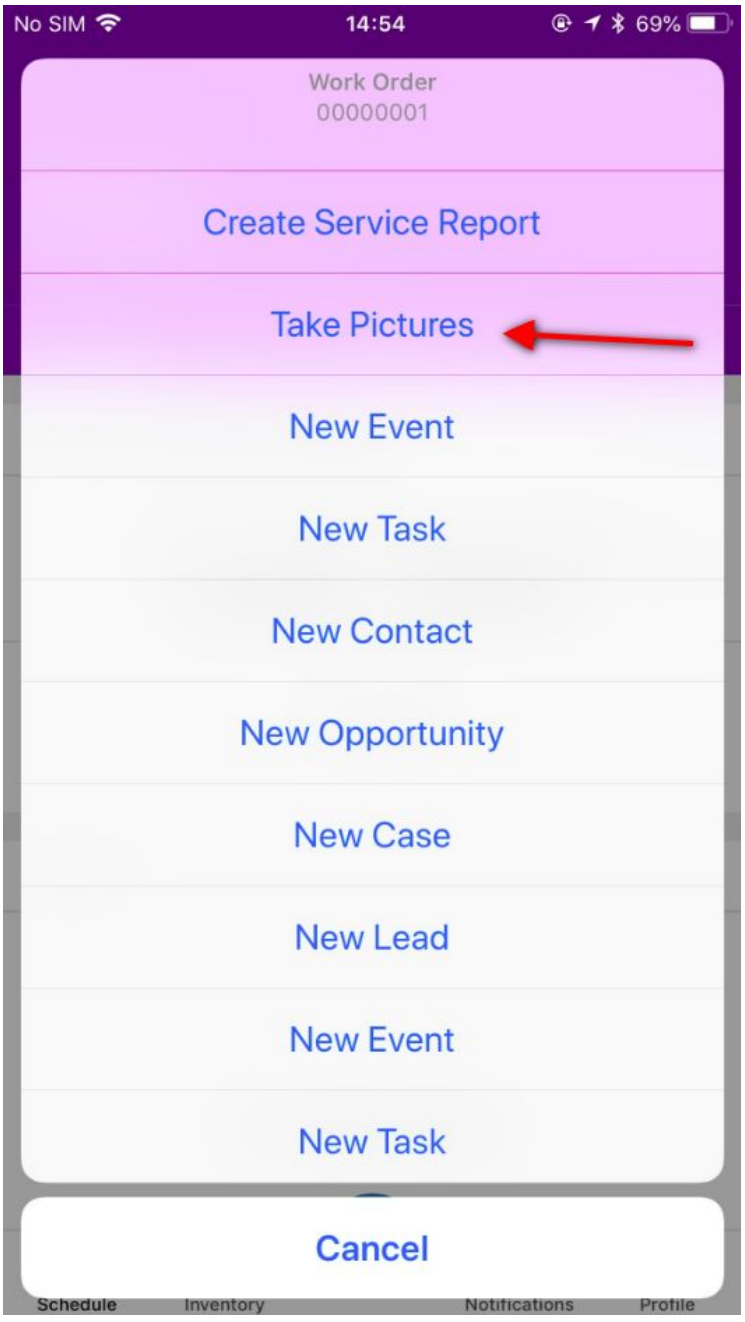
- Access the **Field Service Lightning Mobile Application** and select the **Service Appointment record** related to the newly-inserted WorkOrder record.



- Select the button with the *Lightning* icon.



- All the App extensions created can be viewed. Click on the app extension created in the previous section labelled: **Take Pictures.**



- Upon selecting **Take Pictures**, the **SharinPix Mobile Application** is launched.

Implementation using Flow

It is possible to launch the SharinPix Mobile App from a Flow. Follow the subsequent steps:

Assumptions:

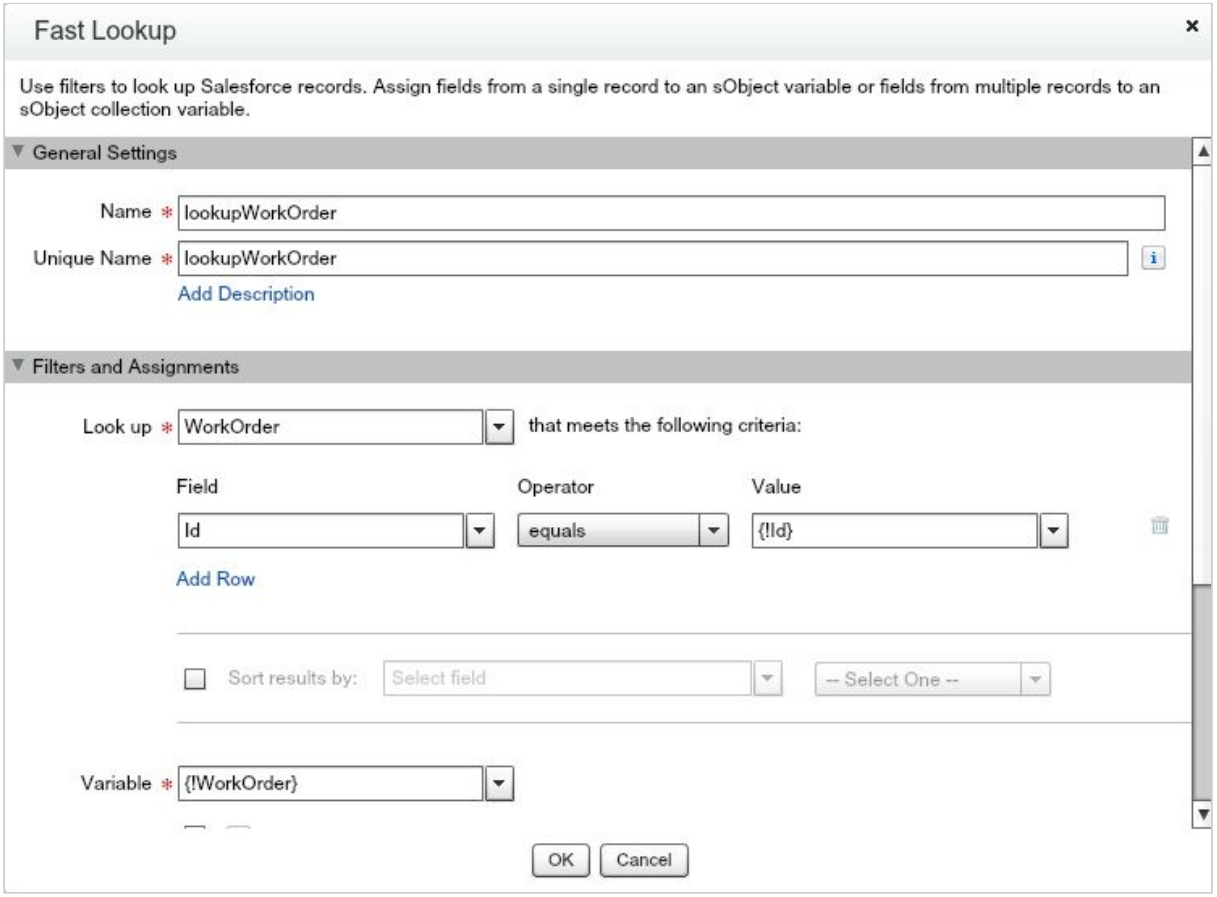
The WorkOrder object has a custom field named **SharinPix_Token__c. (NOTE: To store and retrieve the SharinPix Token, access this section: SharinPix Token).**

The following section will cover 3 key technical areas when building the Flow:

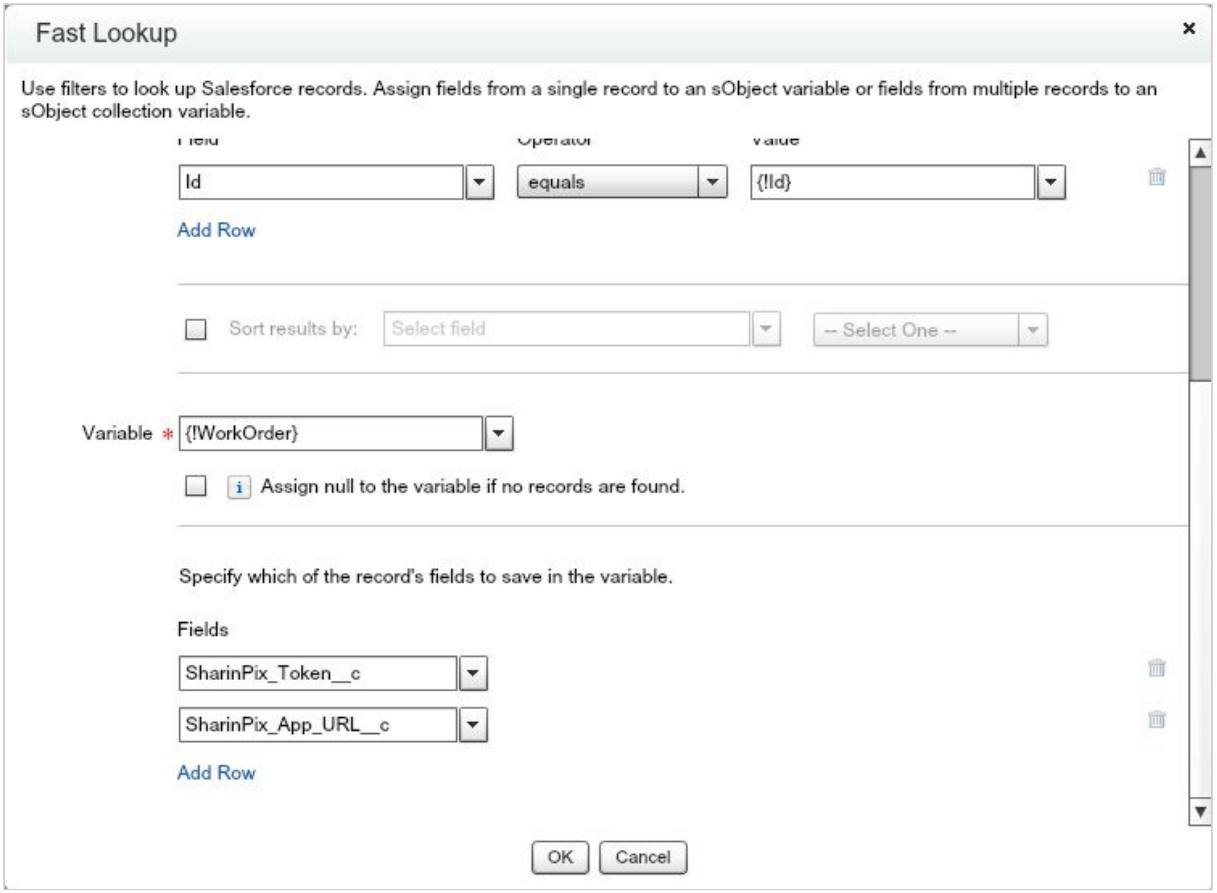
1. A **Fast Lookup** to reference the WorkOrder record containing the Flow.
2. A **Screen** displaying the link that enables the launch of the SharinPix App.
3. Testing on a mobile device to observe the link and auto-tag feature in action.

Fast Lookup

- Go to **Setup**. Type **Flows** in the Quick Find Box. And Access the **Flows** item under **Workflow & Approvals**.
- Click on **New Flow**. You will then be directed to the **Flow Designer**.
- Drag and Drop a **Fast Lookup** element from the **DATA** section onto the blank canvas.
- For the **Fast Lookup:**
    - Name: **lookupWorkOrder**
    - Unique Name: **lookupWorkOrder**
    - Look up: **WorkOrder**
    - Criteria:
        - **Id** (Field) **equals** (operator) **{!Id}** (Value)
    - Variable **{!WorkOrder}**



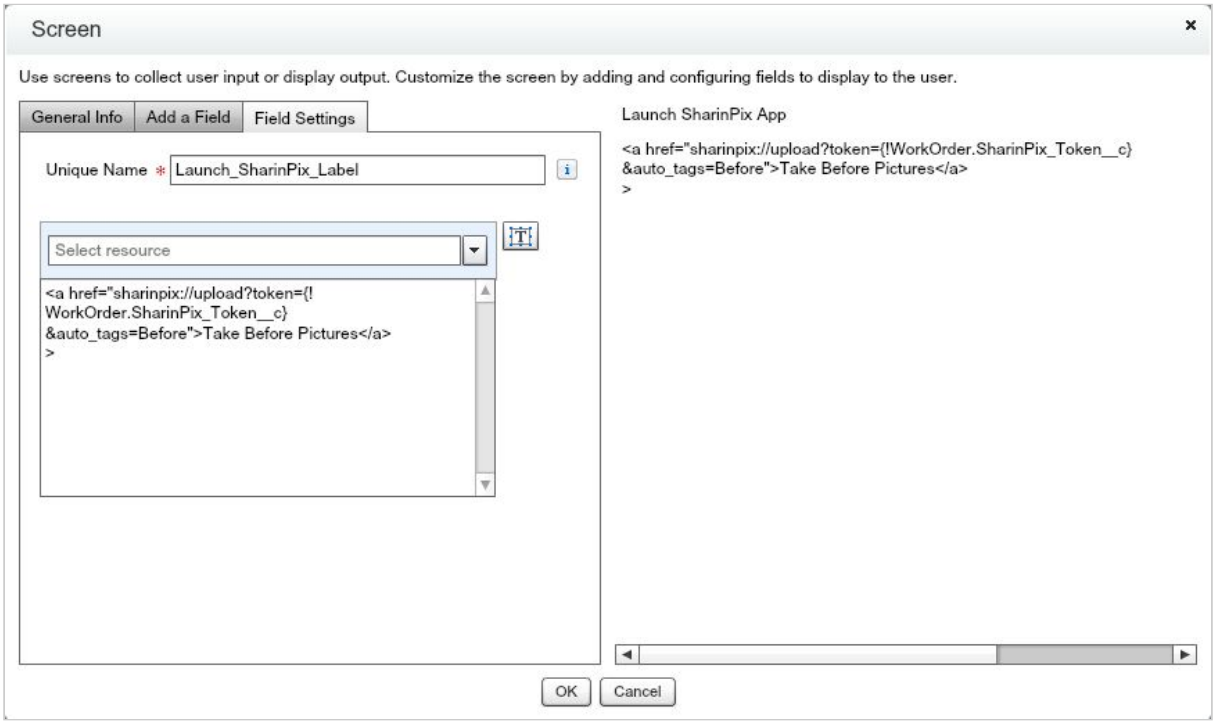
- **Specify which of the record’s fields to save in the variable**



User Interface Screen

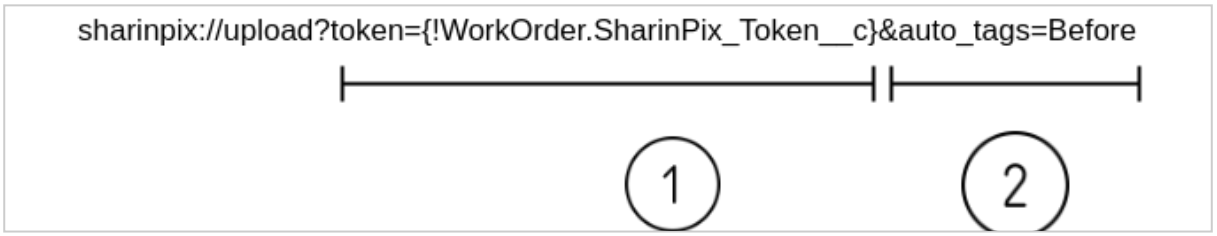
- Drag and Drop a **Screen** from the section **USER INTERFACE** onto the canvas.
- Properties for the **Screen**:
    - Name: **Take Pictures**
    - Unique Name: **Take_Pictures**
    - In the **Add a Field** Tab:
        - Select **Display Text**
    - In the **Field Settings** Tab
        - Set the **Unique Name** to **Take_Pictures_Label**
        - Paste the following code: (which refers to the link containing the SharinPix App Url Launcher)

```
<a href="sharinpix://upload?token={!WorkOrder.SharinPix_Token__c}&auto_tags=Before">Take Before Pictures</a>
```



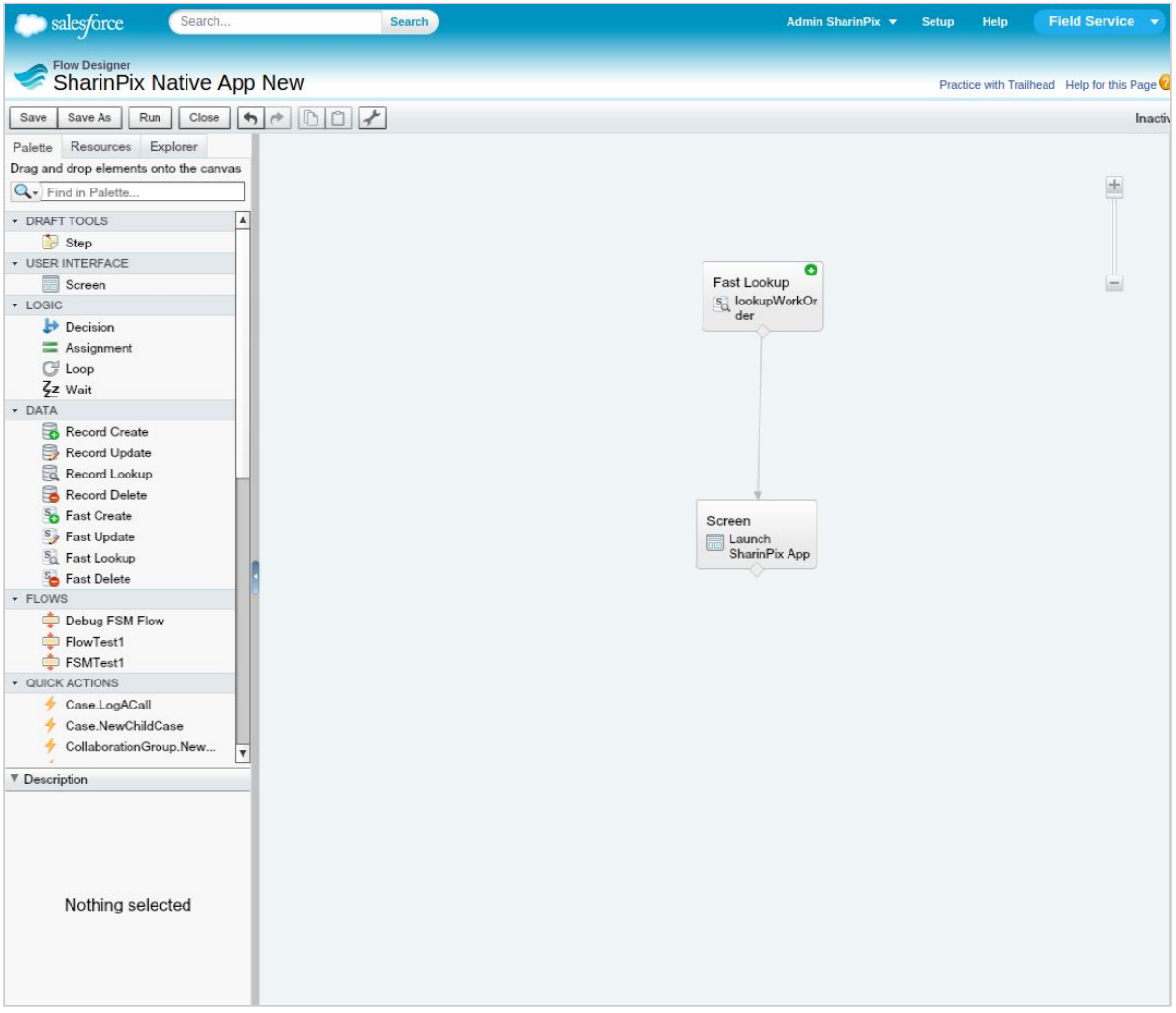
Click on **OK**.

Structure of the SharinPix App Url Launcher

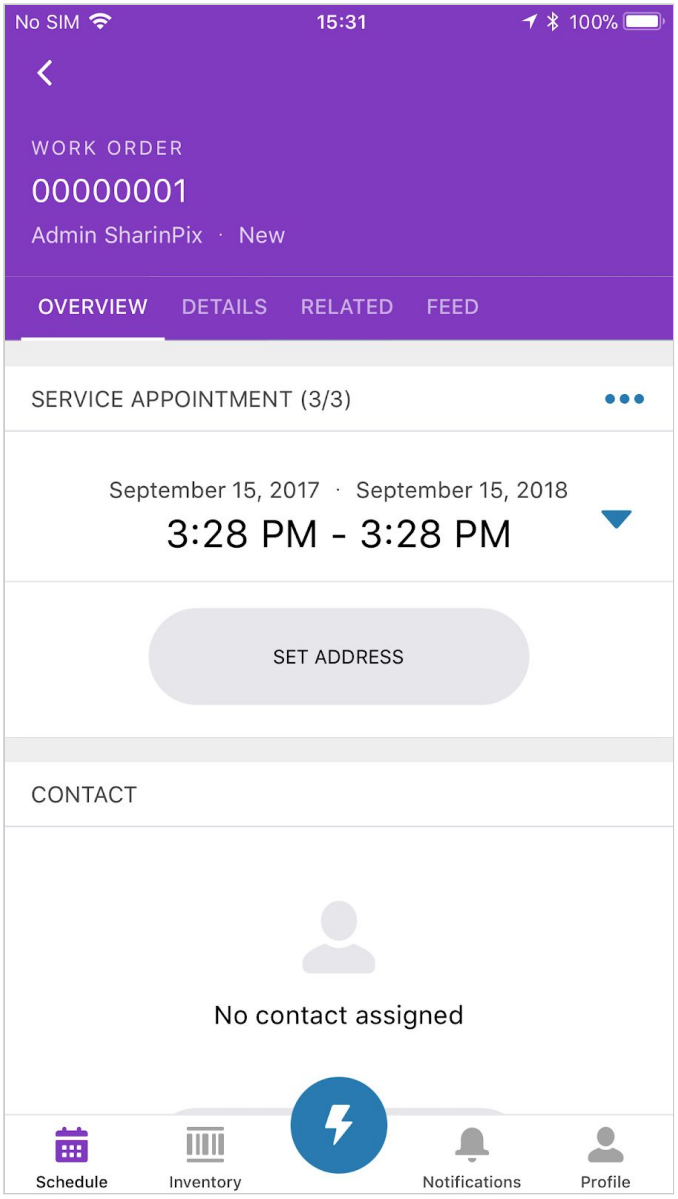


1. The **token** parameter corresponds to the SharinPix Token that will indicate from which WorkOrder record, the images will be referenced, once the SharinPix App is launched and new pictures are taken.
2. The **auto_tags** parameter takes as value the name of the tag we want to automatically apply to all images we take when the SharinPix App is launched.

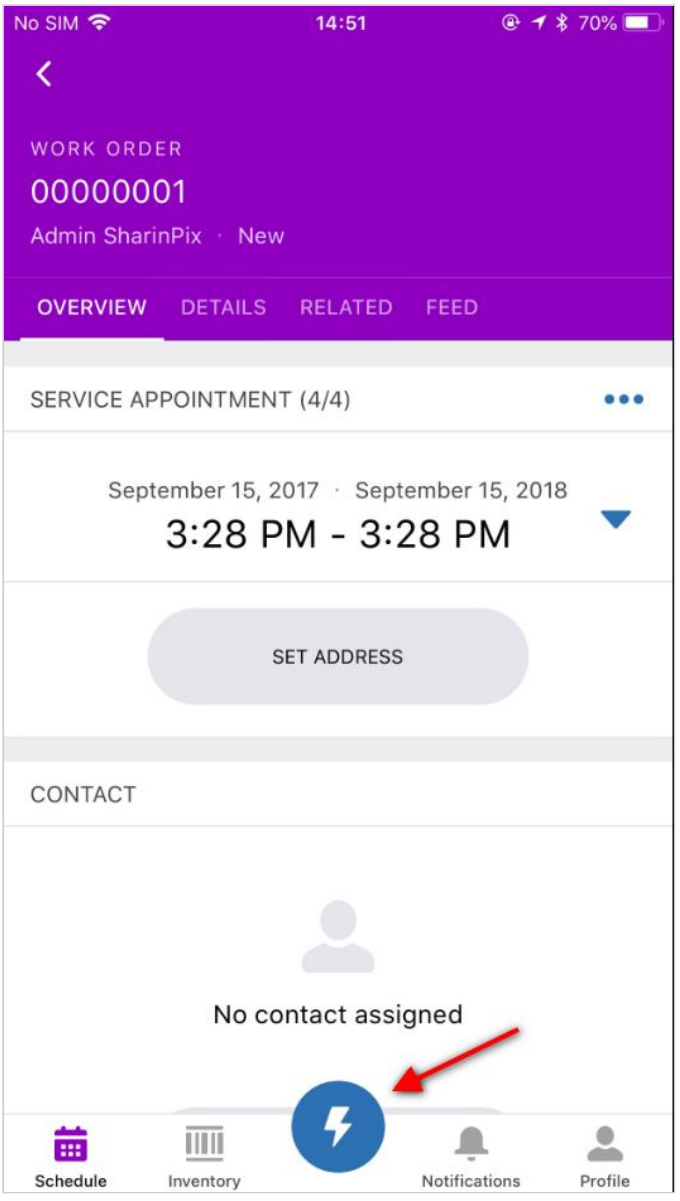
- Connect the **Fast Lookup** to the **Screen**.



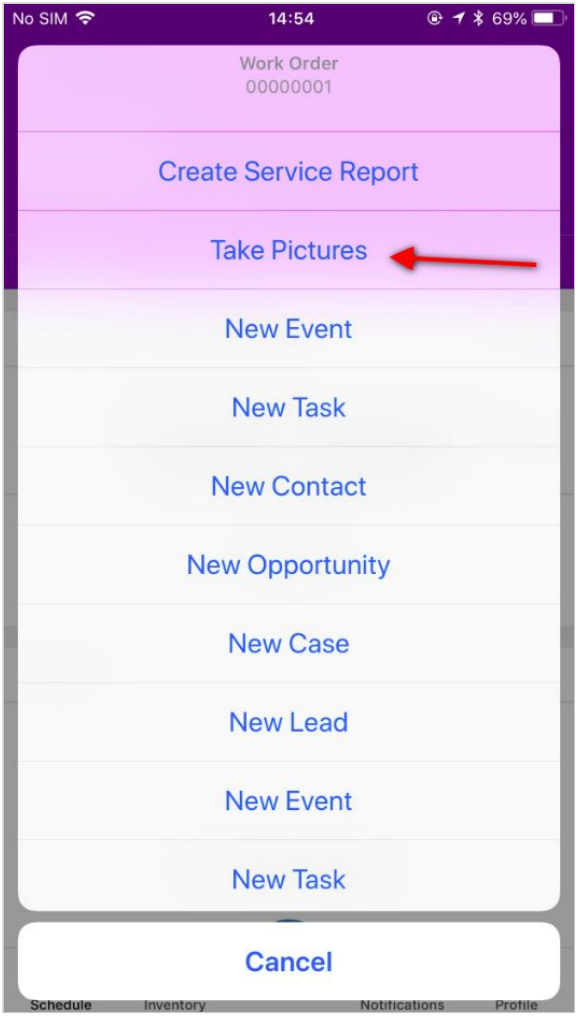
- Click on **Save** and **Activate** the newly-created flow.
- Access a **WorkOrder** record from the **Field Service Lightning Mobile App**.



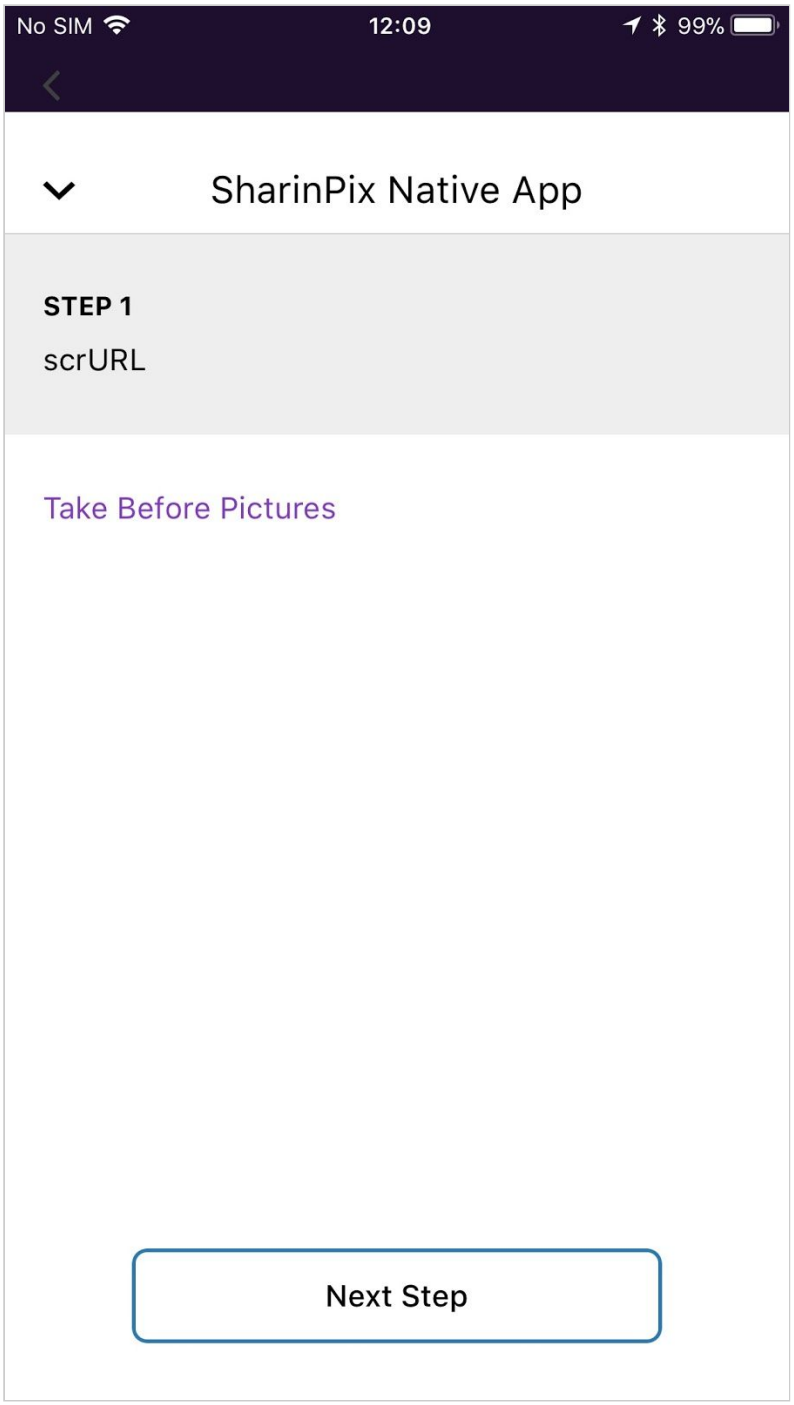
- Select the button with the **Lightning** icon.



- Select the newly-created flow. (In this case, it is **Take Pictures**)



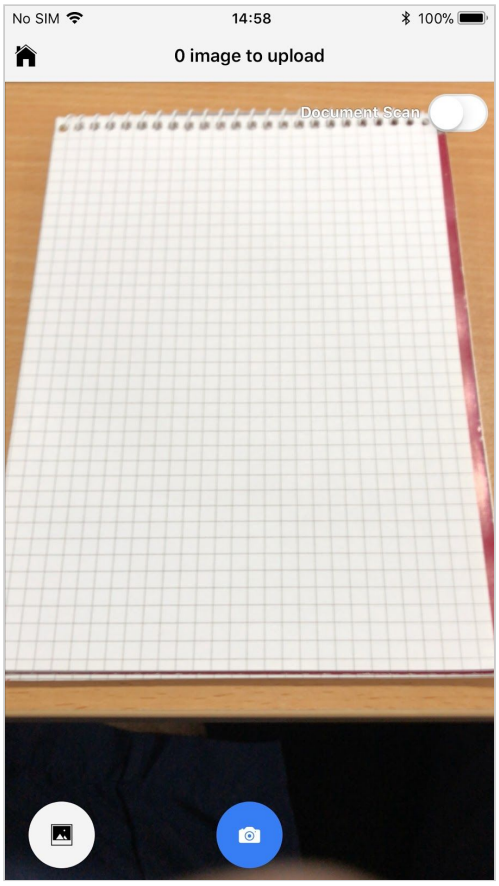
- You will be prompted to the screen as shown below.



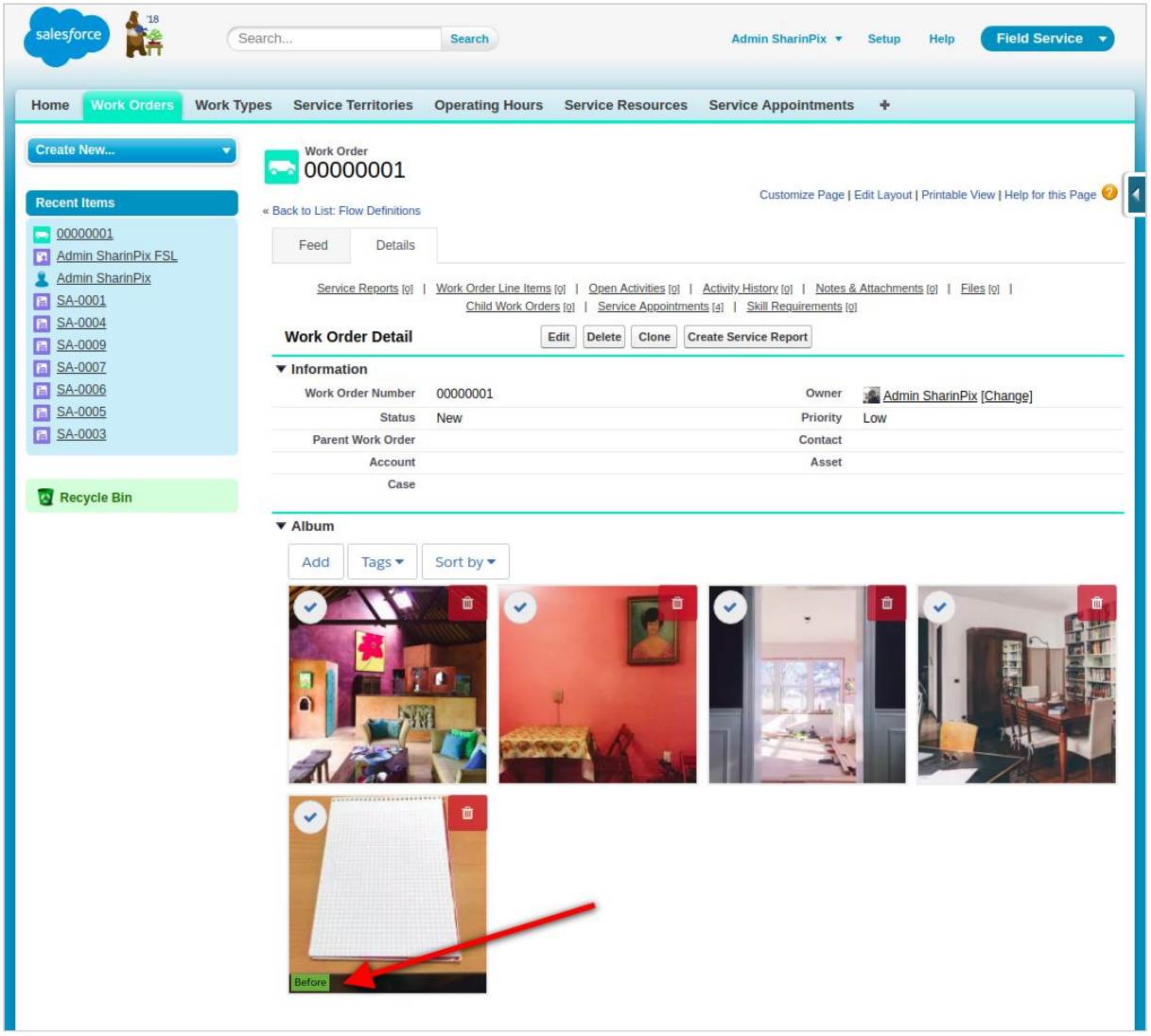
- Upon clicking on the **Take Before Pictures** link, the SharinPix Mobile Application will be launched.

- Once SharinPix App is launched, you will be able to snap pictures that will be automatically tagged with the label “**Before**”.

- For illustrating the auto-tagging feature, the figure below snaps a picture.



- Once the snapped photo has been uploaded; it can been seen that on the SharinPix Album of the corresponding WorkOrder record, the picture is tagged with the label **Before**.



Launch SharinPix Offline Mobile App

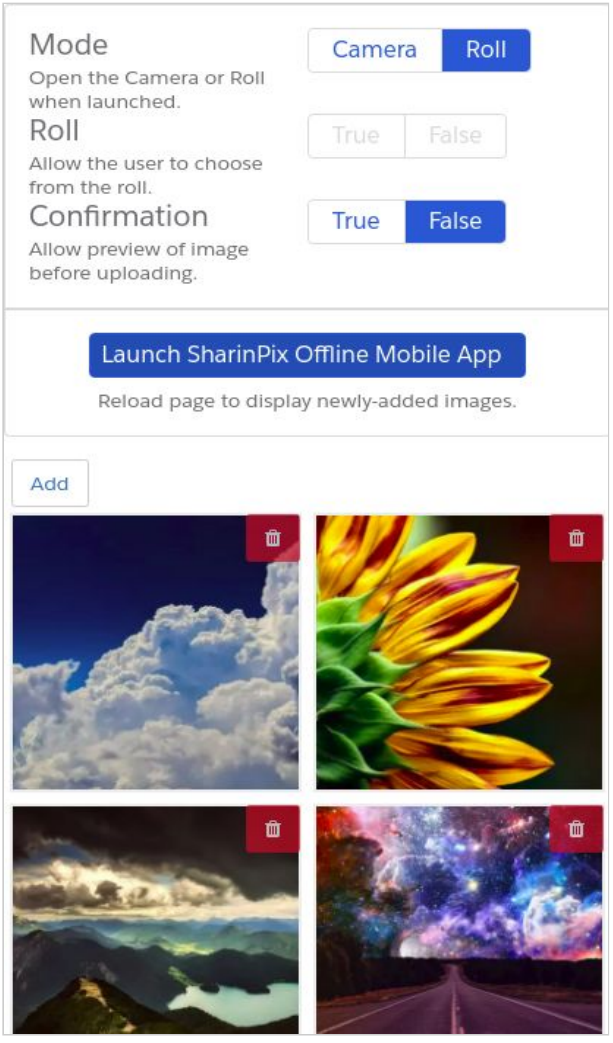
This example will demonstrate how to display a URL, that can launch the SharinPix Mobile App while offline. You will be shown how the parameters can be configured to control how the mobile application is launched.

These parameters are enumerated as follows:

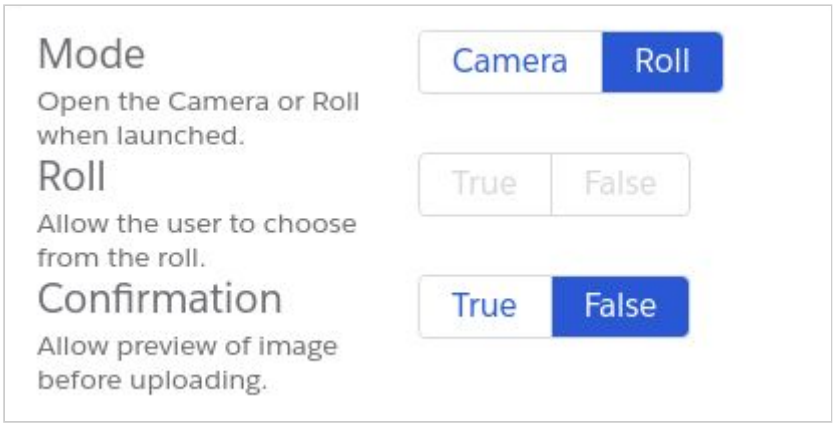
1. mode
   The parameter **mode** takes two values:
   - **camera** - this mode will open the camera for the user once the application is launched.
   - **roll** - this mode will open the roll/gallery of the mobile device once the application is launched.
2. roll
   The parameter **roll** takes two values:
   - **true** - this value allows the user to choose from the roll/gallery of the mobile device.
   - **false** - this value DOES NOT allow the user to choose from the roll/gallery of the mobile device.
3. confirmation
   The parameter **confirmation** takes two values:
   - **true -** this value previews the image on the display before uploading it.
   - **false** - this value DOES NOT preview an image on the display before uploading it.

The following screenshots demonstrate the offline launcher URL in action and its possibilities.

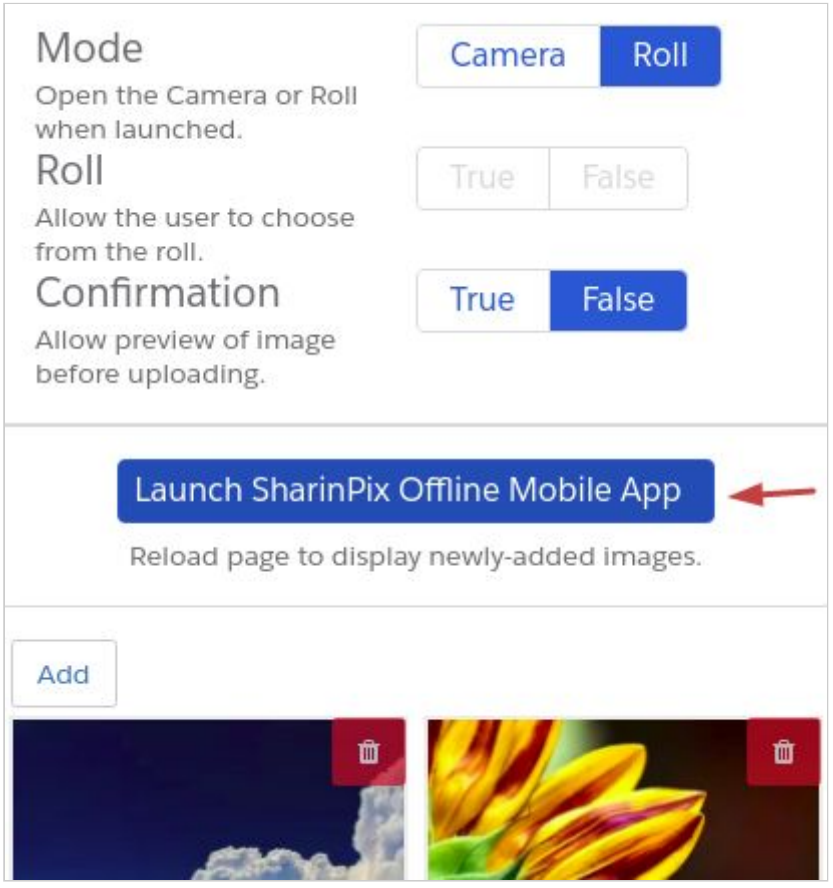
1. The figure below shows an interface rendered on a Visualforce Page, comprising of different input elements that help to configure the parameters and a button that can launch the SharinPix Mobile Application while offline. (**NOTE:** The exact implementation and technical explanation on how the URL can be displayed will be shown in the next sections below.)



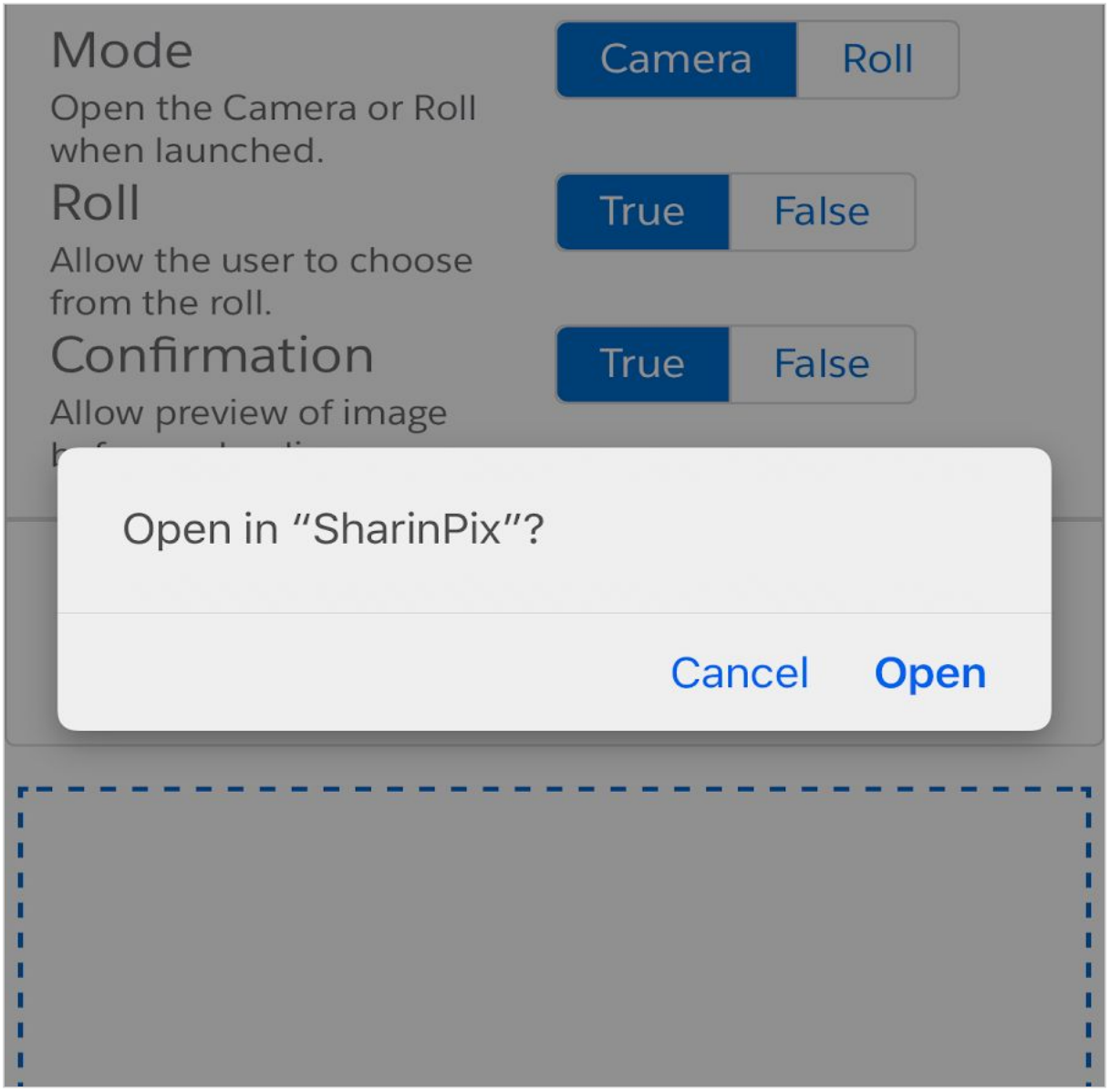
2. The figure below shows the input controls that specify how the SharinPix mobile app is launched.



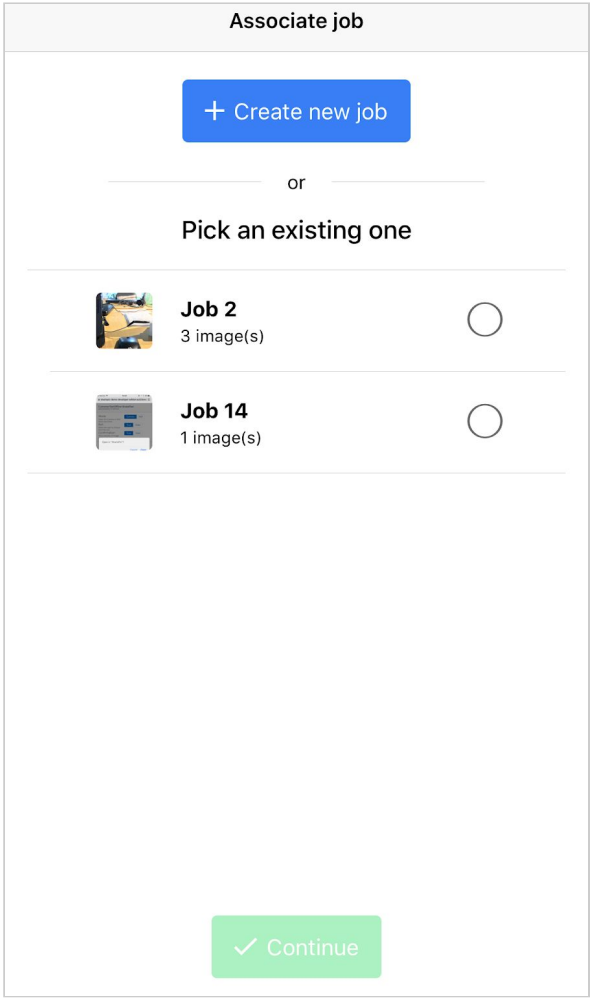
3. The figure below shows the button containing the URL which will be used to launch the SharinPix mobile application.



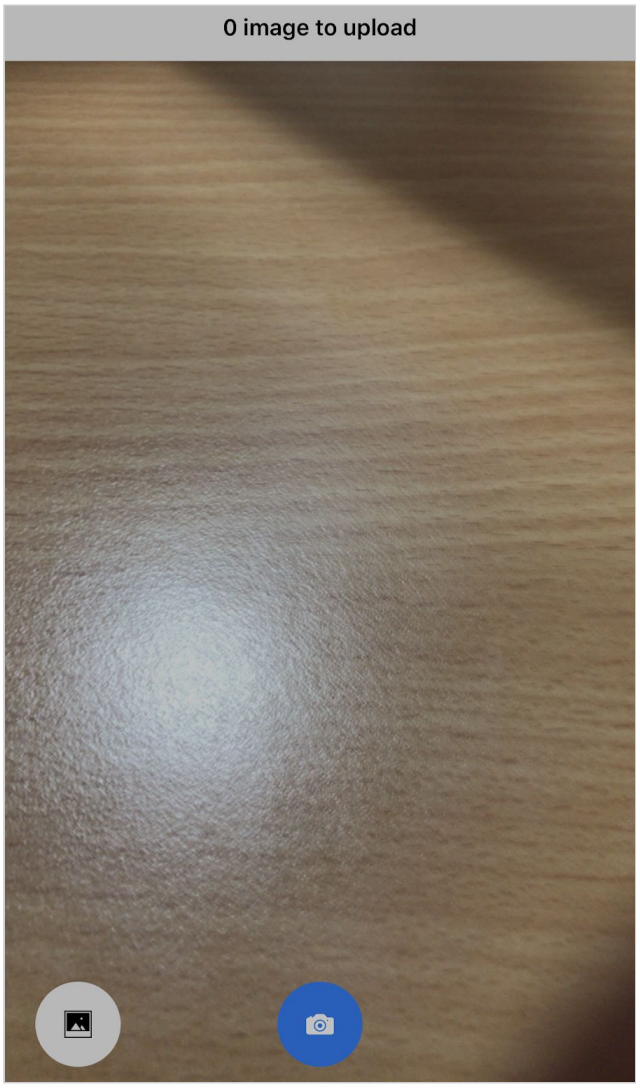
4. Launching the application. After you click on the button, you will be prompted like in shown in the figure below. (**NOTE:** The behaviour might differ across mobile devices, some platforms display a prompt and others might open another browser instantly.)



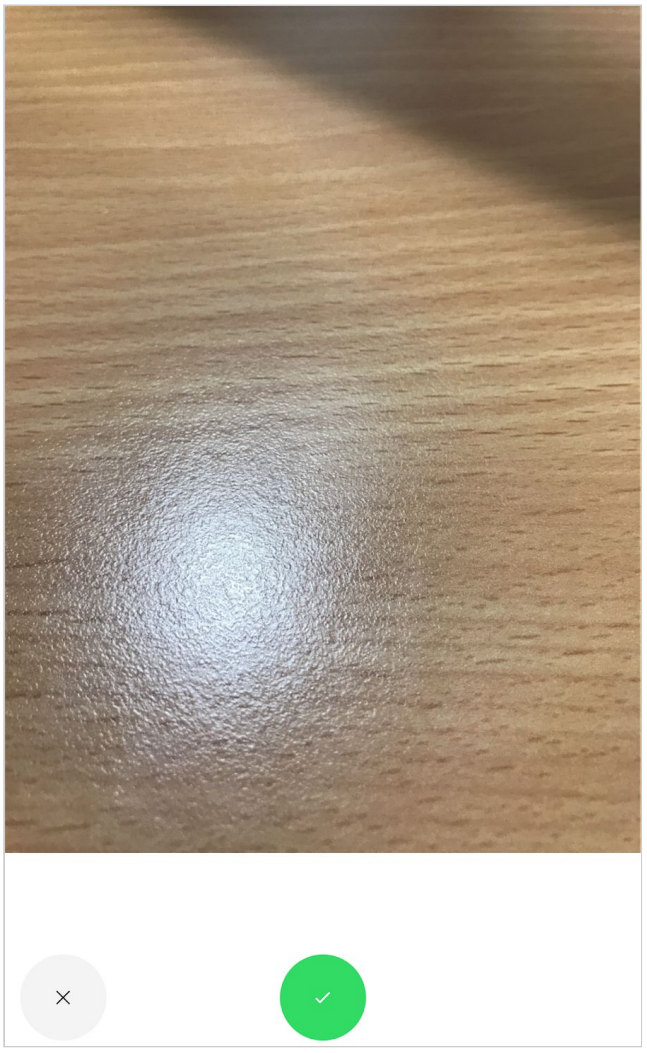
5. When the SharinPix App is launched, you will be prompted to either create a new job or select an existing one. (A “job” refers to a collection of images and is related to an album.)



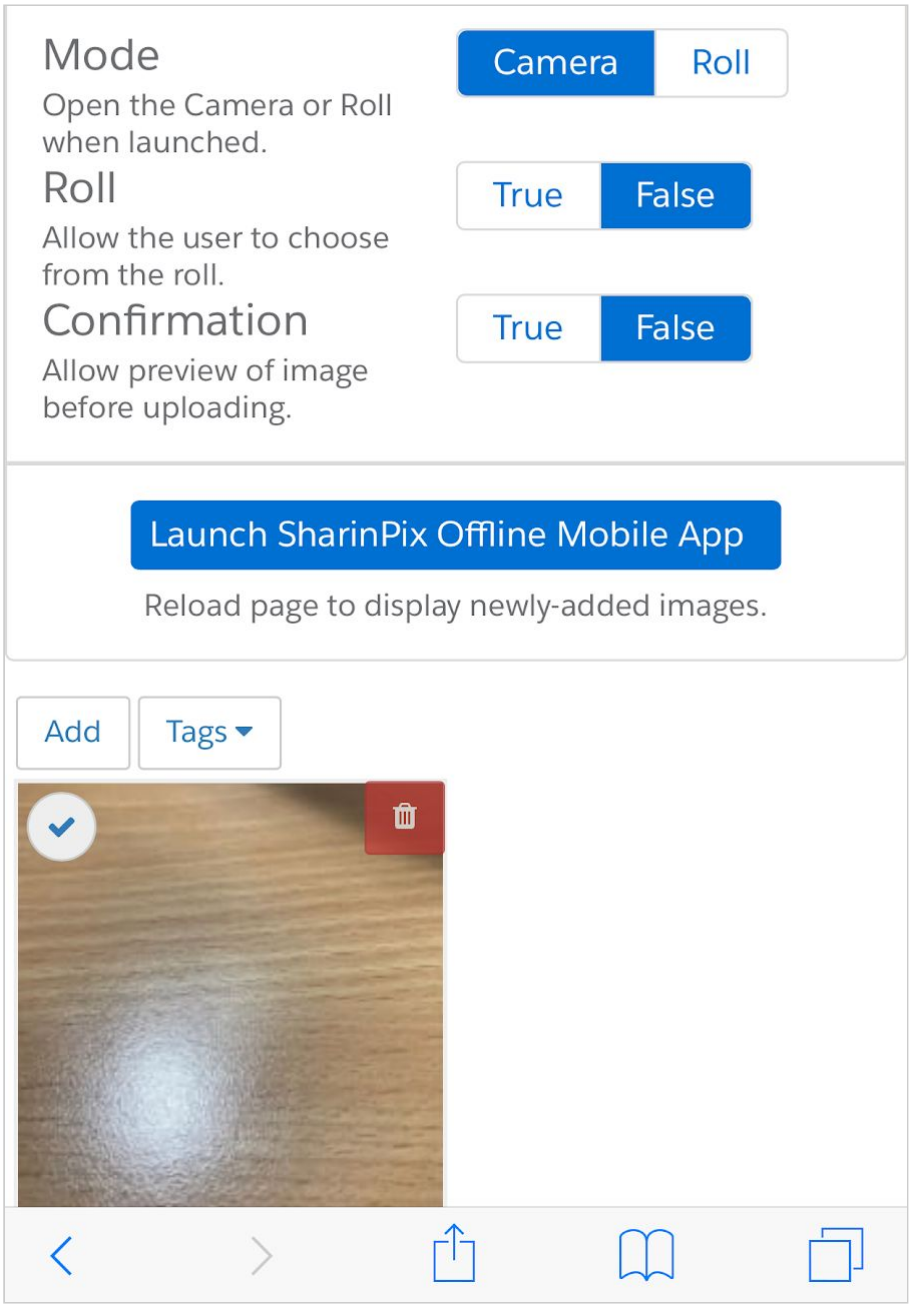
6. After you choose one of these two options, the application will open the camera view (since the parameter **mode** has the value **camera** as shown in the previous figures.)



7. After snapping a picture, a preview will be displayed(since the parameter **confirmation** was set to **true** as shown in the previous figures).

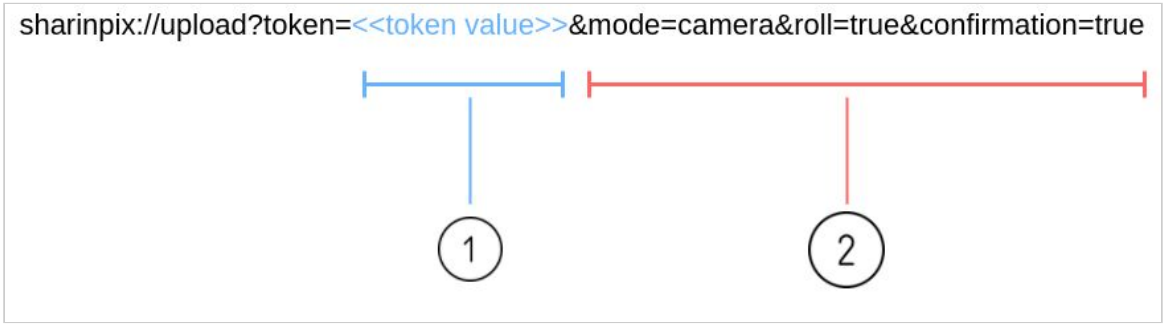


8. After the picture has been uploaded, you will need to refresh the SharinPix Album to display the newly-uploaded image(s).

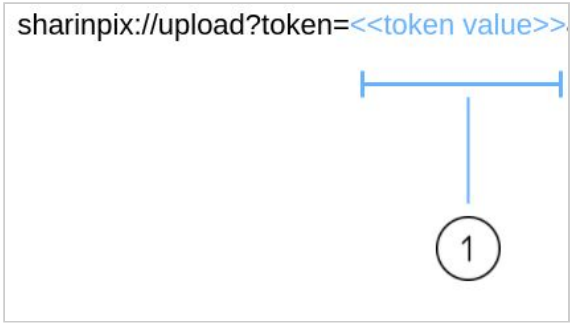


Structure of the SharinPix Launcher URL

As demonstrated before, the button labelled **Launch SharinPix Offline Mobile App**, contains a url which can be used via the concept deep linking to launch the SharinPix mobile application.
Find below an explanation on the different elements that make up the SharinPix Launcher URL.



1. The **token value** refers to a long string hash of characters that contains the different specifications of an album including its id, abilities, Access permissions, actions, etc…



Find below a code snippet that generates a token value which is specifically configured to help launch the SharinPix mobile application.

```
String token = sharinpix.Client.getInstance().token(
            new Map<String, Object>{
                'album_id' => recId,
                'exp' => 0,
                'path' => '/pagelayout/' + recId,
                'name' => 'JobVisit',
                'native_app_options' => new Map<String, String>{},
                 'abilities' => new Map<String, Object>{
                    recId  => new Map<String, Object>{
                        'Access' => new Map<String, Boolean>{
                            'see' => true,
                            'image_list' => true,
                            'image_upload' => true,
                            'image_delete' => true,
                            'image_annotate' => true
                        }
                    },
                    'Display' => new Map<String, Object>{
                        'tags' => true
                    }
                }
            }
        );
```

Referencing a token value from a Salesforce Field

Using a merge-field syntax, it is possible to reference a token value from a record field in Salesforce just as shown in the code snippet below.

Syntax for IOS platforms:

```
sharinpix://upload?token={!$ WorkOrder.SharinPix_Token__c }
```

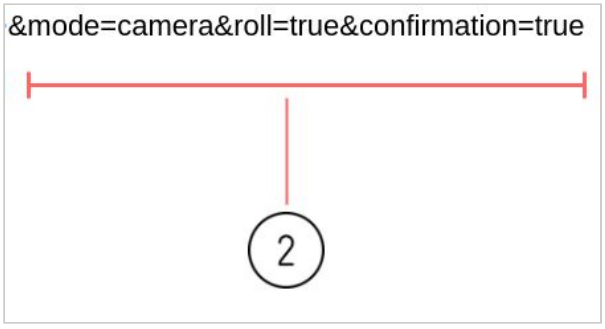
Syntax for Android platforms:

```
sharinpix://upload?token={! WorkOrder.SharinPix_Token__c }
```

Note: Pay attention to the “$” sign after the exclamation point in the IOS syntax.

In both code snippet, the custom field **SharinPix_Token__c** is referenced from the Standard Object **WorkOrder**.

2. The set of **parameters** (as mentioned in the previous section) defines how the SharinPix Mobile application will be launched.



To create a SharinPix URL that launches the SharinPix mobile application both the **token value** and the set of **parameters** are appended to the end of the following partial URL:

sharinpix://upload

SharinPix Image Sync

- What is Image Sync ?

Image Sync is a SharinPix functionality that allows the user to automatically extract useful data from an image when it is uploaded on the SharinPix Album of an object.

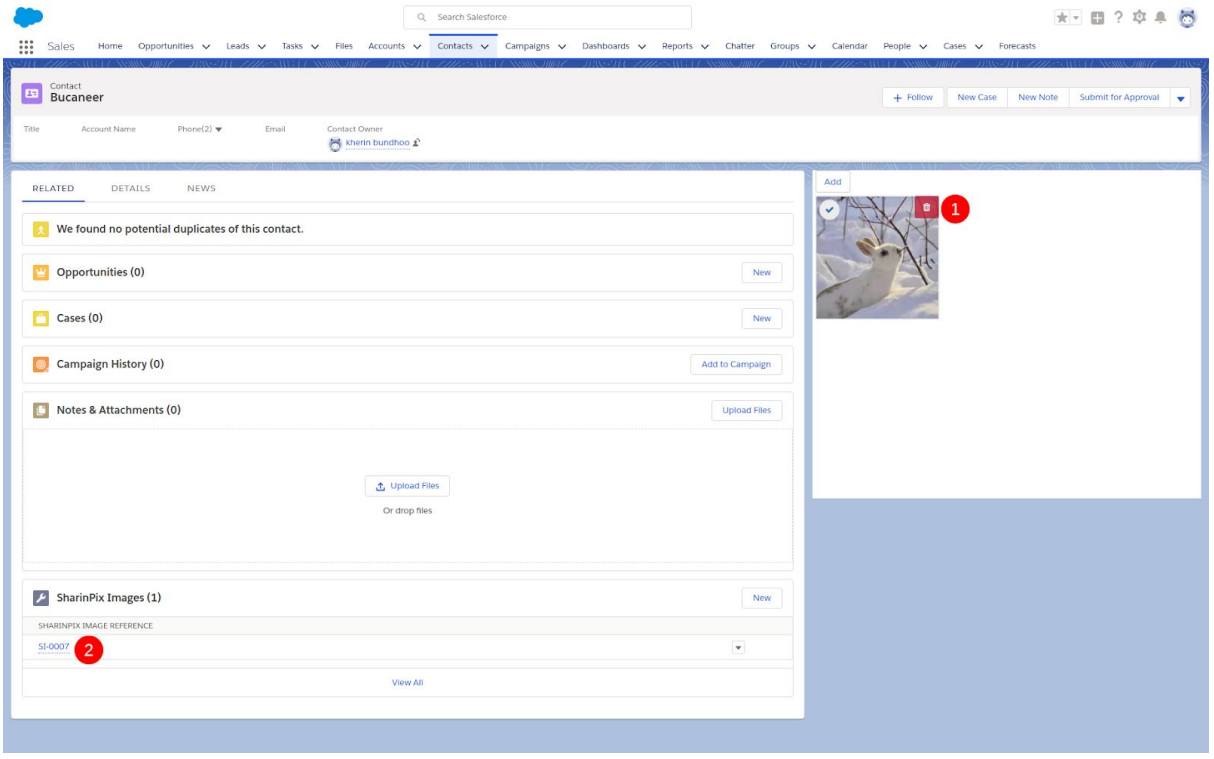
These useful data include:

- ❏ Filename of image
- ❏ Image Width
- ❏ Image Height
- ❏ Image Format
- ❏ Tags on the image
- ❏ URL of the original image
- ❏ URL of the Thumbnail-sized image
- ❏ URL of the Mini-sized image
- ❏ A preview of the Image
- ❏ Geolocation data embedded within the image
- ❏ Date the image was uploaded
- ❏ Date the image was taken
- ❏ Exifs data embedded within the image

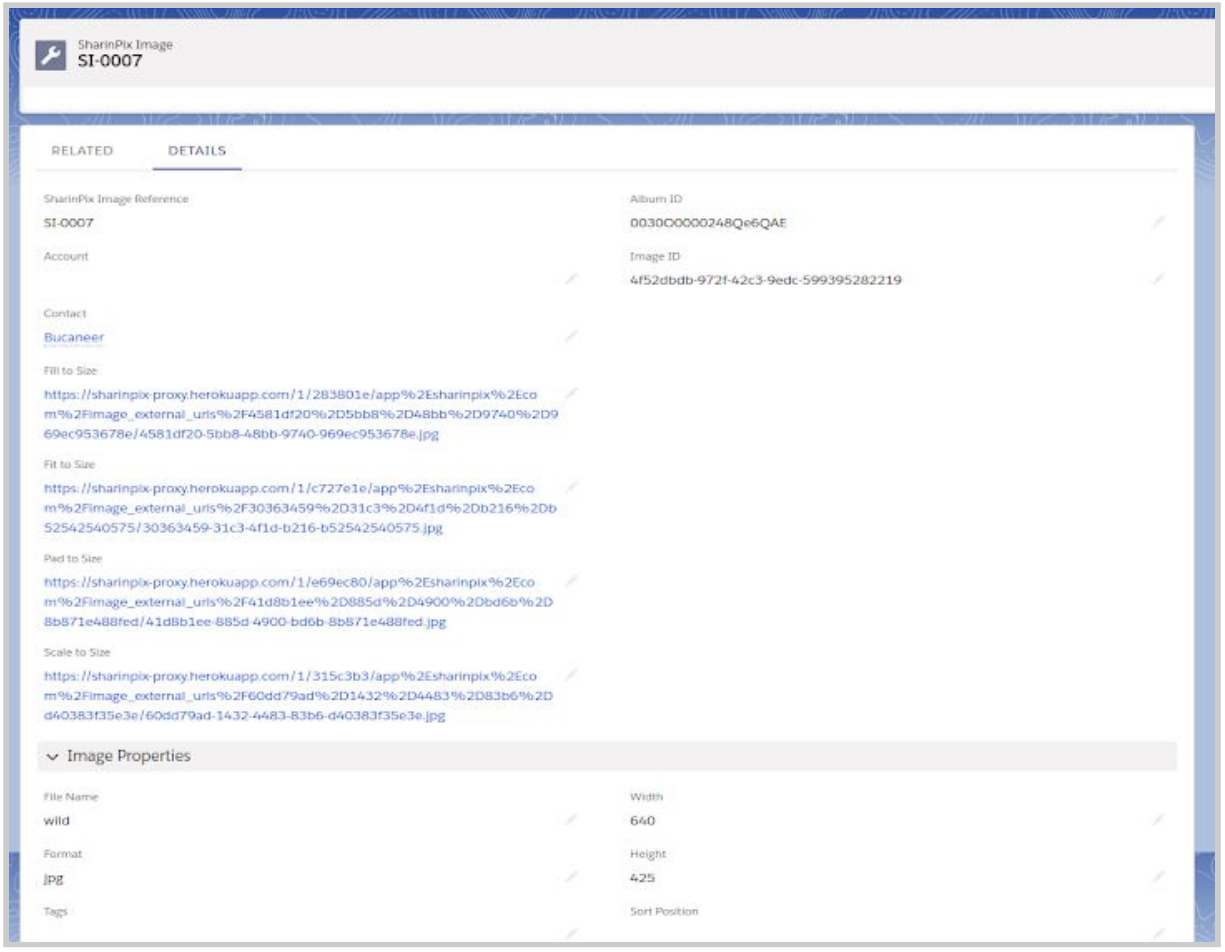
In the following sections, you will be able to see examples of how the above data are displayed on SharinPix Image record.

- What are the uses of Image Sync ?

The main use of Image Sync is to extract data from a newly-uploaded image and present it in a clear form. The process is automatic and any image that has been uploaded**(1)** is instantly “synchronized” with its corresponding SharinPix Image record**(2)**. **(NOTE: In this case, the Contact object is used as an example, however Image Sync can be used to display a related list of Sharinpix Image records on other objects.)**



An example of the data extracted from the image can be seen below.

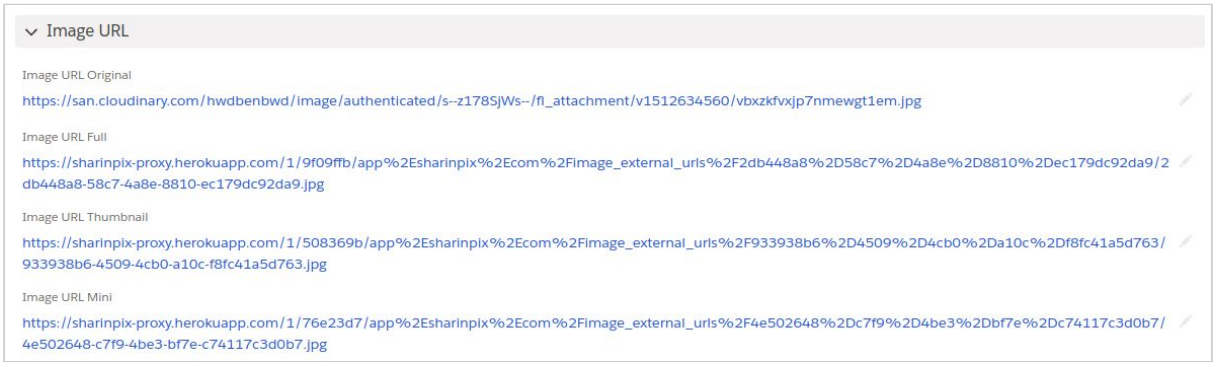


- What data are inside the Image Sync object ?

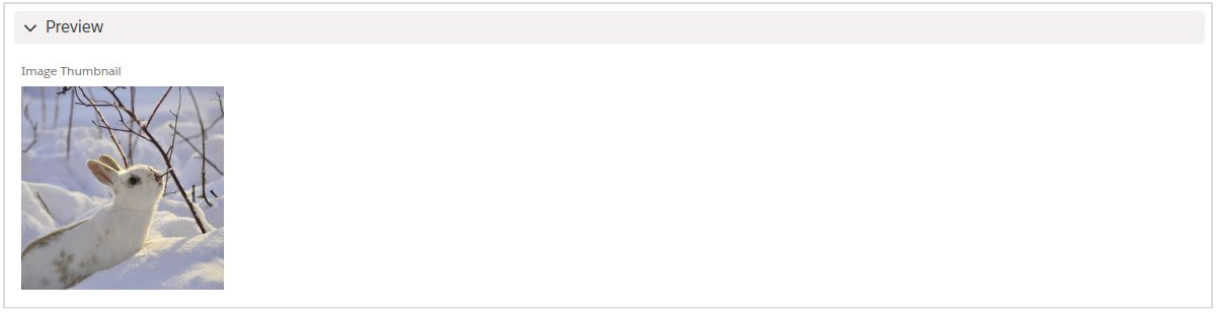
The view page of the SharinPix Image record consists of the following sections and containing details on the newly-uploaded image.

- Image Properties

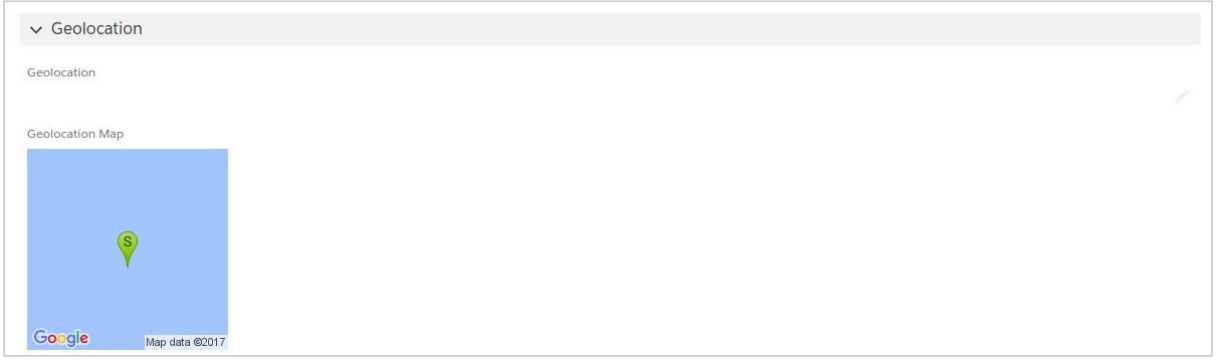
- Image URL



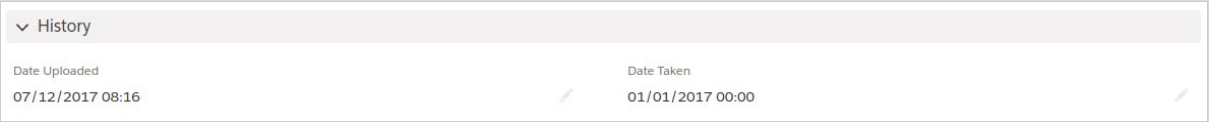
- Preview



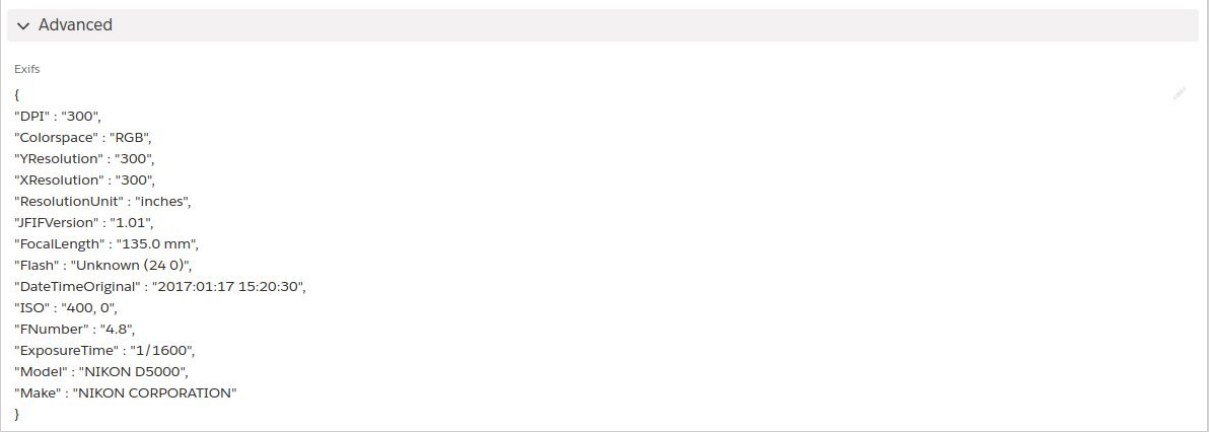
- Geolocation



- History



- Advanced



- Other data can be displayed, on the SharinPix Image record. This can be selectively displayed by modifying its page-layout. See figure below.

The following demo will show the different Image Sync functionalities within the SharinPix package. (version 1.42 +). Follow the steps below:

1. Setup SharinPix Image Sync

Depending on the Object you want to use SharinPix Image Sync on, you must create a lookup relationship field on the SharinPix Image Object which links to the parent object. This demo will make use of the Contact object.

Go to *Setup**(1)***, then access Object Manager. Click on *Fields and Relationships**(2)*** and create new relationship ***(3).***

Select the Data Type: **Lookup Relationship**

Select the related object: **Contact**

Proceed to the next steps and finally save the new relationship. When you back to the *Fields & Relationships* section, you should see the new relationship displayed as shown in the figure below.

Copy the field name: “Contact__c”

Go to the Custom Metadata.

Select the **Manage Records** action next to the label **SharinPix Sync Setting**.

Create a new record.

Enter the values into their corresponding fields as shown in the figure below.

2. Configuring Image Sync to work on Salesforce Lightning

Select any Contact record.

Select **Edit Page**

Edit the object layout and add the SharinPix Component on the layout.
On the right-hand-side, configure the component and enable Image Sync.

Select the *SharinPix Component(1)* and drag it onto the layout (2).
Select *Enable Image Sync (3).*

Upload an image

Click on the album. Select an image from your computer/device.

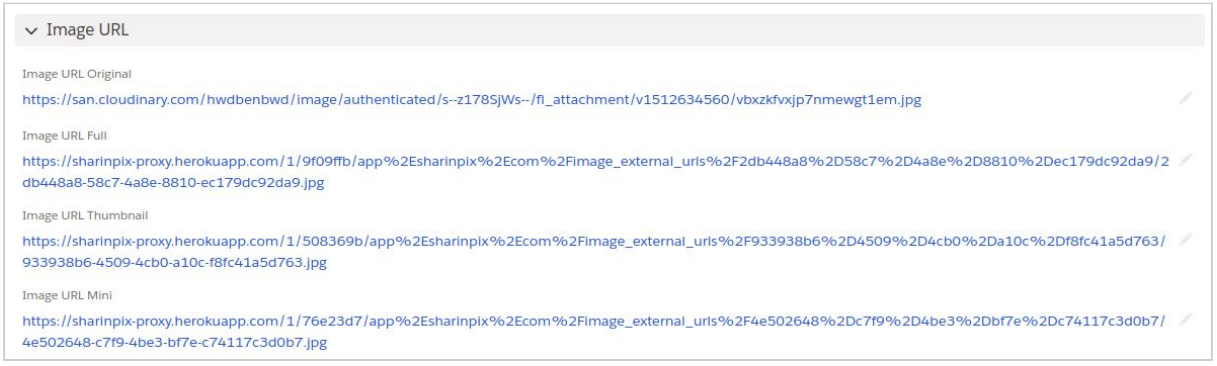
Refresh your current browser page. You should see a new entry inside the **SharinPix Images** section.

When you click on the new SharinPix Image, you will be directed to the view page of the new record as shown in the figure below.

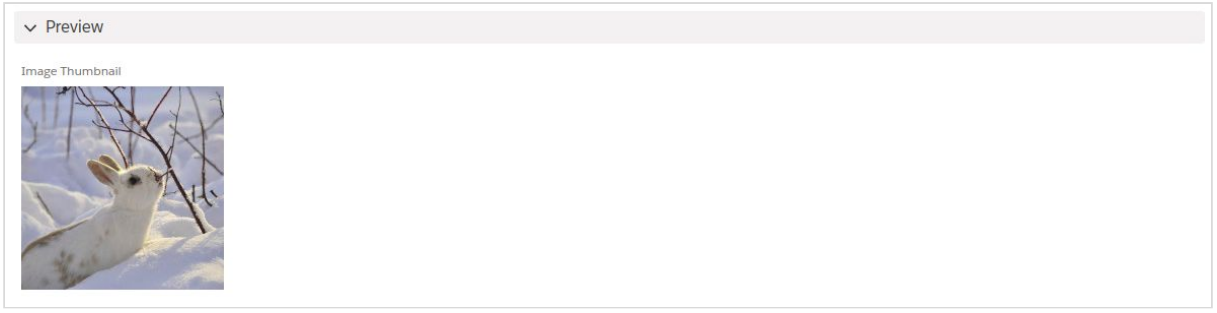
The view page consists of the following sections and containing details on the newly-uploaded image.

- Image Properties

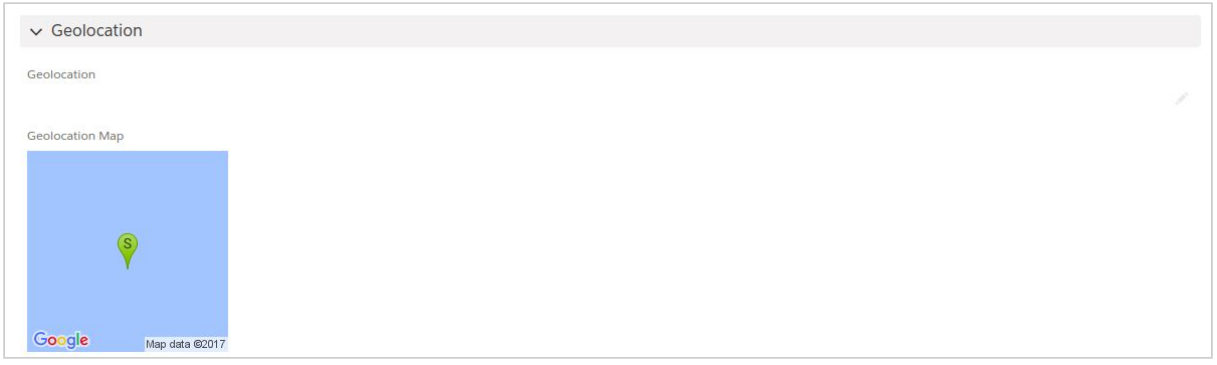
- Image URL



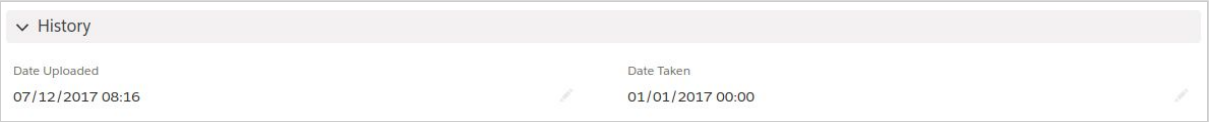
- Preview



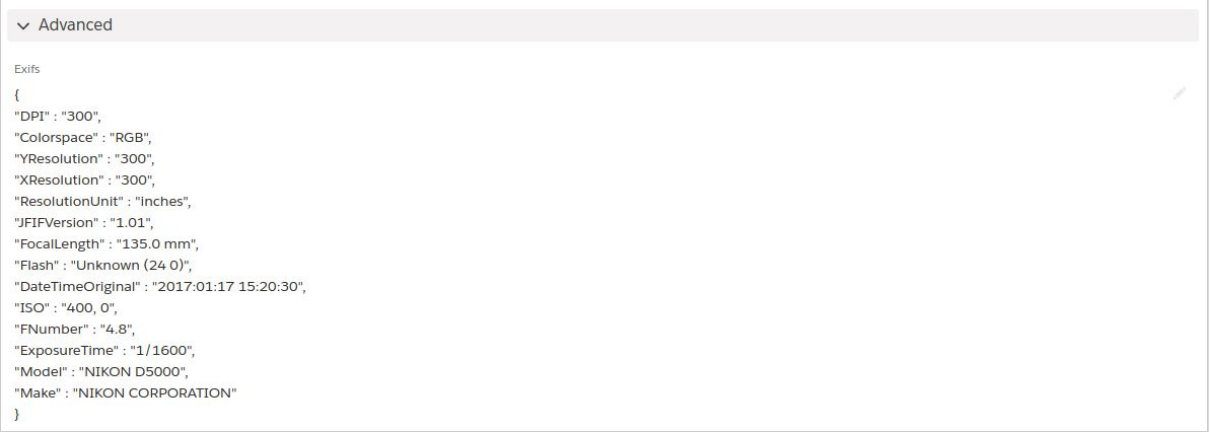
- Geolocation



- History



- Advanced



Configuring for Salesforce Class with a Visualforce page

To use Image Sync on Salesforce Classic, Visualforce integration needs to be done.

The following code needs to be added to your Visualforce code containing the SharinPix album.

```
<sharinpix:ImageSync recordId="{! $CurrentPage.Parameters.Id }"/>
```

Sample code with Album:

```
<apex:page standardController="Contact">
    <apex:canvasApp developerName="Albums" height="500px"
parameters="{abilities:{'{!CASESAFEID($CurrentPage.Parameters.Id)}':{Access:{image_rotate:t
rue,image_crop:true,image_delete:true,image_upload:true,image_list:true,see:true}}}}"
width="100%"/>

    <sharinpix:ImageSync recordId="{! $CurrentPage.Parameters.Id }"/>
</apex:page>
```

Follow the subsequent steps to add the Visualforce Page to Salesforce Classic:

1. Add the previous code snippet to a new Visualforce Page. And save the new Visualforce Page.

2. Add the newly-created Visualforce Page to the Layout of the Contact Object.

Add an image.

Refresh the current browser page. Go to the **SharinPix Images(1)** section. You should see a new entry(2).

Click on the new entry. You will be directed to the SharinPix Image Record of the newly-uploaded image. You should see the details of the image as displayed in the figure below. (**Note: The image values will differ depending on the image you uploaded)**

Transformations

The following examples will illustrate how it is possible to perform image transformations as listed below:

- **Fill to Size**

Create an image with the exact given width and height while retaining the original aspect ratio.
(height = width = 250 pixels)

- **Fit to Size**

The image is resized so that it takes up as much space as possible within a bounding box defined by the given width and height values.
(height = width = 250 pixels)

- **Pad to Size**

Resize the image to fill the given width and height while retaining the original aspect ratio and with all of the original image visible.
(height = width = 250 pixels)

- **Scale to Size**

Change the size of the image exactly to the given width and height without necessarily retaining the original aspect ratio.
(height = width = 150 pixels)

Transformation Setup

(This part assumes that you have followed the steps in the SharinPix Image Sync section. If not, find these steps here).

1. The first step is to create a new field on the SharinPix Image Object, that will contain the URL towards the image that has been transformed/manipulated.

- Go to the Object Manager inside Setup, and select the **SharinPix Image** object.
- Select **Fields & Relationships**. Click on **New**.
- Select the Data Type of **URL.**
- Enter the field name of your choice and proceed to the next steps. The field label should ideally reflect the Image Transformation carried out on the image.
- Finally click on **Save**.

2. Transformation: **Fill to Size.**

Follow the above steps to create a field on the SharinPix Image object. In this example, a field with label **Fill to Size** has been created on the SharinPix Image Object(1).

3. Go to Setup, type “Custom Metadata Types” in the **Quick Find** box(1). Select **Manage Records** right next to the **SharinPix Transformation** label (2).

2. Click on the **New** button.

4. Fill in the values.
   - In the **Field name** text input, enter the name of the field that was created above in the SharinPix Image Object(1).
   - Select SharinPix Sync Setting item that was previously selected in the *SharinPix Image Sync* Section above.(2)
   - Choose the **Transformation** of your choice(3).
   - Enter the **Value**, that corresponds to the width/height parameters, in pixels, relative to which the image will be transformed(4).

5. Go to Contacts and select an existing record or create a new one.
   - Upload a new image.
   - Refresh the current browser page.
   - Select the new entry in the SharinPix Image section.
   - You should see that a URL has been added to the field **Fill to Size**(or in your case, the field you defined for the SharinPix Object).

When you follow the URL, you will be directed towards the new transformed image.
In the figure below, the image has been subjected to a **Fill to Size** transformation with value **350 pixels**.

The following screenshots illustrate the other types of transformations. To reproduce, the results, the steps are identical to those evoked above.

- Transformation: **Fit to Size**. Value: **500**.

- Transformation: **Pad to Size**. Value: **500**.

- Transformation: **Scale to Size**. Value: **500**.

Troubleshoot common Image Sync issues

1. SharinPix Token Error

Resolution:

A. Edit permitted users for the SharinPix Album

Go to Setup(1), Type “Manage Connected Apps” inside the *Quick Find* search bar.
Select Manage Connected Apps(2).
Select the Edit action(3) next to the **Albums** master label.

Within the OAuth policies section(1), make sure that the value for **Permitted Users(2)** is set to **Admin approved users are pre-authorized.**

B.

Click on the App Launcher and select **SharinPix Settings**.

Click on **Grant.**

You will then be directed to a prompt as shown in the figure below. Click on **Allow.**

Refresh the browser page where the **SharinPix Token Error** was initially showing up. You should see that the precedent error has disappeared and the SharinPix Album is now displaying as it should be.

Download a PDF file from SharinPix within a Visualforce Page in the Salesforce IOS App

Find out about the Visualforce Page and Apex Controller Class used here: https://github.com/SharinPix/demo-apex/tree/pdf-download

1. Objective

The objective of the following demo is to enable the download of a PDF file from SharinPix when it is viewed on Salesforce App on an IOS device.

2. Setup Visualforce and Apex Controller

The following code snippet corresponds to the Visualforce implementation on the Contact object.

The implementation consists of:

- An Apex controller and its corresponding SharinPix Abilities. An explanation on this concept can be found here: **SharinPix Image Abilities using an Apex Controller.**
- A javascript script that listens to SharinPix events: An explanation on this concept can be found here: **Capturing SharinPix events using Javascript**.

```
<apex:page standardController="Contact" extensions="DownloadPDFCtrl">
    <script>
        var eventMethod = window.addEventListener ? "addEventListener" : "attachEvent";
        var eventer = window[eventMethod];
        var messageEvent = eventMethod == "attachEvent" ? "onmessage" : "message";
        eventer(messageEvent,function(e) {
            if (e.origin !== "https://app.sharinpix.com") return;

            if(e.data.name == "viewer-image-viewed"){
                if (e.data.payload.image.infos.format == 'pdf') {
                    document.getElementById('download-link').href =
e.data.payload.image.original_url;
                    document.getElementById('download-link').text = 'Download PDF';
                }else {
                   document.getElementById('download-link').text = '';
                }
            }else if (e.data.name == "viewer-closed") {
                document.getElementById("download-link").text = '';
            }
        }, false);
    </script>
    <apex:canvasApp developerName="Albums" height="500px" parameters="{! parameters }"
width="100%"/>
    <p id="container">
       <a id="download-link" href="" download=""></a>
    </p>
</apex:page>
```

In the above example, the purpose of the JavaScript code is to:

- Listen to when an image is viewed.
- Detect if the viewed image is of extension **.pdf**.
- Activate link to download the pdf file.

The code snippet below embodies the Apex Controller used for the Visualforce Page.

```
global with sharing class DownloadPDFCtrl {
    global String parameters { get; set; }

    global DownloadPDFCtrl (ApexPages.StandardController stdCtrl) {
        Id contactId = stdCtrl.getId();
        Map<String, Object> params = new Map<String, Object> {
            'abilities' => new Map<String, Object> {
                contactId => new Map<String, Object> {
                    'Access' => new Map<String, Object> {
                        'see' => true,
                        'image_list' => true,
                        'image_upload' => true,
                        'image_delete' => true,
                        'image_crop' => true,
                        'image_rotate' => true,
                        'image_tag' => false,
                        'image_download' => true
                    },
                    'Actions' => new List<String> {
                        'Send an email'
                    }
                }
            }
        };
        parameters = JSON.serialize(params);
        System.debug(parameters);
    }
}
```

3. Salesforce App Setup (Lightning)

- Go to Setup and Object Manager.
- Select the **Buttons, Links, and Actions** section.
- Select **New Action**.
- As **Action Type**, select **Custom Visualforce**.
- Select the name of the Visualforce Page containing the code mentioned above. (If the name of the Visualforce Page is absent from the list, ensure that the Page is enabled for the **Lightning and Mobile Experience**).
- Adjust the height to a minimum of **600px.**
- Enter adequate values for the **Label** field. In this case it is **PDF Test.**
- Click on **Save**, when you are done.
- Go the **Page Layouts** section.
- Select the layout corresponding best to your context.

- From the **Mobile & Lightning Actions** panel, drag the newly-created action to the section **Salesforce Mobile and Lightning Experience Actions**.
- Click on **Save** when you are done.

4. Demo in Salesforce App

- Access a Contact record on the Salesforce IOS App.

From the **Action Bar,** it can be seen that the newly-created action is present.

Once the **Action** is selected, the Visualforce Page opens up to display the SharinPix Album.

Once a PDF file is viewed, the download link is activated.

Upon selecting the **Download PDF** link, the PDF file is previewed within Safari internal as shown below.

From this preview, you will be able to save the PDF file via the methods shown below (depending on the IOS version of the device).

Manage delete permission with Tag

Objective

A **tag** has the possibility of granting or restricting a permission on a given image. In this demo, we will see how it is possible to allow images, with a particular tag, to be deleted. In this context, we will use a tag named **delete**, to allow the deletion of an image, while images without the **delete** tag cannot be deleted.

Find out about the Visualforce Page and the Apex Controller used in this demo by following this GitHub link:
https://github.com/SharinPix/demo-apex/tree/api-tag-delete

Implementation

In this context, we will make use of an Apex Controller(**SharinPixTagDelete**) and a Visualforce Page (**SharinPixTagDeletePage**).

Implementation of **SharinPixTagDelete**

```
global with sharing class SharinPixTagDelete {
   global String parameters { get; set; }
   global String url {get; set; }
   global Id contactId {get; set;}

    global SharinPixTagDelete(ApexPages.StandardController stdCtrl) {
        contactId = stdCtrl.getId();
       Map<String, Object> params = new Map<String, Object> {
           'abilities' => new Map<String, Object> {
               contactId => new Map<String, Object> {
                   'Access' => new Map<String, Object> {
                       'see' => true,
                       'image_list' => true,
                       'image_upload' => true,
                       'image_delete' => false
                   }
               }
           }
```

```
        };
        parameters = JSON.serialize(params);
        url = 'https://app.sharinpix.com/pagelayout/' + contactId + '?token=' +
sharinpix.Client.getInstance().token(params);
    }

    @RemoteAction
    global static String deleteImage(String imageId, Id recordId) {
        String result = 'failed';
        sharinpix.Utils spUtils = new sharinpix.Utils();
        List<Object>  tagList = spUtils.getTagsOnImage(imageId);
        for(Object tag: tagList) {
            Map<String, Object> tagMap = (Map<String, Object>) tag;
            Map<String, Object> tagDetails = (Map<String, Object>) tagMap.get('tag');
            if(tagDetails.get('name') == 'delete') {
                Map<String, Object> claims = new Map<String, Object> {
                    'abilities' => new Map<String, Object> {
                        recordId => new Map<String, Object> {
                            'Access' => new Map<String, Object> {
                                'image_delete' => true
                            }
                        }
                    }
                };
                try{
                    sharinpix.Client.getInstance().destroy('/images/'+imageId, claims);
                    result = 'success';
                    return result;
                }catch(Exception e) {
                    result = e.getMessage();
                    return result;
                }
            }
        }
        return result;
    }
}
```

In the above code snippet, the constructor defines the **access** of the SharinPix Album. The following code snippet denies the album the ability to delete an image.
`'image_delete' => false.`

The Remote Action Method **deleteImage** takes as parameters: **Image Id** and **Record Id** respectively. The present method uses this image id to identify the current image, which the user is currently viewing and whether this image possesses the tag **delete**. Using the method **getTagsOnImage** from the class **Utils** found in the SharinPix Package, it is possible to retrieve all the tags on a given image by supplying its corresponding image id.

Implementation of **SharinPixTagDeletePage**

```
<apex:page standardController="Contact" extensions="SharinPixTagDelete">
   <script>
       var image_id;
       var eventMethod = window.addEventListener ? "addEventListener" : "attachEvent";
       var eventer = window[eventMethod];
       var messageEvent = eventMethod == "attachEvent" ? "onmessage" : "message";
       eventer(messageEvent,function(e) {
           document.getElementById("message").innerHTML ="";
           if (e.origin !== "https://app.sharinpix.com") return;
           if(e.data.name === "viewer-image-viewed") {
               document.getElementById("deleteBtn").style.visibility = "visible";
               image_id = e.data.payload.image.public_id;
           }
           if(e.data.name === "viewer-closed"){
               document.getElementById("deleteBtn").style.visibility = "hidden";
           }
       }, false);

       function deleteImage() {
           Visualforce.remoting.Manager.invokeAction(
               '{! $RemoteAction.SharinPixTagDelete.deleteImage}',
               image_id,
               '{!contactId}',
               function(response) {
                   if(response === "success") {
                       var iframe = document.getElementById('sharinpix-iframe');
                       iframe.src = iframe.src;
                       document.getElementById("deleteBtn").style.visibility = "hidden";
                   }else {
                       document.getElementById("message").innerHTML ="Error! Unauthorized
to delete this image.";
                   }
               }
           );
       }
   </script>
   <iframe id="sharinpix-iframe" src="{!url}" height="500px" width="100%" style="border: 0"
/>
   <button id="deleteBtn" onclick="deleteImage()" style="visibility: hidden;">Delete
Image</button>
   <p id="message" style="color: red;"></p>
```

```
</apex:page>
```

The above code snippet corresponds to the Visualforce Page which embeds the SharinPix album and observes whenever an image is viewed and provides a **delete** button if ever the **delete** tag is present upon the currently viewed image.

The following code snippet corresponds to a technique to listen to JavaScript events emitted by the SharinPix Album.

```
var eventMethod = window.addEventListener ? "addEventListener" : "attachEvent";
      var eventer = window[eventMethod];
      var messageEvent = eventMethod == "attachEvent" ? "onmessage" : "message";
      eventer(messageEvent,function(e) {
          document.getElementById("message").innerHTML ="";
          if (e.origin !== "https://app.sharinpix.com") return;
          if(e.data.name === "viewer-image-viewed") {
              document.getElementById("deleteBtn").style.visibility = "visible";
              image_id = e.data.payload.image.public_id;
          }
          if(e.data.name === "viewer-closed"){
              document.getElementById("deleteBtn").style.visibility = "hidden";
          }
      }, false);
```

To get an in-depth understanding on how to capture JavaScript, please refer to this section: Capturing SharinPix events using Javascript

Capturing JavaScript events, enable us to determine whether an image is being currently viewed and which image it is. In this context, this event, is called **viewer-image-viewed.**

```
Visualforce.remoting.Manager.invokeAction(
              '{! $RemoteAction.SharinPixTagDelete.deleteImage}',
              image_id,
              '{!contactId}',
```

The above code snippet invokes the **deleteImage** method found on the previously mentioned Apex controller. The **image_id and contactId** are sent as arguments. The image_id corresponds to the id that the controller method **deleteImage** will use to determine whether the current image can be deleted or not. The **contactId** corresponds to the current Id of the record, on which the Visualforce page is present.

Setting up the Album

1. Add both Apex Controller and Visualforce Page to your current organisation.
2. Ensure that the Visualforce Page is **Available for Lightning Experience, Lightning Communities, and the mobile app.**
3. In this context, we will add the Visualforce Page to the Lightning Page on the **Contact** record.

4. Tag an image with the **delete** tag.

5. View the recently-tagged image and click on the delete button.

6. Once you click on the **delete** button, the Album will refresh. It can therefore be observed that the image has indeed been deleted.